AQEF

AI Quality Engineering Framework

A Textbook on Testing AI Systems

From Quality Contracts and Evidence to Reliable Assessment and Release Decisions

Based on the AQEF Specification 1.0-CR1 and AQEF Conformance Test Specification 1.0-CR1

First Edition · Version 1.0 · October 2026

For QA Engineers and QA Leads

A conceptual, coding-free treatment using one continuous RAG and agentic customer-support case study: Aster Support AI.

This textbook explains AQEF concepts and design choices. The published specification—not this textbook—is authoritative for normative requirements.

Publication and status note

This is the first educational edition prepared against AQEF 1.0-CR1, dated October 2026. The underlying specification is a Candidate Recommendation, not a Standard. It may change when implementers identify ambiguous or untestable requirements. The textbook should therefore be read as an edition-specific explanation, not as a statement about all future versions of AQEF.

Source, adaptation and license. Based on Igor Akymenko, AQEF — AI Quality Engineering Framework, Candidate Recommendation 1.0-CR1 (October 2026), https://github.com/igorakymenko-create/AQEF. AQEF is licensed CC BY 4.0 (https://creativecommons.org/licenses/by/4.0/). This textbook is an educational adaptation and commentary, not the normative specification. Textual explanations, selected diagrams and editorial organization have been adapted for this edition.

What the edition establishes

AQEF 1.0-CR1 describes a non-negotiable implementation core and separately claimable capabilities. Its published status notes an independent reference implementation of the core and the human_review, multi_judge and seeded_controls capabilities, with no CTS failure reported for that declared scope. These are bounded claims about implementability and specific checks. They are not certification of a QA professional or a quality guarantee for the AI system under test.

Maturity language in this book

Label	Meaning in this textbook
Core	Requirements applying to the non-negotiable implementation core.
Implemented	Scoped capability with reported implementation evidence in CR1.
At Risk	Specified capability without the implementation evidence reported at CR1 entry.
Reserved	Named capability not specified for a conformant claim in CR1.
Conceptual	Explanatory teaching material, not an extra AQEF requirement.

A maturity label is a status of a specification mechanism in this edition, never a score for the safety, correctness or usefulness of an assessed AI product.

Preface: Why this textbook exists

Traditional deterministic checks remain essential for schemas, tool arguments, access policies and operational limits. Yet they cannot, by themselves, determine whether a probabilistic system has faithfully interpreted context, selected a defensible action or sustained a coherent dialogue. An AI response can vary in phrasing without violating its contract, and can appear persuasive while violating it. Quality engineering therefore needs a richer relationship among expectations, observed behaviour, reliable assessment and decisions.

AQEF defines a structured vocabulary and a set of mechanisms for that relationship. It connects a defined Quality Contract to an Execution, its Conversation and Artifacts, Oracle Results, Aggregation and Quality Gates. The textbook approaches those mechanisms through the reasoning of the people who design tests, interpret evidence and make quality decisions. It does not teach programming or reproduce the specification as a list of configuration fields.

The same example runs throughout the book. Aster Support AI answers policy questions using retrieved information, carries context through dialogue and can call authorised tools to assist a customer. Its failures are plausible in both ordinary and exceptional situations: incomplete retrieval, unsupported claims, lost conversational context, uncertain tool outcomes and inconsistent assessment. We return to these situations from different engineering perspectives so that the framework develops as one coherent model rather than fifteen disconnected topics.

A principle shapes the narrative: a favourable verdict is useful only when its basis can be inspected. If the relevant Evidence is absent, if the Oracle is unavailable or if the Gate lacks enough actionable Results, the responsible conclusion is not an invented pass or an invented product defect. It is an explicitly limited claim.

Purpose, audience and boundaries

What the reader will learn

This book develops the ability to reason from business and product risks to Scenarios, Quality Contracts, recorded Evidence, qualified Oracle Results and release decisions. It explains why Validators, Judges and Human Reviewers make different kinds of claims; why an assessment instrument must itself be questioned; and how Aggregate Basis and Governance preserve the meaning of a decision over time.

Who should read it

The primary audience is QA Engineers and QA Leads who understand functional testing but do not necessarily write automation code. Experience with test design, defect analysis and release risk is helpful. No prior experience with LLM development, data science or statistical programming is required. Technical terms are explained before their mechanism is discussed.

What this book does not promise

AQEF does not prescribe the business risks a team must accept, the organisation of its testing team, model training or fine-tuning, prompt-writing techniques, a universal model ranking or an applicable legal regime. The framework can make judgments and approvals traceable; it cannot substitute for deciding what matters. This book likewise does not teach how to build an AI agent, establish a production sandbox or configure a CI/CD integration.

Conformance belongs to an AQEF implementation. It does not attach to a tester, an organisation or the Aster AI assistant merely because a Suite passed. Readers should keep implementation conformance, product quality findings and governance decisions as distinct assertions.

How to read and use this book

The text is designed for self-paced reading. Each chapter begins with a small set of learning objectives and moves from a quality problem to the AQEF mechanisms that address it. Aster Support AI provides one bounded example where a mechanism needs interpretation. Common Misconceptions challenge attractive but incorrect shortcuts; Chapter Summary consolidates the argument; Review Questions ask for reasoning rather than recall.

Distinguishing a rule from an explanation

Normative quotations appear with their source location in AQEF 1.0-CR1. Explanatory statements interpret those requirements; conceptual tables, analogies and diagrams help learning but must not be mistaken for a newly specified feature or threshold. MUST, SHOULD and MAY have the meaning assigned in Front Matter §9 of the specification. Where the edition leaves choices to a Project, the textbook does not invent mandatory defaults.

An effective learning routine

- Read the chapter objectives and name which existing QA concept each introduces or extends.
- Follow the Aster example and ask which observation, evidence item and Contract clause would support the conclusion.
- Study each diagram as an explanation of responsibilities, not automatically as a required deployment architecture.
- Answer the Engineering Reflection and Review Questions before checking the answer appendix.
- When implementing AQEF, return to the cited specification section and the CTS rather than treating educational prose as the conformance authority.

Learning emphasis

The textbook is deliberately coding-free and theoretical, but not detached from practice. The core practical skill is to distinguish a product failure from an instrument failure, a confirmed result from uncertainty and a release decision from the evidence and permissions that justify it.

The learning journey

The fifteen chapters form five connected learning arcs. Their order follows the path from the meaning of quality to the evidence needed for a release decision.

Part	Chapters and focus
I · Foundations	1–4: the AI quality problem; six dimensions and risk; seven principles and five layers; Conformance and maturity.
II · Test Definition	5–8: Scenario design; Quality Contracts; Datasets and Evidence; coverage and traceability.
III · Trustworthy Assessment	9–12: Validators, Judges and Human Review; Admission and Calibration; Seeded Controls; Multi Judge.
IV · Applied AI Quality	13–14: RAG, agent trajectories, multi-turn context; safety, Action Boundary, regression and operations.
V · Decisions	15: Confidence and Aggregation; Quality Gates; Reports; overrides, governance and Release Decisions.

The appendices

Appendix A provides the edition-specific capability maturity map. Appendix B gathers AQEF terminology for convenient reference. Appendix C provides concise answers to chapter review questions. Appendix D lists the primary source materials. These appendices support study; they do not replace the specification or its Conformance Test Specification.

The central question

Across every chapter, ask: What exactly do we know about the AI system, how was that knowledge produced, and what decision does it justify? AQEF supplies a structure for making the answer testable and reviewable. It cannot decide the acceptable level of product risk on the Project's behalf.

Begin with Chapter 1: Why AI Systems Need a Different Approach to Quality.

Contents

Front Matter	1
Chapter 1. Why AI Systems Need a Different Approach to Quality	8
Chapter 2. Dimensions of Quality and the Role of Risk	21
Chapter 3. Principles and Architecture	34
Chapter 4. Conformance, Maturity, and Limits of Claim	50
Chapter 5. Scenarios and Test Design	64
Chapter 6. Quality Contracts	77
Chapter 7. Data and Evidence	90
Chapter 8. Coverage and Traceability	103
Chapter 9. Validators, Judges, and Human Review	114
Chapter 10. Judge Reliability: Reference Set, Admission, and Calibration	128
Chapter 11. Seeded Controls: Testing the Instrument	144
Chapter 12. More Than One Judge	157
Chapter 13. RAG, Agents, and Multi-Turn Dialogue	168
Chapter 14. Safety, Action Boundaries, Regression, and Operational Quality	181
Chapter 15. Aggregation, Gates, Reports, and Release Decisions	196
Appendix A	214
Appendix B	216
Appendix C	218
Appendix D	228

Chapter 1. Why AI Systems Need a Different Approach to Quality

Maturity: Conceptual · Textbook First Edition v1.0

Textbook edition: First Edition v1.0 · Based on AQEF 1.0-CR1 (October 2026)

Learning Objectives

By the end of this chapter, the reader should be able to:

1. Explain why the same AI request may have several acceptable responses without concluding that correctness is arbitrary.
2. Distinguish variation in the AI system from changes in its environment or the process used to assess it.
3. Separate defects in the subject being tested from failures or uncertainty in the instrument assessing that subject.
4. Explain why an isolated passing verdict is weaker than a traceable body of evidence.
5. Describe the central problem AQEF addresses and the boundaries of what the specification does not decide.

1.1 A familiar testing instinct meets an unfamiliar subject

A quality engineer receives a request: verify that an assistant correctly answers a customer asking how to cancel a subscription. The expected answer seems simple. It should explain the cancellation policy, state the relevant steps and avoid promising a refund that the policy does not offer. Yet the response can meet these expectations in many ways. One version may open with the steps. Another may first clarify which subscription the customer has. A third may briefly cite the policy before offering the appropriate account action. The wording differs; the quality question remains the same.

Traditional testing has long dealt with variable inputs, asynchronous behavior, changing data, human interfaces and complex business rules. It would be misleading to say that older software testing assumes every execution always returns the same bytes. Its established techniques include properties, invariants, equivalence classes, boundaries and explicit tolerances. The difficulty here is more specific: for a substantial portion of generative AI behavior, the intended property concerns meaning, relevance and contextual appropriateness. It cannot necessarily be determined by comparing output with one expected string or by applying one simple structural rule.

Consider two answers to the same question. One says, “You can cancel from Account Settings; cancellation takes effect at the end of the current billing period.” Another says, “Open your account settings and choose Cancel. Your access continues until the billing period ends.” Suppose both statements are supported by the same policy. An exact-text check could reject the second, even though the difference is merely expression. That does not make exact matching bad. Exact matching remains appropriate for a fixed identifier, a legally

prescribed label or a particular machine-readable field. The mistake is to mistake one appropriate technique for a universal definition of correctness.

The opposite reaction is equally problematic. A tester might say that because wording varies, every plausible-looking response should be accepted. This confuses freedom of expression with freedom from obligations. An assistant may paraphrase, but it must not invent refund rights. It may choose a different conversational route, but it must not perform an unauthorized action. Meaningful variability is compatible with strict boundaries.



The first step in AI quality engineering, therefore, is not to replace every familiar test with a model-based judge. It is to identify which aspects of behavior admit mechanical verification and which require evaluation of meaning. The two kinds of checks can coexist. A response can be structurally valid and semantically misleading, or semantically accurate and unusable because the receiving system cannot parse it. A complete account of quality must be able to explain both situations.

This chapter is about the intellectual problem beneath those checks. Later chapters give names to the different quality dimensions, formalize the expectations in Quality Contracts and describe the instruments that evaluate Evidence. Here, the important insight is that a test result is an observation mediated by a method. It is not direct access to truth.

The expectation behind an expected result

Whenever a test says “expected,” it quietly answers three questions. What behavior matters? What variation is acceptable? Who or what can decide whether the observed behavior falls within the acceptable range? For a numeric calculation, the answers may be straightforward: a formula defines the value, a tolerance defines allowable rounding and an exact calculation checks the result. For a customer-support answer, the first question may require a policy owner, the second requires a description of acceptable meanings and the third requires an evaluator competent to interpret those meanings.

Without such separation, the test can fail for the wrong reason. A formatting change is labelled a product defect; an invented claim is overlooked because it contains the expected keywords; or a check reports success despite being unable to inspect the retrieved policy. These are not variations of one error. They are failures in different places: the subject, the test definition and the evaluation instrument.

The useful question is not, “Can this AI response be tested?” It is, “What evidence would justify this particular quality claim, and what would make that evidence untrustworthy?” That question begins the shift from checking an output to governing a quality claim.

1.2 One request, more than one possible outcome

The expression *probabilistic subject* is central to AQEF. A subject is the AI system whose behavior is being assessed. Probabilistic does not mean random in the everyday sense of meaningless or uncontrollable. It means that a repeated request may legitimately produce different observable outcomes. The explanation may involve sampling, context, changed retrieval material, model updates or the routes an agent chooses. Some of those factors are intrinsic to inference; others are properties of the wider system. A quality engineer needs to distinguish them rather than placing every difference under one broad label of AI unpredictability.

Imagine asking an assistant to summarize an internal policy. In one execution it returns three concise points. In another it returns four points, separating exceptions from the main policy. Both may be faithful to the source. If the expected result is a fixed three-item list, the second is rejected for a superficial difference. If the expectation is merely "a sensible summary," both are accepted without a credible check of omitted exceptions. The meaningful standard must describe the essential content, prohibited claims and permissible compression of information.

Variability has more than one face. *Surface variation* changes words and ordering without changing the meaning. *Substantive variation* changes which facts, qualifications or actions appear. *Context variation* arises when the system is provided different information, even though the user's question looks identical. *Execution variation* arises in workflows with tools, retries, different branches and intermediate decisions. These categories are an explanatory aid in this textbook, not a new AQEF classification or a list of conformance requirements.



Table 1.1 — Four useful kinds of variation (textbook classification, not AQEF requirements)

Type	What changes	Example in Aster	What the reviewer must ask
Surface variation	Wording, order, formatting	Two equivalent explanations of a refund exception	Did meaning or obligation change?
Substantive variation	Facts, omissions, conclusions	One answer excludes the exception entirely	Is any material claim inaccurate or missing?
Context variation	Evidence supplied to the system	Policy document or retrieval version differs	Are executions based on comparable information?
Execution variation	Tool path, attempts, intermediate actions	Different valid sequences to obtain cancellation status	Were the required outcome and boundaries respected?

A good quality argument treats the categories differently. Surface variation may require no intervention. Substantive variation can change the verdict. Context variation may invalidate a comparison unless the

inputs are aligned. Execution variation may or may not matter, depending on whether the chosen path violates a Contract or produces a different outcome. The same visible difference can have different significance under different requirements.

Why repeating a test helps, but does not settle the question

Repeated executions reveal possible instability. A single response tells us what happened once. Several responses can reveal a pattern that one would not detect. Suppose an assistant gives an accurate policy answer nine times and omits the refund exception once. The ten observations tell a materially different story from one accurate answer. They do not, however, establish the exact long-run error rate. A small collection of executions is still a limited sample, and the distribution of real requests may differ from the sampled requests.

This is a reason to distinguish a *Scenario*, which defines what is to be tested, from an *Execution*, which is one realized run of that Scenario, and from the resulting *Conversation*, which records the interaction. Their formal relationships appear later in the book. Conceptually, the Scenario captures the intention, the Execution captures the event and the Conversation captures observed content. If these collapse into one record, a reviewer may be unable to tell whether a changed answer came from a changed test, a new environment or the subject itself.

Repeated execution must also be interpreted in context. Running the same visible prompt after changing the model, system instructions, retrieval index or connected tools is not the same experiment as running the original environment again. Any difference might come from the changed environment. Conversely, identical wording does not prove identical behavior when a different source document was consulted or an agent took a new action behind the scenes.

Drift, hidden change and the illusion of stability

A system may appear stable in a demonstration but behave differently after a model version, a prompt template or the underlying knowledge base changes. These changes do not have to be malicious or even accidental. A knowledge base can receive an updated policy; a model provider can deploy a new version; a business can change what the assistant is permitted to do. The challenge is that a test result belongs to a specific configuration and moment. It cannot be transferred automatically to another configuration and moment.

This also works in the opposite direction. A test may seem to fail after a change even when the system has become more correct. If a policy used to say cancellations were immediate and now says they take effect at period end, comparing current behavior to an old answer will punish compliance with the new policy. A baseline is useful only when we understand which historical conditions it represents.

For this reason AQEF places unusual emphasis on reproducibility of the testing infrastructure. Reproducibility does not demand identical generated language. It means the execution conditions and evidence collection are sufficiently defined that differences in the subject can be studied rather than confused with differences in the test apparatus. The principle will be developed in Chapter 3; here it answers a practical question: when a run differs from another, what information would let a reviewer understand *why*?

Engineering Reflection. Two Aster responses have different wording but the same refund conclusion. What evidence would you preserve before deciding whether they are equivalent?

Aster Support AI: a first encounter

Our continuing example, **Aster Support AI**, is a fictional customer-support assistant. It uses retrieval to find product policies and an agent to call tools for account-related operations. A customer asks whether a subscription can be cancelled and whether the last payment will be refunded. The retrieval component finds policy passages; the language model composes an answer; the agent may offer to start cancellation after the necessary authorization.

Aster can fail in several independent ways. It may retrieve an obsolete refund policy, retrieve the correct policy but misrepresent it, give a faithful answer but omit the warning about continued service, or initiate an account action without the required permission. It may also perform all of these steps correctly while the test's evaluator incorrectly flags a safe paraphrase as wrong. The same incident label—"cancellation test failed"—would conceal materially different causes.

The running example matters because AQEF is intended to support analysis across a complex AI system, not to produce a favorable score for one text response. Throughout the book, we will return to the same fictional service and ask what evidence can distinguish these explanations.

1.3 Two kinds of error: the subject and the instrument

Suppose Aster tells a customer they are entitled to a refund even though the applicable policy clearly says they are not. If the actual information given to the system was correct and its output contradicted that information, there is evidence of a subject failure. Now suppose Aster correctly explains the policy, but the evaluator reports a failure because it expects the exact phrase "non-refundable." That is an instrument failure: the checker mistakes a valid expression for an invalid answer. A release decision based on either finding would look similar on a dashboard. The engineering interpretation is different.

The distinction can be stated with two questions. First: did the AI system satisfy the requirement? Second: was the method used to determine that trustworthy and available? The second question cannot be answered merely by looking at the first question's verdict. An instrument can be confident and wrong, unavailable and silent, or correct about the limited evidence it saw while missing information necessary for a broader claim.

This is familiar in other fields. A measuring instrument may be miscalibrated. A thermometer that reads too high does not make the room warmer; a scale that cannot measure does not prove the object weighs zero. Similarly, a faulty Judge does not make Aster incorrect, and a missing retrieval trace does not show that the answer was inaccurate. These analogies illuminate a conceptual distinction; they do not imply that quality evaluation can always be calibrated as precisely as a physical sensor.

The oracle problem

An *Oracle* is the means by which an observed result is assessed against expectations. In AQEF an Oracle may be a deterministic Validator, a probabilistic Judge or a Human Reviewer. A *Validator* applies a rule to Evidence; a *Judge* assesses criteria that require interpretation; a Human Reviewer supplies a human assessment where one is required or useful. Each has a different error profile. The details are the subject of Chapters 9–12.

For an easily observed property—say, whether a required tool-call field exists—a Validator can be decisive if the evidence is available and the rule is correct. For whether a policy answer is faithful to a source, a Judge may offer a useful assessment, but that judgment is not automatically ground truth. A human can review a difficult case, yet human reviewers too may misunderstand a policy, overlook a detail or disagree about whether wording is misleading. Calling an evaluator “human” does not remove the need to ask what qualifies its judgment.

Instrument reliability is not one thing. An instrument may be accurate for common examples but weak on exceptions. It may work well for billing disputes and poorly for access-control cases. It may become unreliable after its own instructions change. It may report a plausible verdict despite losing the Evidence on which the verdict should depend. Consequently, trust in an Oracle is not established by the pleasant appearance of its final score.

Four situations that should not be collapsed

Consider four distinct outcomes for the same criterion. In the first, reliable evidence indicates that Aster violated the refund policy. In the second, the evaluator completes its check and finds the answer acceptable. In the third, the evaluator cannot perform the check because the source document is unavailable. In the fourth, there is evidence to inspect but not enough basis for a defensible conclusion. These cases should not all be forced into pass or fail.

The specification gives particular attention to the difference between an unavailable Oracle and a failed subject. The chapter's foundational statement is simple: **an unavailable instrument is not a failed system**. Conversely, the absence of a demonstrated failure does not establish a pass. The interpretation must preserve what has actually been observed.

An unavailable instrument may require the system to withhold a positive decision, but the reason matters. “Not enough trustworthy assessment was available” is a different claim from “the tested product violated its requirement.” If a report erases that distinction, the team may waste effort fixing a product defect that never occurred, or, worse, approve a release on the assumption that all unassessed cases are harmless.

False alarms and missed defects

A false alarm occurs when an instrument reports a problem that is not present under the stated expectation. A missed defect occurs when the instrument fails to identify a violation that is present. Both degrade the evidentiary value of a Result, but they have different consequences. Excessive false alarms can make legitimate signals harder to distinguish from noise. Missed defects can make untrustworthy behavior appear acceptable. It is tempting to solve the first problem by relaxing criteria until nothing fails; that strategy can exacerbate the second.

Aster illustrates the trade-off. A judge that rejects every paraphrase may trigger many review requests. A judge told simply to accept any polite-sounding cancellation guidance may overlook fabricated exceptions. The way forward is not to select whichever version yields the highest pass rate. It is to articulate the requirement, characterize the instrument's behavior against known examples and preserve evidence of uncertainty. AQEF's later Admission and Seeded Controls mechanisms are designed around this separation of the tested subject from its judge.

The larger lesson is that *quality of the product* and *quality of the measurement process* must be examined separately. A report of 98 percent passing checks, by itself, does not tell us whether the checks measure meaningful requirements, whether the assessors are trustworthy or whether serious risks were represented in the sample. It is data, but not yet an adequate argument.

1.4 “Passed” is a verdict; “correct” is a claim

In everyday testing conversation, “the test passed” often serves as shorthand for “the behavior is correct.” Shorthand can be efficient when all participants know exactly which assertion passed and what that assertion covers. It becomes dangerous when the word “passed” is interpreted as a blanket guarantee about behavior that was only partially observed. A Result is an output of a defined assessment. Correctness is a wider claim about conformance to relevant expectations. The two overlap when the assessment is appropriate, trustworthy and sufficiently scoped; they are not synonymous.

A passing result always has a boundary. A Validator might establish that Aster returned a properly structured cancellation confirmation. It does not follow that cancellation was authorized, that the refund statement was accurate or that the tool applied changes to the correct account. A Judge might establish that an answer is faithful to one retrieved document. It does not follow that the retrieved document was current or that all required documents were consulted. A Human Reviewer might approve the wording but lack access to the tool-call trace. Each result has evidentiary value for particular assertions, not for every desirable property of the product.

This is why broad quality labels can be misleading. The label “good response” compresses many separate judgments, sometimes with conflicting evidence. An answer may be accurate but unhelpful, compliant with policy but slow, safe in wording but unsafe in tool use. Future chapters introduce Quality Dimensions to keep these concerns visible. At this stage, it is enough to recognize that a combined verdict requires a defensible explanation of what was combined and what was left unexamined.

Three ways a green result can mislead

First, **the requirement may be too weak**. If the check asks only whether the response mentions cancellation, a fabricated refund promise could still pass. The evaluator performed the defined check correctly; the problem lies in defining the wrong expectation. A checker cannot discover obligations that were never given to it merely by checking more accurately.

Second, **the evidence may be incomplete**. Aster's final message might say “Your cancellation is complete,” while the account tool failed before committing the change. A text-only assessment could pass a message that does not correspond to the actual outcome. The relevant evidence must include the tool operation and its result. This is not a reason to record every possible datum without restraint; it is a reason to capture evidence required by the claims we intend to make.

Third, **the instrument may not be qualified for the case**. A semantic Judge could reliably detect whether the answer restates a policy but struggle to identify subtle authorization problems in a sequence of account operations. A pass within its demonstrated competence must not be silently promoted into a pass outside that competence. The qualification question is about the instrument, not about whether the product looks plausible.

What a negative result does and does not say

The same care applies to failure. A failed result can establish that an observed execution violated a defined clause, assuming the evaluation was valid. It does not automatically establish how common the failure is, whether it will occur in production, which underlying component caused it or what release decision should follow. Those additional conclusions require their own evidence and risk context.

For example, a single Aster execution may return an unsupported claim about refunds. This deserves investigation even if the assistant succeeded on other runs. Yet the single observation does not determine the overall failure probability. It could be a rare but severe defect, a common systematic problem, an artifact of outdated source data or a case triggered by an unusual prompt. Good quality work avoids claiming more certainty than the observations support.

What the word “confidence” should prompt us to ask

A numerical confidence value can make a verdict feel scientific. But the existence of a number does not prove that the number corresponds to the instrument's actual accuracy. If a Judge reports high confidence when it is frequently wrong, the apparent precision increases risk rather than reducing it. AQEF later distinguishes whether Confidence applies, how it is represented and how the reliability of a Judge is established. Here the practical lesson is more modest: every apparent degree of certainty deserves an explanation of its source and limits.

A mechanically determined result is different. Where a Validator has the required data and an unambiguous rule, the verdict need not be assigned a probabilistic confidence score merely for symmetry. The important issue is whether the rule and the evidence are appropriate—not whether every instrument can display the same kind of number. The specification makes the distinction explicit:

> “the Result MUST instead explicitly declare that Confidence does not apply” — *AQEF 1.0-CR1*, *Volume I*, *Chapter 3*, *Everything has Confidence* (excerpt).

This is an actual obligation on an implementing engine, not merely a suggestion about good reporting. This helps prevent false equivalence between a parsed field check and a nuanced semantic judgment.

In short, a pass is a statement that *a particular check completed and found its defined condition satisfied*. The stronger phrase “the system is correct” would require broader qualification: correct with respect to what requirements, within which environment, for which cases and with what basis for trusting the checks?

1.5 From isolated evaluation to an evidence-based quality discipline

A benchmark score can be informative, just as one accurately checked response can be informative. Yet an engineering discipline must support something beyond a standalone assessment: the ability to explain the route from an expectation to evidence, from evidence to a verdict and from verdicts to an accountable decision. AQEF was written to specify that route for AI systems with probabilistic behavior.

The specification names a *Quality Contract* as the explicit artifact defining the expectations under assessment. Evidence records what happened in an Execution, including the Conversation and relevant Artifacts. An Oracle evaluates that Evidence against a clause in the Contract and produces a *Result*. Results

can then be considered by a *Quality Gate*, a mechanism that evaluates whether defined criteria for proceeding are satisfied. A *Release Decision* is the governed decision about whether to proceed, based on evidence and applicable controls. Precise rules for these objects appear later; the point here is that each link answers a different question.

A Contract answers: what should count as acceptable? Evidence answers: what did we actually observe? A Result answers: what did an identified evaluator conclude about a defined part of that evidence? A Gate answers: does the available basis satisfy a stated decision condition? A Release Decision answers: what is the authorized action, given the conclusions and remaining risks? Treating these as interchangeable would remove the traceability that makes a claim reviewable.

There is an important human boundary in that chain. AQEF can specify how expectations and approvals are represented and checked. It does not choose which business risks an organization ought to accept, which policies a support system should enforce or what level of customer harm is tolerable. Nor does a passing Gate erase responsibilities that exist outside the test. The quality engineer and decision owner still have to make reasoned judgments about context and residual risk.

Engineering Reflection. A high-confidence Judge approves an answer that cites an outdated policy. Which records would you need to decide whether the subject, the retrieval source or the instrument is responsible?

The distinction between evaluation and governance

A one-time evaluation might answer: “How often did this assistant answer the sampled questions as expected?” A governed quality process asks additional questions. Who determined what was expected? Which cases were sampled, and which known risks remain unrepresented? Which model and knowledge-base versions produced the answers? Who assessed them, and what evidence qualifies that assessor? What changed since the last review? Which unresolved failures are important enough to block release? Who is authorized to accept residual risk?

These questions do not negate the value of evaluation. They reveal why evaluation alone cannot support all the claims commonly attached to a release. A carefully designed evaluation becomes stronger when its limits, assumptions and relationship to decisions are explicit. In AQEF terms, evaluation is one operation within a wider system of evidence and governance.

The five purposes described by the specification

AQEF's Purpose section identifies five ambitions: common vocabulary; a governed lifecycle; confidence-qualified, auditable evidence; extensibility and interoperability; and a traceable basis for Release Decisions. These are the stated purposes of the framework, not measured proof that implementing it improves quality outcomes. In this book we examine the reasoning behind the structure that attempts to serve them.

Common vocabulary makes a claim interpretable beyond one person's dashboard. A governed lifecycle ties checks to a sequence of definable records rather than occasional demonstrations. Auditable evidence permits later inquiry into why a verdict was issued. Extensibility allows different systems to satisfy the same conceptual responsibilities. Traceable decisions connect the observations to an accountable release choice. None of these purposes makes a bad requirement good by itself. All depend on the quality of the underlying judgment that selects what matters.

Boundaries matter as much as mechanisms

AQEF's Scope section is explicit that the specification does not prescribe model training, general prompt-writing technique, product business logic, the laws applicable to a project or an organization's staffing structure. It also does not select a model for a team or determine which risks are worth testing. These limits protect an important division of responsibility: the specification provides a model for representing and evaluating expectations; practitioners and organizations supply the expectations and their context.

It would therefore be inaccurate to describe AQEF as a product that makes AI systems safe, a certification of tester competence or a set of guaranteed scores. Equally, it would be inaccurate to say it renders conventional QA obsolete. Deterministic components in a support platform—account identifiers, access controls, billing transactions, data serialization—still benefit from established testing methods. AQEF addresses the behavior and assessment challenges created by the probabilistic parts of the system and their interactions with the surrounding application.

One further qualification is essential. The specification's current status, **Candidate Recommendation 1 (1.0-CR1)**, identifies a text open to implementation feedback. That is not evidence of universal acceptance, formal standardization or outcome effectiveness. The framework makes explicit claims about what an implementation must do if it claims Conformance. It does not certify people, and a passing conformance test of an implementation is not a quality verdict on an AI product evaluated by that implementation.

What a trustworthy claim sounds like

Compare two statements about Aster. The first is: “Our AI assistant passed 94 percent of tests, so it is ready.” The second is: “For the specified environment and sampled customer-support Scenarios, recorded assessments found that the stated clauses were satisfied at the observed rate; these higher-risk cases and these limitations remain, and the Release Decision is documented separately.” The second is more cumbersome, but it also makes fewer unsupported leaps. It identifies the boundary of the evidence and leaves the decision visible.

Nothing in this comparison implies that a particular pass rate is a required AQEF measure, or that a release ought to proceed at a specific percentage. Those decisions depend on the Contract, the Gate, what was measured, what remains unknown and the actual consequences of errors. The disciplined move is to connect each conclusion to what the evidence supports.

1.6 Aster Support AI: following one question all the way through

We end by following a single customer question without turning it into a tool tutorial. A user asks Aster: “Please cancel my plan, and will I get my last payment back?” Two obligations are entangled. The user needs reliable information about the refund policy; the user also requests an action that may require authentication and an explicit confirmation. The quality claim must therefore cover both what the assistant *says* and what it *does*.

Suppose Aster retrieves the correct policy and says the subscription is not refundable, but it also issues a cancellation tool call before the customer confirms. A text-only evaluator may judge the response faithful; a tool-aware check may identify a violation of the authorization boundary. The case does not have one natural, universal score. It has multiple meaningful expectations and potentially conflicting observations that the decision process must preserve.

Now change the facts. Suppose Aster seeks the required confirmation, explains the policy accurately and the account tool succeeds, but the semantic Judge flags the wording as incorrect because the answer uses “payments aren't returned” instead of “non-refundable.” If we label this a product defect and rewrite Aster's response generation rules to satisfy the Judge, we are optimizing against an inadequate instrument. The correct next inquiry is whether the Judge understood the clause and the reference policy.

In a third version, the account tool trace is missing. Aster's final message states that cancellation succeeded, but the available Evidence cannot establish whether the action actually occurred. Calling the case failed would assert that the subject did something wrong. Calling it passed would assert that the intended operation succeeded. Neither is supported by the record. The responsible outcome preserves the uncertainty and makes its decision consequence explicit.

Finally, suppose the system meets every assessed expectation on the observed runs. This is meaningful evidence. It is not a proof that no other customer phrasing, hidden account condition or future policy update will expose a failure. The assessment says something specific, contingent and worth recording. The release owner may decide that evidence is sufficient for the relevant risk context, or may request more. The distinction between observation and decision is deliberate.

One case, several lines of inquiry

The Aster example establishes a pattern for the chapters that follow. First identify the requirement, not the metric that is easiest to compute. Then identify the Evidence needed to examine the requirement. Determine which Oracle can assess it, and what may make that Oracle unreliable or unavailable. Preserve the result at its actual scope. Finally ask what the available basis can support as a decision, rather than treating a positive display as its own justification.

This pattern is not an added normative AQEF workflow. It is the textbook's way of reading the problem that the specification addresses. Chapter 2 develops the dimensions and risk categories that make expectations precise. Chapter 3 shows how the fundamental principles and logical layers fit together. Later chapters take the reader inside the mechanisms that make a Result actionable or prevent an unsupported pass from flowing into a Release Decision.

The most important conclusion of this opening chapter is that AI quality is not merely an attribute of a generated answer. It is also a claim made through evidence, interpretation and judgment. When the tested subject varies, the need for clarity about the evaluating instrument grows rather than disappears. **A verdict is only as good as the instrument that produced it.** An unavailable instrument is not a failed system; a passing instrument is not proof that every relevant quality condition has been covered.

Common Misconceptions

1. “AI is nondeterministic, so precise testing is impossible.” Not so. Some properties remain mechanically checkable, and others can be expressed as explicit semantic expectations. The difficulty is choosing an appropriate Oracle and defining the limits of its evidence, not abandoning rigor.

2. “A different answer is a regression.” A difference may be an acceptable paraphrase, a meaningful defect or a consequence of changed context. The relevant requirement and environment determine its significance.

3. “A Judge's high confidence makes its verdict true.” A confidence value is itself a claim about the instrument's assessment. It does not replace evidence that the instrument is reliable for the problem under consideration.

4. “When evaluation cannot run, the AI system failed.” Evaluation unavailability tells us that a check could not provide its intended evidence. It is not the same observation as violating a Contract clause.

5. “A passed collection of tests makes a release decision automatically.” Assessment findings must still be interpreted against coverage, instrument qualification, known gaps and the relevant Gates and governance. A release is an accountable decision, not a synonym for a green score.

Chapter Summary

- AI systems can produce multiple acceptable answers to the same request; their variability does not make requirements optional.
- Traditional deterministic checks still matter wherever an observable property is rule-checkable.
- A Scenario, its Execution and the produced Conversation represent different things, helping isolate sources of change.
- Defects of the AI subject are distinct from false alarms, missed defects and unavailability of the evaluation instrument.
- A pass describes the outcome of a particular check, not universal correctness of the product.
- Credible quality claims identify their Contract, Evidence, Oracle, Result and scope of application.
- AQEF describes mechanisms for traceable assessment and decisions; it does not choose acceptable business risk or guarantee better outcomes.

Review questions

1. Two Aster executions give different wordings but convey the same cancellation policy. What evidence would you need before deciding whether the difference matters?
2. An answer contains the required refund phrase but omits a critical exception. Why can an exact-match check pass while the response is unacceptable?
3. A semantic Judge rejects an answer that a policy owner considers accurate. Which possible explanations would you investigate, and why should you avoid immediately changing the system under test?
4. A cancellation confirmation is visible in the Conversation, but the tool trace is unavailable. What can you conclude, and what cannot you conclude?
5. How does changing the retrieval index between two Executions complicate the interpretation of regression?
6. Aster answers correctly in each of five sampled runs. What claim do those observations support, and which stronger claims would remain unjustified?
7. A team reports only a single aggregate pass percentage. What additional questions would reveal whether that number is decision-worthy evidence?
8. Why is the reliability of an Oracle a separate concern from the quality of the AI system it assesses?

9. A product owner asks whether AQEF itself guarantees compliance with a company's refund policy. How would you explain the responsibility boundary?

Sources

Primary specification (verified against the GitHub repository's published 1.0-CR1 text on 24 October 2026):

1. AQEF, Front Matter §4, Preface — nature and organization of the specification.

2. AQEF, Front Matter §5, Purpose — the five stated purposes.

3. AQEF, Front Matter §6, Scope — in-scope and out-of-scope concerns.

4. AQEF, Front Matter §0, Document Status — Candidate Recommendation status.

5. AQEF, Volume I, Chapter 1, Why AI Quality Engineering Exists — deterministic checks, probabilistic subject, problems of exact matching and limits of isolated evaluation.

6. AQEF, Volume I, Chapter 2, Quality Model for AI Systems — overview of dimensions (introduced, not specified, in this chapter).

7. AQEF, Volume I, Chapter 3, Fundamental Principles — deterministic infrastructure, probabilistic subject, independent oracles.

8. AQEF, Volume II, Domain Model — Scenario, Execution, Conversation, Evidence and Result relationships.

Chapter 2. Dimensions of Quality and the Role of Risk

Maturity: Core · Textbook First Edition v1.0

Learning Objectives

After reading this chapter, you should be able to:

1. Explain why the six AQEF Quality Dimensions answer different questions and cannot be replaced by one undifferentiated quality score.
2. Distinguish a non-negotiable Safety veto from weighting that expresses differing business significance.
3. Reason from risks and defect classes to the evidence a Suite ought to seek, without confusing test count with coverage.
4. Classify an unexpected observation as a possible subject defect, uncertainty, unavailable instrument, or insufficient evidentiary basis.
5. Explain the principal quality risks of Aster Support AI and why distinct risks require distinct forms of assessment.

2.1 Why quality needs more than a single score

The previous chapter proposed an uncomfortable question: what exactly does a passing AI test tell us? A second question follows immediately. When a team reports that its AI system achieved a quality score of 92%, which kind of quality was measured? A well-formed answer may be factually wrong. An accurate answer may arrive after the customer has abandoned the conversation. A fluent agent may correctly explain a policy but initiate a transaction the customer never authorized. All three observations could receive an attractive numerical rating if the rating hides the characteristics that distinguish them.

The problem is not that numbers are inherently misleading. Numbers are useful when a reader knows what was measured, which observations entered the calculation and which values cannot compensate for one another. The problem arises when different questions are treated as if they were interchangeable units. Semantic faithfulness, execution latency and action authorization are not three versions of the same thing. Their failure mechanisms, evaluators, consequences and acceptable margins are different. Adding them into a single average before understanding those differences loses information precisely where an engineer needs it.

AQEF's response is a model of six *Quality Dimensions*: Functional Quality, Semantic Quality, Operational Quality, Safety, Reliability and Business Quality. A Quality Dimension identifies the kind of concern being assessed. It is not a universal score, a compulsory dashboard tile, or a declaration that every product must give equal weight to every dimension. It is a way to preserve distinctions that matter for evidence and decisions.



Figure 2.1 — Six quality dimensions as complementary lenses. Functional, Semantic, Operational, Safety, Reliability and Business Quality describe different properties of the same Aster interaction; an acceptable result on one does not automatically establish another.

Quality Dimension	Central question	Typical evidence and assessment
Functional Quality	Did the required structural behavior occur?	Format, required fields, permissible tool names and arguments; often a Validator
Semantic Quality	Was the meaning accurate, relevant and faithful?	Claims compared with intent and sources; usually a Judge
Operational Quality	Did the running system meet measurable operating expectations?	Recorded latency, cost, throughput and errors; often a Validator or aggregate Metric
Safety	Did behavior stay within unacceptable-action and content boundaries?	Mechanical constraints plus context-sensitive judgment
Reliability	Does acceptable behavior remain stable across runs and time?	Results aggregated across multiple Executions
Business Quality	How consequential is the behavior to the outcome served?	Scenario importance and Contract-level criticality/weighting

This table summarizes the model from *AQEF 1.0-CR1*, *Volume I*, *Chapter 2*. It should not be read as a one-to-one assignment of every Quality Dimension to one Oracle. A safety issue can have functional and semantic aspects. Operational signals can need aggregated interpretation. And a single case can engage several dimensions. The separation is conceptual: it protects the integrity of the questions we ask.

For Aster Support AI, imagine a customer asking for a cancellation and explanation of refund eligibility. One can observe a structurally valid API call (Functional), a correct explanation of the policy (Semantic), an acceptable response time (Operational), confirmation before an irreversible action (Safety), consistent decisions across repeated requests (Reliability), and the seriousness of incorrectly cancelling a paid account (Business). None is simply a more detailed way of saying “the assistant was good.” Each corresponds to a different potential failure and a different reason an organization might refuse to proceed.

The model also prevents a familiar substitution. If a product team improves stylistic fluency, that improvement should not be counted as evidence that tool authorization became safer. If faster inference reduces latency, it says little by itself about whether the answer cites the correct policy. The discipline is not to discard aggregate indicators, but to insist that the components remain intelligible and available for inspection.

2.2 Functional Quality: what can be decided by a rule

Functional Quality concerns properties of an interaction that can be stated as observable conditions. Did Aster produce a response in the agreed structure? Were the required fields present? Did it choose an allowed tool, and did its tool arguments satisfy the constraints of that tool? If a workflow requires verification before a cancellation action, did the relevant sequence of recorded events satisfy that requirement? These questions can often be answered deterministically from the captured Conversation and Artifacts.

A *Validator* is AQEF's deterministic Oracle: a mechanism that applies a rule-based check to specified evidence. The term *Oracle* denotes the role of deciding whether evidence satisfies a Quality Contract; an actual Oracle is a Validator, Judge or Human Reviewer. An *Artifact* is supporting evidence such as tool traces and recorded durations. These are not synonyms for a test case, and their separation matters. A rule may be perfectly deterministic while assessing the wrong artifact or an outdated contractual expectation.

The specification states a testable obligation:

> "A Validator MUST return the same Result for the same input, and MUST NOT carry a Confidence value" — *AQEF 1.0-CR1*, *Front Matter §8*, *Validator*.

The textbook's interpretation is narrower than "all deterministic tests are correct." The rule constrains an implementation of a Validator. It does not guarantee that the selected rule represents what stakeholders actually needed, that the evidence contains all relevant events, or that a failure is attributable to the subject without further diagnosis. A Validator can consistently answer a poorly framed question. A precise but irrelevant test can be reliable as a computation and weak as quality evidence.

Consider a rule requiring an account action to have a recorded authorization. If the trace includes the actual authorization event, checking its presence may be mechanical. But a record saying that "consent was discussed" may not prove the user actually granted it. Translating the business requirement into a verifiable condition requires analysis before implementation. The importance of Functional Quality is not that it avoids professional judgment; it is that, *after* the criterion has been defined and evidence identified, the check can be repeatable.

A second danger is over-assigning functional checks to semantic questions. A response containing the words "refund policy" is not necessarily a faithful explanation of the policy. A perfectly formatted citation can reference the wrong document. A schema cannot determine by itself whether an explanation omits an important exception. Whenever the required property depends on meaning rather than mechanically checkable structure, a different assessment question is being asked.

2.3 Semantic Quality: correct meaning, not expected wording

Semantic Quality examines whether Aster's statements mean what they should mean. Faithfulness to the retrieved source is different from grammatical fluency. Factual correctness is different from confident delivery. Relevance is different from politeness. These distinctions matter because systems can succeed on the most visible aspects of communication while failing on the claim that customers rely upon.

A customer asks, “If I cancel today, will the charge from yesterday be refunded?” Suppose Aster cites a paragraph that says cancellation prevents renewal but does not automatically refund prior payments. One answer paraphrases it accurately. Another says the payment will certainly be refunded. Both could be fluent, helpful-sounding and appropriately formatted, yet the second changes the customer's understanding and may induce a costly decision. A purely lexical comparison is a poor instrument for this difference.

AQEF uses a *Judge* for probabilistic, criteria-based assessment of semantic or subjective qualities. The specification distinguishes the verdict from the Judge's uncertainty about that verdict:

> “A Judge's Result MUST carry a Confidence value, reflecting the Judge's own uncertainty in the verdict — separate from the verdict itself.” — *AQEF 1.0-CR1*, *Front Matter §8*, *Judge*.

This statement is normative for the Judge Result. It does not turn Confidence into a guarantee of accuracy. A confident Judge can be wrong. A cautious Judge can identify a real defect. The quality engineer needs to ask whether the criteria were explicit, which evidence was available, how the Judge was qualified for this kind of assessment and how disagreements can be resolved. Later chapters treat those safeguards in depth; here they show why a Semantic Quality result cannot be interpreted as an ordinary keyword match wearing a numerical score.

Semantic Quality is also not one indivisible property. A policy answer can be relevant but inaccurate, accurate but omit an essential qualification, faithful but unhelpfully phrased, or precise but inappropriate for the customer's language needs. One broad “answer quality” score risks concealing a critical defect if harmless strengths compensate for a consequential omission. The engineer should therefore think about the claims and criteria that actually determine acceptable behavior. AQEF's Quality Contracts give those expectations an explicit home.

For a retrieval-based system, the evaluator must ask which source was available and which portion of the final response is supported by it. If Aster was handed an obsolete policy, its answer might be a faithful rendering of wrong context. If it was handed a current policy and contradicted it, a different failure hypothesis becomes plausible. Semantic assessment does not by itself solve the attribution problem. It identifies which assertions need evidence and which distinctions a later investigation must preserve.

2.4 Operational Quality: a correct answer can still fail as a service

An AI assistant is an operating service, not merely a generator of text. Aster may produce a thoroughly correct cancellation explanation only after an excessive delay. It may consume expensive resources on routine requests, become unreliable under concurrent load or repeatedly retry a failed dependency. A quality model concerned solely with the final wording would overlook failures customers and operators experience directly.

Operational Quality covers measurable behavior such as latency, cost, throughput and infrastructure-level reliability. When a duration or resource quantity has already been captured as evidence, checking it against a Contract threshold is ordinarily a deterministic task. The test does not need a semantic Judge to decide whether the recorded latency exceeded a defined bound. However, the interpretation still depends on the chosen service objective and on whether the measurement represents the intended scope.

Suppose Aster spends most of its time waiting for a policy search service rather than generating its answer. A slow response is observable. Yet the observer should not immediately diagnose the language model as the cause. Operational quality asks first what happened; root-cause analysis can then use traces, timings, infrastructure errors and environment versions to distinguish a slow tool, congestion, repeated calls or model inference. Mixing those layers can lead to an expensive “fix” that does not address the problem.

Averages alone are often inadequate here. Two services may have the same mean response time, while one delivers almost all responses consistently and the other alternates between very fast and extremely slow outcomes. Depending on the risk, the tail of a latency distribution can be more relevant than its center. Similarly, an average cost can hide a rare route that calls external tools many times. The choice of summary is a product-level judgment; AQEF does not prescribe one universal operational metric for every system.

Operational measurements also interact with the evidence model. An infrastructure-level timeout can leave too little evidence for a substantive semantic verdict. The correct conclusion may be that the execution did not complete or that an assessment is unavailable, rather than that the answer was semantically wrong. Treating every kind of interruption as a failed content check confuses the quality of the service with the quality of a nonexistent response. The dimensions remain connected, but their explanations must remain distinct.

2.5 Safety: when compensation is the wrong model

Some failures should not be traded against excellent results elsewhere. If Aster initiates a cancellation without appropriate authorization, excellent wording and low latency do not erase the unauthorized action. This illustrates the difference between a *hard veto*, a rule whose failure blocks a relevant decision independently of compensating scores, and a weighted criterion, where relative importance influences an aggregate.

Safety includes both mechanically recognizable prohibited behavior and situations that require contextual judgment. An attempt to invoke an unauthorized tool can leave a concrete trace. Whether a polite-sounding request is an attempt to bypass a restriction may require interpretation of the conversation and its context. Restricting all safety assessment to a pattern match misses subtle violations; using only a probabilistic Judge can make explicitly prohibited actions appear negotiable if they coexist with otherwise favorable behavior.

In Volume I, Chapter 2, AQEF describes Safety as drawing on both Validator-bound checks and context-sensitive judgment by a Judge or Human Reviewer. In the specification's decision model, a veto is not simply a large negative weight. A sufficiently high weighted average cannot necessarily compensate for violating an independently blocking condition. This protects the meaning of boundaries that an organization treats as non-negotiable.

The boundary of the current edition matters: **Action Boundary** is an *At Risk* capability in AQEF 1.0-CR1. Aster's unauthorized action is a meaningful risk and the idea of constraining tools is important, but this chapter does not present the At Risk Action Boundary mechanism as a settled, implemented feature. The conceptual concern with unauthorized action and the maturity status of a particular enforcement capability are separate statements.

Safety decisions are not always simple refusal questions. Consider a customer who asks for a cancellation status but provides inconsistent identity details. Aster should not invent a successful action merely to be helpful. Nor should it treat the absence of a tool response as proof that the customer is ineligible. The relevant evidence includes what was requested, what permissions were available, which action was attempted and what outcome was recorded. A strong safety assessment preserves the difference between *attempt*, *execution* and *reported result*.

Engineering Reflection. An Aster run has perfect semantic scores and excellent latency but attempts a prohibited account action. Why would a weighted average alone be inadequate evidence for permitting release?

2.6 Reliability: behavior across runs, not within one answer

Reliability asks whether acceptable behavior is stable under repetition and over time. A single successful answer cannot establish it. Aster might accurately explain refund eligibility on Monday, omit a crucial exception on Tuesday and recover on Wednesday without the visible customer question changing. The pattern deserves analysis even when the majority of individual runs pass. One result is an observation; reliability is a property inferred from a collection of observations and their circumstances.

This makes *Execution* and *Scenario* different counting units. A Scenario is a defined testable situation. An Execution is one realization of that Scenario inside an Environment. Repeating one Scenario twenty times can reveal instability in that situation but does not provide the breadth of twenty distinct Scenarios. Likewise, twenty cases run once each explore more breadth but may reveal little about within-case variation. Neither denominator is inherently superior; they answer different questions.

Reliability also has a temporal dimension. If a policy index is updated or a model version changes, new behavior must be compared with the conditions actually relevant to the change. A shift in the answers is not automatically a regression: it may reflect an intentional correction. Conversely, unchanged top-line pass rate can conceal a redistribution of defects into more serious scenarios. The reference point and the risk classification matter as much as the visible numerical trend.

Volume I describes Reliability as a Metric-level property, assembled across Results. This matters because a single Oracle verdict does not have the scope to certify stability. A separate result might assess one answer's correctness, but the reliability conclusion needs cross-execution evidence and an honest description of the sample. The model's purpose is not to require identical generated text; it is to expose whether the system preserves acceptable behavior when variation is legitimate.

Aster provides an instructive distinction. Five runs may all call slightly different search paths but consistently explain the same policy and refrain from unauthorized actions. Those differences could be acceptable reliability behavior. By contrast, five identical-looking responses could all depend on an outdated policy and consistently mislead customers. Consistency is not the same thing as correctness. A reliable failure is still a failure.

2.7 Business Quality: why failures differ in consequence

An engineering team cannot investigate every possible AI mistake with equal intensity. It must decide which failures would materially affect customers, operations, obligations or organizational outcomes. Business Quality gives the AQEF model a way to express how much a Scenario matters relative to others. It does not assert that financial cost is the only kind of value. Harm, trust, reversibility and significance to the product's purpose can matter even when they are difficult to monetize.

Take two Aster failures. In one, a low-stakes response uses an awkward phrase. In another, the system incorrectly asserts that cancelling a subscription guarantees a refund and then proceeds to an account action. A flat count calls both “one failed scenario.” That arithmetic can be mechanically correct yet unsuitable for a decision if stakeholders consider the consequences radically different.

Business Quality therefore intersects with *risk*. Risk is not only a probability of an undesirable event; for quality planning it also concerns what follows if the event happens, who is affected, and whether it can be detected or reversed. This textbook uses these questions as a practical reasoning aid. It does **not** propose a numerical risk formula or claim that AQEF requires one. Different products may legitimately choose different priority schemes while retaining explicit evidence and reviewable decisions.

The specification connects criticality and weighting to aggregation. A higher-stakes Scenario can exert more influence on an aggregate than a low-stakes Scenario when the Contract and decision mechanism express that priority. But weighted aggregation is not permission to blur distinct legal, safety or authorization boundaries. A hard veto and a high weight encode different decisions about whether compensation is allowed. Their effects must remain interpretable to the reader of a Report.

The human boundary deserves emphasis. A specification can make a risk decision visible and traceable; it cannot infer the acceptable harm threshold from the test output. Stakeholders must decide what the AI system is allowed to do, what consequences merit a veto and which trade-offs can reasonably be considered. Those decisions belong to the product's governance context. Engineering helps make them explicit, testable and reviewable; it does not automatically make them ethically or commercially correct.

2.8 Hard vetoes, weighting, and misleading aggregates

Suppose Aster is assessed against ten different behaviors. Nine are routine FAQ responses and one concerns authorization for an account cancellation. Nine FAQ cases pass; the authorization case fails. A dashboard showing “90% passed” is not false, but it is dangerously incomplete if the authorization failure is a release blocker. A reader needs the dimension, the contractual treatment of that result and the count and scope of the evidence behind the number.

Conversely, some requirements are legitimately compensable or weighted. If a product values concise answers but permits a slightly longer explanation when it is more faithful to source material, the assessment may allow trade-offs within explicit criteria. The critical point is to decide the compensation rule consciously rather than allowing a general average to create it by accident. Where one property must always hold, a gate or veto must reflect that. Where relative importance matters, weighting can make consequences visible.

The evidence base itself can also mislead. Imagine a pass rate of 95% from twenty repeated executions of one easy Scenario, compared with 95% from a diverse set of two hundred scenarios. Those figures do not

support the same breadth claim. AQEF's *Aggregate Basis* requires reporting the relevant counts of actionable and non-actionable Results, distinct Scenarios and Executions. The idea is simple: the denominator is part of the claim, not fine print beneath it.

> “Every aggregate a Quality Gate reads MUST therefore state its Aggregate Basis” — *AQEF 1.0-CR1*, *Volume I*, *Chapter 6*, *Aggregate Basis* (excerpt).

The textbook's interpretation is that the reader needs to see both the numerical outcome and what produced it. Reporting a strong aggregate while hiding missing evaluation, concentrated coverage or non-actionable results can falsely imply comprehensiveness. A report that exposes weak evidence may be uncomfortable, but it preserves the decision maker's ability to judge the limitation honestly.

2.9 Risk is the starting point of coverage

Traditional test plans sometimes use test count as the easiest visible indicator of progress. Counting cases can serve planning, but it says very little about which important failures can be detected. If one hundred scenarios all ask nearly equivalent FAQ questions, they may exercise a small portion of the product's meaningful risks. If ten carefully chosen scenarios explore authorization, policy grounding, exceptions, tool failure and inconsistent user instructions, they may cover a broader range of high-consequence failure hypotheses. Neither number proves coverage by itself.

A *defect class* is a category of failure the assessment is intended to detect: for example, unsupported policy claims, skipped authorization, invalid tool arguments or missed exceptions. A risk identifies why the class matters in a particular context. An effective traceability conversation can then proceed in stages: what could go wrong; why would it matter; what evidence would reveal it; which Contract criterion applies; which Oracle can assess that criterion; and which decision depends on it? This chain helps distinguish merely executing activity from obtaining relevant evidence.

Volume VIII develops test design around risks and defect classes. The practical lesson for this chapter is that coverage is a claim about important possible failures, not a trophy for producing many test cases. A team may reasonably state that a risk is not yet covered. The problem is when a report's green appearance leads readers to assume the gap does not exist. Coverage is credible when its omissions are visible.

Risk-based planning also guards against a subtler mistake: using one powerful evaluator as a universal substitute for diverse evidence. A Judge that reliably examines the final explanation cannot by that fact alone prove that no unauthorized tool attempt occurred. A latency measurement cannot establish policy faithfulness. A semantic assessment of one conversation cannot demonstrate stability over time. To cover a risk, the evidence must be capable of revealing the corresponding defect class. Otherwise the test may be sophisticated but irrelevant.

In Aster, a useful coverage discussion starts with a cancellation/refund policy risk inventory. The team asks whether the test set includes outdated policy retrieval, correct retrieval followed by incorrect explanation, unauthorized cancellation attempt, failure to complete a tool call, misleading claim of completed action, and unstable treatment of exceptions. It also asks which cases are not represented and whether the instrument

can detect each class. This is not a prescribed universal list for customer-service systems; it is a reasoned example of how coverage is derived from a product's actual behavior and stakes.

Engineering Reflection. A report shows fifty passing tests, all concerned with response wording. Which cancellation-related risks remain unsupported by that evidence, and what different artifacts would be needed to assess them?

When the risk map changes, the coverage claim changes

A risk-based test set is not a permanent inventory of danger. Aster may add a new payment method, enter a new jurisdiction, change from read-only responses to tool-triggered actions, or incorporate a different customer-identity workflow. Each change can alter the consequence of a defect even when the user interface seems largely the same. An earlier set of passing scenarios remains historical evidence about the environment in which it ran; it is not automatically evidence that the new version covers the same risks.

Consider the transition from merely suggesting cancellation to executing it. The semantic question, “Did the assistant describe the cancellation policy faithfully?”, remains relevant. But an entirely new class of concern appears: the assistant might take action without valid authorization, execute twice after retry, or claim that a failed action succeeded. A team that simply reruns the old explanation tests may receive a reassuring pass rate while never exercising the new exposure. The source of the gap is not an incorrect Oracle verdict; it is an incomplete definition of what requires assessment.

The reverse can happen too. Suppose a once-critical tool is retired, and Aster no longer acts on accounts. Some blocked-path or unauthorized-action risks may no longer apply to the current product. Keeping every historical case in the active release denominator may obscure current priorities. Historical results remain valuable for audit and regression investigation, but active coverage should correspond to the risks of the present subject and environment. This distinction makes a coverage statement time-bound rather than timeless.

A QA lead reviewing a release should therefore ask what changed in the model, sources, permissions, tools and product policy; which risk hypotheses were added, retired or reprioritized; and whether the evidence-producing instruments still assess the intended criteria. This is not a new AQEF-prescribed change-control checklist. It is a practical implication of describing coverage against risks rather than against a static quantity of test cases.

2.10 Classifying what we observed

An unexpected observation should not be hurried into a single bin labelled “AI defect.” The source of uncertainty determines which next action is defensible. Aster may genuinely violate a policy; the observed answer may be ambiguous; a Judge may be unavailable; or the team may lack enough cases to justify a broad conclusion. These situations can look similar in a dashboard while carrying different epistemic meanings.

Possible subject defect. The available evidence supports that Aster violated a defined expectation. For example, the recorded output asserts a refund despite contrary policy evidence. The conclusion remains scoped to the specific criterion and execution; root cause and frequency require additional inquiry.

Uncertainty or inconclusive assessment. Available observations do not support a stable verdict with sufficient confidence. Perhaps the policy text is ambiguous, the Judge's assessments disagree, or relevant context was not preserved. A careful process communicates the uncertainty rather than silently transforming it into success or failure.

Instrument unavailable. An Oracle could not perform its assessment. This is a failure to obtain an evaluation, not proof the subject behaved incorrectly. AQEF distinguishes such a state, including the disposition `oracle_unavailable`, from a failed clause. The difference should survive aggregation and reporting.

Insufficient basis. Individual observations may be valid while the collection is too narrow, incomplete or non-actionable to support a Quality Gate's required basis. This is a decision-level limitation rather than a product defect discovered in one conversation. More sampling, additional scenarios or resolving non-actionable Results may be necessary before a supported decision is possible.

These classes should not be confused with an exhaustive normative list of AQEF Result dispositions. They are **the textbook's explanatory decision categories** for a QA engineer investigating an observation. Their purpose is to prevent a causal mistake: changing the subject because the instrument failed, changing the threshold because the evidence is thin, or declaring the product good because no evaluation took place.

A useful diagnostic question is, “Where did the chain stop?” If the system produced an observable contractual violation, assessment has substantive content. If the Oracle did not run, the chain stopped at the instrument. If Results exist but do not support the Gate's minimum basis, the gap appears at aggregation. If the requirement itself is ambiguous, the chain may never have had a stable target. Different failures call for different kinds of correction.

2.11 Aster Support AI: a risk-centered view

We can now read Aster as a collection of quality claims rather than as one “chatbot test.” The customer wants to understand cancellation and refund eligibility; Aster must retrieve the applicable policy and may request a tool-mediated account operation. The primary risks include misleading policy claims, wrongful actions, missing authorization evidence, unstable treatment of policy exceptions and operational interruption during account-changing workflows.

The following is a **teaching example**, not an AQEF-required risk matrix or a universal prioritization model:

Aster risk	Most relevant dimensions	Evidence question	Why a green overall score may conceal it
Unsupported refund promise	Semantic, Business	Does the explanation match the applicable policy?	Fluency and schema checks may still pass
Unauthorized cancellation attempt	Safety, Functional, Business	What account action was attempted and authorized?	Many ordinary answers can average above one critical violation
Invalid tool-call arguments	Functional, Safety	Do captured arguments satisfy declared constraints?	Correct final wording may hide a failed action

Aster risk	Most relevant dimensions	Evidence question	Why a green overall score may conceal it
Unstable handling of an exception	Reliability, Semantic	Do repeated comparable executions respect the same policy?	One successful run shows no stability
Slow or interrupted transaction	Operational, Reliability	What duration and error traces were recorded?	Semantic correctness alone omits service behavior
Missing trusted assessment	Evaluation trust across dimensions	Was the responsible Oracle available and qualified?	Absence of evidence can masquerade as absence of defects

Imagine Aster's report states that all ten sampled FAQ responses were relevant. This supports a narrow claim about relevance in those samples, assuming the evaluator was trustworthy. It does not resolve authorization, tool arguments, cancellation accuracy or repeated behavior. If the decisive cancellation Scenario was never run, the quality conclusion must state that gap. If it ran but the instrument was unavailable, the gap is different. If it ran and exposed an unauthorized action, the finding is different again. The six dimensions help keep those findings from collapsing into one attractive headline.

This prepares the next chapter's architectural question. If different quality questions demand different evidence, how can a testing system preserve the contracts, observations, evaluators and decisions without allowing them to drift into an opaque score? AQEF's principles and layers address that problem. The present chapter has supplied the essential prerequisite: before designing the apparatus, determine which risks matter and what would count as credible evidence about them.

Common Misconceptions

1. “A high overall score proves quality.” A high aggregate can coexist with a hard-veto failure, a missing risk class or a small evidentiary basis. Ask which dimensions and decision rules the score actually represents.

2. “A Safety failure can be compensated by better fluency.” Not when the relevant requirement is a hard veto. Whether compensation is acceptable must be explicit, not inferred from an average.

3. “Functional testing is unnecessary for generative AI.” The subject can be probabilistic while its tool arguments, schema and recorded steps are mechanically checkable. Traditional rule-based checks retain a real place.

4. “Many tests mean strong coverage.” A large test count can explore one narrow class of behavior. Coverage must be explained in terms of risks, defect classes and relevant evidence.

5. “An unavailable or uncertain evaluation means the product failed.” Instrument unavailability, uncertainty, subject violation and insufficient aggregate basis are different states with different implications.

Chapter Summary

1. AQEF distinguishes six Quality Dimensions so that materially different kinds of quality are not hidden inside a single score.
2. Functional and Operational concerns often support deterministic checks; Semantic concerns generally require criteria-based judgment.

3. Safety can combine mechanical prohibitions with contextual assessment and may demand a hard veto rather than compensable weighting.
4. Reliability concerns repeated or temporal behavior; Business Quality makes differences in consequence explicit.
5. Weighting expresses relative importance, while a veto expresses that a particular failure cannot be compensated by successes elsewhere.
6. Coverage is about risks and defect classes rather than the number of test cases or a mandatory percentage.
7. Subject defects, inconclusive assessments, unavailable instruments and insufficient aggregate basis support different conclusions.
8. Every aggregate must be read with the evidence and denominator that support it.

Review questions

1. Aster returns a policy answer that is grammatically excellent, accurately formatted and factually wrong. Which Quality Dimensions do the observations concern, and why would averaging them be misleading?
2. A release has 99 passing low-risk FAQ scenarios and one failure involving unauthorized account action. Explain why a 99% pass rate may be insufficient for the decision.
3. How would your risk argument change between an assistant that only drafts refund guidance and one permitted to execute refund actions?
4. Why is a repeated Execution of one Scenario not equivalent to an additional distinct Scenario when describing coverage?
5. An Aster Judge cannot run after the retrieval service changes. What can the team conclude about the product's correctness and about the available evidence?
6. A report shows a stable overall pass rate but a rising number of failures on high-criticality Scenarios. What questions should a QA lead ask before accepting the report's headline?
7. Compare the evidence needed to establish that an answer is faithful to policy with the evidence needed to establish that a tool call was authorized.
8. When might the absence of a definitive assessment result be more important for a Gate than the presence of several successful results?
9. What information must be made visible to compare a pass rate derived from hundreds of repeated runs of one case with a rate derived from many distinct cases?

Sources

Normative and conceptual sources checked against the 1.0-CR1 repository text:

1. AQEF, Volume I, Chapter 2 (Quality Model for AI Systems)

2. AQEF, Volume I, Chapter 6 (Quality Decision Model, especially Aggregate Basis and Gates)

3. AQEF, Front Matter §8 (Terminology)

4. AQEF, Volume II (Domain Model)

5. AQEF, Volume VIII (Test Design, risk-based testing and Seeded Controls)

6. AQEF, Volume XI (Reporting & Analytics)

7. AQEF, Front Matter §0 (Document Status) for the At Risk status of Action Boundary.

Chapter 3. Principles and Architecture

Maturity: Core · Textbook First Edition v1.0

Learning Objectives

After reading this chapter, you should be able to:

1. Explain the seven Fundamental Principles of AQEF in their specified order and identify the failure each is designed to prevent.
2. Distinguish the five logical layers of the reference architecture and describe what evidence passes between them.
3. Follow a quality claim from Quality Contract through Evidence, Result and Quality Gate to a governed Release Decision.
4. Differentiate Project, Environment, Suite, Scenario, Execution and Conversation without collapsing definitions into runtime observations.
5. Explain which decisions AQEF constrains and which choices remain with an implementation or the people governing a Project.

3.1 From quality concerns to an architecture of evidence

The first two chapters established two linked difficulties. An AI system can produce several acceptably different responses to one request, and even a perfectly recorded response can be misjudged by the instrument evaluating it. Quality therefore cannot be reduced to a comparison of two strings, nor to a single score detached from how the score was obtained. Once this is acknowledged, the engineering challenge becomes more precise: how can an organization preserve a reliable connection between what it expected, what actually happened, how it was assessed and the action it chose to take?

AQEF addresses that connection through principles and a reference architecture. A principle explains why certain shortcuts are unacceptable; an architecture identifies which responsibilities must remain distinguishable so that those principles can be made inspectable. Neither is a substitute for the other. A beautifully drawn architecture can still conceal shared model failures, weak requirements or retrospective alteration of results. Conversely, a list of excellent principles without a stable information model leaves participants unable to agree on which record a decision actually used.

Consider Aster Support AI, the customer-support assistant introduced earlier. It retrieves policy documents, produces explanations and may use tools to update a service account. Suppose the team reports that the new release 'passed.' This sounds like an outcome, but it conceals several questions. Did the test describe authorized behavior before it was run? Was the observed tool action recorded? Was the retrieval context preserved? Was the semantic judgment made by an independent mechanism? Was the judge permitted to decide? Did the gate have enough actionable evidence? Was an exception approved by the right decision-maker? These are questions about a chain, not a single test.

The chapter's organizing idea is that a release-level claim must be reconstructible. Reconstructibility is not the same as correctness: a fully traceable but badly chosen requirement can still produce a poor product decision. The benefit of an architecture is more modest and more defensible. It makes specific claims inspectable, separates kinds of uncertainty and identifies the point at which judgment enters the process. The specification's constraints concern the implementation that represents these objects, not a certification of the team's judgment or the AI product itself.



Figure 3.1 — A chain of evidence.

A Quality Contract defines the relevant expectation; an Execution produces Evidence; an Oracle produces a Result; Quality Gates evaluate the usable Results; and a Release Decision records the outcome of governance. This figure is a pedagogical simplification: the full architecture includes aggregates, snapshots, reports and approvals, covered later in this chapter.

The principles below appear in **exactly the order used by AQEF 1.0-CR1, Volume I, Chapter 3**. They are not interchangeable slogans. Each preserves a different boundary. Read them as safeguards against seven different ways a superficially successful testing process can produce misleading evidence.

3.2 Principle one: Contracts over Assertions

A conventional assertion can be extremely effective when the expected property is simple. A service should return a particular status; a calculation should obey an invariant; a payload should match a schema. The limitation appears when the assertion is also the only accessible record of the requirement. Someone reviewing the release may know that a check passed, without knowing who chose the criterion, why its threshold is appropriate, or whether a developer quietly weakened it during a repair. The issue is not code as such. It is the fusion of *what must be true* with the implementation detail of *how to check it*.

AQEF gives these responsibilities different homes. A *Quality Contract* is a declarative statement of the conditions a given behavior must satisfy, including the Oracles that will assess those conditions and the thresholds relevant to their Results. An *assertion* is one possible expression of a check inside a technical

implementation. The Contract survives changes of testing mechanism; the mechanism does not thereby acquire the authority to change what the Contract means. In Aster, a requirement that refund eligibility be explained faithfully should exist as an inspectable expectation, not only as a phrase hidden in a judge prompt or a test harness.

The normative direction appears in Volume I, Chapter 3, under **Contracts over Assertions**:

> *"AQEF requires expectations to be expressed as a Quality Contract: a declarative, inspectable artifact, separate from any Execution's implementation" — AQEF 1.0-CR1, Volume I, Chapter 3, Contracts over Assertions.*

The textbook interpretation is that separation protects the integrity of the question being tested. A Contract does not guarantee a wise question; it guarantees a place where that question can be reviewed independently of a particular run. The later chapter on Contracts explains how requirements compose between organizational levels and how conflicts must surface rather than silently disappear.

An instructive failure occurs when Aster occasionally answers a refund question with a correct paraphrase that fails a brittle text comparison. The team could change the assertion to accept almost any answer mentioning a refund. The build becomes greener, yet no one has shown that the system is faithful to policy. By making the substantive criterion explicit, a Contract forces the team to confront the underlying decision: should the check evaluate a fixed phrase or fidelity to a source? The second may require a Judge rather than a simplistic pattern. Making the distinction visible is the principle's purpose.

A Contract's explicitness also makes disagreements productive. Product owners may care most about explaining policy clearly; compliance stakeholders may insist that a specific exception never be omitted. Those concerns can be expressed separately instead of competing in an opaque score. This does not remove governance or professional judgment. It creates an artifact through which they can be exercised and audited.

3.3 Principle two: Validation before Evaluation

There are two substantially different kinds of question about AI behavior. A *Validator* can apply a deterministic rule to evidence such as a schema, a forbidden pattern or a recorded latency. A *Judge* evaluates a criteria-based semantic property that cannot necessarily be settled by a fixed rule. AQEF calls the first process *Validation* and the second *Evaluation*. Their distinction has practical consequences for cost, ordering and interpretation.

The specification's explicit requirement is short:

> *"Validators MUST run before Judges for a given Execution." — AQEF 1.0-CR1, Volume I, Chapter 3, Validation before Evaluation.*

The reason is not that semantic quality matters less than structural quality. Rather, a deterministic failure may make later assessment irrelevant or misleading. Imagine Aster generating an eloquent explanation inside a malformed payload that the application cannot display. A Judge might call the prose accurate, but the product still failed a required functional condition. The recorded sequence should make the functional

failure visible before spending resources on semantic evaluation. That is the familiar fail-fast idea applied to two different kinds of Oracle.

There is an important boundary here. 'Validate first' does not mean 'every Validation failure proves that the AI model is faulty.' A Validator can be unavailable, can receive incomplete evidence or can reveal a harness problem rather than a defect in generated behavior. The Results model must distinguish these situations rather than turning a failed assessment process into a failed subject. Similarly, the principle is not permission to omit semantic assessment merely because convenient mechanical checks passed. A structurally valid explanation can still be false.

Volume V develops this ordering into an execution pipeline: Validators precede Judges and, where multiple Validators apply, cheapest-first is recommended. The chapter's teaching point is the underlying dependency. Before asking whether an answer means the right thing, establish that its mechanically required form and action boundaries are assessable. An implementation may optimize how separate checks are scheduled, but it cannot claim the same evidentiary meaning if semantic verdicts are silently substituted for failed mandatory validation.

The result is a richer meaning of 'passed.' Aster might pass all structural Constraints, but that is not the same as passing the Quality Contract in full. It merely means the path to the next kind of assessment is not blocked by those checks. Quality evidence accumulates through distinct obligations; it is not created by treating a single favorable verdict as universal.

3.4 Principle three: Independent Oracles

An *Oracle* is the mechanism that determines whether a Conversation satisfies a Quality Contract. AQEF realizes that abstract role through Validators, Judges and Human Reviewers. The Oracle is an instrument: it can be well designed, badly designed, unavailable or insufficiently qualified. When the same mechanism that produced the disputed behavior is effectively allowed to certify it, apparent agreement may tell us little about correctness.

Volume I, Chapter 3 makes the boundary explicit:

> *“An Oracle MUST NOT be the same system, weights, or decision process as the AI system it evaluates” — AQEF 1.0-CR1, Volume I, Chapter 3, Independent Oracles.*

This rule is stronger than an informal suggestion to 'have AI check AI.' Independence is about the process that can fail, not merely about the visible names of two services. Two different prompts sent to the same underlying decision process may reproduce the same blind spots. Two evaluators trained on related material may agree on an unsupported claim because both share the same misconception. Agreement becomes most informative when the evaluator is capable of making an error different from the subject's.

Aster offers a concrete case. Suppose the support agent fabricates an exception to a refund policy because it mistakes a customer-forum post for official guidance. A Judge with the same retrieval source and a closely related reasoning process may repeat the mistake. Both may speak with high Confidence. The joint verdict creates an impression of corroboration, yet the supposed independent check has merely echoed the error.

The meaningful question is whether the evaluator had an independently justified basis for recognizing an unsupported claim.

Independence is not an absolute guarantee of correctness. The specification also distinguishes a MUST-level prohibition on identity of the decision process from a SHOULD-level preference to avoid shared failure modes where feasible. Those are not equivalent obligations. An evaluator may be distinct yet correlated with the subject. Good engineering practice examines the residual risk rather than declaring independence because the service endpoints differ.

This principle also clarifies the role of human review. A Human Reviewer is another kind of Oracle, not an automatically perfect ground truth. Human judgments can disagree, contain bias or rely on incomplete information. Their value comes from an independently inspectable judgment basis and an appropriate place in the assessment lifecycle, not from an assumption that humans are infallible. AQEF's later treatment of Judge Admission and seeded controls strengthens the case by asking for evidence that the instrument can actually recognize the errors it is supposed to catch.

3.5 Principle four: Deterministic Infrastructure

The subject may vary; the test environment should not introduce unexplained variation into the record of what happened. This is the distinction between the probabilistic behavior being assessed and the deterministic infrastructure surrounding it. If evidence collection, Contract selection or Result storage depend on uncontrolled state, it becomes impossible to distinguish product drift from test-harness drift.

AQEF requires the surrounding mechanism to be reproducible:

> *"Everything surrounding the AI system — how an Execution is triggered, which Environment it runs in, how a Conversation and its Artifacts are captured, how a Contract is resolved, how a Result is stored — MUST be fully deterministic and reproducible." — AQEF 1.0-CR1, Volume I, Chapter 3, Deterministic Infrastructure.*

This is a requirement on the testing implementation. It is not a promise that invoking the subject twice will lead to identical responses. A clear test history records which Environment, model and prompt versions were used, what the subject received and which Evidence the Oracles assessed. Two runs can differ in output while the surrounding operations remain identically specified. Without this discipline, a changed evaluator configuration or missing artifact could be mistaken for a behavioral regression.

Imagine a series of Aster tests where an occasional policy document is omitted from the recorded retrieval evidence. One response appears to invent a fact. But was the assistant given a different document, did the retrieval mechanism change, or did the harness simply fail to save the relevant source? If the execution record cannot answer, the apparent semantic failure is ambiguous. A system that returns an attractive dashboard while obscuring such changes has not provided an adequate basis for diagnosis.

Determinism here should not be confused with fixed ordering of every concurrent infrastructure event. It means the governed operations and their inputs must remain reproducible and accountable within the implementation's design. When the subject depends on external services, their responses and relevant

context become part of the observed conditions that need faithful representation. A traceability record that silently absorbs uncontrolled changes is not made trustworthy by assigning it a timestamp.

The deeper insight is that an engineering team can tolerate variation when it knows *where* that variation comes from. In ordinary debugging, reproducibility helps isolate causes. In AI quality engineering, the boundary is even more valuable because some variation is an expected property of the subject. The infrastructure must not add another unidentified source of variation to an already difficult inference problem.

3.6 Principle five: Probabilistic Subject

AQEF does not demand a single literal answer to every request. The AI system under test is a *Probabilistic Subject*: its outputs can legitimately differ across repetitions of an otherwise comparable situation. This is not a universal statement that every component of an AI product is stochastic. Deterministic application logic and tool schemas remain testable by ordinary rules. The principle identifies the part of the product whose acceptable behavior cannot be defined as one immutable response string.

The specification explains the boundary as follows:

> *"The AI system under test is not required, and generally SHOULD NOT be expected, to produce identical output for identical input." — AQEF 1.0-CR1, Volume I, Chapter 3, Probabilistic Subject.*

Aster may explain a policy with two different sentences. If both preserve the relevant meaning, variation alone is not a defect. It may choose different permissible tool sequences that reach the same authorized state. Yet the principle must not become an excuse for every inconsistency. A system that sometimes grants an ineligible refund and sometimes refuses it exhibits a substantive difference. A tool path that crosses an authorization boundary is not harmless just because its final response sounds acceptable.

The important task is to distinguish *permitted variation* from *violated expectation*. A Quality Contract states the meaningful invariant and the acceptable outcome; different Oracles examine the aspects they can legitimately assess. When behavior cannot be decided from a single Execution, Reliability is assessed across repeated observations. The challenge is not to suppress uncertainty but to frame it so that an evidentiary claim can acknowledge what remains unknown.

The two neighboring principles are complementary. Deterministic Infrastructure ensures comparable runs are recorded coherently; Probabilistic Subject makes clear that equivalence of runs does not imply equivalence of generated text. Without the first, the engineer cannot say what changed. Without the second, the engineer mistakes harmless variation for failure or responds by weakening checks beyond usefulness.

Engineering Reflection. Aster produces two different but apparently valid policy explanations. What minimum evidence would allow you to distinguish stylistic variation from a changed policy basis, without asserting that identical words are required?

3.7 Principle six: Everything is a Test

A traditional test suite begins with authored scenarios. AI behavior can also be observed during exploratory work and real use, often revealing interactions no author thought to enumerate. AQEF's principle of *Everything is a Test* enlarges the conceptual boundary of testable material: an interaction becomes relevant when a Quality Contract can be applied to its Conversation. The key idea is applicability of explicit expectations, not the claim that every production interaction has already been assessed.

The specification states:

> *"Any Conversation that a Contract can be applied to is worth treating as a test, whether or not it originated from a deliberately authored Scenario." — AQEF 1.0-CR1, Volume I, Chapter 3, Everything is a Test.*

This principle has a particular maturity caveat in the current edition. Production Replay is **At Risk** in AQEF 1.0-CR1, Volume IV. The principle may be explained as part of the conceptual foundation, but the textbook must not suggest that a conformant implementation necessarily includes a working replay mechanism. The practical infrastructure for bringing production observations back into a Suite has its own requirements and maturity status. The principle remains useful even before that capability is available.

For Aster, an unusual customer dialogue may show how a correctly phrased initial answer becomes misleading after several follow-up turns. The observation is valuable because it exposes a risk, but its mere occurrence is not a passing or failing formal Result. Someone must determine which expectation applies, identify the relevant Evidence and establish whether the Oracle can assess it. Without those steps, 'we tested everything that happened' is a slogan rather than a defensible claim.

The principle is also not a demand for indiscriminate collection of customer data. What is permissible to observe, retain and replay depends on the project's own responsibilities and applicable constraints, not on an implied license contained in this AQEF phrase. Testing value and data-governance authority are different questions. A disciplined quality program can recognize the epistemic value of observed interactions without claiming entitlement to every record.

Conceptually, this principle closes the loop between learning and specification. A defect that appears outside the authored Suite can motivate a new Scenario and a clarified Quality Contract. What began as an unexpected observation can become repeatable evidence. That is a stronger use of the idea than simply attaching the label 'test' to everything the system has ever done.

3.8 Principle seven: Everything has Confidence

A verdict and our justification for trusting that verdict are different facts. AQEF calls the first a *Result* and uses *Confidence* to represent an Oracle's stated uncertainty where it applies. A Judge can judge an answer to be wrong while being uncertain about that conclusion. Another Judge can judge it correct with high Confidence yet still be mistaken. The score describes the instrument's assessment, not independent proof of truth.

The governing rule is explicit:

> *“Every Result MUST be explicit about whether Confidence applies to it and, if so, what value it carries” — AQEF 1.0-CR1, Volume I, Chapter 3, Everything has Confidence.*

Crucially, 'everything has Confidence' does not mean that every Result contains a numerical Confidence value. Validators are deterministic Oracles and must state that Confidence is **inapplicable**; they must not manufacture a score or silently omit the matter. Judges produce Confidence-qualified Results. Human Reviewer Results may have a Confidence value where the review captures it, and otherwise declare it inapplicable. This distinction prevents a consumer from confusing 'not provided,' 'not relevant' and 'highly certain.'

Consider Aster giving an ambiguous answer about an uncommon account exception. A Judge may see plausible but incomplete support in the retrieved material and return a passing verdict with low Confidence. Treating the word 'pass' as a complete fact loses important information. On the other hand, a deterministic schema check needs no artificial probability number simply to fit beside the semantic verdict. The results can coexist without pretending that they express the same kind of uncertainty.

One subtle consequence is that Confidence cannot repair poor Oracle design. If a Judge shares the model's false assumption, a confident verdict may be confidently wrong. This is why the principles operate as a system: Independent Oracles addresses correlated errors; Deterministic Infrastructure preserves what was assessed; Contracts over Assertions makes the criterion inspectable; Everything has Confidence prevents uncertainty from being erased from the Result. None replaces the others.

Across all seven Fundamental Principles, the teaching test is the same: what bad release decision becomes possible when this boundary is ignored? Hidden assertions can let a threshold move unnoticed; poor evidence can make an unsupported pass seem valid; a dependent Oracle can repeat the subject’s error; conflated confidence can disguise uncertainty; absent provenance can erase the reason for a verdict; ignored variability can hide instability; and a non-governed decision can accept risk without an accountable owner. These hypothetical failure modes illustrate why the seven principles matter; they are teaching examples, not additional normative AQEF requirements.

A second distinction belongs to the later treatment of aggregation. Confidence on one Result is not a statistical confidence interval around an estimated rate across many Results. A dashboard showing a mean score is not automatically telling us how precisely it represents the product's behavior. The later chapter on Gates and Aggregate Basis examines why a threshold cannot be interpreted without knowing how many actionable Results and distinct Scenarios support it.

3.9 Why the seven principles work together

The principles are easiest to misunderstand when treated as independent virtues. A team might comply with validation ordering while judging its subject with the same model that generated the response. Another might preserve a perfect audit log around a weak, hidden requirement. A third might have independent Judges but no way to distinguish unavailable assessment from failed behavior. Each partial design protects a narrow part of the evidence chain while leaving another weak point exposed.

A useful diagnostic exercise is to imagine seven distinct failure stories. Without Contracts over Assertions, Aster's rubric is loosened invisibly after test failures. Without Validation before Evaluation, a Judge approves

words in an unusable payload. Without Independent Oracles, the agent and Judge agree on the same invented policy. Without Deterministic Infrastructure, retrieval evidence disappears and the incident cannot be reconstructed. Without Probabilistic Subject, benign paraphrases are misreported as regressions. Without Everything is a Test, unexpected production-like conversations never inform the Suite. Without Everything has Confidence, uncertain semantic judgments are presented as certain passes. Every story breaks a different link in the quality argument.

These examples are illustrative reasoning, not empirical measurements of how often a failure occurs or how much AQEF prevents it. Their purpose is to show why each principle was included. A proposed architecture change should be tested against all seven questions, not just against whichever principle is easiest to implement.



Figure 3.2 — Seven safeguards around the evidence chain.

The figure places the seven principles around the same central sequence: definition, execution, capture, assessment and decision. The safeguards do not form another mandatory processing order; they are conceptual constraints that govern the meaning of the chain.

3.10 The five layers: responsibilities rather than servers

AQEF's reference architecture organizes work into **Definition, Execution, Evidence, Assessment and Decision**. These are five logical layers, not five required software services. A compact implementation may place several responsibilities in one process, while a larger implementation may distribute them. What

matters for Conformance is that the implementation satisfies the applicable requirements, not that its deployment diagram resembles a textbook figure.

The Definition layer holds the description of what to test and what counts as acceptable behavior. The Execution layer runs a Scenario against an Environment. The Evidence layer captures what occurred: the Conversation and accompanying Artifacts. The Assessment layer applies Oracles and records Results. The Decision layer uses the Results, Metrics, Baselines, Snapshots and Reports needed for governed choices. The layers express different epistemic roles. Definition states the question; Execution makes it happen; Evidence preserves the observation; Assessment judges it; Decision establishes the consequence for the release.

This separation helps diagnose disagreement. If two teams disagree about whether Aster should cancel an account without confirmation, their dispute belongs initially in Definition. If the intended tool was never called, Execution and Evidence are relevant. If the tool trace is incomplete, Evidence is the problem. If a Judge interpreted the policy incorrectly, Assessment is implicated. If a failed safety result was overridden without appropriate governance, the issue sits in Decision and its surrounding controls. Saying merely that 'the AI quality pipeline failed' conceals the repair location.

Logical layer	Question it answers	Illustrative AQEF entities
Definition	What behavior is examined, under which expectations?	Project, Suite, Scenario, Dataset, Contract
Execution	In what runtime context did the attempt occur?	Execution, Environment
Evidence	What interaction and traces actually resulted?	Conversation, Artifact
Assessment	Which Oracle assessed the evidence, and what did it report?	Validator, Judge, Human Reviewer, Result
Decision	What do the Results justify at the release level?	Metric, Baseline, Snapshot, Report, Release Decision

Table 3.1 — The five logical layers. Adapted as an explanatory summary from AQEF 1.0-CR1, Volume I, Chapter 4, Logical Layers. The list of entities is illustrative rather than an exhaustive schema.

The layers should not be mistaken for an organizational workflow. In practice, a quality problem often travels backward through them. A Release Decision exposes a gap, which prompts a revised Contract or a new Scenario. A disputed Result can trigger scrutiny of the underlying Evidence or of the Oracle itself. The architecture is therefore a **closed loop** rather than a one-directional assembly line. Feedback does not rewrite history: a new Contract should govern new assessments rather than silently changing what the earlier record meant.

There is also a sixth concern that is not a sixth layer: *Governance*. Governance wraps the architecture by constraining who can approve, change or override material decisions. A permission model does not itself decide whether a policy explanation is faithful. It does specify who is authorized to weaken a requirement, approve a Baseline or act on a Gate failure. Conceptually, governance forms the boundary around the flow, not another station through which the Conversation passes.

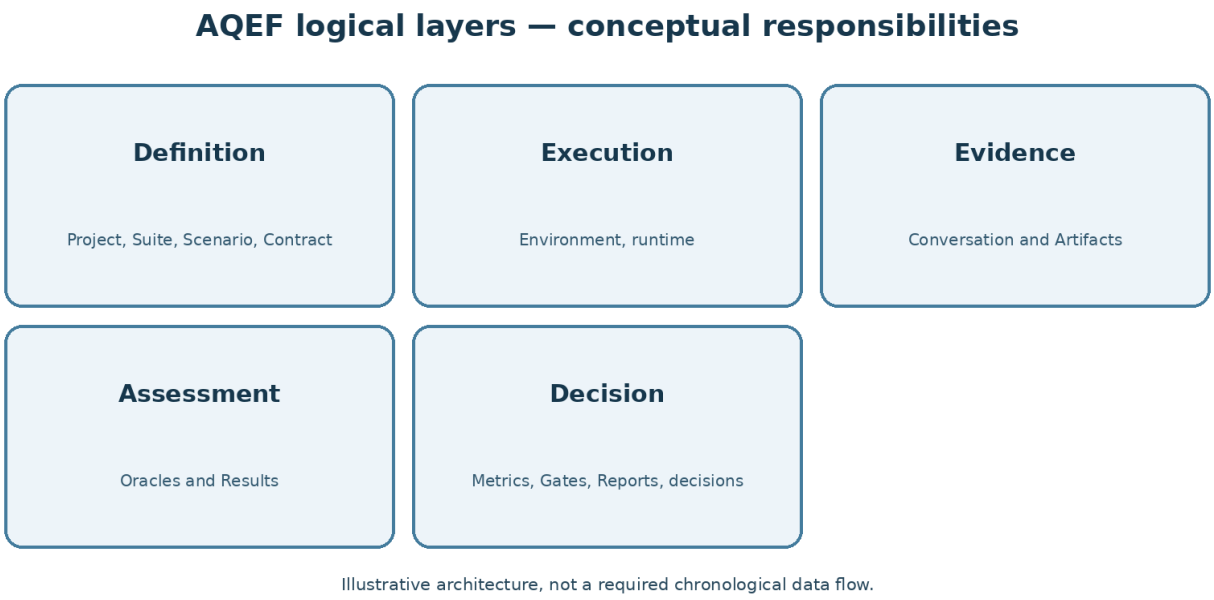


Figure 3.3 — Five layers with Governance around the loop.

The flow begins at Definition, passes through Execution, Evidence and Assessment to Decision, then feeds learning back to Definition. Governance is drawn as a perimeter rather than a sixth stage. Logical layering is not a deployment blueprint.

3.11 The path from Contract to Release Decision

To see the entire architecture, follow one assessment of Aster's refund policy explanation. At Definition, the Quality Contract states which behavior counts as acceptable and which Oracle will judge it. A Scenario provides the situation to examine. In an Environment identifying the relevant runtime configuration, an Execution occurs. The resulting Conversation contains prompts and responses; Artifacts can include retrieved context, tool calls and timing evidence. The Evidence is assessed under the Contract. The Oracle produces a Result, potentially qualified by Confidence and other disposition information.

At this point, a single passing Result is not a release decision. A Gate is a decision criterion applied to a specified basis of Results, potentially across dimensions and Scenarios. A *Quality Gate* must not be confused with an individual assertion: it considers aggregate evidence and decision policies appropriate to its scope. Later chapters explain actionable Results, minimum basis, independent failure reasons and the governance of overrides. For now, the important conceptual distinction is that a Result is about an assessment and a Gate Result is about whether decision-level requirements were met.

The traceability record links Evidence, Results, applicable Gate Results, the relevant Snapshot, the frozen Report, and the governed Release Decision; these links express required provenance and decision dependencies, but the diagram does not prescribe an exclusively linear execution schedule. The Decision Pipeline applies the Confidence Model and Aggregation Model before evaluating Gates. A Snapshot freezes relevant point-in-time state. A Report preserves the information needed to inspect the basis for a choice. A Release Decision records the governed decision, not an objective declaration that the product can never fail.

Returning to the original Contract closes the chain: the decision should be explainable with reference to which expectation and which observations justified it.

Imagine one hundred Aster checks appearing green on a dashboard while none covers unauthorized cancellation. That dashboard provides no evidence that the absent risk is controlled. Imagine instead that all checks cover only one narrow policy question, repeated many times. High result count may establish something about consistency within that question but says little about breadth of coverage. This is why the chain must preserve the Scenarios and Executions underlying its aggregates, rather than reducing the entire run to an attractive percentage. The detailed treatment of Aggregate Basis arrives in Chapter 15.

The Release Decision also belongs to a human and organizational governance context. AQEF supplies a structured mechanism through which evidence is presented, gates evaluated and authorized overrides recorded. It does not replace the Project's decision about acceptable risk. A person may approve a release with known residual risk, but the record must not conceal the failed evidence by relabeling it as passing.

3.12 The domain model: definition, event and produced data

Several nouns in AQEF sound similar to concepts that conventional QA teams use loosely. Their differences are important because they identify what can be repeated, what can be compared and what is preserved after a run. A *Project* provides the top-level container for an AI system under quality governance. An *Environment* identifies a runtime context: model version, prompt version, configured tools and supporting infrastructure. A *Suite* is a curated grouping of Scenarios; the same Scenario may belong to multiple Suites. A *Scenario* is a reusable definition of one testable situation and its governing expectations.

An *Execution* is not a Scenario. It is the runtime event that realizes a Scenario at a specific time and in a specific Environment. A *Conversation* is not an Execution. It is the ordered prompt-and-response transcript produced by the event. A supporting Artifact is not a Conversation either: a tool trace or latency record can describe what happened without being a conversational turn. AQEF preserves these separations because each answers a different question: what was meant to be examined; what actually ran; and what data came out of the run.

> *"Scenario (the static definition of what to test) → Execution (the runtime event of testing it) → Conversation (the data that event produced)." — AQEF 1.0-CR1, Volume II, Relationship Overview.*

The distinction is easy to see in Aster. A Scenario asks the assistant to explain refund eligibility for an account containing an exceptional subscription condition. One Execution may use yesterday's model version and produce a terse answer. A later Execution may use an updated model version and produce a more complete response. The Scenario is the same reusable test definition. The two Executions are separate historical events. Their Conversations are different evidence records. If this vocabulary collapses, comparisons across releases become ambiguous: a statement that 'the test changed' might refer to the definition, its runtime context or its generated response.

The Contract is not the same as the Scenario, although a Scenario references one. A Contract captures the quality expectations and relevant Oracles, and may be attached at higher levels and reused. A Dataset provides versioned inputs or Variables when many instances of a situation must be examined. Results are

separate from both the response content and the Scenario because different Oracles may assess different clauses against the same observed interaction. This domain model gives traceability a precise grammar: one can ask which Contract clause, Oracle, Execution, Conversation and Evidence support each Result.

A further point guards against careless use of Baselines. A *Baseline* is a specific historical Execution or Result approved as a reference for later comparison; a *Snapshot* captures a broader frozen state. Neither is the whole meaning of an Execution. A Baseline records a chosen reference, not an eternal assertion of correctness. As the product evolves, a reviewer must understand both the reference and the grounds on which it was approved.

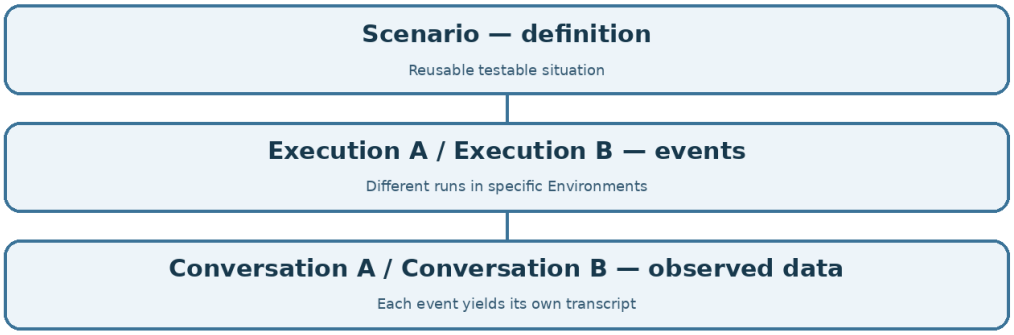


Figure 3.4 — Definition versus event versus data.

A Scenario points to repeatable Executions, each producing its own Conversation and Artifacts. Results connect the Evidence back to the governing Contract. The visual emphasizes multiplicity: one reusable definition can yield many observed runs and assessments.

3.13 What AQEF fixes and what it leaves open

A specification must decide enough to make independently built implementations intelligible to one another, but not so much that it prescribes every team structure, test strategy or implementation technique. AQEF constrains the semantics of key artifacts, their relationships, the required behavior of applicable capabilities and the principles that make the evidence chain coherent. It does not require a particular user interface, number of microservices or language in which an engine is written.

The logical-layer model is a good illustration. A small implementation could store Contracts, run checks and produce Reports in one program. Another could distribute execution workers, Evidence storage and assessment services across different processes. Both can express the same conceptual relationships if they satisfy applicable normative requirements. The specification's architecture is therefore not a mandatory cloud diagram, and visual resemblance is not a valid Conformance test.

The boundary extends beyond deployment. AQEF does not specify the product's business logic, the organization's preferred risk formula or a universal minimum number of test cases. Human decisions establish which quality properties matter. The framework makes those choices explicit and governs how they are represented and evaluated; it does not claim to make the choices on everyone's behalf. This is particularly important for Business Quality: knowing that risk has different consequences does not supply an objective cross-industry weighting formula.

There is also a maturity boundary. AQEF 1.0-CR1 is a **Candidate Recommendation**, not a finalized standard. The non-negotiable Core and applicable scoped capabilities are specified, while some named capabilities remain **At Risk** or **Reserved**. A diagram showing a Dataset Engine must not be read as a claim that Generated or Production Replay datasets are already implemented. The architectural role and the maturity of each extension are separate facts. The current edition's exact maturity map belongs in Appendix A.

Likewise, Conformance is exclusively a property of an implementation: the engine or platform executing the specification. It is not a badge for a tester, a team or the Aster product itself. An implementation passing the CTS supplies evidence about the implementation's conformity to a scoped set of rules. It does not show that Aster makes accurate decisions on real customer cases. Confusing those claims would reverse the fundamental purpose of the evidence chain.

The most useful way to read the architecture is as a disciplined allocation of responsibilities. A developer remains free to choose how to store data or distribute work. A Project remains responsible for its own expectations and decisions. An implementation claiming AQEF Conformance is not free to silently change the meaning of a Result or collapse the distinction between subject error and unavailable assessment. This combination of flexibility and checkable boundaries is the engineering compromise behind the model.

Engineering Reflection. A dashboard shows green release status, but the supporting team cannot name which Scenario or Oracle produced its strongest safety claim. Which links in the evidence chain must be recovered before that status can be treated as an intelligible quality claim?

Common Misconceptions

1. 'A principle is the same as a processing stage.' The seven principles constrain the meaning of the lifecycle; they are not seven runtime steps. By contrast, the five layers separate major responsibilities. Governance wraps them rather than becoming another stage.

2. 'Validation before Evaluation means a passing Validator makes the answer correct.' A Validator checks its assigned deterministic criterion. Passing structure does not establish semantic faithfulness, while an unavailable Validator is not a proved subject defect.

3. 'Independent Oracles means two AI prompts rather than one.' The normative prohibition concerns using the same system, weights or decision process. Distinct prompts can still preserve correlated error mechanisms.

4. 'Everything has Confidence requires a probability score on every Result.' A Judge carries Confidence; a Validator explicitly declares that Confidence is inapplicable. High Confidence itself is not proof of correctness.

5. 'Five layers require five separate deployed components.' The layers are logical. Conformance constrains applicable behavior and artifacts, not a preferred service topology. Nor does Conformance certify the AI product being tested.

Chapter Summary

AQEF's seven Fundamental Principles preserve distinct safeguards across the quality-evidence lifecycle.

Contracts make expectations inspectable; validation ordering and independent Oracles protect the meaning of assessment.

Deterministic Infrastructure separates test-harness behavior from legitimate variation in a Probabilistic Subject.

Everything is a Test describes the possibility of contractual assessment beyond authored Scenarios, not a claim that replay is implemented.

Everything has Confidence requires explicit treatment of uncertainty, including inapplicability for Validators.

The five logical layers are Definition, Execution, Evidence, Assessment and Decision, with Governance acting across them.

Scenario, Execution and Conversation are different entities; their separation makes historical claims traceable.

A Release Decision rests on evidence and governance, not merely on a passing assertion or the visible color of a dashboard.

Review questions

1. Aster's Judge is a differently prompted copy of the same model used by the subject. Which principle is implicated, and why is the visible prompt difference insufficient by itself?
2. A team modifies a hidden assertion until an unstable natural-language test becomes consistently green. Why can this create a weaker quality claim even when execution becomes more repeatable?
3. Explain why Validation before Evaluation and Independent Oracles cannot be collapsed into one rule.
4. Two Executions share one Scenario but produce different valid explanations. Which domain-model objects are the same and which are different, and what evidence would support a regression claim?
5. What failure of reasoning occurs when a release dashboard treats thirty repetitions of one Scenario as equivalent to thirty distinct Scenarios?
6. A safety observation is discovered outside the authored Suite. Explain the steps needed before it can support a Quality Gate, and state the maturity caveat about Production Replay.
7. Why does the phrase 'Everything has Confidence' not justify adding artificial Confidence numbers to deterministic Validator Results?
8. The release sponsor approves Aster despite an advisory Gate warning. Which parts of the evidence chain should remain intact, and why does approval not transform the warning into a pass?
9. A proposed implementation has one service rather than five. What evidence would matter more for Conformance than the number of deployed components?
10. Choose one of the seven principles, describe a case where it is obeyed but another principle is broken, and explain the resulting weakness in the release decision.

Sources

Primary normative and conceptual sources, verified against the public AQEF 1.0-CR1 repository:

1. AQEF 1.0-CR1, Volume I — Architecture & Concepts, Chapter 3, Fundamental Principles; Chapter 4, AQEF Reference Architecture; Chapters 5–6, lifecycle and decision context.
2. AQEF 1.0-CR1, Volume II — Domain Model, Relationship Overview and Entities.
3. AQEF 1.0-CR1, Front Matter, §6 Scope, §8 Terminology, §9 Normative Language, §10 Conformance; §0 Document Status.
4. AQEF 1.0-CR1, Volume V — Validator Engine, Execution Order and Validator Architecture.
5. AQEF 1.0-CR1, Volume XI — Reporting & Analytics, Reports and Quality Gates.
6. AQEF 1.0-CR1, Volume IV — Dataset Engine, Production Replay maturity cross-reference.
7. Answers to Review Questions belong to Appendix C of the textbook. All fictional Aster situations and the diagrams in this chapter are educational explanations, not additional requirements of AQEF.

Chapter 4. Conformance, Maturity, and the Limits of a Claim

Maturity: Core + Conceptual · Textbook First Edition v1.0

Learning Objectives

By the end of the chapter, readers should be able to:

1. Distinguish the Conformance of an AQEF implementation from the quality of an AI subject, its test strategy, and its practitioners.
2. Interpret MUST, SHOULD and MAY without either weakening requirements or inventing obligations.
3. Explain the non-negotiable core, scoped capabilities, and the role of a Conformance Statement.
4. Differentiate Working Draft, Candidate Recommendation, Proposed Standard, At Risk, Reserved, and the textbook's label Implemented.
5. Identify what an independent implementation and a CTS result demonstrate, and where their claims stop.

4.1 What exactly is claiming to conform?

An organisation developing an AI customer-service assistant will often want a simple answer to a complicated question: “Are we AQEF-compliant?” The apparent simplicity of that question is dangerous. Is the organisation asking whether its engineers followed a workflow, whether its testing tool implements a defined contract, whether the assistant behaves acceptably, or whether a release was properly authorised? These statements need different evidence. If they are merged into a single badge, the badge can appear stronger than any of the underlying findings.

Conformance is deliberately narrow in AQEF. It describes an *implementation*: the engine, tooling or platform that carries out the specification. The implementation has observable obligations. It must represent definitions, record executions and evidence, apply assessments and produce decision artifacts in the ways specified. A human tester's skill, an organisation's process maturity, and the quality of a product being tested cannot be established by proving this mechanism exists.

> *“Conformance is a property of an AQEF implementation — the engine, tooling, or platform executing this specification — not of any AI system under test.” — AQEF 1.0-CR1, Front Matter §10, Conformance.*

This sentence is a normative boundary, not an insult to professional competence. AQEF has no mechanism for certifying engineers or entire QA departments. Its requirements address what a conformant implementation does. A project can still demand excellent engineering practices and assign reviewers with appropriate authority, but those expectations belong to the project's own governance rather than a personal qualification invented by the specification.

Consider Aster Support AI, our fictional assistant that retrieves support policy and may call customer-service tools. Aster might correctly explain its refund policy in every examined case, while the engine that evaluated it silently omitted an unavailable Judge from its evidence. In that situation, apparently good product performance cannot vindicate a faulty implementation. The reverse is equally important: an engine could conform to AQEF and honestly report that Aster failed a critical safety gate. Conformance of the measuring apparatus is not a favourable verdict about what it measures.

This distinction mirrors the problem from Chapter 1. There are always at least two potential sources of error: the behaviour of the subject and the behaviour of the instrument. When testing AQEF conformance, the implementation becomes the subject; its own instrument is the Conformance Test Specification and accompanying harness. When using an AQEF implementation to evaluate Aster, Aster is the subject and the implementation supplies the assessment machinery. Evidence from these two boundaries should never be exchanged as if it were interchangeable.

The right engineering question is therefore not "does the team use the right label?" It is "what property of which object does this evidence actually support?" A robust quality claim specifies its object, scope, version, evidence and limitations. No authority is gained by removing those qualifiers.

4.2 Normative language: obligation, recommendation and choice

A specification needs vocabulary that makes implementation obligations testable. Ordinary English is full of sentences such as "normally run validation first" and "consider keeping an audit trail." These may be helpful recommendations, but they are poor substitutes for a rule that another party could apply to an implementation. AQEF uses a small normative vocabulary precisely to distinguish binding conditions from preferred and permissible behaviour.

> *"MUST — An absolute requirement. Conformance (§10) treats every MUST as binding on whichever capability of an implementation it applies to." — AQEF 1.0-CR1, Front Matter §9, Normative Language.*

MUST is binding, and **MUST NOT** is the corresponding prohibition. **SHOULD** or **RECOMMENDED** conveys a strong expectation that may be departed from as a deliberate, understood choice. **SHOULD NOT** expresses its negative. **MAY** or **OPTIONAL** indicates genuine discretion. These are distinctions about what the *implementation* must do when the requirement applies, not prescriptions about how a tester should think.

Term	How to read it	Wrong interpretation
MUST / MUST NOT	Binding applicable requirement or prohibition	A preference waived by a project without changing the claim
SHOULD / SHOULD NOT	Strong expectation; informed deviation possible	Either indistinguishable from MUST or entirely meaningless
MAY / OPTIONAL	Permitted but not compulsory	Required just because the specification describes it

For example, Validators running before Judges is a MUST-level principle of AQEF. A vendor cannot decide to run an expensive semantic Judge first, call the result "good enough," and still describe that as the same

required order. In contrast, recommending a particular optimisation of how Validators are scheduled is a different kind of provision. If a reviewer reads every recommendation as mandatory, the resulting test suite will accuse valid implementations of violating rules the specification did not write. If the reviewer treats every requirement as optional, the Conformance claim loses definition.

Scope is just as important as the capitalised word. A requirement belonging to an additional capability applies when the implementation claims that capability. Failing to supply a capability that an implementation does not claim is not necessarily a failure of its core Conformance. But a core requirement cannot be switched off by declaring a tiny scope. We will examine this distinction next.

The specifications' treatment of a SHOULD-level deviation is particularly instructive. It expects a deliberate, understood choice, but Front Matter §10 does not demand a recorded justification as an additional Conformance condition. A book may reasonably recommend keeping such records because the explanation helps an auditor; it must not rephrase that advice as if AQEF required the log. Here is a recurring lesson in standards-based engineering: a sensible practice and a normative requirement are not automatically the same thing.

4.3 The common foundation and scope-dependent capabilities

AQEF permits variations in what an implementation offers. An implementation may expose Human Review, Multi Judge, Seeded Controls, particular dataset sources, extensions or integrations, but it does not have to offer the entire catalogue to claim Conformance. However, it cannot offer only a disconnected fragment and call that AQEF. The framework therefore distinguishes a non-negotiable core from capabilities whose additional obligations depend on an implementation's claim.

The first part of the core is the five logical layers: **Definition, Execution, Evidence, Assessment and Decision**. Each responsibility must exist somewhere, whether in one executable or multiple services. The second part is the seven Fundamental Principles explained in Chapter 3. An implementation cannot waive them by describing itself as a minimal engine. The source states:

> *“The five logical layers (Chapter 4 — Definition, Execution, Evidence, Assessment, Decision) MUST each be met somewhere.” — AQEF 1.0-CR1, Front Matter §10, The Non-Negotiable Core.*

Aster provides a useful illustration. Imagine a small, self-hosted AQEF engine that defines Scenarios, executes Aster conversations, captures evidence, assesses it and produces quality decisions. It offers a single Judge but no Human Review, no external plugins and no CI/CD integration. Its lack of optional facilities does not automatically make it nonconformant. The relevant question is whether its core works as specified and whether any additional features it actually claims meet their own applicable requirements.

Now suppose a marketing page begins advertising “AQEF Human Review.” That claim adds obligations. It should be possible to show that the implemented capability behaves according to the specification, not merely that a human can read a log and leave a comment. The concrete distinction between a person looking at a dashboard and a Human Reviewer functioning as an Oracle matters because the framework defines evidence, result and lifecycle semantics for the latter.

The concept of *topology* must be kept separate from *conformance scope*. A Minimal Engine can combine layers in one process, while an Enterprise Engine might distribute them. Cloud and self-hosted deployments are likewise architecture choices. Neither topology nor hosting arrangement forms a second tier of Conformance. A small tool is not judged under weaker rules because it has a small user base. A large platform does not become more conformant because it has separate microservices.

This independence of requirements from size is an important design decision. Otherwise, compatibility would depend on deployment style rather than on behaviour. The same contract should mean the same thing whether its evaluation is executed by a single machine or a fleet of workers. The specification constrains outcomes and records, while leaving substantial room for differing technical implementations.

Figure 4.1 — Core plus declared capabilities. In the figure, the mandatory core supports every implementation; optional claims add corresponding requirements without replacing that foundation.



4.4 A Conformance Statement is a claim, not proof

A *Conformance Statement* describes the version and scope for which the implementation asserts conformity. It must state the specification version, its claimed scoped capabilities, and how Projects and Suites are bound to Teams, or that there is no such binding. It may name capabilities deliberately not offered, along with SHOULD-level deviations. The statement is an indispensable starting point because tests cannot be selected sensibly until the target of the claim is defined.

The essential distinction is between stating an obligation and supplying evidence that it has been met. An engine can publish a beautifully formatted Conformance Statement while its execution code violates a requirement. Conversely, a CTS report with many passes may be too ambiguous to interpret without knowing what the engine claimed. These are complementary artifacts, not substitutes.

In Aster's organisation, a manager could be shown a passing test report and reasonably ask: was this test of the mandatory core, or only of a selected capability? Was it performed against version 1.0-CR1 or an earlier Working Draft? Were some tests inapplicable because the implementation did not claim the relevant capability? Those questions do not undermine the report; they explain the information needed to read it responsibly.

A Conformance Statement is also version-specific. Version labels are not cosmetic because requirements can change between editions. Candidate Recommendation permits particular changes under defined conditions.

An engine demonstrated against the requirements of one edition does not automatically inherit an identical claim for another. A textbook that describes the statement therefore needs the same editorial discipline as the engine: it must say which edition its explanation covers.

A clear statement can be modest and still valuable. “The engine claims AQEF 1.0-CR1 core, Human Review and Seeded Controls; it does not offer Multi Judge or CI/CD integration” is far more informative than “fully AQEF-ready.” It leaves no false impression that feature lists are quality rankings, or that omitted claims have silently passed their tests.

Engineering Reflection

An implementation advertises “100% AQEF-compatible.” What precise information would you ask for before assigning meaning to that claim, and why would a passing release report for Aster not answer the same question?

4.4.1 Why declared scope protects both implementers and reviewers

Scope is sometimes misunderstood as a strategy for avoiding work. In fact, a precise scope does the opposite: it binds an implementation to the work that it publicly claims to perform. Without such boundaries, two reviewers can evaluate the same platform and reach different conclusions because one assumes that every feature described in the specification is mandatory, while the other assumes that only the minimal path matters. Neither assumption reflects AQEF's declared division between its core and scoped capabilities.

Imagine Aster's organisation considering two possible AQEF engines. The first offers the core, Human Review and Seeded Controls; the second offers the core and Multi Judge. These platforms are not interchangeable in what evidence they can produce. But neither is automatically less conformant because it has fewer scoped capabilities. For each claim, the reviewer needs to inspect the relevant implementation requirements and their demonstrated behaviour. Breadth of features is an acquisition question; correctness of claimed behaviour is the Conformance question.

That difference also changes how procurement language should be written. “Must offer Multi Judge for our planned workflow” is a project requirement for choosing a tool. “Must conform to AQEF for the capabilities it claims” is a statement about the specified implementation contract. Both can appear in a purchasing decision, but one must not be disguised as a universal requirement of AQEF. A tool that does not supply the feature your organisation requires may be unsuitable without being nonconformant.

A second confusion arises when partial support is advertised. Suppose a vendor claims Human Review but implements little beyond a comments field. Reviewers may find an interface that looks like the required concept, yet the semantic obligations—whether a Human Reviewer is represented as an Oracle, how Results relate to a Contract, and what a later resolution means—may be absent. This is a failure within claimed scope, not evidence that the capability is optional. The distinction between “we do not offer it” and “we offer it incompletely” is crucial.

The Conformance Statement therefore creates a shared question rather than supplying an automatic answer. It tells the reviewer which requirements to examine. That is the same basic pattern used elsewhere

in AQEF: a Quality Contract defines what is to be assessed, and a later assessment determines how the observed behaviour compares. Defining the assessment boundary is not the same as satisfying it.

4.5 Maturity of a specification and maturity of a capability

AQEF 1.0-CR1 calls itself **Candidate Recommendation 1**, dated October 2026. That status matters because the document is believed sufficiently complete for independent implementers to build against it, while remaining open to correction. It is not described as a final Standard. The history includes Working Draft editions; future stages are Proposed Standard and Standard, each with its own stated progression criteria.

> *"A Candidate Recommendation is not a Standard: it may still change, under the rules below, in response to what implementing it shows." — AQEF 1.0-CR1, Front Matter §0, What 'Candidate Recommendation' Means.*

Maturity is about the standing of the normative text and the evidence available for its implementation. It is not a universal ranking of software products, people or organisations. A Candidate Recommendation can contain carefully specified requirements that an implementation satisfies, while the specification itself continues receiving feedback. Conversely, publication of an impressive diagram does not demonstrate that every capability has been implemented independently.

The distinction becomes sharper for particular capabilities. **Core** denotes obligations that apply to every conformant implementation. **Implemented**, used by this textbook as an edition-specific educational label, refers to independently implemented scoped capabilities identified in the CR1 evidence: Human Review, Multi Judge and Seeded Controls. **At Risk** describes features specified in CR1 that have no implementation yet under its published status; the edition says they may be removed from 1.0 or deferred if a conforming implementation is not demonstrated by the end of the Candidate Recommendation period. **Reserved** describes concepts named but not specified for claims in version 1.0. **Conceptual** is the textbook's label for explanations that are not new features or obligations.

Category in this book	How a reader should interpret it	Appropriate claim
Core	Always applicable to an AQEF implementation	Core obligations must be met
Implemented	Claimed scoped capabilities with CR1-published independent implementation evidence	Evidence exists for that bounded implementation and run
At Risk	Specified, but lacking the published implementation evidence at CR1	Do not describe as established or universally available
Reserved	Named but not specified for claiming in CR1	Cannot be claimed under this edition
Conceptual	Explanatory or educational framing	No separate Conformance identifier

The list of At Risk items in CR1 includes Action Boundary, Generated and Production Replay datasets, plugin categories, CI/CD integrations, SDK and CLI facilities, and Reference UI. Synthetic datasets and dataset mechanisms are Reserved. The distinction is not academic: a book may discuss the value of replaying production situations without stating that Production Replay is a proven component of CR1. A Reserved dataset mechanism cannot be taught as though it is already part of the available implementation contract.

The source states the restriction plainly:

> *"No implementation can claim them, and every implementation rejects a configuration that uses them (Appendix A §A.8)." — AQEF 1.0-CR1, Front Matter §0, Reserved.*

One other classification needs attention. **Test types** are not automatically capability identifiers. RAG Testing, Agent Testing and regression scenarios describe the phenomena being tested. A capability claim instead describes behaviour offered by the AQEF implementation. Confusing them would let an implementation claim additional conformity simply by placing a test type name on a report.

For the textbook, the risk of future change is real. If CR2 revises normative content or At Risk capabilities are removed, the maturity map must be checked again, along with every passage that assumes the CR1 list. Chapter 4 is deliberately edition-bound; this is a strength, not an embarrassment. Versioning preserves what was asserted and why at each stage.

4.5.1 Version-sensitive reading is part of technical literacy

An edition of a specification is not simply a date stamped on a cover. It represents a fixed set of obligations, definitions and maturity judgments. The words in a candidate edition may be quite stable, but readers still need to know which edition they have. Otherwise, a sentence quoted from an earlier Working Draft can be treated as if it were a requirement that still binds a 1.0-CR1 implementation.

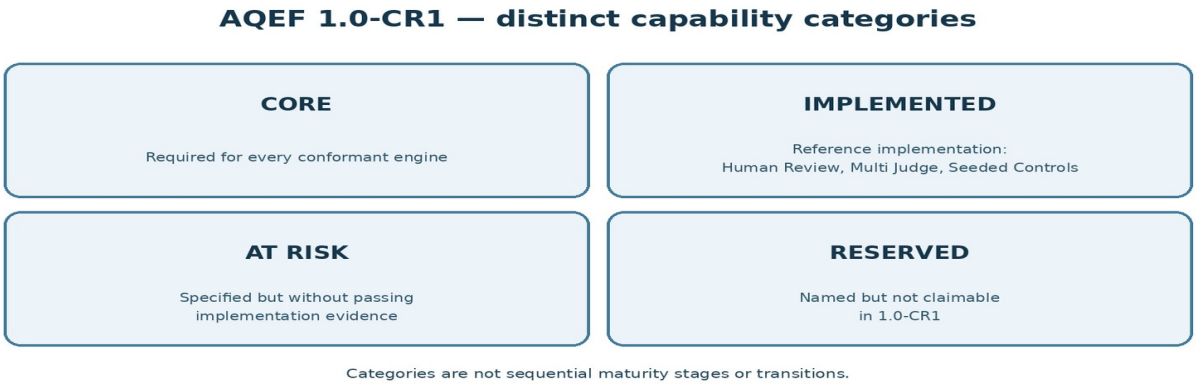
CR1 itself distinguishes editorial correction from normative change. An editorial patch does not change what the implementation is required to do. A normative change to a non-At-Risk feature requires a new Candidate Recommendation with matching CTS revision. The status rules allow an At-Risk feature to be removed if the stated exit condition is not met. These are different types of change, and they warrant different editorial responses in the textbook.

Consider a fictional revision that clarifies a phrase without changing its meaning. A textbook might update the quotation and location while preserving its explanation. Now consider a revision that changes which Result qualifies as actionable. This would affect the teaching of evidence, aggregation and Gate interpretation. The author could not merely replace a word in Chapter 4: cross-chapter implications would require inspection. The book's version control is therefore a way to keep the educational argument internally consistent rather than a bureaucratic afterthought.

For reviewers, this also explains why a CTS pass cannot always travel forward unchanged. If the test checks the old rule, its result might not cover the new rule. Re-running the same named test against a different normative baseline is not automatically equivalent evidence. The meaningful unit is an edition-specific requirement and the case used to examine it. A result without that connection may be historically accurate yet irrelevant to the current claim.

This careful reading is particularly important for the book's own category “Implemented.” That label is an educational snapshot of evidence recorded for CR1, not a permanent promise about every engine or every future edition. If a later independent implementation fails a test, or a capability changes, readers should not infer from a previous snapshot that the new state has already been settled. A maturity label is useful only when its underlying time and evidentiary scope are visible.

Figure 4.2 — CR1 status is an edition snapshot. The figure groups Core, independently implemented scoped capabilities, At Risk and Reserved. Its purpose is to prevent accidental promotion of a described feature into an established capability.



4.6 The CTS and what a passing test can actually demonstrate

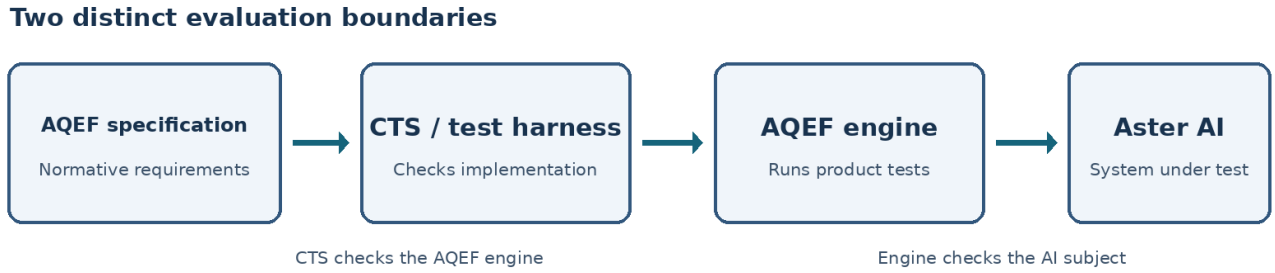
The AQEF **Conformance Test Specification** (CTS) is a separate, non-normative set of tests against implementations. Its purpose is to check whether a platform behaved as the AQEF specification requires in the cases exercised. The CTS does not define additional MUST-level rules. If a test inadvertently asks for more than the specification, the specification is authoritative and the test requires correction.

> *“A test that passes shows that an implementation behaved as the specification requires in that one situation.” — AQEF CTS 1.0-CR1, §0.2, What a test result means.*

That sentence describes the appropriate size of the conclusion. Passing a test provides evidence for a requirement under the specific circumstances that were examined. It does not establish that every possible future interaction of the implementation is correct. Failure of an applicable test is likewise meaningful evidence of nonconforming behaviour in that case; a test not applicable to an unclaimed capability does not magically become a pass.

The same epistemic distinction governs Aster's product testing. Aster passing its policy-accuracy checks is evidence about its observed behaviour relative to chosen Contracts. The engine passing CTS checks is evidence about its handling of AQEF obligations. There are therefore *two* evaluation boundaries, and a report from one must not stand in for a report from the other.

Figure 4.3 — Two boundaries of evaluation. CTS and its harness examine the AQEF engine; the AQEF engine examines Aster under the project's Contracts. The arrows represent different tests of different subjects.



The CTS maps test cases to normative requirements, includes core checks and scoped-capability checks, and states conventions for test outcomes and harness behaviour. It also explains that an independently developed implementation was used in earlier runs. These features matter because a test of compliance must itself be sufficiently reproducible for its conclusions to be reviewed. A CTS suite is an instrument. It can contain mistakes, omit cases or rely on ambiguous fixture conditions; acknowledging those limitations is part of rigorous testing rather than an argument against testing.

A strong claim about a CTS run names the version of the CTS, the implementation under examination, the capabilities claimed, the tests actually executed and the recorded outcomes. A vague statement that “the CTS passed” strips away precisely the information a quality engineer needs to decide what was learned.

4.7 Independent implementation and the problem of ambiguous requirements

A specification may feel entirely clear to the person who wrote it because unstated choices are already resolved in that person's mind. An independent implementer does not inherit those assumptions. It must decide, using the text alone, how to order records, treat unavailable outcomes, reject invalid changes and distinguish cases that appear similar in casual prose. When two reasonable readings produce incompatible implementations, the specification needs clarification rather than praise for being flexible.

The CR1 publication records a reference implementation written from the specification by a party that had not seen the CTS. It claims the core and the three scoped capabilities Human Review, Multi Judge and Seeded Controls. The published CTS found no failure against that implementation for the stated scope. The implementation, harness and evidence are described as public. None of these statements means every capability was implemented, that every interpretation is beyond dispute or that any AI product was thereby validated.

The AQEF specification’s Revision History documents corrections made during independent implementation and subsequent independent reading of capabilities not yet built. These revisions include the 0.41.2–0.41.5 series and changes incorporated in 1.0-CR1. The public revision record, rather than unpublished editorial working materials, is the appropriate basis for tracing those corrections. Independent implementation and review expose ambiguity; neither proves the quality of an AI product.

To understand the mechanism, consider a hypothetical rule saying that “the latest accepted record decides.” Does latest mean the highest timestamp, the last accepted sequence or something else when timestamps

coincide? Implementers need one inspectable answer. If the author assumes one choice but does not write it down, two independent programs may disagree while both claim to follow the document. A separate test may then inadvertently privilege the author's hidden interpretation. The remedy is to resolve the ambiguity in the normative source and align the test with that rule. This hypothetical illustration is not a claim that it was one of the numbered historical defects.

Independent review of unimplemented capabilities offers a related but different kind of evidence. Reading a feature as a future implementer can expose missing cases before anyone builds it. It cannot replace the evidence of actually implementing and running it. The distinction explains why CR1 can report corrected wording for an At Risk feature while still labelling that feature At Risk.

The generalisable lesson is modest but important: independent implementation and independent review make assumptions visible. They provide occasions for correction and an inspectable record of decisions. They do not demonstrate that the framework has been proven across every future context. AQEF is the work of a single author, and the Document Status section explicitly avoids presenting the specification as an industry consensus or as endorsed by a standards body.

4.7.1 Different instruments answer different verification questions

When an independent implementer reads a specification, the primary question is whether its prose can be translated into reproducible behaviour without private interpretation. When the CTS examines that implementation, the primary question is whether the resulting behaviour matches the specified expectations in selected cases. When an external reviewer repeats a CTS run, the question is whether the published harness, fixtures and evidence support the reported result under a separately executed process. These acts are related, but each challenges a different point in the chain.

The distinction helps explain why a “no failure found” report is both significant and limited. Significant, because an implementation built without advance access to the tests has been exercised against requirements that can be independently inspected. Limited, because the result does not sample every possible configuration, environmental failure or sequence of operations. A reader should resist replacing the observed statement—no failure was found in the published run—with a stronger invented statement that the implementation is incapable of failing.

Consider a hypothetical CTS case about a configuration change. If the suite only tests a full initial load, an engine might correctly reject invalid configurations at startup while accepting the same invalid condition through a later targeted update. A separate test for the update path addresses a new behavioural situation. The defect in this story is not necessarily an ambiguous source requirement; it could be incomplete test coverage. Distinguishing gaps in the specification from gaps in its CTS prevents teams from fixing the wrong artifact.

There is a second subtlety in independent implementation. A report can be reproducible without being conclusive. Repetition establishes that others can obtain the same recorded results under comparable conditions; it does not transform a finite sample into a proof of universal behaviour. This is not a special weakness of AQEF. It is the ordinary logic of software verification applied consistently to the framework's own implementing tools.

The author of a specification therefore needs a disciplined response to issues. Some questions require changing normative wording; some require correcting test expectations; others are editorial or based on a misunderstanding that can be answered with an explicit reference. Treating every issue as a specification defect would be as inaccurate as dismissing every issue because the first implementation passed. What matters is whether the recorded resolution gives future implementers a checkable, consistent rule.

4.8 Aster: separating three legitimate but different claims

Imagine Aster Support AI is evaluated before a new support-policy release. Its project team runs a collection of Scenarios, gathers retrieval and tool-use evidence and produces a Report. The report says every blocking Gate passed, but two advisory findings remain. The person authorised to make a Release Decision approves the release and records the reasoning. The team would like to announce that Aster is "AQEF certified." What has gone wrong?

The immediate issue is the target of Conformance. Aster is an AI system under test. Even if all the project's Quality Gates passed, that would support a bounded conclusion about Aster relative to its Contracts, not an AQEF Conformance statement. Secondly, the report gives no direct proof that the engine implementing AQEF has followed the specification. For that, one would inspect its declared scope and implementation evidence, including CTS results. Thirdly, neither finding eliminates the residual risks and gaps created by the project's choice of scenarios, thresholds or evidence sources.

Three statements can be appropriate when supported separately. *The testing implementation claims AQEF 1.0-CR1 Conformance for its core and declared capabilities. The published CTS evidence found no failure in the applicable recorded cases. The project released Aster after evaluating the recorded quality evidence and documenting exceptions.* All three statements may be true without their conjunction proving that Aster is always correct or safe.

There is also an important counterexample: an AQEF-conformant engine may report a failed release Gate, and that should be regarded as useful evidence rather than as a failure of the framework. A correct instrument need not produce favourable measurements. Conversely, a seemingly successful release cannot excuse an implementation's incorrect result handling. The point of separating the claims is to preserve their meaning under inconvenient results.

Engineering Reflection

Aster's product report is green, but the AQEF engine has no published Conformance Statement. What would you say confidently to a release stakeholder, and what would you decline to infer?

4.8.1 Evidence can support a narrow claim while leaving a larger risk open

Aster's release decision provides another useful distinction between a *complete record* and a *complete picture of risk*. AQEF can require a result to be traceable to its Oracle and Contract, and a release decision to be supported by its evidence. A faithfully implemented chain means that one can reconstruct which cases and instruments informed the decision. But traceability cannot fill a missing business scenario: if the team never considered account recovery fraud, the record may be fully accurate about the cases it did consider while offering no answer about that neglected risk.

This matters especially to QA Leads, who are often asked to translate complex findings into one management sentence. The correct response is not to reject concise communication; it is to make concision precise. “All defined blocking Gates passed for the recorded scope” is short and bounded. “The AI system is safe” is far broader. If the second statement is intended, the team needs a defensible account of what safe means, which risks were considered and what evidence speaks to those risks. The existence of a conformant engine does not supply that account on its own.

One might imagine four combinations. A conformant engine can assess a poor product and return failures. A conformant engine can assess a product that meets the chosen Contracts. A nonconforming engine can by chance emit favourable results about a genuinely acceptable product. Or it can emit misleading results about a poor one. From a single product score alone we cannot know which situation holds. This is why the instrument and subject must have independent claims and independent evidence.

The ethical obligation is to preserve inconvenient distinctions. A non-actionable Result should not be presented as a confident pass. An uncovered risk should not disappear from an executive summary because the engine's conformance checks are green. A human Release Decision should not be restated as a guarantee from the framework itself. These safeguards are about the integrity of language as much as the integrity of data. Good engineering practice demands that the words used in a report not promise more than the mechanism was designed to establish.

4.9 Claims across editions and over time

Conformance claims have a time dimension. The status of the specification, the features offered by an engine, the CTS edition and the product conditions used in a release review can all change. A statement that was properly bounded last month may be misleading today if its qualifiers are silently removed. This is one reason why both specification and CTS have their own versioning schemes and why this textbook names its source edition prominently.

A Candidate Recommendation does not become a Standard simply because enough time passes. AQEF's published maturity progression describes evidence and review conditions for later stages. Similarly, a feature that is At Risk is not made Implemented because it has a diagram or a test plan; its status depends on the independent implementation evidence that the edition requires. A historical chapter should not write tomorrow's conclusion into today's record.

For quality engineers, the practical reasoning is straightforward. First, state the exact object: engine, product, test run or governance decision. Second, name the edition against which the claim was made. Third, describe the relevant scope and evidence. Fourth, preserve the unresolved limitations. None of these qualifiers weaken the result. They allow another person to decide whether the result applies to the question they actually have.

The distinction also prevents the authority of a specification from becoming authority over a person's decisions. AQEF provides a vocabulary and checkable implementation requirements; it does not dictate which business risk is acceptable or how many scenarios make a project sufficiently tested. These are judgments the project must own. Conformance of the mechanism should make those judgments easier to inspect, not conceal them behind a label.

Common Misconceptions

1. “A chatbot passes AQEF, so it is AQEF-conformant.” Conformance belongs to the testing implementation. The chatbot has outcomes against project-specific Contracts and Gates.

2. “Everything described in AQEF is mandatory for every engine.” The non-negotiable core is universal; other requirements attach to capabilities the engine claims. Reserved capabilities cannot be claimed in this edition.

3. “A passing CTS run proves universal Conformance.” It supplies evidence about the requirements and cases actually tested; the CTS itself warns against the stronger inference.

4. “At Risk means an idea has been prohibited.” At Risk features are specified but lack CR1's published implementation evidence. Reserved features are explicitly unavailable for claims in 1.0.

5. “A SHOULD is either a MUST or mere decoration.” Its strong recommendation allows a deliberate, understood deviation, which is distinct from both mandatory and discretionary behaviour.

Chapter Summary

Conformance is a property of an AQEF implementation, not a product, person or testing organisation.

The five logical layers and seven Fundamental Principles form the non-negotiable core of every Conformance claim.

Scoped capabilities add applicable obligations; they do not lower the core or create different Conformance grades for Minimal and Enterprise Engines.

A Conformance Statement defines what an implementation claims. By itself, it does not show that the claim was tested successfully.

Candidate Recommendation is a version-specific maturity stage, not a finished Standard or proof of product quality.

Core, Implemented, At Risk, Reserved and Conceptual must not be mistaken for interchangeable kinds of implementation evidence.

The CTS tests the implementation in bounded cases; it does not certify the AI subject, and finite passing tests do not prove universal correctness.

Independent implementation and review expose ambiguities, but claims must remain proportionate to the evidence they produced.

Review questions

1. Rewrite “Aster is AQEF-certified” into distinct implementation, product and governance claims that could be checked independently.
2. A Minimal Engine has no CI/CD integration. What information would you need before concluding whether this affects its Conformance claim?

3. Why can a MUST-level obligation not be dismissed as an approved exception while keeping the same Conformance claim?
4. What does a Conformance Statement contribute that a CTS report alone does not provide?
5. An implementation claims Multi Judge but cannot perform its required consensus behaviour. Explain why the missing capability cannot be treated as optional in this case.
6. Why is a Cloud deployment neither inherently more nor less conformant than a self-hosted one?
7. How should a textbook distinguish discussion of Production Replay from an assertion that it is already independently implemented in CR1?
8. If every applicable CTS test passes, what can a reviewer conclude, and what remains unproven?
9. Compare the evidence from independently implementing a specification with the evidence from independently reviewing text that has not yet been implemented.
10. A later candidate edition changes a normative requirement. What needs to be rechecked before reusing an old engine's Conformance claim?

Sources

1. AQEF 1.0-CR1, Front Matter §0: Document Status, At Risk, Reserved and Maturity Progression.
2. AQEF 1.0-CR1, Front Matter §9: Normative Language.
3. AQEF 1.0-CR1, Front Matter §10: Conformance, Conformance Statement and Non-Negotiable Core.
4. AQEF 1.0-CR1, Volume I, Chapters 3–4: Fundamental Principles and logical layers.
5. AQEF 1.0-CR1, Volume XVII: Reference Implementations.
6. AQEF CTS 1.0-CR1, §0: Purpose, scope and limits of a passing test.
7. AQEF CTS 1.0-CR1, §90: Traceability of requirements to CTS cases.
8. AQEF 1.0-CR1, Revision History and release-status notes — corrections from independent implementation and independent reading of scoped capabilities.

Chapter 5. Scenarios and Test Design

Maturity: Core · Textbook First Edition v1.0

Learning Objectives

After reading this chapter, readers should be able to:

1. Explain how a Project, Environment, Suite, Scenario, Execution and Conversation relate without treating them as interchangeable.
2. Explain why meaningful Scenario design begins with a risk and ends with a sufficiently explicit Quality Contract, rather than merely collecting prompts.
3. Compare positive, negative, boundary and blocked-path tests, identifying what each can and cannot establish.
4. Use AQEF's test-type taxonomy without mistaking a test type for a claimable Conformance capability.
5. Reason from priority risks to a purposeful Suite, including the gaps and limitations that its Evidence will leave.

5.1 From a quality concern to something testable

The earlier chapters established why AI Quality Engineering requires more than an interesting collection of AI responses. A program can produce fluent, plausible answers whose meaning is wrong; an instrument can be unavailable, misconfigured or unreliable; and a large number of successful checks can still miss the defect class that matters most. These difficulties are often described as a problem with evaluation. They are also, fundamentally, a problem of **test design**. Even a trustworthy Oracle cannot establish what a test never asks it to examine.

Test design is the engineering activity that converts a concern about behaviour into a situation that can be reproduced and assessed. The concern may be broad: Aster Support AI must not misrepresent a company's refund policy. It may be operational: when an account lookup is unavailable, Aster should not claim that it successfully verified the customer's account. Or it may concern how a dialogue evolves: after a user corrects an account detail, Aster must not treat the superseded value as current. The problem is not that these expectations are unimportant. It is that none, as initially phrased, identifies a particular observable situation, the evidence needed to understand the response, and the check by which an outcome is classified.

A **Scenario** gives a reusable definition of one such situation. In AQEF, it is not identical to a script, a transcript or a prompt. It expresses an intent, the Prompt or Variables that create the stimulus, and the **Quality Contract** whose clauses specify how the resulting behaviour is assessed. A Contract is a declarative account of expected quality; it is not the code that runs the test. Design succeeds when the chosen situation and its contract form a meaningful pair. A beautifully described prompt paired with an empty or ambiguous expectation remains a weak test. Conversely, a precise rubric without an input situation does not yet say when or what to assess.

The specification states the pairing principle directly:

> *"All three entry points MUST produce the same thing: a Scenario specific enough to be realized by an Execution, and a Contract specific enough for an Oracle to check it against." — AQEF 1.0-CR1, Volume VIII, Test Design Process.*

The "three entry points" are forward authoring from prioritised requirements, retroactive authoring from production interactions, and retroactive authoring from exploratory sessions. They describe conceptual routes into Test Design, not promises that every dataset source has the same implementation status. In particular, Production Replay is **At Risk** in 1.0-CR1. The fact that the specification describes the idea does not make an unimplemented feature available. The stable lesson of the quoted requirement is the common destination: an authored Scenario with a usable Contract.

For Aster, "test refunds" is not a Scenario. A better definition identifies the behaviour at issue: a customer asks whether a particular purchase qualifies for a refund, the applicable policy is supplied through the retrieval context, and Aster must avoid unsupported eligibility promises. The Contract can then distinguish a deterministic requirement—perhaps that a structured response contains a required information field—from a semantic expectation that the explanation accurately reflects the retrieved policy. These are two separate kinds of claim; the fact that they concern one customer question does not collapse them into one check.

The reader should notice what this approach does **not** dictate. AQEF does not decide that refunds are inherently more important than account security or response latency. It does not replace the work of discovering the risks that matter. It makes the resulting expectation explicit enough to be exercised and inspected. An engineer's judgment is required before any Scenario exists, and after a Result has been recorded. AQEF provides a common means of representing that judgment's testable consequences.

5.2 The container, context and authored unit

Several AQEF terms sound like ordinary organisational labels. Their separation is more rigorous than it first appears. A **Project** is the top-level container for the AI system under quality governance. It scopes the system's Environments and Suites as well as the historical and reporting artifacts that belong to them. A **Suite** is a curated collection of Scenarios grouped for a testing purpose. An **Environment** is the versioned runtime context in which an Execution takes place, including the relevant model, prompt, tools and infrastructure. A **Scenario** is the static test definition; it is realised by an **Execution**, the runtime event that produces the concrete Conversation and other Evidence.

These distinctions answer different questions. Project asks *whose quality governance is this?* Environment asks *which configured system was exercised?* Suite asks *which set of concerns have we assembled for this test purpose?* Scenario asks *what situation is defined?* Execution asks *what actually happened at this point in time?* Conversation asks *what was said during that execution?* Artifact, another AQEF term, captures evidence beyond the transcript, such as tool-call traces, recorded duration or usage. A single interaction can be interpreted correctly only if these boundaries remain visible.

> *"A single-turn Scenario yields a one-turn Conversation; a multi-turn or agentic Scenario yields a multi-turn Conversation." — AQEF 1.0-CR1, Front Matter §8, Conversation.*

The definition matters because a Scenario can be stable while its Conversations vary. Aster may explain a policy in different words on different Executions without any change to the Scenario. At the same time, two Conversations that happen to read identically might have been produced under different Environment versions. They do not therefore establish the same evidence about current behaviour. Reproducibility of the test machinery and versioning of the subject's context make such distinctions traceable.

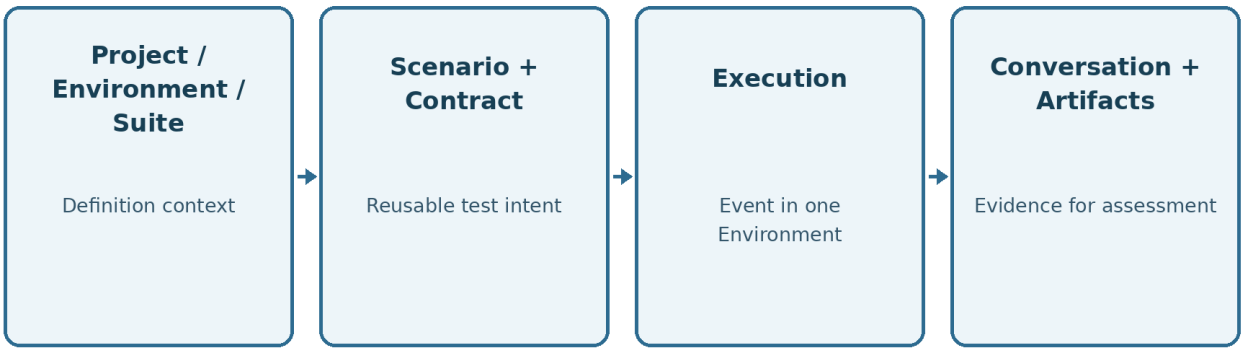


Figure 5.1 — Definition, execution and evidence. The diagram separates Project, Environment and Suite/Scenario from the runtime Execution, then shows the Conversation and Artifacts as its evidence outputs. Arrows do not imply that one Conversation is a static property of its Scenario.

In Aster, imagine a Suite for payment-support policy. It contains one Scenario about refund eligibility and another about payment disputes. Each may be executed against staging and a candidate release environment. The same Scenario is therefore realised more than once; the recorded Conversations may differ both because of probabilistic output and because the Environments have distinct model or policy versions. If the resulting report collapsed all these objects into a single list of “passed prompts,” it could conceal which release was evaluated and which definition each observation addressed.

A **Dataset** supplies named, versioned inputs or Variables that can instantiate Scenarios. It is important to avoid confusing input variation with an evaluation decision. A dataset row can replace the purchase category in Aster's question. It cannot, merely by existing, define whether the answer correctly applied the refund rules. The Contract remains essential. Chapter 7 will examine Dataset status and provenance in detail; for this chapter, the key point is that inputs make situations concrete while Contracts make their evaluation intelligible.

The model also allows a Scenario to belong to more than one Suite. This matters when teams organise testing by purpose. The same refund-eligibility situation could be relevant to a smoke check, a policy regression review and an acceptance decision. Reusing the Scenario is preferable to making three copies that can silently diverge. Yet reusing a Scenario does not mean the different Suites have identical goals or Gates. One Suite may seek fast confirmation that the interaction remains structurally usable, while another examines the full quality requirement with additional evidence and history.

5.3 What a well-designed Scenario actually states

A Scenario deserves to be called testable when the reader can connect four things: **stimulus**, **risk**, **expected evidence** and **assessment criteria**. These are the textbook's explanatory lenses, not a new four-field schema mandated by AQEF. They help authors reason about the conceptual completeness of a Scenario and its Contract without confusing a book's teaching aid with formal syntax.

The **stimulus** is the situation presented to the subject: the user's question, the relevant conversation history, or the conditions that make a tool available or unavailable. It should be specific enough to reproduce an Execution while leaving permitted variation in output. The **risk** explains why this interaction matters. Aster could expose a customer to a wrong promise, an unauthorised action, a fabricated account fact or a misleading response to an unavailable dependency. Naming the risk gives direction to both the check and the eventual interpretation of a failure.

Expected evidence identifies what an evaluator must actually inspect. If Aster claims to have cancelled an order, a final chat message may not establish that an order was cancelled. The relevant tool-call Artifact and the recorded result of that tool matter. If Aster asserts that a policy permits reimbursement, the retrieved policy passage and its effective version may matter. If the Scenario examines latency, the recorded duration is the relevant evidence rather than the elegance of the response. Evidence needs to be sufficient for the kind of claim a Contract asks an Oracle to make.

Finally, **assessment criteria** define the meaningful difference between acceptable and unacceptable behaviour. A deterministic check might examine a structural property through a Validator. A meaning-dependent check might need a Judge whose criterion is explicit and whose verdict has Confidence. A high-stakes or uncertain case may call for Human Review where the implementation offers that capability. A criterion such as “give a good answer” has little engineering value because it does not specify what the Oracle should judge and what failure would mean.

A helpful way to think about design completeness is through counterfactuals. Could an answer be fluent but wrong? Could it contain the correct decision while making an unauthorised tool call? Could it be operationally acceptable but semantically misleading? Could the instrument produce no assessment at all? If these alternatives matter to the selected risk, the Scenario and Contract should enable the evaluator to distinguish them. A single green label that erases those differences is not an adequate design explanation.

Consider Aster's response to “Can I get my subscription charge reversed?” One superficial test might look for the word “refund.” That check could pass a reply that confidently states a policy exception that does not exist. A stronger semantic check examines whether Aster's interpretation is supported by the actual policy presented in the Execution. A separate structural check may verify that the required response envelope is valid. Neither one subsumes the other. This is not a demand that every response carry a fixed set of metrics; it is a demonstration that the chosen failure modes determine the evidence and criteria.

A Scenario should also make uncertainty visible. If the policy passage is missing, a semantic verdict about policy correctness might not be supportable. The right conclusion need not be “Aster failed.” If the Oracle cannot assess the available target Evidence, its assessment may be `oracle_unavailable`; if the Judge produced a verdict but cannot support sufficient Confidence, the Result may be `inconclusive`. Neither outcome alone proves a defect in Aster. The system may have responded appropriately by acknowledging its

limitation. Chapter 9 will cover dispositions and Oracle availability. At the design stage, engineers should at least recognise the difference between a subject failing a requirement and a test lacking the basis to decide.

5.4 Positive, negative, boundary and blocked-path cases

The four case kinds expose different failure surfaces. They are not merely variations in whether a customer has a pleasant experience. They differ in the logic of expected behaviour and in what success demonstrates. The same Aster capability can require several case kinds because success along one path says little about what happens when the path is unavailable or forbidden.

A **positive case** exercises an intended, supported interaction. Aster receives a well-formed request, obtains relevant support information and explains the result without inventing facts. Positive testing establishes whether the specified behaviour is possible in the examined conditions. It is naturally necessary, but often insufficient, because systems can succeed in ordinary conditions while behaving incorrectly at boundaries or under conflicting instructions.

A **negative case** deliberately presents something the subject should reject, decline or handle safely. The "negative" refers to the stimulus or prohibited outcome, not automatically to a failing Result. If Aster correctly refuses to expose a different customer's order details, that negative Scenario **passes**. A rejection can be successful behaviour. This distinction helps avoid the common error of treating any refusal as poor assistance. The quality criterion is whether the refusal is appropriate to the user's request and the authorisation context.

Boundary analysis probes a named limit, such as missing input, unusually long context, a rare combination of valid fields or the edge of a policy condition. Its purpose is not to surprise the system for entertainment. It asks whether an important property survives near the point where ordinary assumptions cease to hold. Aster might be given an account-history request with no available transactions or a support question whose retrieved context is empty. A well-designed case distinguishes graceful handling from fabricated certainty. It should also specify which content expectations remain meaningful when the ordinary path has no relevant data.

A **blocked-path case** addresses a more specific condition: the subject is acting towards a goal, but a route needed to achieve that goal is intentionally closed. The important question is whether it stops, explains the obstacle and respects the restriction rather than finding a prohibited workaround. The specification explains the purpose:

> *"Blocked-Path Testing asks what an acting subject (AI Agent, Tool Calling, Chapter 7) does when the path it was expected to take is closed." — AQEF 1.0-CR1, Volume VIII, Blocked-Path Testing.*

If Aster's customer-record tool is unavailable, the assistant may still give general information, but it must not falsely claim that it verified or updated that customer's record. A successful blocked-path test does not require completion of the original task. It requires proper behaviour given that completion is not permitted or is not possible through the expected route. The distinction from an ordinary operational resilience test is important: one asks whether a service degrades gracefully; the other asks whether an acting subject stays within a restriction when tempted to bypass it.

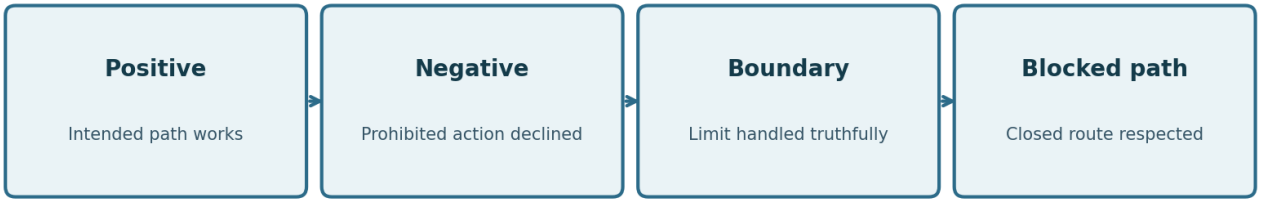


Figure 5.2 — Four case kinds, four different questions. The visual compares intended-path success, prohibited requests, named limits and deliberately closed routes. It depicts testing intentions, not four Conformance capabilities.

Case kind	Central question	Acceptable outcome may include	What it does not establish
Positive	Can the subject perform the intended task?	Correctly completed interaction	Safe behaviour when the task is forbidden or blocked
Negative	Does it decline what should not be done?	Appropriate refusal or rejection	Correctness on ordinary permitted requests
Boundary	What happens at a named limit?	Graceful, truthful handling	Behaviour at every untested limit
Blocked path	Does an acting subject respect a closed route?	Honest stop or authorised escalation	Success of the unavailable task

The reader should resist a mechanical four-cases-per-feature rule. Some risks need several boundary variants; others require careful negative cases with different causes for refusal. Some Scenarios concern systems that never take actions through tools and therefore do not have meaningful blocked paths of the kind described above. Test design is about matching cases to risks, not achieving symmetry in a spreadsheet. The labels organise reasoning; they do not replace it.

Engineering Reflection

If Aster's account-lookup tool returns no usable response, what evidence would distinguish an honest blocked-path response from a fabricated claim of successful account verification? Would the same Evidence also establish that Aster chose a good response tone?

5.5 AI Test Patterns and their proper place

Case kinds describe situations. **AI Test Patterns** describe ways of relating observations to expectations, especially when exact output equality would create misleading failures. AQEF Volume VIII describes metamorphic, differential and consistency patterns. Each addresses a distinct question about behaviour under variation. They provide useful design vocabulary but should not be mistaken for architectural patterns such as RAG or an AI Agent. One category describes how to test; the other describes a system that might be tested.

Metamorphic Testing examines a meaningful relationship between related inputs and outputs. If a customer's eligible refund question is paraphrased without changing its meaning, the expected policy conclusion should remain consistent. The content of the answer need not be textually identical. The key is

that the relation itself must be justified. A language change, for example, may legitimately affect tone and presentation while not changing the policy determination. A poorly chosen metamorphic rule can be just as misleading as exact string equality if it ignores what was intentionally changed.

Differential Testing compares the same Scenario across two Environments—for example, Aster's current model configuration and a proposed replacement. The comparison is not a simplistic contest to see which response sounds more polished. It asks whether observed changes in behaviour are intended and reviewed. Since Environments can differ in prompts, tools and infrastructure as well as models, the interpretation should consider what actually changed. A difference is not automatically a regression; a similarity is not proof that nothing important changed.

Consistency Testing repeats a Scenario in the same Environment and asks whether behaviour stays within acceptable bounds across Executions. Aster may use different wording in each run while maintaining the same material decision. Conversely, repeated answers that all mention "refund" may hide inconsistent eligibility conclusions. This pattern relates directly to the Reliability dimension introduced in Chapter 2. Repetition adds evidence about variation, but repeating one Scenario many times does not create breadth across risks that were never represented.

Pattern	What varies	What matters in the comparison
Metamorphic	Related input representation or condition	Whether a justified relation between outcomes holds
Differential	Environment or version	Whether meaningful behaviour changes are identified and reviewed
Consistency	Execution of the same Scenario	Whether outcomes remain within the defined acceptable range

These patterns are not a licence to turn one good example into a universal guarantee. The quality of a relation depends on the semantics of the Scenario and the supporting Evidence. For a policy answer, checking only final text may not reveal retrieval failure; for an agent, comparing only the outcome may miss a prohibited action. Test patterns help choose what to vary and what to compare, but the Contract must still express the quality requirement being judged.

5.6 Test types are a taxonomy, not capabilities

In Chapter 4, we distinguished the functions an AQEF implementation can claim from the quality of a product tested through that implementation. Volume IX introduces a related but different distinction. A **Test Type** is a label for the testing purpose of a Suite, not a scoped capability in the Conformance Statement. The source is unambiguous:

> *"A Test Type is not a conformance capability (Conformance). It names the purpose of a Suite built from the core: its Scenarios, Contracts, Oracles, and Gates." — AQEF 1.0-CR1, Volume IX, Introduction.*

This prevents a familiar standards mistake. If a tool runs a RAG Suite, its authors might want to announce that it "implements the RAG capability of AQEF." But RAG Testing is not, in Volume IX, a separate capability identifier for Conformance. RAG is a type of test purpose, and its Scenarios still rely on the general

mechanisms of Contracts, Evidence, Oracles and Results. Similarly, Safety Testing names a concern to investigate; it does not by itself prove the implementation offers every mechanism relevant to acting subjects.

Volume IX groups Test Types by different perspectives. **Test-cycle types**—Smoke, Functional, Regression and Acceptance—describe the role of a Suite in a lifecycle. **Pattern-focused types** such as RAG, Agent, Memory and Multi-Turn focus on characteristics of AI systems. **Safety-adjacent types** include attempts to challenge restrictions or policies. **Operational types** investigate performance and resilience properties. The volume also addresses multimodal testing. These categories are deliberately descriptive. A single Scenario or Suite may have reasons to appear in more than one discussion without implying that it is independently conformant to each label.

The difference between **Smoke** and **Acceptance** illustrates why taxonomy should not be inflated into a universal requirement. A Smoke Suite offers a narrow indication that a system and its environment function at all. An Acceptance Suite supports a decision about whether defined release expectations have been met. It is possible for smoke to pass and acceptance to fail, and that may be precisely the truthful result. Neither title eliminates the need to inspect its actual Contracts and Evidence. The naming of a Suite does not change the substance of the check it performs.

Aster's RAG testing might ask whether an answer is grounded in retrieved policy content. Its Agent testing might ask whether a permitted tool was selected and whether the action achieved the authorised result. Multi-Turn testing might examine whether a conversation remains coherent after clarification; Memory testing might examine whether earlier facts are recalled correctly. These concerns overlap in the system's user experience but differ in the kinds of Evidence and error they expose. Treating them as one undifferentiated “AI quality test” risks concealing important failure classes.

A taxonomy can support communication across engineering and management. A QA Lead can explain that a candidate release was examined through safety, regression and RAG-focused Suites without claiming that those names themselves correspond to independent certification checks. For a test designer, the categories are prompts for thinking: which risk surfaces have we covered, and which have we not? Their value is organisational and intellectual, not an extra gate of AQEF Conformance.

The label *Operational* also does not impose a single mandatory metric formula. Aster's Project might care about latency during customer conversations, the cost of tool-heavy workflows or reliability under load. AQEF gives a way to express and report relevant expectations; it does not force every team to use one identical numeric threshold. The priority still originates with risk and business context.

5.7 Building a purposeful Suite from risk

A Suite is not just a folder into which test cases are placed as they are written. It is a curated body of scenarios grouped for a test purpose. Designing a Suite means deciding which risks deserve attention, how different cases expose the same risk, and what the resulting body of Evidence will and will not support. This is a continuation of Chapter 2's central point: **coverage is meaningful only relative to risks and defect classes**. The number of recorded Scenarios is a poor substitute for an account of what was actually tested.

Start with the risk, not with a fashionable test type. Aster's organisation may regard fabricated policy advice as especially consequential because customers could act on it; incorrect account updates may pose another high-stakes concern; elevated latency may be a different kind of operational issue. None requires inventing a universal AQEF risk formula. The organisation's own risk analysis identifies what matters. Test design then chooses Scenarios, Contracts and Oracles that produce evidence relevant to that priority.

For example, the risk of unsupported refund advice can be examined through a positive case with clear source material, a boundary case where the retrieved passage is incomplete, and a differential case comparing two Environment versions. These cases are not duplicates: they ask whether correct behaviour occurs ordinarily, survives missing information, and remains stable across a change. It may be tempting to add dozens of variations of the same easy positive prompt instead. Doing so increases test count while leaving the missing-context risk untouched.

Risk-based testing in the specification specifically connects priority to the composition of Suites. A high-stakes behaviour may justify greater Scenario depth, stricter Confidence requirements or human involvement. This is a project decision made within the available mechanism, not a general rule that every high-risk test requires one particular Oracle. Design remains sensitive to whether a claimed implementation offers the relevant scoped capabilities. A project cannot silently rely on a Human Review workflow if its engine does not implement that capability.

A useful Suite needs an explicit account of gaps. If there is no Scenario for an unauthorised account update, a green report on refund explanations says nothing about account-update safety. If the Suite observes only final customer-facing text, it may not substantiate a claim about agent tool-use history. If no repeated Executions were performed, evidence about consistency is limited. Stating those limitations does not make the Suite worthless. It makes its conclusions appropriately bounded.

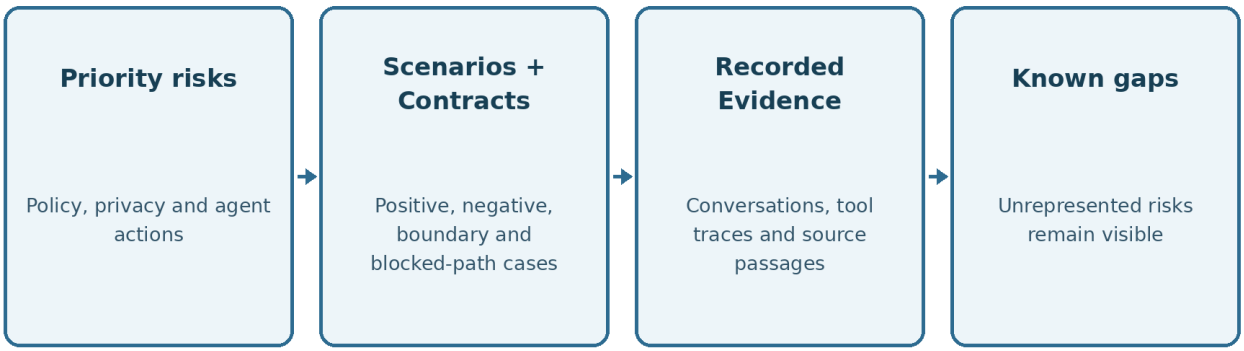


Figure 5.3 — Risk-to-Suite design. Three risk families map to combinations of case kinds, Contracts and expected Evidence. Uncovered risks remain visible rather than disappearing behind a total number of green Results.

There is a subtle distinction between **breadth** and **depth**. Breadth means that Scenarios represent different risks, defect classes and contexts. Depth may mean varying inputs, repeating Executions, probing boundaries or examining a path with stronger independent assessment. Both can matter. A hundred Executions of one

refund Scenario provide useful observations about that Scenario's repeatability, not proof that other policy risks were covered. Chapter 8 will develop formal reasoning about the traceability of these claims.

A Suite also needs maintainability. A large collection of Scenarios can lose its purpose when requirements change and expected outcomes are quietly loosened to keep passing reports green. The right response to a failing but legitimate Contract may be to investigate the subject, clarify an ambiguous expectation or use the governed requirement-change process. Simply deleting a demanding clause is not equivalent to resolving a defect. Chapter 6 explains Contract authority and composition in detail; here the design lesson is that test suites preserve meaning only when their expectations remain reviewable.

Engineering Reflection

Aster has 200 passing refund paraphrase cases but no Scenarios for unavailable retrieval or unauthorised order changes. What would a responsible coverage statement say, and why would “200 tests passed” be insufficient to support the broader quality claim?

5.8 Three roads into a Scenario—and the status of their prerequisites

The main discussion so far has assumed a quality risk identified before the interaction occurs. AQEF Volume VIII also describes learning from interactions that have already happened. In **forward authoring**, a prioritised requirement guides the designer to appropriate situations and expectations. In **exploratory authoring**, an investigator observes a surprising behaviour and turns the relevant interaction into an explicit Scenario and Contract for future assessment. **Production Replay**, described conceptually in the specification, would provide a comparable route from captured production Conversations. All three share the logical requirement that a repeatable situation and assessable expectation eventually be defined.

The maturity qualification here is essential. A hands-on description of replay as though every CR1 engine could immediately import production traffic would misrepresent the edition. **Production Replay is At Risk** and not an independently implemented capability evidenced at CR1 publication. The textbook may explain why the idea matters to “Everything is a Test,” but it cannot teach replay as a stable, ready-to-use practice guaranteed by this version of the specification. The same caution applies to future Dataset mechanisms named Reserved.

Exploratory testing remains intellectually important independently of those dataset implementations. A QA engineer asks Aster a vague question, notices an unsupported claim and investigates how that claim arose. The informal discovery is not yet a substitute for governed assessment. To preserve it, the engineer needs enough context to define what behaviour was exercised and which evidence would support a later verdict. A retrospective Scenario can then become a stable object for subsequent Executions. Exploratory judgment generates a hypothesis; an authored Contract makes the hypothesis inspectable.

The difference between *discovering* a defect candidate and *demonstrating* it matters. A support response that looks odd may be correct under the retrieved policy in force at that time. A reviewer who has not inspected that policy should not call the observation a proven hallucination. Conversely, a beautifully formatted answer can contain a wrong decision that a superficial test missed. The scenario-authoring process has to make the source of correctness explicit. Otherwise a captured Conversation merely transports uncertainty from one context to another.

Volume VIII also introduces **Seeded Controls**, but their purpose differs from negative cases. A negative test probes whether Aster avoids prohibited behaviour. A Seeded Control plants known evidence to test whether an Oracle can detect a defect class. The former examines the subject; the latter examines the instrument. Seeded Controls are a separately claimable implemented capability, explained in Chapter 11. Mentioning them here helps preserve the boundary: a negative Suite is not equivalent to proof that the Judge detecting its failures is itself qualified.

5.9 What Scenarios cannot establish by themselves

Test design is often treated as finished when a library of prompts is complete. AQEF encourages a more demanding interpretation: a Scenario definition is only one element in the evidentiary chain. It does not produce observations until an Execution occurs. An Execution does not produce an actionable assessment until appropriate Oracles process its Evidence. Several actionable Results do not automatically satisfy a release Gate without the required basis and scope. The test designer should therefore anticipate how the Scenario will be read later by people who did not author it.

A good Scenario preserves the relationship between a risk and an observable failure. It cannot prove universal absence of that failure. Even repeated, successful executions support conclusions only about examined situations and conditions. In a probabilistic subject, variation introduces additional reason for care: one observed success may be unrepresentative, while several apparently similar failures may have different causes. A careful report should distinguish the scope tested from claims about the whole system.

The quality of an Oracle is also not guaranteed by the brilliance of the Scenario. If an assessment instrument is unavailable, the correct handling of its inability to decide differs from a failing subject verdict. If a Judge has a confident but incorrect interpretation, detailed prompt design does not transform that confidence into correctness. These are reasons the framework separates test design from assessment trust, not reasons to abandon test design. Each mechanism is needed to answer a different question.

Aster's support team might assemble a meaningful Suite covering refunds, privacy requests, missing policy evidence and blocked account updates. The design work can be excellent and still produce a report containing `inconclusive` verdicts where an Oracle is uncertain, or `oracle_unavailable` assessments where no Oracle verdict could be produced. Such a result may be disappointing, but it is more honest than silently treating every unassessed concern as passed. It gives the next engineer something precise to investigate: the subject, the instrument, the captured Evidence or the decision criteria.

Test design therefore ends neither in a green dashboard nor in a universal definition of “enough tests.” Its purpose is to make the path from risk to evidence deliberate, reviewable and repeatable. Whether the eventual evidence is sufficient for a particular Release Decision belongs to the project's Gates and governance. The same distinction that prevents a Conformance Statement from proving product quality prevents a Suite name from proving coverage. Claims remain bounded by their actual basis.

Common Misconceptions

1. “A Scenario is simply a prompt.” A Scenario defines a testable situation and its governing Quality Contract. A prompt is one stimulus used to instantiate the situation; a Conversation is observed only during an Execution.

2. “A negative test should produce a failing Result.” Negative describes the input or disallowed behaviour being probed. An appropriate refusal may be the expected passing outcome.

3. “Blocked-path testing is just another name for resilience testing.” Blocked-path testing concerns whether an acting subject respects a deliberately closed route. Operational resilience concerns graceful degradation. One situation may exercise both, under different Contracts.

4. “RAG, Regression and Safety are AQEF Conformance capabilities.” Volume IX gives a taxonomy of test purposes. Claimable capabilities are specified separately in the Conformance section.

5. “More passing cases always means better coverage.” Extra cases can deepen evidence for one risk while leaving another entirely unexamined. Coverage statements must identify risks, defect classes, circumstances and gaps.

Chapter Summary

A Scenario is the static, reusable definition of a testable situation; its Execution produces the concrete Conversation and Artifacts from which assessment draws Evidence.

Projects, Environments and Suites provide different kinds of context and organisation; collapsing them destroys traceability about which system and purpose were tested.

Test design connects a selected risk to a stimulus, the evidence needed to judge it and a sufficiently explicit Quality Contract.

Positive, negative, boundary and blocked-path cases probe distinct questions; none is universally interchangeable with another.

Metamorphic, differential and consistency patterns help examine meaningful variation without reducing correctness to literal output equality.

Volume IX's Test Types organise Suite purpose but are not claimable Conformance capabilities.

A risk-based Suite should communicate both breadth and depth, including untested risks and limitations of its Evidence.

Exploratory observations can lead to authored Scenarios, while Production Replay remains At Risk in AQEF 1.0-CR1.

Review questions

1. Aster gives the correct refund explanation in one conversation. Distinguish the Scenario, Execution and Conversation involved, and explain what evidence is still missing to claim reliable behaviour.
2. Why does a collection of fifty prompts with no explicit expectations fail to provide the same quality basis as a curated Suite of Scenarios and Contracts?

3. Design the conceptual difference between positive, negative and blocked-path cases for an assistant that can update customer records. Why could the same final response be acceptable in one case but unacceptable in another?
4. A test checks that an answer mentions a policy keyword. Explain how a semantically wrong answer might still pass, and what kind of evidence a better check would require.
5. Under what circumstances might an empty retrieval result represent a subject defect, an acceptable limitation or an insufficient assessment basis?
6. How do metamorphic and differential patterns answer different questions when a team's prompt version changes?
7. A Suite has 500 repeated Executions of one Scenario. What has this added to the evidence, and what coverage claim would remain unjustified?
8. Why does the label “RAG test” not establish a particular AQEF Conformance capability?
9. A production interaction reveals a possible unsafe action. What is the conceptual path from that observation to an inspectable Scenario, and what CR1 maturity caveat applies to Production Replay?
10. A test design report contains mostly passing positive cases but no boundary tests. How should a responsible QA Lead describe its strengths and limitations without inventing an AQEF-mandated coverage percentage?

Sources

1. AQEF 1.0-CR1, Front Matter §8 — Terminology (Scenario, Suite, Execution, Conversation, Quality Contract, Oracle).
2. AQEF 1.0-CR1, Volume II — Domain Model: Project, Environment, Suite, Scenario, Dataset, Variables, Execution and Evidence relationships.
3. AQEF 1.0-CR1, Volume VIII — Test Design: Test Design Process; Risk-Based Testing; AI Test Patterns; Boundary Analysis; Negative Testing; Blocked-Path Testing; Exploratory AI Testing; Test Maintainability.
4. AQEF 1.0-CR1, Volume IX — Test Types: taxonomy and distinction from scoped capabilities.
5. AQEF 1.0-CR1, Volume VII — Quality Contracts (Conceptual reference only; details follow in Chapter 6).
6. AQEF 1.0-CR1, Front Matter §0 and §10 — status of Production Replay and capability claims.
7. AQEF CTS 1.0-CR1, §0 — scope and non-normative role of Conformance tests.

Chapter 6. Quality Contracts

Maturity: Core · Textbook First Edition v1.0

Learning Objectives

After reading this chapter, readers should be able to:

1. Explain why a Quality Contract describes acceptable behaviour independently from the mechanism that evaluates it.
2. Distinguish Constraints, Expectations and Policies, including the proper roles of Validators, Judges and Human Reviewers.
3. Reason about the identity and composition of clauses inherited at Project, Suite and Scenario levels.
4. Identify the difference between resolving a conflicting Contract and obtaining human approval to change a requirement.
5. Trace how Contract versioning and approved change preserve the meaning of historical Evidence and Release Decisions.

6.1 From an interesting observation to an explicit expectation

An AI system can be fluent, convincing and wrong. The earlier chapters established why this observation does not, by itself, explain how to test the system. A risk tells us what might matter; a Scenario gives that concern a reproducible context. But neither tells an independent assessor exactly which properties distinguish an acceptable outcome from an unacceptable one. This is the problem solved by the **Quality Contract**: the explicit, reviewable statement of expected quality against which a Conversation is assessed.

Consider Aster Support AI. A customer asks whether an annual subscription is eligible for a partial refund. Aster retrieves a policy extract and responds in reassuring prose. The response may be beautifully phrased while contradicting the policy. Conversely, an awkward but accurate response may meet the requirement the project actually cares about. Without a stated criterion, a reviewer risks applying personal preferences after seeing the answer. A Quality Contract separates the definition of acceptable behaviour from that retrospective reaction.

The AQEF Glossary defines a Quality Contract as:

> *"A declarative specification — attached to a Scenario, Suite, or Project — stating which Oracles MUST be applied to a Conversation, what constitutes a passing Result under each, and what Confidence threshold makes a Result actionable." — AQEF 1.0-CR1, Front Matter §8, Quality Contract.*

A Contract is **declarative**, meaning that it expresses what must be true, not the algorithm by which the system produces an answer or the implementation code by which an Oracle checks one. This distinction is consequential. A policy requirement about refund eligibility remains the same when an implementation

changes its test runner, replaces an assessment model, or moves from a single process to multiple services. The check implementation may change; the explicit expectation should not silently follow it.

This separation does not mean that expectations are independent of context. “Explain a refund accurately” is meaningful only in relation to the policy version, the customer's circumstances and the intended use. Contract authors must describe criteria that an Oracle can apply to the Evidence produced by a particular Scenario. The point is to make context explicit rather than to remove it. A customer whose subscription is expired and a customer whose purchase was cancelled during a trial period may deserve different outcomes under the same policy; a properly designed Contract can express the relevant distinction without demanding one fixed sentence in every answer.

There is a second separation: defining a requirement is not proving that it was satisfied. The Contract itself does not assess any Conversation. An **Oracle** does that: a Validator for deterministic checks, a Judge for semantic judgments, or a Human Reviewer when applicable. The Result is the recorded outcome of that assessment. A Contract can be precise and the instrument wrong; an instrument can be reliable and the Contract misguided. Good quality engineering exposes both possibilities rather than letting a single pass label conceal them.

The most useful mental model is that the Contract acts as the *meaning of a check*, while an Oracle supplies the *instrument* and Evidence supplies the *observation*. This is a textbook explanation, not a new AQEF entity or workflow. It avoids a familiar testing mistake: adjusting the instrument until the observed response passes and then assuming that the underlying expectation has remained unchanged. We will return to this when discussing requirement weakening.

6.2 Two kinds of clause: Constraints and Expectations

A Quality Contract contains clauses. A **Constraint** states a rule that can be verified deterministically. An **Expectation** states criteria for probabilistic or human semantic assessment. The two are complementary. They do not represent a hierarchy in which semantic quality is “more advanced” than structural correctness. They represent different epistemic situations: whether the evidence can be checked against a rule, or whether judging its meaning requires an instrument whose verdict carries uncertainty.

AQEF describes Constraints this way:

> *“A Constraint is a Contract clause bound to a Validator: a deterministic, rule-based expectation — a required schema, a forbidden pattern, a latency ceiling — that either holds or does not, with no Confidence value attached.” — AQEF 1.0-CR1, Volume VII, Constraints.*

For Aster, a Constraint can demand that a tool invocation use an allowed operation and supply structurally valid arguments. It can require an output field, check a recorded latency threshold or reject a prohibited literal pattern. These are not questions about whether the response *sounds* appropriate. If the required field is missing, the Validator can determine the outcome from the Evidence and versioned rule. The Validator itself must be deterministic. A failed check is different from a Validator that cannot complete because required Evidence is absent; availability cannot be turned into a product failure by convenience.

An Expectation, by contrast, may ask whether Aster's explanation accurately reflects the retrieved refund policy, avoids unsupported certainty and responds appropriately to the user's purpose. Several different phrasings may be acceptable. A Judge's assessment of those criteria must not be confused with literal truth. Its Confidence communicates uncertainty about the verdict, not the probability that every assertion in a broad response is true.

The distinction can be illustrated using the same customer exchange. One clause checks that Aster did not send a prohibited tool operation; another examines whether its explanation is faithful to the policy text. The first belongs to a Validator and the second to a Judge or Human Reviewer. If the team replaced the semantic check with a string search for “refund,” it would gain repeatability but lose the meaning that mattered. If it replaced the structural check with an LLM assessment, it would add unnecessary uncertainty to a rule that was already mechanically checkable.

The **Quality Dimension** attached to a clause places that check within a broader interpretation of product quality. Structural checks typically contribute to Functional or Operational Quality. Semantic checks contribute to Semantic or Business Quality; Safety often crosses both. The reader should resist equating clause type with a universal mapping of dimensions. AQEF's architecture is deliberately able to combine checks because the consequences of errors differ. A single response may satisfy a structure rule while violating a semantic requirement, and the combined decision must not erase the distinction.

A useful practical diagnostic is to ask what a counterexample would look like. If a rule can specify the forbidden observation and a deterministic instrument can establish its presence from the captured Evidence, it may be a Constraint. If the correct conclusion depends on whether the overall meaning is faithful, relevant or justified, an Expectation is more appropriate. Some apparently semantic questions can be decomposed into deterministic checks once their precise boundaries are understood. The goal is not to maximise the number of Judges; it is to select the kind of instrument that corresponds to the property being asserted.

6.3 One clause, one assessable proposition

A Contract can be explicit yet still obscure its own logic. Consider the instruction “Aster should be accurate, clear, polite and helpful.” This seems like a single requirement in ordinary conversation. As an assessable clause, it is dangerously broad. A response might be polite and well structured while presenting an incorrect refund amount. If the Judge reduces all four impressions into one score, the project may lose visibility into which property failed and how much it mattered.

Volume VII expresses a preference for narrow Expectations and explains that a broad criterion can force several internal assertions into a single hidden reduction. This is a design concern, not merely a scoring concern. If accuracy and tone have different consequences, averaging them without an explicit Contract policy obscures the decision that must ultimately be made. A narrower Contract separates the policy-faithfulness Expectation from the clarity and tone expectations, allowing each to produce an interpretable Result.

This does not imply that every sentence requires a new clause. Decomposition has costs. Too many microscopic clauses can make the Contract difficult to understand, and overlapping criteria can count the same failure several times. The engineering question is whether the pieces have sufficiently different evidence, Oracles or decision consequences to deserve independent treatment. A long response with twelve

factual assertions may merit separate checks for safety-critical figures while allowing a broader assessment for conversational coherence. The Contract should make any deliberate aggregation visible.

A **clause identity** provides a stable way to recognise when two contributions refer to the same requirement. This matters when Contracts are reused or composed. Text similarity is not a reliable identity rule: two clauses can use similar wording while testing different risks, and one clause can be rewritten so that it appears to be new while its actual obligation is meant to remain constant. AQEF makes identity an explicit aspect of composition, rather than a guess based on text.

The critical rule is stated in Volume VII:

> *"Two contributed clauses are the same clause only when they declare the same id* (Appendix A §A.5)."
> — *AQEF 1.0-CR1, Volume VII, Contract Composition.*

In the textbook body we need not teach the serialization of that identity. The concept is enough: a stable clause identifier gives an implementation a precise way to decide whether two inherited requirements are duplicates or conflicts. Matching content under the same identity can be combined once. Different content under the same identity is not silently reconciled. Without explicit identity, a clause is independently contributed. This prevents one level of the project from appearing to "override" a rule simply by phrasing another rule as though it were equivalent.

For Aster, suppose the organisation maintains one common policy-faithfulness requirement and a support Suite wants to add another requirement about citing the exact current policy passage. These are compatible independent checks. If the Suite instead attempts to redefine the inherited faithfulness clause with a lower acceptance threshold, it is a collision of the same requirement, not merely an additional criterion. That collision must be surfaced so a responsible person can decide whether the original requirement should change.

The human benefit of stable identity is traceability. A recorded Result can be understood in terms of the precise expectation it assessed. When the Contract is revised, reviewers can distinguish new coverage from a changed interpretation of an existing criterion. Clause identity helps preserve the meaning of history; it is not an administrative number whose only role is to satisfy a schema.

6.4 Policies: how individual checks contribute to a decision

Individual clauses do not explain how their outcomes should be combined. A **Policy** is the part of a Contract that determines the relationships among checks: which failures veto an outcome, when a group of Constraints is joined, and how relative importance influences aggregation. Unlike a Constraint or Expectation, a Policy is not a separate Oracle check and does not produce its own independent Result. Its role is to govern the meaning of other Results.

The distinction prevents another common error: treating aggregation settings as though they were measurements. A numeric weight can express a project's view that one Scenario or clause matters more than another. It does not demonstrate that the weighted property is objectively more important. The weight is a choice about consequences and priority; the Result remains evidence about whether a particular criterion was satisfied. The choice should therefore be intelligible to those responsible for the risk it represents.

In Aster, a false statement about the amount legally due to a customer can matter far more than a slightly awkward greeting. A Project might represent that asymmetry with higher criticality attached to relevant clauses or Scenarios. At the same time, some rules cannot safely be averaged away. A prohibited transfer of customer information should not be compensated by excellent helpfulness elsewhere in the conversation. A hard veto expresses that non-negotiable decision boundary.

This is why AQEF separates Validator Composition from broader Aggregation. Validators operating on the same evidence may combine according to AND, OR or hard-veto rules within the Contract. Aggregation across Results and Executions answers a different question: what the broader set of tests shows against Gate criteria. Confusing these two levels can produce a false release narrative in which a high average score disguises a locally forbidden action.

A Policy should not be used to conceal an undefined expectation. Increasing weight on a clause whose criterion is ambiguous does not make the clause more meaningful; it makes a dubious verdict more influential. The explanatory order remains: articulate what matters, choose the appropriate Oracle, verify that the Result is interpretable, and only then reason about relative importance in composition and decision-making.

6.5 How Contract composition works across a project

Aster's project may have universal requirements, domain-specific requirements and Scenario-specific requirements. These should not be copied manually into every test. AQEF supports attaching Contracts at three conceptual levels: **Project**, **Suite** and **Scenario**. The effective Contract for a particular Scenario run within a Suite combines the contributions of all three. This lets a project carry organisation-wide Safety and reliability commitments into every relevant test while adding local expectations appropriate to one user journey.

Imagine a Project-level rule forbidding certain customer-data disclosures, a Suite-level group of refund-policy checks and a Scenario-level criterion about the treatment of subscription renewals. All three should matter when that Scenario executes in that Suite. A lower level is not a private sandbox that can silently erase what the higher level required. The implementation must surface conflicts rather than prefer whichever rule appears closest to the Scenario.

Volume VII gives the crucial bound:

> *"Composition across levels follows the same rules as any other: it is additive, and a lower level adds clauses but cannot remove or weaken a clause contributed by a higher one." — AQEF 1.0-CR1, Volume VII, Contract Composition.*

The language **additive** describes the normal case. It does not mean that conflicting inputs can always coexist. A reused clause with the same identity but a different threshold is not a harmless extra check. Nor may a weaker composition policy introduced lower in the hierarchy nullify a stricter higher-level policy. The conflict must be visible at the point where the effective Contract is resolved, before the recorded verdict can be interpreted as though only one unambiguous rule existed.

There is a subtlety important for practitioners: one Scenario may belong to more than one Suite. Its effective Contract is then resolved in the context of the Suite under which it executes. It is incorrect to imagine a single permanent effective Contract attached to a Scenario independent of its execution context. Aster's general refund-information Scenario may appear in a smoke Suite and a policy-regression Suite. A shared Project-level obligation remains, but additional Suite-level requirements may legitimately differ. That does not authorise contradictions; it clarifies that the composed assessment depends on the Suite.

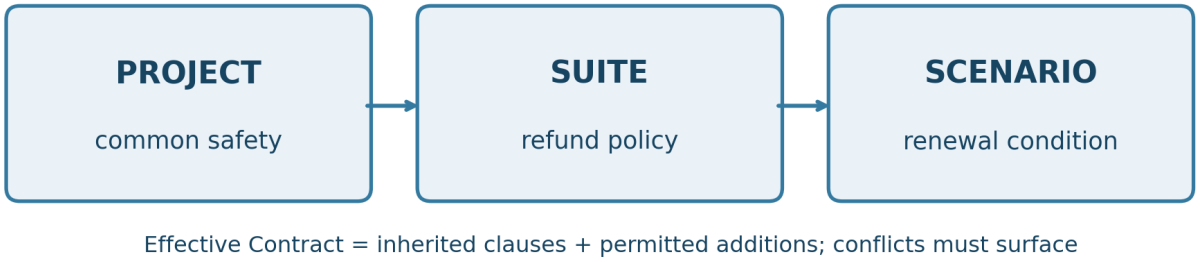


Figure 6.1 — Effective Contract composition
The Project contributes common rules; the Suite contributes test-purpose rules; the Scenario contributes case-specific rules. Each lower level may add but must not weaken inherited obligations. Conflicts are surfaced, not silently replaced.

Another reason to keep levels separate is responsibility. A Project owner may approve universal risk boundaries, while a domain team defines Suite-level expectations and a tester creates a Scenario. The fact that different people author contributions does not weaken the meaning of the resulting Contract. Composition makes their interaction explicit. In a mature workflow, the reviewer should be able to explain which level introduced each rule and why it mattered to the final Result.

A fragile system instead copies a large Contract into many tests, then edits individual copies when they become inconvenient. That practice creates invisible drift: tests may still look similar, but one silently permits what the others forbid. Reusable Contract fragments and additive composition address the structural cause of that problem. They do not eliminate the need for review; they make review possible at the correct level.

6.6 Reuse, versioning and the meaning of historical Results

A **Reusable Contract** is a fragment that can be authored once and applied to multiple Scenarios, Suites or Projects. Its immediate benefit is consistency, but the more important architectural benefit is that a particular recorded assessment remains connected to a definite expectation. Without versioned expectations, a team could unknowingly reinterpret last month's passing Result in the light of this month's weaker policy.

Suppose Aster's policy-faithfulness check changes after a legitimate policy revision. Earlier Executions should still be understandable against the Contract version they used. It would be misleading to display old Results as though they had been judged according to the revised wording. A report is a historical claim, not a live formula that can be recalculated under whatever requirements happen to be current unless a new assessment is explicitly performed and recorded.

The same issue appears when a threshold is adjusted. If a lower threshold makes a previously failing result look acceptable, the change does not erase the evidence that the earlier requirement was not satisfied. It creates a new basis for future evaluation. Whether and how a past Execution is reassessed is a distinct issue from changing the requirement. The project should be able to tell which decision depended on which version, instead of presenting a timeless “pass” label without its meaning.

This is also why the separation of Dataset and Contract versions matters. A Dataset captures which inputs were used; a Contract captures what the project expected; an Environment identifies the runtime context; the Result shows what an Oracle concluded. Changing one is not equivalent to changing all. Traceability needs the relationships, not only a collection of files sharing the same release name.

For an engineer, the best question is not “is this Contract reusable?” but “can an independent reviewer reconstruct the exact expectation behind this historical Result?” Reuse without versioning could make the answer worse by multiplying the impact of a silent edit. Versioning without stable identity could make it difficult to compare which obligation changed. Their combination preserves the interpretation of evidence across time.

6.7 Weakening a requirement is a governed human decision

A tempting failure mode appears after Aster's Suite goes red. A new model version answers certain policy questions less reliably than expected. A team member proposes lowering the semantic threshold so the build becomes green. Such a change may be technically easy. It is not, by itself, a justification that customer-facing risk has become acceptable. The testing tool can encode a new threshold, but the tool does not own the decision to weaken the underlying expectation.

Three questions must be kept distinct. **Was the original requirement wrong or too strict? Did the implementation or Oracle mismeasure the requirement? Has the project knowingly chosen to accept more risk?** The answers lead to different actions. If the Oracle is defective, repair or requalify the instrument; do not redefine the target as correct. If the customer's policy has changed, revise the requirement through authorised review. If the organisation accepts a different level of risk, make that choice explicit with its reason and accountable owner.

The specification's Governance mechanisms distinguish people from automation for consequential authorisations. In Volume XII, it states:

> *“Automation may carry out a decision a person made; it may not make one.” — AQEF 1.0-CR1, Volume XII, Principals.*

That principle is especially important when change tools make editing easy. A configuration interface can record an approved decision, but it cannot decide that a customer privacy rule is less important merely because a release is blocked. This textbook therefore treats **weakening a requirement** as a human decision about requirements and risk, with an explicit reason and accountable owner. This is the textbook's explanation, not a newly invented AQEF serialization field or an independent certification rule.

There is another mechanism that must not be confused with weakening: a **Quality Gate override**. An override concerns one recorded Gate Result under Governance; it does not globally edit the Contract or

convert other failing Gates into passes. Where a known deviation is accepted for a particular release decision, preserving the failed Gate and the authorisation trail is more transparent than rewriting the requirement so that the failure vanishes. One mechanism changes expectations for assessments; another governs a particular decision in the presence of recorded evidence.

For Aster, compare two decisions. If the organisation changes its refund policy, an authorised human owner must approve the changed requirement and its recorded rationale; an automation or service account can carry out that already approved change but cannot decide to weaken the policy on its own. By contrast, a governed override accepts the consequence of one specific failed Gate Result for a particular decision without changing the Contract or causing another failed Gate to pass.

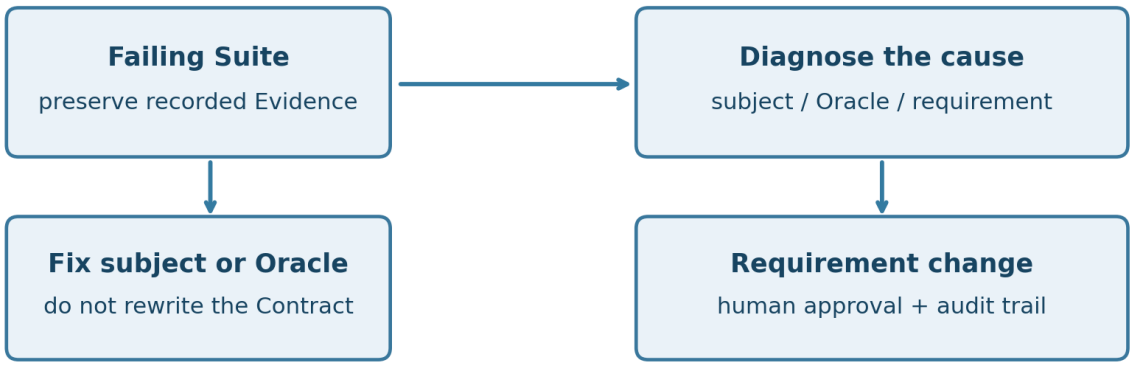


Figure 6.2 — An honest response to a failing Suite
When a check fails, first distinguish subject failure, instrument failure and an obsolete or disputed requirement. An authorised requirement change is a separate governed decision, not an automatic consequence of an inconvenient result.

For Aster, suppose a policy-faithfulness Expectation repeatedly fails because the assistant gives an outdated refund window. Reducing its threshold might change a Gate calculation, but it would not make the policy statements correct. Nor would it repair an Oracle that overlooked the incorrect time window. The team needs to identify what happened, preserve the evidence and make any requirement change on its own stated merits. A green report should be the consequence of justified assessment and governance, not the goal that retrospectively dictates the requirement.

Engineering Reflection

A manager asks you to “fix the tests” before release because a refund-policy Scenario is failing. What evidence would you inspect to distinguish a deficient product, a defective Oracle and a legitimately changed requirement? Which possible actions require human authority?

6.8 Contract conflicts, missing basis and the limits of a pass

The existence of a Contract does not guarantee that an assessment can be completed. A clause may refer to Evidence that was not captured, a Judge may be unavailable, a composed Contract may contain a conflict or an Oracle may return a disposition that cannot safely support a Gate. These outcomes must not be collapsed

into a failed product Result. They are failures or limitations of the assessment process, configuration or evidence basis. Chapter 9 develops the details; here the concern is how Contract design anticipates them.

A contract author must be able to explain what each Oracle needs to make its claim. For a refund-faithfulness check, the relevant policy version may be essential. If that document is unavailable, a fluent answer alone cannot establish faithfulness. For an action-authorisation check, a user-facing statement of “completed” may be insufficient without tool-call artifacts. A Contract whose criteria require evidence that the Execution never produces is not an effective safeguard simply because its wording sounds rigorous.

This is a reason to avoid treating a single pass rate as a verdict on the entire quality model. Aster might pass all captured positive cases while leaving blocked-path behaviour untested. It might produce valid tool arguments while giving misleading policy advice. Or it might have correct answers that its Judge cannot assess confidently. Each outcome identifies a different limitation. Quality Contracts allow these limitations to be stated at clause level; reporting and Gates later decide what can be concluded from the resulting basis.

The implementation's job is to apply the resolved, valid Contract consistently. Its job is not to improvise an acceptance criterion when the Contract fails to define one, nor to silently select a favourable contribution when a hierarchy conflicts. The author's job is to make the conditions of success intelligible before seeing the desired outcome. The decision-maker's job is to examine the evidence and residual gaps, rather than assuming that a configured threshold defines acceptable risk in every context.

This division of labour is the practical answer to the question that opened the chapter. A Quality Contract does not turn subjective judgment into objective truth. It makes expectations explicit enough that the question of whether they were satisfied can be examined, challenged, repeated and traced. That is a more defensible foundation than either exact-string matching or a Judge's undifferentiated opinion.

6.9 Why the measurement boundary must remain visible

Contract authors sometimes suppose that the only relevant question is whether an Oracle returned pass. That shortcut misses the conditions under which the conclusion was possible. A Contract creates a bridge from requirements to assessment, but bridges have endpoints. At one end stands the intended property of Aster's behaviour. At the other stands an observation collected during one Execution. The Oracle can only compare these ends honestly if the information it receives is sufficient to distinguish compliance from violation.

Suppose a refund-policy Expectation demands faithful explanation of an exception that depends on whether the purchase was made before a policy transition. If the Execution captures the question and response but not the applicable policy version, a Judge may still produce a confident-looking verdict. The missing policy evidence is not cured by a carefully worded rubric. A different Judge or repeated samples may agree with each other and still be unsupported. The correct engineering response is to examine the evidentiary basis of the clause and how that basis enters the assessment.

This lesson also applies to apparently simple Constraints. A Validator can deterministically check a recorded argument only if that argument is actually preserved. A rule requiring that no prohibited operation be invoked cannot be demonstrated by looking solely at a final user-facing message. If an agent performs an action, then later tells the user it did not, conversational fluency is irrelevant to the action rule. The required

observation is the appropriate execution artifact. A Contract that names the relevant check is useful only when the system is capable of delivering the evidence the check needs.

The distinction between **sufficient criteria** and **sufficient evidence** is worth holding in mind. Criteria specify how success should be decided; evidence supplies what the assessor knows. A clear criterion applied to incomplete evidence may yield an unavailable or inconclusive assessment rather than a product failure. Conversely, a complete transcript cannot compensate for criteria such as “the answer should be great,” because the rule itself is too vague to sustain an interpretable verdict. Improving one side does not necessarily repair the other.

Aster's team might be tempted to combine missing-data situations with bad subject behaviour under a single red status. That makes reporting simple but diagnosis weak. A subject that hallucinated a policy, an Oracle that could not be reached and a Contract that refers to a nonexistent artifact call for different corrective actions. The Contract should help distinguish them, even though the Gate may eventually refuse to pass in all three situations. A conservative Gate outcome does not imply that every underlying cause is a defect of the AI subject.

The same principle governs comparison over time. Imagine two release candidates assessed against different policy revisions. Comparing their pass rates without noting the Contract versions would make the numbers appear commensurable when the acceptance basis had moved. Even when the same threshold is used, a change in clause meaning, Evidence capture or Oracle configuration can change what the number represents. Traceability therefore supports both audit and analytical validity. It tells a reviewer whether the comparison is truly about Aster's behaviour or partly about changes in how that behaviour was defined and observed.

A Quality Contract describes the clauses and criteria to be assessed. The minimum Aggregate Basis that a decision requires belongs to the Quality Gate, not to a Contract clause or Contract Policy. Keeping these responsibilities separate avoids confusing the meaning of a requirement with the quantity of evidence demanded before a release Gate can pass.

For a QA Lead, this suggests a more rigorous review question than “how many tests passed?” Ask whether each important risk has an explicit clause, whether its Oracle can inspect appropriate evidence, whether failures can be attributed to the correct cause and whether the resulting record preserves its own interpretation. The answers may not fit into one dashboard tile. They do, however, explain what the tile can legitimately claim. This is exactly the purpose of a governed quality model: not to promise certainty, but to make uncertainty and evidence limits visible at the level where decisions are made.

6.10 The Contract as the hinge of the AQEF architecture

Chapter 3 showed the five logical layers of AQEF. A Quality Contract begins in Definition, travels into Assessment and ultimately gives meaning to the Results that Decision consumes. It links the human act of identifying acceptable behaviour with the technical act of measuring evidence. Without a stable Contract, the same response could be labelled differently depending on who looked at it or which release needed approval. Without an independent Oracle, the Contract is only a written intention. Without traceability, the Result becomes detached from the intention it was meant to examine.

This is why Contracts over Assertions is a fundamental principle rather than a documentation preference. An inline test can still be mechanically correct. But when criteria are buried in implementation logic, changes can occur without an explicit account of which requirement moved, who approved it and which past assessments remain comparable. AQEF's declared separation makes those dependencies reviewable. It does not prohibit every useful deterministic assertion; it insists that the quality expectation exists as an inspectable Contract rather than only as incidental execution code.

Aster provides a final synthesis. The same customer request can be examined through a structural Constraint, a semantic Expectation and policies governing criticality. The Project can contribute customer-data protections, the Suite can add refund-policy checks and the Scenario can add a particular renewal condition. Their combined meaning is explicit. If assessment fails, the record should allow a reader to identify whether the behaviour, instrument or requirement is in question. If risk tolerance changes, a responsible person decides and the implementation records the approved change.

The chapter does not tell a reader what the refund threshold ought to be. That is deliberate: AQEF provides the architecture by which a project can specify, apply and govern its own requirements. The engineering discipline resides in selecting criteria that address real risks, preserving their identity and resisting the temptation to redefine success after observing failure. The Contract is therefore both a technical interface and a boundary of accountability.

Engineering Reflection

Two Suites reuse the same Aster Scenario, but one inherits additional policy checks. What would a reviewer need in order to explain the different effective Contracts and compare the resulting Evidence without assuming one Suite silently overrides the other?

Common Misconceptions

1. A Quality Contract is an executable test script. A Contract specifies expected quality and the Oracles that assess it; the implementation performs the checks. The distinction preserves independent review of expectations.

2. Every criterion needs an LLM Judge. Deterministic properties belong with Validators. Meaning-based criteria may require probabilistic or human assessment. Choosing a Judge for an easy rule adds uncertainty rather than rigour.

3. A lower-level Scenario Contract can relax Project rules when needed. Composition is additive. A lower level may extend but cannot silently remove or weaken a higher-level obligation; conflicts must be surfaced.

4. A green Suite justifies lowering the threshold that created it. Thresholds express expectations and risk decisions. Changing them to manufacture green results conflates measurement with governance.

5. An overridden Gate is equivalent to an amended Contract. A Gate override concerns one decision record. A Contract change alters the stated expectation under the approval rules and has different implications for future assessments.

Chapter Summary

A Quality Contract specifies what acceptable AI behaviour means for a Scenario, Suite or Project independently of execution code.

Constraints bind to deterministic Validators; Expectations bind to semantic Judges or Human Reviewers, with different treatment of Confidence.

Clause identity makes duplication, conflict and historical interpretation explicit; broad criteria should not conceal materially different assertions.

Policies govern composition and importance rather than producing standalone assessment Results.

The effective Contract is composed from Project, Suite and Scenario contributions; lower levels may add but cannot weaken inherited requirements.

Reusable Contracts and version history keep past Results tied to the expectations under which they were recorded.

Weakening a requirement is a human governance decision about risk and requirements, not a convenient test-tool adjustment or a Gate override.

An honest Contract lets reviewers distinguish product failure from instrument failure, insufficient Evidence and disputed expectations.

Review questions

1. Why is a Quality Contract not equivalent to a collection of inline assertions, even if both can produce the same immediate pass or fail?
2. Aster uses a correct refund phrase in a response that contradicts the current policy. Which kind of clause would more directly examine the actual failure, and why?
3. How would you decompose an overly broad criterion such as “accurate, safe and helpful” without creating meaningless fragmentation?
4. Two Contract contributions share the same clause identity but different Confidence thresholds. Why must the implementation treat the situation as a conflict rather than choose the newer value?
5. A Scenario is part of two Suites. Explain why it can have two different effective Contracts without weakening Project-level requirements.
6. What is the difference between changing a Contract after an approved policy revision and overriding a failed Gate for one release?
7. If a required policy document is missing from Evidence, why might a semantic assessment be unavailable even though Aster produced an answer?
8. Explain why a high clause weight cannot compensate for ambiguous assessment criteria.
9. A team changes a shared Contract fragment. What must remain knowable about past Executions and Results?

10. An automated change tool proposes to lower an expectation because a Suite failed. Which decision belongs to a human and what evidence should inform it?

Sources

1. AQEF 1.0-CR1, Front Matter §8 — Terminology (Quality Contract, Oracle, Validator, Judge, Confidence).
2. AQEF 1.0-CR1, Volume VII — Quality Contracts (Contract Language, Constraints, Expectations, Confidence, Policies, Contract Composition and Reusable Contracts).
3. AQEF 1.0-CR1, Volume I, Chapters 3 and 6 — Contracts over Assertions; aggregation and Quality Gates.
4. AQEF 1.0-CR1, Volume V — Validator Architecture and Validator Composition.
5. AQEF 1.0-CR1, Volume VI — Judge Architecture and Confidence.
6. AQEF 1.0-CR1, Volume XII — Governance, Principals, Permissions, Approval Workflow and Traceability.
7. AQEF 1.0-CR1, Appendix A — Contract and Clause concepts.

Chapter 7 — Data and Evidence

Maturity: Core (Static Datasets and Versioning); At Risk (Generated and Production Replay); Reserved (Synthetic Datasets and Dataset Mechanisms) · Textbook First Edition v1.0

Learning Objectives

By the end of this chapter, the reader will be able to:

1. Distinguish Dataset, Scenario, Variables, Execution, Conversation, Artifact and Result without conflating data inputs with assessment evidence.
2. Explain why Dataset versions must be pinned and why a Dataset version differs from a release Snapshot.
3. Describe the Execution Pipeline's treatment of Variables, Context, Prompt Construction and Artifact Collection.
4. Recognise the status and limits of Generated and Production Replay Datasets and reserved Dataset mechanisms in AQEF 1.0-CR1.
5. Trace a disputed Aster Support AI outcome back to its input data, original Evidence and applicable Contract without overstating what the record proves.

7.1 Why test data is part of the quality argument

An AI test does not begin when a prompt reaches a model. It begins earlier, when an engineer decides which situations deserve to be represented, how to express them without distorting their meaning, and which evidence will make the resulting assessment defensible. The data used to instantiate a Scenario is therefore not a neutral ingredient. It carries decisions about which customers, policies, failures and ambiguities are visible to the test system. A carefully constructed Oracle cannot compensate for an input population that systematically omits the situations in which the subject tends to fail.

Consider Aster Support AI, the customer-support assistant introduced earlier in this book. A test set containing only straightforward questions about published return periods may produce persuasive success rates. Yet real support work includes purchases outside the standard period, conflicting customer statements, inaccessible order details, missing policy passages and attempts to make the agent take actions it has not been authorised to take. If such cases are absent from the data, the results describe the narrow test set accurately while leaving the project's important risks largely unexamined. A high percentage is not an answer to a missing population.

AQEF separates the Scenario, which defines a testable situation, from the Dataset, which can supply Variables that instantiate that situation, and the Execution, which runs it in one Environment at a given time. That separation matters because a single carefully described situation may be exercised with multiple variable sets. It also matters because the same Scenario may be compared across product versions without quietly changing the test data at the same time. The architecture makes data choices visible rather than burying them inside a prompt assembled ad hoc.

Data quality in this sense is not synonymous with realism. A deliberately artificial case may be exactly what is needed to test a rare boundary. A realistic production example may be unusable because the project lacks permission to retain its contents. A broad sample may still be biased, while a small fixed collection can be highly informative for specific contract clauses. The engineer's task is to explain the relationship between the data, the intended risk and the limits of any generalisation drawn from the resulting evidence.

7.2 The three objects engineers must not collapse

Figure 7.1 | From defined data to assessable evidence



Educational schematic — consult AQEF Volumes II–IV for normative relationships

A **Dataset** supplies reusable rows of inputs. A **Scenario** specifies the situation, its intent and the Quality Contract that governs assessment. An **Execution** is the particular runtime event that realises the Scenario, producing a Conversation and zero or more Artifacts. These entities answer different questions: which values were available, what was to be examined, and what actually occurred. A Dataset row does not become a test result just because it contains a plausible question; assessment requires an Execution and the relevant Oracles.

The distinction becomes concrete when Aster receives a variable named purchase_date. One Dataset row represents a recent purchase, another an old purchase and a third a purchase date that conflicts with the user's text. The Scenario may consistently ask Aster to explain return eligibility using the policy available in its Environment. Each Execution resolves one row, constructs a Prompt and receives behaviour from the subject. The resulting Conversations may legitimately differ even though the method used to construct the literal prompts is deterministic.

Treating the Scenario and its data as the same object makes maintenance difficult. Changing a policy clause might require updating the Contract; discovering an untested customer boundary might require adding a Dataset row; diagnosing a retrieval failure might require preserving the Artifact containing the retrieved

passage. Those changes operate at different levels, and records should make clear which level changed. Without such separation, an apparently stable test name can conceal a different case on every run.

The central lifecycle is therefore not simply ‘data in, answer out.’ It is a traceable transition from chosen and versioned input to an executed interaction, captured evidence and a qualified Result. Not every item in that chain is interchangeable. In particular, a Result is not raw Evidence: it is a determination made by an Oracle under a Quality Contract, and it may be unavailable, inconclusive or otherwise not actionable.

7.3 Static Datasets: the ordinary core

A **Static Dataset** is a fixed collection whose rows are authored or otherwise frozen at a known point. AQEF treats Static Datasets and Dataset Versioning as core. Their value is not that they cover every possible input, but that their contents remain intelligible and reproducible. When a Scenario uses a fixed row, an engineer should be able to identify the row's meaning and why it was included. This provides a stable starting point for understanding the probabilistic behaviour of the AI subject.

Static does not mean trivial or uniform. A curated collection for Aster may include conflicting policy excerpts, non-English wording, misleading customer assumptions and ambiguous order states. The rows remain static because the collection changes only through explicit edits, not because the questions are easy. Strong static data is often deliberately heterogeneous: engineers choose representative ordinary cases alongside risk-driven exceptional cases, then state what the collection is intended to support.

A static collection is especially helpful when a model is replaced, a prompt is revised or a retrieval index changes. With the same Scenario and Dataset version, a difference in results can be investigated against a reasonably stable input basis. Without pinned data, a team may attribute a change to the new model even though a Dataset row was silently corrected or replaced. Deterministic Infrastructure cannot rescue an experiment whose supposedly fixed inputs are not actually fixed.

Still, repeatability of inputs is not proof of completeness. A static suite may go stale as policy, user behaviour and product boundaries evolve. The proper response is not to secretly mutate an old version, but to author a new version and document how its coverage differs. It may also be appropriate to compare the old and new collections side by side so that a change in measured quality is not misreported as a pure model effect.

7.4 Dataset versioning and the identity of evidence

Versioning is the foundation of credible historical comparison. AQEF requires every Dataset, regardless of source, to have a version. A Scenario references a Dataset using its name and version, rather than a mutable name alone. This enables an investigator to reconstruct which variable values a particular Execution was meant to resolve. A label such as ‘support_cases’ without its edition would be too weak: the contents might have changed between the test and the later release review.

“Every Dataset, regardless of source, MUST be versioned” — AQEF 1.0-CR1, Volume IV, Versioning.

A data version and a release Snapshot are not synonyms. The Dataset version identifies the contents of that particular collection. A Snapshot freezes a wider release-oriented state that can include configuration and

multiple Baselines. Confusing the two obscures where responsibility for change resides. One Dataset may be revised several times while release decisions refer to different Snapshots; a Snapshot may refer to several Dataset versions that were each independently maintained.

Versioning is meaningful only when the version is a stable identity for the relevant contents. A team should be able to explain what changed between versions: was a policy date corrected, a new risk scenario added, an accidental duplicate removed or a source document withdrawn? These are not equivalent edits. A correction to an invalid input improves the test population but may reduce comparability with previous Results; adding a difficult boundary changes the composition of coverage and can legitimately change aggregate outcomes without product regression.

For Aster, imagine a release investigation showing lower faithfulness scores. If the current Dataset version adds several previously absent ambiguous-refund cases, a direct comparison of raw aggregate scores with the earlier version would mix product change and test-design change. An engineer should report the composition difference, use matching subsets where meaningful and explain that coverage grew. The honest comparison preserves the evidence chain; it does not insist that every statistic from two different populations is directly comparable.

7.5 Variables and deterministic prompt construction

Variables are named parameters supplied from Dataset rows or fixed on a Scenario. AQEF's Execution Pipeline resolves them and constructs the Prompt. The substitution mechanism belongs to Deterministic Infrastructure: identical templates and identical resolved variable values must produce the same literal prompt text. What the model does after receiving that text is part of the probabilistic subject. This is an important boundary because it prevents test-harness variation from being mistaken for model behaviour.

Suppose a Scenario says that Aster should explain return eligibility for an order with a specified purchase date, region and product category. A row supplies those facts; Prompt Construction makes the literal message. If punctuation, omitted values or locale conversions unpredictably change between runs, the test harness has inserted noise. A different answer might be reasonable in light of a different literal prompt, but that would not be evidence about variation under identical input. The harness must not create uncontrolled alternative cases.

Context is not the same as Variables. Variables are selected inputs to the Execution; Context accumulates within it, including earlier turns, tool outputs and retrieved passages. In an agentic conversation, the next Prompt can depend on the subject's earlier Response. The Conversation Engine's procedure for building that next Prompt must still be deterministic given the preceding Context and Responses. In short, the subject may choose a different path; the harness may not improvise an unrecorded one.

Variable resolution also requires editorial attention to meaning. A test value can be syntactically valid but inconsistent with the Scenario's intent. Inserting a customer who is legally ineligible for a return into a case whose Contract assumes eligibility is a test-design contradiction, not useful negative coverage. Better design names the intended boundary and adjusts the Contract to assess it. The technical mechanism must be accompanied by a coherent description of what that data instance is supposed to test.

7.6 From a run to Evidence

AQEF uses **Evidence** for the Conversation and relevant Artifacts an Oracle examines. The Conversation preserves the ordered Prompt/Response record of what the subject did. Artifacts preserve associated material: retrieval passages, tool-call traces, intermediate agent state and operational measurements are typical examples. A Result follows from assessment of that evidence, not directly from a team's memory of what seemed to happen.

Evidence collection is part of execution, not a cosmetic logging step after the model has finished. Volume III specifies that Artifact Collection runs alongside the Conversation Engine because some materials exist only transiently during a turn. If Aster retrieved a policy document and then wrote an unsupported statement, a later investigator needs to know what the retriever actually returned, not simply which document currently resides in the knowledge base. Reconstructing the retrieved passage afterward could substitute changed information for the evidence that the subject received.

The same applies to agents. A final response that says ‘I have completed your refund’ may be impossible to assess reliably without a trace of whether any tool was called, with which arguments and what outcome. Conversely, a well-formed tool trace does not prove that the customer-facing explanation was faithful. Different Quality Contract clauses can require different portions of the evidence, and those portions should be captured in time to support independent assessment.

An absence of evidence must never be silently converted to a passed or failed product verdict. If a necessary Artifact was not captured, an Oracle may be unable to evaluate its clause. That is different from observing the subject violate the clause. The resulting reporting must preserve this distinction so that a missing measurement channel cannot be used either to accuse the product unfairly or to create an undeserved green assessment.

7.7 Evidence provenance, custody and interpretability

Provenance asks where evidence came from and through which transformations it reached the assessor. A test may begin with a Dataset row, pass through Prompt Construction, retrieve documents, call tools and end with a Result. At each stage the relevant record should allow an investigator to reconstruct what was known and when. Provenance is therefore a way of making an explanation falsifiable: someone else can challenge the assessment by inspecting the material on which it rested.

Consider two different Aster failures with identical final text. In one, the retriever supplied the wrong return policy. In the other, it supplied the correct policy and the generator invented an exception. Without the retrieved passage as an Artifact, the failures look alike. With it, the team can distinguish a retrieval defect from a generation defect and decide whether the next action concerns indexing, prompt design, policy content or evaluation. The objective is not to assign blame to a component before assessment; it is to preserve the evidence needed to separate hypotheses.

Provenance also matters when a Dataset comes from outside the project. A row copied from a public example, a customer conversation or an internal knowledge-base test needs sufficient context to understand its status. The textbook recommends recording origin, purpose, known restrictions and the reason the row is considered suitable. That is editorial guidance rather than a new AQEF schema requirement; the actual normative obligations remain those in the cited volumes and appendices.

Integrity does not require a unique database topology. A small self-hosted engine and a distributed service may preserve equivalent evidence if each respects the applicable requirements. What matters is that the execution record cannot silently resolve to a different Dataset version, different Contract or different past Conversation simply because the storage system was refactored. Trust depends on preserved relationships, not on a prestigious infrastructure diagram.

7.8 Generated Datasets — specified, but At Risk in CR1

A **Generated Dataset** uses an AI model to create test rows from a seed set, specification or existing Dataset. It can discover phrasing and combinations that a person did not enumerate. Yet generation also creates a distinctive epistemic problem: the model producing the tests may share weaknesses with the model being tested. If both avoid the same inconvenient cases, a large generated collection can create an illusion of diversity without expanding coverage of the risks that matter.

Volume IV recommends drawing generation from an independent source, using SHOULD-level language rather than promoting that recommendation to the MUST that specifically governs independent Oracles. A generator is not itself an Oracle. This distinction is valuable because good intentions often turn analogies into invented requirements. Independent generation can mitigate correlated blind spots, but the exact requirement depends on the role the component plays in AQEF.

The specification also defines a strong all-or-nothing rule for generated output. A generation yielding malformed rows must not be silently cleaned into a valid Dataset version. A rejected generation yields no new version. This avoids a subtler source of evidence corruption: if the engine drops inconvenient or malformed generated rows without disclosure, it has changed the sample while still presenting the result as the generator's output. The composition of the test population should not be reconstructed through hidden repair.

However, **AQEF 1.0-CR1 lists `dataset:generated` as At Risk**, with no implementation reported at Candidate Recommendation entry. Teaching the intended mechanics does not imply that CR1 has proven interoperability or production readiness for the capability. An implementation claiming it would need to satisfy its applicable requirements; the textbook's hypothetical examples are not conformance evidence.

For Aster, a generated collection might attempt to invent customer questions that refer to overlapping refund rules and ambiguous delivery times. The promising educational lesson is to assess both the input population and the resulting subject behaviour. The question is not merely whether the generated cases sound realistic; it is whether their origin, validity, intended risk and eventual version are sufficiently clear to make downstream evidence meaningful.

7.9 Production Replay — a powerful principle with a qualified status

Production Replay begins with actual observed Conversations rather than a pre-authored Scenario collection. Under AQEF's ‘Everything is a Test’ principle, an interaction can become a test when a relevant Quality Contract is applied to it. A captured production case often contains complexity that design sessions miss: a customer changes the subject, an unfamiliar phrasing collides with a policy, or a tool fails at an inconvenient point. Such evidence can lead to new Scenarios and improve the team's understanding of real risk.

The idea must be handled with temporal precision. A production Conversation was produced under the original service's circumstances; replaying or retroactively assessing it later is not automatically equivalent to recreating that environment. Model versions, retrieved documents, customer permissions and external systems may have changed. The responsible question is what exactly is being preserved: the original Conversation as fixed Evidence, a reconstructed Scenario with controlled inputs, or a new live Execution inspired by the observation. Different choices answer different testing questions.

Privacy and data-handling permissions are not solved by calling a production interaction test data. Volume IV requires captured production material to be handled under the project's applicable policies and explicitly does not define those policies itself. A customer record might be useful for diagnosing Aster, but usefulness does not authorise copying it into a long-lived test fixture. Before reuse, the project must determine what may lawfully and responsibly be retained, redacted or excluded.

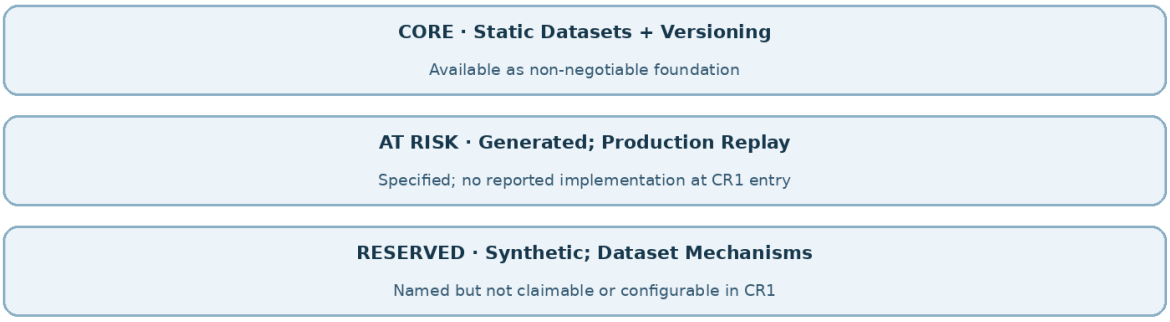
In CR1, **`dataset:production_replay` is At Risk**. The principle and its intended mechanics are present in the specification, but the CR1 status statement reports no implementation yet. It would be inaccurate to describe this as a widely implemented core feature of AQEF 1.0. That qualification does not reduce the educational value of the idea; it tells the reader which assertions are architecture and which are evidence of delivered capability.

Reader caution: At Risk dataset sources must not be assumed to be implemented or available in every AQEF 1.0-CR1 engine. Reserved dataset mechanisms are not available capabilities. A testing plan that depends on either needs explicit implementation evidence or a different supported source of test data.

Aster's support logs might reveal that users frequently ask about returns after changing countries during an order. Such a pattern could motivate a new static Scenario without retaining personal transcript data. The resulting test is based on a production observation but does not pretend to reproduce the original private conversation. Responsible test design distinguishes learning from data and reproducing data verbatim.

7.10 Reserved sources and mechanisms are not usable configurations

Figure 7.2 | Capability status in AQEF 1.0-CR1



Edition-specific map. Test-design techniques are not dataset capability claims.

AQEF names **Synthetic Datasets** and four mechanisms—Data Mutation, Fuzzing, Edge Cases and Adversarial—but reserves them in 1.0-CR1. The descriptions explain why a future implementation might want mechanical expansion or transformations of rows, while the configuration semantics are intentionally not available for a CR1 conformance claim. This is a stronger restriction than At Risk. It does not merely say that evidence is thin; it says that the capability cannot be claimed in this edition.

"No implementation can claim them, and every implementation rejects a configuration that uses them" — AQEF 1.0-CR1, Front Matter §0, Reserved.

Readers may already know these techniques from conventional testing. That familiarity does not alter their status as AQEF Dataset mechanisms. One can manually author a static boundary case and evaluate it under the core, without claiming support for the reserved *dataset_mechanism:edge_cases* configuration. The **test-design technique** of boundary analysis is not identical to the **Dataset mechanism** reserved in Volume IV. This is the same distinction Chapters 4 and 5 made between test types and scoped implementation capabilities.

The same reasoning applies to adversarial inputs. A team may include a deliberately hostile Prompt in a static Dataset and assess it under a Safety Contract. This does not mean the engine implements the reserved Adversarial Dataset mechanism. The content of a row and the mechanism by which AQEF generates or transforms rows are different matters. Confusing them would either falsely restrict ordinary testing or falsely advertise an unsupported capability.

Readers should therefore treat the descriptive sections on reserved mechanisms as design orientation, not instructions for a CR1 implementation. When a later edition specifies them, its configuration rules and CTS will determine what a conformant claim actually entails. Until then, a textbook that presents those mechanisms as available knobs would be inventing operational functionality the source explicitly withholds.

7.11 Dataset selection as a risk and coverage decision

Dataset selection should begin with risk, not with the number of rows. Chapter 2 distinguished quality dimensions and business importance; Chapter 5 showed how risk becomes Scenarios and Contracts. This chapter adds that data selection determines which instantiations of those Scenarios are actually exercised. A large Dataset repeating easy cases can provide narrower risk coverage than a modest collection deliberately spanning critical boundaries.

For Aster, a useful design conversation might name several risks: incorrect eligibility conclusions, unsupported policy claims, wrong tool arguments, exposure of sensitive customer details, misleading confirmation of an action and unstable behaviour across repeated runs. Different rows target those risks in different ways. A date close to the policy threshold probes eligibility; a conflicting retrieved passage probes faithfulness; an unavailable tool probes the agent's blocked path. Their value lies in the intended evidence, not just their distinct strings.

Risk-based selection also demands clarity about what is *not* covered. If a static Dataset contains only English prompts and simple single-turn conversations, a report should not imply that multilingual or multi-turn behaviour has been adequately assessed. A well-defined coverage gap is information. It can support a cautious Release Decision or a plan for additional Scenarios. A hidden gap, by contrast, allows a polished pass rate to stand in for a much broader claim than the evidence permits.

AI test data also creates a tension between comparability and novelty. Fixed rows make regressions easier to investigate, while unfamiliar rows help reveal generalisation failures. AQEF's model accommodates this tension by separating Dataset identity, versions, Scenario definitions and Results. An engineer can compare a stable baseline collection while also tracking a deliberately expanded version. There is no need to sacrifice one objective to pursue the other, provided the two populations are not merged into a statistic that conceals their differences.

7.12 Evidence gaps, Oracle limits and honest conclusions

A final distinction is necessary: having enough data to run a Scenario is not the same as having enough evidence to decide whether its Contract was satisfied. A RAG response may be captured while the retrieval trace is missing. A tool call may be recorded without the tool outcome. A Judge may be unavailable after a completed Execution. Each condition affects what can responsibly be concluded, but not all are product failures.

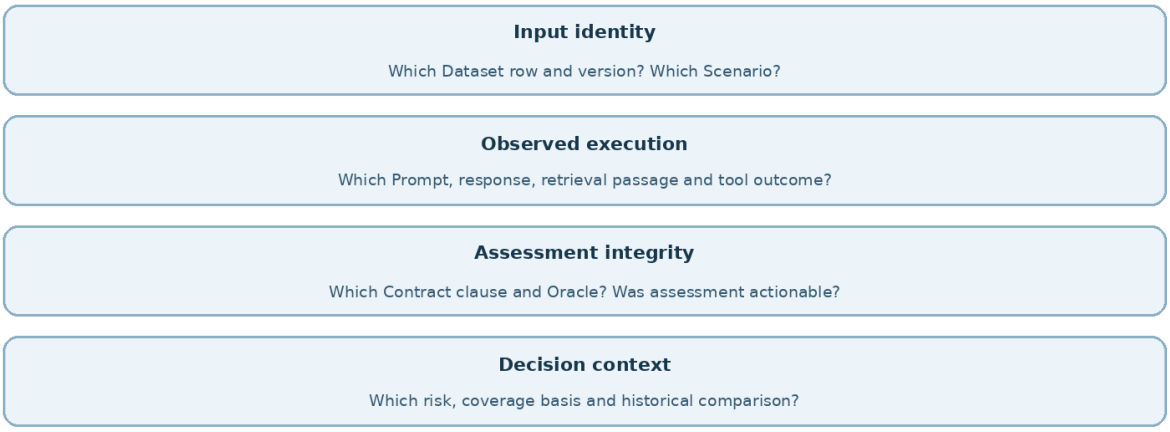
In Aster, suppose the final answer correctly quotes a refund rule while the policy passage returned by retrieval was not retained. The team may be able to check the statement against the current policy, but it cannot prove from the original Execution what evidence the model saw. If the test specifically measures grounding in retrieved context, the absence of that Artifact is central. A passing final answer is not interchangeable with evidence of a faithful retrieval-grounded process.

The reliability of the Oracle is another independent dimension of evidence quality. A confident-sounding Judge may be mistaken. An unavailable validator does not demonstrate that Aster broke a deterministic rule. A missed Seeded Control, if that capability is used, can invalidate otherwise plausible outcomes for an affected defect class. These are reasons to keep the assessment record and its provenance distinct from the underlying Conversation.

The broader engineering principle is to preserve uncertainty at the level where it arises. An unknown Dataset origin should not become assumed coverage. An absent Artifact should not become a clean bill of health. An unavailable Oracle should not become a subject failure. The strength of AQEF is not that it abolishes ambiguity, but that its architecture provides places to record what happened, what was checked, and where the evidentiary chain remains incomplete.

7.13 Aster Support AI: tracing one disputed refund response

Figure 7.3 | Investigating Aster’s refund answer



Four distinct questions — the final answer alone does not resolve them.

Imagine a controlled test in which Aster tells a customer that a return is permitted. The customer bought a product near the end of the published return period. The first investigative question is not whether the answer sounds helpful, but which Scenario and Dataset row produced the situation. Were the purchase date and region fixed inputs? Was the Prompt template identical to the one used last week? Was the policy version used in the test the intended one?

Next, examine the Execution and Evidence. The Conversation shows the customer's question and Aster's response. Where captured, the Artifacts document which policy passage was retrieved, whether order-lookup tools were invoked, and what latency or errors were recorded. Absent or incomplete Artifacts cannot be treated as proof that the corresponding event did not occur. If the right policy passage was present but Aster drew the wrong conclusion, the defect may lie in the generation or reasoning path. If the passage itself was wrong, the retrieval or source-data path deserves attention. Both could contribute; preserving evidence allows the team to test that hypothesis.

The third question concerns the assessment instrument. Which Validator or Judge applied which Contract clause? Did the result carry its required disposition and Confidence status? Was the Oracle available, and is the Result actionable? The same factual conclusion about Aster can be supported strongly, weakly or not at all depending on the evidence and the instrument's state. A team must resist the temptation to infer a reliable finding merely because the final output looks obvious.

Finally, the project connects findings to decision. Does the issue affect a high-risk Scenario? Is the evidence base sufficient to interpret aggregate performance? Did the Dataset change relative to the Baseline? The chain from fixed data to Result allows an accountable explanation of why a Gate passed, failed or lacked sufficient basis. It does not demand that every answer be identical. It demands that the grounds for judgment be reconstructable.

Engineering Reflection

Aster passes a Scenario that asks it to explain a return rule, but the recorded retrieval passage is absent. What evidence would you need before calling this a grounded answer? Which parts of the record can still be assessed and which claims must remain qualified?

A manager requests that a Generated Dataset be “cleaned” by silently removing malformed rows before a version is registered. Which AQEF requirement is implicated? How could the failure be reported without corrupting the identity of the generated output?

Common Misconceptions

A Dataset row is already a test. It may instantiate a Scenario, but the test lifecycle still requires execution, Evidence and assessment under a Contract.

More rows automatically mean better coverage. Repetition of low-risk cases can conceal important gaps; coverage must be interpreted against the risks and purposes of the Suite.

A Dataset version and a Snapshot mean the same thing. A Dataset version identifies one collection's content; a Snapshot captures a wider release-oriented state.

Production Replay is already core in CR1. It is specified but At Risk. Its conceptual importance does not make it an implemented core capability.

Reserved dataset mechanisms prevent ordinary boundary or adversarial testing. They restrict claims and configurations for the reserved mechanisms, not the manual authoring of meaningful static cases.

Chapter Summary

Dataset design determines which risks become visible to assessment; an accurate Oracle cannot diagnose cases never supplied.

The Dataset, Scenario, Execution, Conversation and Result are distinct entities connected by a traceable lifecycle.

Static Datasets and Versioning are part of the AQEF core; Dataset identity must remain pinned for historical comparison.

Variables are selected inputs, Context accumulates during execution, and Prompt Construction belongs to deterministic infrastructure.

Evidence includes the Conversation and Artifacts captured while the subject runs, not reconstructed after the fact.

Generated and Production Replay sources are At Risk in CR1, while Synthetic Datasets and Dataset Mechanisms are Reserved.

A dataset mechanism's status must not be confused with the freedom to manually create static cases using familiar test-design techniques.

Claims about an AI system must be no stronger than the preserved input, execution and assessment evidence allows.

Review Questions

1. A Dataset receives fifteen new cases between two runs of the same Suite. Why might a lower overall pass rate not prove product regression? How would you separate the effects?
2. Explain why a Scenario that references only a mutable Dataset name undermines the value of a Baseline.
3. For a RAG answer, how do a Conversation and a retrieved-document Artifact support different conclusions?
4. Aster's Prompt changes slightly between two Executions with otherwise identical declared inputs. Under what conditions would this be a harness defect rather than subject variation?
5. Why is an AI-generated test population vulnerable to correlated blind spots, and how does the applicable SHOULD differ from the Independent Oracles MUST?
6. Explain why malformed Generated Dataset output must not silently be repaired into a new version.
7. A project wants to test prompt injection using hand-authored Static Dataset rows. Why does the Reserved status of the Adversarial Dataset mechanism not automatically prohibit this?
8. What is the difference between a production interaction inspiring a new static Scenario and a conformant Production Replay Dataset capability?
9. When Aster's response is plausible but a required Artifact is absent, which product claims remain unsupported?
10. Describe the complete evidence trail needed to investigate an incorrect return-eligibility decision while distinguishing input data, subject behaviour and Oracle failure.

Sources

1. AQEF 1.0-CR1, Front Matter §0 — Document Status, At Risk and Reserved.

2. AQEF 1.0-CR1, Front Matter §8 — Terminology.

3. AQEF 1.0-CR1, Front Matter §10 — Conformance and scoped capability claims.

4. AQEF 1.0-CR1, Volume II — Domain Model and relationships.

5. AQEF 1.0-CR1, Volume III — Execution Pipeline, Variables, Context and Artifact Collection.

6. AQEF 1.0-CR1, Volume IV — Dataset Engine, sources, mechanisms and Versioning.

7. AQEF 1.0-CR1, Volume VIII — Test Design and relationship to Dataset techniques.

8. AQEF 1.0-CR1, Appendix A §A.4 and §A.8 — Scenario Dataset reference and reserved mechanism restrictions.

Chapter 8 — Coverage and Traceability

Maturity: Core · AQEF 1.0-CR1 · Textbook First Edition v1.0

Learning Objectives

By the end of this chapter, readers should be able to:

1. Explain why counting tests is not equivalent to covering the risks that matter.
2. Evaluate coverage across risks, defect classes, data variants, Oracles and Quality Gates without inventing a mandated numeric target.
3. Distinguish Scenario breadth, repeated Executions and actionable Results when interpreting Aggregate Basis.
4. Build an explanatory risk-to-evidence map for Aster Support AI and expose its missing links.
5. Explain AQEF Traceability as reconstruction of the authorisation and evidence chain behind a particular Release Decision.

8.1 Coverage as an argument, not a percentage

Coverage is often presented as a number: a fraction of requirements tested, a fraction of cases passed, or the percentage of a suite that has executed. Such counts can be useful, but they are not the complete argument about whether an AI system has been tested adequately. For a probabilistic subject, the interesting question is whether the evidence represents the behaviours and failure mechanisms that matter to the people relying on the system. A thousand near-identical prompts may be less informative about a high-stakes failure than a carefully selected boundary case. The distinction is between **volume of activity** and **coverage of a risk**.

AQEF frames test design in the context of Risk Analysis. Volume VIII describes Risk-Based Testing as the way a Suite's composition acts on priorities already identified by Risk Analysis. More scrutiny can be concentrated on high-stakes behaviours: additional Scenarios, tighter thresholds and Human Reviewer requirements where appropriate. Those are design choices made in response to risk, not a universal algorithm assigning a fixed number of tests to every requirement. An engineer therefore needs a defensible explanation of why particular tests exist, not merely evidence that they ran.

For Aster Support AI, routine questions about store opening hours and requests to initiate a refund do not carry the same consequence. The latter can create an irreversible or customer-visible business action. A risk-oriented coverage model asks whether the team's Scenarios examine authority, eligibility, tool arguments, blocked paths and false claims of completion. Running dozens of paraphrases of the store-hours question would not fill those gaps. Coverage is judged against the declared risk landscape, including what has intentionally been left outside it.

The word coverage also has at least two levels. At design time, it describes whether Scenarios and Contract clauses address an identified concern. At assessment time, it asks whether Executions generated usable Evidence and actionable Results for that concern. A suite can have excellent planned coverage and poor

realised evidence because a tool was unavailable, a required Artifact was not collected or a Judge could not assess a clause. These are different deficiencies and should lead to different corrective actions.

8.2 The five lenses of a coverage review



Figure 8.1 — A risk-to-evidence map (pedagogical, not a prescribed AQEF schema).

A useful educational model reviews coverage through five questions: **risk**, **defect class**, **data**, **Oracle** and **decision Gate**. This is an explanatory checklist drawn from AQEF's relationships, not a new normative five-part coverage formula. The purpose is to prevent the review from stopping at the first apparently reassuring signal. A Scenario may target the right risk but use unrepresentative data. It may have suitable data but no Oracle capable of evaluating the critical property. It may yield a Result that is never consumed by the relevant release Gate.

Risk coverage begins with what could go wrong and why it matters. Defect-class coverage makes the concern sufficiently precise to understand what would count as a failure: an unsupported factual claim, an unauthorised action, a malformed tool argument, a missing citation or a failure to preserve conversational context. Data coverage examines which variants of the intended situation have actually been instantiated. Oracle coverage asks what mechanism will distinguish acceptable from unacceptable behaviour. Gate coverage asks how the assessed findings influence a consequential decision.

The chain is useful because each missing link has a different owner and remedy. If a risk has no Scenario, the test design must change. If the Scenario exists but lacks an appropriate Contract clause, the intended check is not specified. If the clause requires a retrieved-document Artifact that is not captured, the execution evidence is incomplete. If a Judge's result is non-actionable, counting the nominal check as passed would conceal an assessment gap. And if the project's release report omits the finding, governance cannot make a decision informed by it.

Coverage should thus be described in language that admits qualifications. An engineer can say, “The critical refund-eligibility risks are represented by named Scenarios, but we lack a usable assessment of multilingual policy interpretation.” This is more honest and more actionable than saying the suite has achieved an apparently precise percentage of coverage whose denominator no one has defined. The objective is not to eliminate numbers; it is to make their meaning inspectable.

8.3 Requirements, risks and Scenarios must remain distinguishable

An engineering requirement describes an expected property or permitted behaviour. Risk Analysis asks what could happen if that property does not hold, how serious the consequence would be and where scrutiny is justified. A Scenario represents one testable situation selected to examine that concern. These three

concepts are related, but they are not interchangeable. One requirement can motivate several risks; one risk can require multiple Scenarios; and one Scenario may illuminate more than one concern. A sound trace therefore permits many-to-many relationships without pretending every item has one neat parent.

Take Aster's requirement to apply the correct return policy. It raises at least two different risks. Aster could state the wrong eligibility date, or it could call a refund tool with arguments inconsistent with the retrieved order. One Scenario might address a policy boundary and another might address the action trace. The same Contract fragment may provide a general requirement to avoid unsupported claims, but different cases will need different Evidence to make the check meaningful. Grouping all of this under a label such as “refund tested” discards precisely the distinctions engineering review needs.

The Scenario itself is a definition, not proof that a test ran. Volume II keeps Scenario, Execution and Conversation separate for this reason. Coverage reports that count scenarios defined should not be silently interpreted as execution coverage. Similarly, counting a performed Execution does not establish assessment coverage: an Oracle could be unavailable or its evidence incomplete. Reviewers should ask which stage the count refers to before treating it as support for a decision.

This separation also improves maintenance. A new business policy may create a new risk without invalidating every earlier Scenario. A change in test data might expand cases for an existing Scenario. A change in an Oracle can alter the credibility of Results without changing either the requirement or the input. Maintaining separate identities allows each change to be examined for its actual consequences rather than triggering indiscriminate edits or unearned conclusions.

8.4 Coverage by defect class and the limits of a broad label

A defect class is a more precise expression of what an assessment must detect. “Semantic quality” is a useful dimension, but it is too broad to demonstrate that the team can discover every relevant semantic failure. Aster could be factually wrong about a return period, faithful to the wrong retrieved source, misleading about an order status or inconsistent across turns. Each failure requires distinct evidence and sometimes a different Oracle approach. A suite described only as “semantic testing” may contain little evidence for any particular class.

AQEF's Seeded Controls provide a particularly strong reminder of this distinction when that scoped capability is claimed. They can test whether an Oracle still detects a planted defect of a known class, or refrains from falsely reporting a defect in clean Evidence. A missed control affects confidence in the instrument's Results for the affected class. It does not, by itself, prove the AI subject failed. In other words, coverage of the subject and coverage of the measurement instrument must not be merged into one success metric.

The educational conclusion does not require Seeded Controls in every project; their capability remains scoped. Even without them, engineers should articulate which classes a Judge or Validator is meant to recognise and which are outside its competence. A formatting Validator provides deterministic evidence about a schema. It cannot establish that a nuanced policy explanation is faithful. An LLM Judge may assess a semantic clause, but its numerical Confidence is not evidence that the reference cases fully represent all future inputs.

A carefully phrased coverage claim might therefore specify that the suite exercises conflicting policy statements, malformed tool parameters and a known failure of retrieval grounding, while leaving out cross-language equivalence and multi-agent delegation. This statement gives decision-makers a map of exposure and its limits. A broad label without the underlying classes does not.

8.5 Data coverage: variety must serve a purpose

Data determines which instantiations of the risk are observed. The dataset might include multiple languages, regions, policy ages and customer statements, but diversity alone is not an engineering objective. A variant is useful insofar as it probes a distinct assumption or increases confidence about the system's behaviour under relevant variation. A hundred syntactic paraphrases of the same straightforward case may provide repeated reliability information. They do not necessarily add breadth across eligibility boundaries or tool permissions.

For Aster, consider three rows for a return-eligibility Scenario: a purchase comfortably within the period, one exactly at the policy boundary and one beyond the period with an asserted exception. Those rows ask different questions even if their surface wording is similar. By contrast, three phrasings of the within-period case may be appropriate for Consistency or Metamorphic Testing, but their evidentiary contribution should be described differently. The categories can complement one another; neither substitutes for the other.

Chapter 7 established why dataset versions must be pinned. A coverage assessment performed on one version cannot be transferred uncritically to another if the new version changes the population of cases. This affects both interpretation and reporting. An aggregate may change because the product changed, the dataset changed, or both. The causal claim has to be supported by a controlled comparison, not by the mere fact that two percentages differ.

The CR1 maturity boundary is also relevant. Static Datasets and versioning are core. Generated and Production Replay dataset sources are At Risk; synthetic sources and dataset mechanisms are Reserved. Test designers may still author representative static rows manually. They should not, however, describe the reserved mechanisms as implemented AQEF facilities. The design objective—probing relevant variation—is stable even when the available mechanism for obtaining that variation is edition-specific.

8.6 Oracle coverage and actionable assessment

A test can be executed without producing a decision-ready finding. An Execution yields a Conversation and relevant Artifacts; an Oracle applies a Quality Contract clause and produces a Result. If the appropriate Oracle is unreachable, cannot access necessary evidence or is unqualified for the claim being made, the planned assessment may not be actionable. This is not equivalent to the subject passing or failing. Coverage claims must distinguish successful collection of inputs and outputs from successful assessment of the intended property.

The distinction between Validators and Judges matters here. A Validator can check a deterministic constraint such as required tool-argument structure. It must report Confidence as inapplicable rather than pretend to issue a probabilistic certainty score. A Judge evaluates a semantic criterion and carries Confidence about its verdict. Its presence is not sufficient merely because it produced a number: the criteria, independence, applicable admission and other conditions affect whether its result can meaningfully support the intended Gate.

An Oracle-coverage review asks whether every consequential clause has an appropriate assessor and whether its actual evidence is available. If Aster asserts that it completed a refund, an assessment of tool-action correctness requires an authentic trace of the tool outcome. A fluent final answer does not replace it. Where the tool trace is absent, an engineer should describe the evidence gap rather than silently rely on a response-only semantic score.

An additional issue is that Oracles themselves may share systematic blind spots with the subject. Independent Oracles is a Fundamental Principle, not a convenient label for a model with a different prompt. Where the project uses more than one Judge, repeated samples of the same model and genuinely diverse Judges address different concerns. As Chapter 3 explained, a system can supply many nominal checks without substantially increasing independence. Counting Oracles is therefore no stronger as a proxy for coverage than counting prompts.

8.7 Gates, basis and the denominator problem

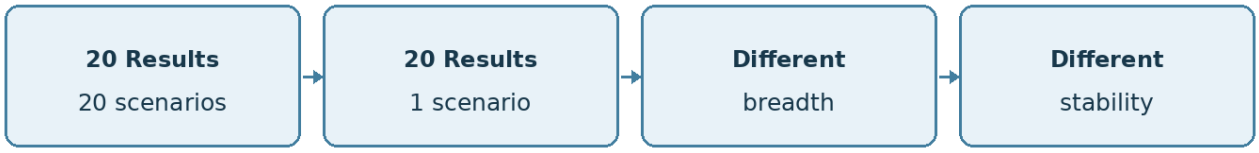


Figure 8.2 — Same Result count, different Scenario breadth and repeated-execution evidence.

Release decisions require evidence at a different level from individual Scenarios. AQEF's Aggregation Model combines Results for relevant quality dimensions, while Quality Gates compare aggregates against configured thresholds and minimum bases. The size and composition of the basis matter because an attractive average can rest on very little information. A pass rate over twenty distinct Scenarios and the same pass rate over one Scenario executed twenty times do not describe the same breadth of tested behaviour.

The specification requires every aggregate that a Quality Gate reads to declare its Aggregate Basis (AQEF 1.0-CR1, Volume I, Chapter 6, Aggregate Basis). The basis includes counts of actionable and non-actionable Results, plus distinct Scenarios and Executions. These counts give the reader a way to reason about breadth, repetition and missing assessments. They do not produce a universal sample-size formula. AQEF explicitly leaves how much basis is enough to the Project, subject to its stakes and design.

Repeated Executions of a single Scenario can expose instability. That is valuable evidence about Reliability, but it does not show that many different risk situations were examined. The reverse is equally important: a collection of distinct Scenarios each executed once may demonstrate breadth but tell us little about variability under repeated calls. Honest denominators prevent these two forms of evidence from being presented as equivalent simply because the number of Result records matches.

For Aster, suppose one hundred run records represent ten repeated attempts at each of ten low-risk FAQ Scenarios. A separate set of twelve records might represent distinct refund-action boundaries that involve high consequences. Neither number should automatically dominate the Gate's interpretation. Contract priorities, weighting and mandatory clauses shape the assessment; applicable Quality Gates separately

define thresholds and any minimum Aggregate Basis required for a release decision. AQEF provides the mechanism for making that argument explicit; it does not decree that the team must have exactly one hundred or one thousand test cases.

8.8 A practical coverage map for Aster

A useful review artifact is a coverage map with rows for the risks that actually matter and columns for the objects that support their assessment. This book uses the chain **Risk → Scenario → Contract clause → Evidence → Result → Gate → Owner** as an explanatory model. It is not a mandatory serialized AQEF object or a replacement for the specification's Domain Model. Its merit is that it exposes a missing link before a release meeting depends on an ambiguous status label.

One risk might be an incorrect refund eligibility decision. A Scenario fixes a purchase near the cutoff, and its Contract requires accurate application of the governing policy. The Evidence includes the customer request, the retrieved policy and any order-lookup trace. The relevant Judge or Validator generates a Result according to the clause it evaluates. A Gate reads the suitable aggregate, and a named business owner decides how to handle residual risk. Every arrow states something that can be questioned: does the clause really address that risk, and did the Execution preserve the Evidence needed to check it?

A second risk is claiming a refund was completed when no action occurred. Its Scenario exercises an acting agent, but the Contract should not be satisfied solely by a reassuring text response. The evidence must capture the tool invocation and the outcome that establishes what happened. If Action Boundary enforcement is part of the question, Chapter 14 will discuss its At-Risk status in CR1; the absence of an implemented boundary capability must not be concealed by the coverage map.

A third risk is privacy exposure. The correct test design may require a deterministic constraint on forbidden disclosure alongside semantic assessment of sensitive context. Data preparation and governance are central; the team must not create harmful exposure in the test itself. The map therefore distinguishes a missing safety control, an unavailable assessor and an unexecuted case, rather than compressing them into a single “not covered” label.

8.9 Traceability is more than a requirements spreadsheet

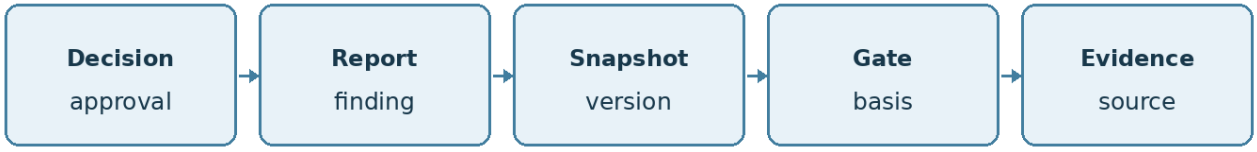


Figure 8.3 — The retrospective path from a decision to its sources and approvals.

A spreadsheet that connects requirements to test-case identifiers can be a useful design aid. But **Traceability** in AQEF has a broader, more demanding meaning. Volume XII defines it as the capability to reconstruct the full authorisation chain behind a specific artifact or decision. The relevant chain can lead from a Release Decision to its Report, Snapshot, Quality Gates, Baselines and the people who approved relevant actions. It is

about explaining an actual outcome after the fact, not merely documenting that someone intended to test a requirement.

Aster illustrates why the difference matters. Imagine a manager asking why a release was authorised although a semantic score was poor. The requirements spreadsheet might show that a faithfulness Scenario existed. It will not explain whether the Result was actionable, whether the Gate was advisory, whether a blocking Gate was overridden, who approved an override or which versioned Contract and Snapshot governed the evaluation. The traceability chain has to expose those links so the decision can be reconstructed.

Audit and Traceability are related but distinct. Audit asks broadly what actions occurred over a time range or for a group. Traceability asks how one particular artifact or decision acquired its recorded basis and authorisation. The same underlying records may support both. A complete event log is not sufficient if it cannot reconstruct which specific approvals and configurations were relevant to the decision under review.

The specification also protects this historical chain against destructive changes. A past decision should not become unexplainable because its referenced Dataset, Contract or Environment was deleted. AQEF requires a referenced entity to be preserved in a suitable frozen historical form or the destructive operation to be rejected. This turns Traceability from a pleasant reporting feature into a durability property of the implementing platform. The documentary chain must survive normal maintenance.

8.10 Gaps, limitations and the duty to report them

A coverage gap should be treated as information rather than an embarrassment. It can be an absent Scenario, an unrepresented defect class, a Dataset without a necessary boundary, an unavailable Oracle, an unexecuted test or a Gate that cannot use the available evidence. These gaps are not identical, and a good report names the relevant layer. Otherwise “coverage incomplete” becomes a vague diagnosis from which no one can decide the next action.

Some gaps are deliberate. An organisation may accept limited language coverage for an early release while ensuring strict control of financial actions. Such a choice requires a clear statement of the risk it leaves and an authorised decision, rather than retroactively redefining success to mean that the gaps never existed. Different teams may set different minimum bases or priorities without violating AQEF, provided the implementation respects its requirements and the project's choices remain visible.

Other gaps invalidate apparent reassurance. If a Gate has no actionable evidence, it must not pass merely because the aggregate is absent or the tool displays zero failures. An assessment blocked by missing Evidence or a failed Oracle is not a subject failure, but it is also not affirmative support for release. AQEF's decision machinery is designed to distinguish insufficiency of basis from a measured threshold failure. That distinction is especially important when management dashboards prefer green or red simplification.

A responsible report can therefore combine a favourable result with explicit limitations. It might say that refund-policy interpretation met the defined Contract for the assessed Scenarios, while tool-action confirmation remains unassessed for one environment. This sentence is longer than “all tests passed” and more useful. It allows a decision-maker to weigh evidence, unresolved risk and governance obligations without pretending that an absent measurement is positive evidence.

8.11 Maintaining coverage as Aster changes

Coverage is not static. Product requirements change, retrieval corpora gain new documents, tools acquire new actions and real customer behaviour reveals previously overlooked failure modes. A Suite that was carefully designed last month may no longer support the same risk claim today. Maintaining coverage means keeping the links from current risks to Scenarios, Contracts, Evidence and Gates intelligible as the environment evolves.

Versioned Datasets and Contracts help preserve this relationship. A policy update may require a new Contract clause and different boundary inputs. A new refund endpoint may create an unauthorised-action risk that the old tool-call tests cannot see. Adding cases is not merely increasing count; it is updating the map of what the product can do and what can go wrong. Change analysis should ask whether an old Result still answers the question now being asked.

Regression design introduces another dimension of traceability. A Baseline is a specific approved historical reference, not whichever past result happens to look favourable. When a Result changes, a reviewer needs to know the compared Scenario, Environment, data version, Contract and relevant approvals. A changed Judge may also alter the comparison independently of subject quality. Without historical identity, a trend can show movement while hiding whether the measuring conditions stayed stable.

Finally, the organisation should periodically challenge its own map. Which high-stakes behaviour has not been exercised under a blocked path? Which semantic defect class lacks convincing reference cases? Which Gates rely on repeated results from very few distinct Scenarios? Which approvals are difficult to reconstruct? These questions turn traceability into a feedback mechanism for improving test design rather than treating it as administrative documentation.

8.12 Reading a coverage report without overclaiming

A coverage report has two audiences whose questions overlap but are not identical. The engineering team needs to know what to investigate next. A decision-maker needs to know whether the present evidence is sufficient for a particular release choice. A report that merely lists percentages satisfies neither audience. It should make the tested population identifiable, disclose cases that could not be assessed and explain how high-risk gaps affect the proposed decision. These are communication objectives derived from the specification's evidence and governance model, not a mandatory AQEF report template.

The difference between *not tested*, *tested but not assessed* and *tested and found failing* is especially significant. An absent Scenario is a design gap. A completed Execution with an unavailable Judge is an instrumentation or assessment gap. An actionable Result showing a Contract violation is evidence of a subject defect under the tested conditions. Although each condition can block a confident release claim, the remedial work differs: design a new Scenario, restore the Oracle, or correct the product. Mixing them into one category called “failed tests” directs investigation toward the wrong cause.

Reports should also make it possible to inspect contradictory evidence. If Aster passes a semantic rubric in one context and fails the same rubric in another, averaging the values may hide the boundary at which correctness deteriorates. A review can compare the relevant environments, retrieved passages and customer constraints before proposing an aggregate. The aggregate remains useful for decision-making, but it should not erase the structure of the evidence from which it was computed. This is one reason the AQEF

domain model retains individual Executions and Results rather than reducing the whole test campaign to a single score.

Coverage language should be calibrated to the actual release scope. “We assessed customer-support grounding for these policies, languages and environments” is bounded and falsifiable. “Aster is safe” is much broader and cannot follow from the first statement alone. Even a well-designed suite can only examine selected circumstances; it cannot enumerate every future input or tool combination. Engineers can nevertheless make meaningful release arguments when they identify important risks, select purposeful evidence, disclose limitations and preserve the authority chain behind decisions.

8.13 Who owns a gap?

A coverage map becomes practical when unresolved gaps have accountable owners, but ownership must not be confused with the claim that AQEF prescribes a particular organisational structure. The Project determines who can accept or change business risk, while Governance enforces Roles and Permissions for defined actions. A test engineer may identify that a high-risk Scenario is missing, a product owner may clarify the intended behaviour, and a platform engineer may restore the missing Artifact channel. One person might hold several of those responsibilities in a small team, but the questions themselves remain distinct.

Consider a failed attempt to verify that Aster did not execute an unauthorised refund. The Scenario may have run and produced a reassuring answer, while its tool-call trace is incomplete. A tester can diagnose the observation as insufficient evidence without alleging that the refund occurred. An implementation owner can repair the capture mechanism and rerun the test. A release authority may address remaining uncertainty only through the applicable Quality Gates and Governance process: a failed blocking Gate cannot simply be accepted informally, and any permitted exception requires an explicit, approved override for that particular Gate Result. The remaininging uncertainty permits deployment according to the applicable governance rules. These are separate decisions, and retaining that separation prevents operational urgency from rewriting the original evidence.

A mature review process also asks who is authorised to change the test's acceptance criteria. If a high-priority Contract clause begins failing, weakening it to restore a passing dashboard would change the question being asked of the product. The governance mechanisms in Volume XII distinguish approval of a Contract change from overriding a particular Gate Result. Coverage records should preserve the difference, including whether a revised Contract is being applied to future evidence or an exception is being considered against evidence already produced.

The objective is a traceable loop: identified risk leads to an accountable test-design choice, evidence reveals strengths or gaps, reporting makes those limits legible, and authorised decisions determine what changes next. The loop does not guarantee a successful release; it enables a later reviewer to understand why a decision looked reasonable or unreasonable at the time it was made. That ability to reconstruct reasoning is a substantial part of quality engineering's professional value.

Engineering Reflection

An Aster release report shows ninety-eight percent passing Results, yet almost all Results came from repeated runs of three FAQ Scenarios. Which additional Aggregate Basis information is necessary before treating the percentage as broad release evidence?

A Release Decision was approved six months ago, but the Contract version used by its Snapshot can no longer be retrieved. Is this merely a documentation problem? Which traceability guarantees are relevant?

Common Misconceptions

“More tests always mean better coverage.” Repeated low-risk tests can increase count without addressing another important defect class or user situation.

“A Scenario defined is a Scenario assessed.” Definition, Execution, Evidence and actionable Result are separate stages; each may be incomplete.

“Passing Results prove the whole application is covered.” Results only support claims about the specified Scenarios, clauses, instruments and evidence basis.

“An Oracle's confidence is the same as aggregate statistical confidence.” AQEF separates confidence in one verdict from the sample properties of an aggregate.

“Traceability is only requirement-to-test mapping.” AQEF Traceability reconstructs the actual chain from a particular decision to its evidence, configuration and approvals.

Chapter Summary

Coverage is an argument about whether important risk situations and failure mechanisms have been examined, not a universal test-count threshold.

Risk, defect-class, data, Oracle and Gate lenses provide a useful **pedagogical** review model rather than an additional AQEF requirement.

A Scenario can exist without an Execution, and an Execution can exist without an actionable assessment.

Repeated Executions support reliability investigation but do not substitute for distinct Scenario breadth.

Every aggregate read by a Quality Gate must disclose its Aggregate Basis, including actionable and non-actionable Results and distinct Scenarios and Executions.

Gaps and unavailable assessments must remain visible; missing basis is not a passing finding.

AQEF Traceability reconstructs the authorisation chain behind specific decisions, while Audit supports broader historical queries.

Changes to data, Contracts and environments require impact analysis so that old evidence is not presented as if it assessed new conditions.

Review Questions

1. Aster runs five hundred paraphrases of one low-risk FAQ question. In what sense has the test effort grown, and what important forms of coverage may not have improved?
2. How can a Scenario be “covered” in a design matrix but remain unassessed in a release report? Name two distinct causes.
3. Explain why defect-class coverage and Quality Dimension coverage are not interchangeable.
4. A team has twenty Results from twenty Scenarios; another has twenty Results from one Scenario repeated twenty times. What can each group of evidence support, and what does it not support?
5. Why does AQEF require Aggregate Basis while declining to specify a universal minimum sample size?
6. Show how a missing retrieval Artifact can break the chain from a RAG faithfulness requirement to an actionable Result.
7. A manager asks why a failed Gate did not prevent a release. What is the minimum kind of traceability path you would attempt to reconstruct?
8. How do Audit and Traceability answer different questions using related records?
9. If a Dataset version and Judge configuration change together, why is a simple score difference inadequate evidence of subject regression?
10. How would you explain deliberately untested risks in a release recommendation without implying that AQEF prescribes whether the product may ship?

Sources

1. AQEF 1.0-CR1, Volume I, Chapter 2 — Quality Model and Business Quality.
2. AQEF 1.0-CR1, Volume I, Chapter 6 — Aggregate Basis, Quality Gates and Release Decision.
3. AQEF 1.0-CR1, Volume II — Domain Model: Scenario, Execution, Conversation, Result, Baseline and Snapshot.
4. AQEF 1.0-CR1, Volume VII — Quality Contracts and clause composition.
5. AQEF 1.0-CR1, Volume VIII — Test Design, Risk-Based Testing and Seeded Controls.
6. AQEF 1.0-CR1, Volume XI — Reporting and Analytics, including display of Aggregate Basis.
7. AQEF 1.0-CR1, Volume XII — Governance, Audit and Traceability.
8. AQEF 1.0-CR1, Front Matter §0 — maturity restrictions on dataset mechanisms.

Chapter 9 — Validators, Judges, and Human Review

Maturity: Core + Implemented (human_review), as applicable · AQEF 1.0-CR1 · Textbook First Edition v1.0

Learning Objectives

After completing this chapter, readers should be able to:

1. Distinguish a Validator, a Judge and a Human Reviewer by the nature of the Oracle and its evidence.
2. Explain Validation before Evaluation and why an unavailable Validator is not a failed subject.
3. Interpret verdict, Confidence, inconclusive, and awaiting-review states without collapsing them.
4. Describe how an effective Result can change without erasing the historical assessment record.
5. Select appropriate Oracle roles for Aster Support AI's structural, semantic and policy-sensitive risks.

9.1 The assessment instrument is part of the system of evidence

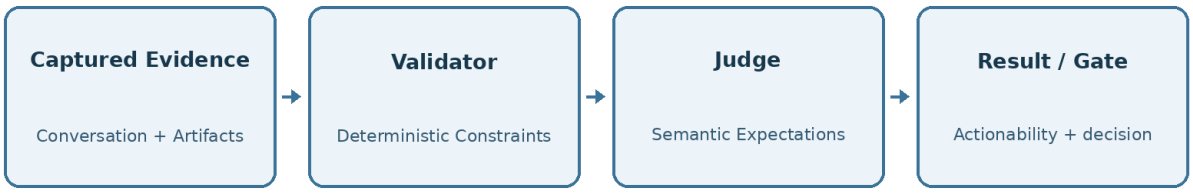


Figure 9.1 — The assessment boundary

A completed interaction is not yet a quality verdict. The Conversation and its Artifacts describe what Aster Support AI said, which passages its retriever supplied, and which tools it attempted to use. Assessment begins only when a defined Oracle reads that Evidence against a Quality Contract. This distinction is crucial because the Oracle can itself be wrong, unavailable, or inappropriate for a particular property. A well-preserved response is a valuable observation even if its assessment has not yet been completed.

The central architectural division is between deterministic Validators, probabilistic Judges, and Human Reviewers. They are not three interchangeable ways to give an answer a score. Each produces a different kind of claim and requires different treatment of Confidence. A structural check can determine whether a tool argument conforms to its declared shape. A semantic assessment may judge whether a policy explanation is faithful to the retrieved text. A person may resolve a genuinely contested interpretation. The design problem is selecting the right kind of evidence-producing instrument for each clause.

The division does not assign virtue to machines or humans. A deterministic instrument is reproducible but may be configured to check the wrong property. A probabilistic instrument may assess sophisticated meanings but can disagree with a competent reference reviewer. A human can examine context carefully

while also being inconsistent or biased. Trust does not come from choosing a universally superior Oracle. It comes from making the mechanism, criteria, limits and record inspectable.

Aster's return-policy explanation illustrates the stakes. If a response lacks a mandatory field, a Validator should make that structural determination. If it contains every field yet invents a refund exception, the correct shape tells us almost nothing about semantic correctness. A Judge or Human Reviewer must read the substantive claim against the governing policy evidence. The two checks can produce different conclusions without contradiction, because they answer different Contract clauses.

9.2 What makes a Validator deterministic

A Validator is an Oracle whose check is rule-based and deterministic. AQEF binds the check to target Evidence and to one or more Quality Contract clauses. For the same Evidence and its own versioned configuration, a Validator must reach the same Result. In Volume V the requirement is expressed in terms of a pure function: a Validator cannot obtain different answers merely because the current time changed, another service returned different data, or randomness entered the check.

> *"A Validator's Result MUST depend only on its target evidence" — AQEF 1.0-CR1, Volume V, Validator Architecture.*

A latency Validator demonstrates the subtlety. Latency itself is measured during Execution and recorded as an Artifact. Later Validation reads that recorded measurement against a threshold. The Validator must not substitute the time it takes to execute the validation check. Otherwise an identical archived Execution could pass on one day and fail on another because the assessment infrastructure was busy. Separating measurement from assessment protects reproducibility.

Determinism is a property of the instrument's check, not of the subject's output. Aster may generate different replies in two genuine Executions. The Validator can return different Results for those different pieces of Evidence without violating purity. What it may not do is vary its verdict for the same recorded Evidence and unchanged check configuration. This is why the phrase deterministic Validator is compatible with a probabilistic AI subject.

Validators are appropriate where correctness can be stated as an unambiguous rule: required structure, schema conformance, tool selection and argument format, forbidden strings or patterns, step ordering, and recorded cost or latency thresholds. Determinism does not imply that the chosen rule captures the entire business requirement. A Validator confirming that a refund action has the right argument fields cannot establish that the customer was entitled to that action.

9.3 Validation before Evaluation, and what short-circuiting means

AQEF gives Validation a defined place in the Execution-to-Result pipeline. Validators run before Judges for the same Execution. The rationale combines efficiency and evidentiary clarity. There is little reason to spend resources asking a probabilistic Judge whether a malformed payload communicates a policy correctly if the payload already violates a mandatory structural Constraint. More importantly, a favorable semantic opinion should not blur a hard deterministic failure that prevents downstream use.

> *"Validators MUST run before Judges for a given Execution." — AQEF 1.0-CR1, Volume I, Chapter 3, Validation before Evaluation.*

This principle is not an arbitrary convention. It preserves the difference between what can be established mechanically and what requires interpretation. In Aster, imagine a tool-call argument containing a field with the wrong type. An elegant customer-facing explanation does not compensate for the invalid action request. Validation identifies the failing clause and the pipeline records the corresponding outcome before any Judge has an opportunity to make that defect appear less important.

Inside the Validator Engine, cheapest-first ordering is a SHOULD, not an unconditional MUST. A schema check may precede a more expensive structural analysis, but the implementation can have a deliberate reason to order its deterministic checks differently. The normative requirement is that the Validator stage precede the Judge stage. Conflating those two levels is a common source of invented conformance rules.

A failed Validation can short-circuit later Evaluation. Yet the meaning of the short-circuit depends on *why* Validation stopped. If a Validator completed and returned fail, the product Evidence violated the checked Constraint. If a Validator could not run, or lacked indispensable Evidence, no such verdict exists. Both can prevent the next stage, but only one constitutes a demonstrated product failure. A trustworthy report keeps the distinction visible.

9.4 Composition is not aggregate quality scoring

Several Validators may contribute to the assessment of one clause or Execution. Their results can be composed through AND, OR, or a hard-veto condition as permitted by the Contract. A clause requiring both correct argument structure and a valid tool name may use conjunction. Another may accept any one of several approved formats. A safety-critical condition may veto the clause regardless of what other checks passed.

This composition concerns deterministic assessments within a particular Execution. It is not the same operation as the later Aggregation Model, which combines Results across Executions and quality dimensions for Quality Gates. Aster could have several Validators composing one structural verdict while a release Gate separately considers hundreds of scenario results. Collapsing the two operations produces misleading claims about what a number represents.

The boundary also matters for human review. Human judgment is not a replacement for an explicit hard-veto Validator when the Contract demands an unconditional rule. Nor should a rigid Validator be used to pretend a contextual semantic criterion has become deterministic. Contracts over Assertions means the Project specifies which concern each instrument addresses; it does not mean every concern can be made a boolean string comparison.

Engineers reviewing a composition policy should ask whether the logical combination matches the original intent. An OR across two Validators may be appropriate when either of two approved representations is acceptable. It is not appropriate simply because an AND caused failures in a release candidate. The decision about acceptable behaviour belongs in the Contract and its governance process, not in a hidden attempt to obtain a favorable outcome.

9.5 Unavailable is not failed

The most consequential distinction in the Validator lifecycle may be the difference between a failed check and a check that never completed. A Validator can crash, time out, or discover that necessary Evidence was not captured. In these cases the Oracle is unavailable for that assessment. AQEF records the condition rather than manufacturing a fail verdict against the AI subject.

> *"A Validator that cannot complete it … has reached no verdict: its Result is oracle_unavailable … never fail" — AQEF 1.0-CR1, Volume V, Validator SDK.*

Consider a completed Aster interaction for which the required retrieval trace was not saved. A structural check that needs the trace cannot know whether Aster selected the proper document. Calling the subject wrong would assert a fact that no instrument established. Calling it correct would be equally unjustified. The appropriate engineering response is to preserve the captured Evidence, identify the missing assessment basis and show that the Result cannot support a passing Gate.

The same principle applies to infrastructure outages. A Validator service may be unreachable while the stored Conversation remains intact. Retrying the same Oracle later against the original Evidence can resolve an unavailable outcome, subject to AQEF's Result Resolution rules. That is different from rerunning a genuinely failed check repeatedly until it happens to pass. The first resolves absence of a verdict; the second risks discarding an unfavorable observation.

Unavailability is not harmless. It can block release because a required Constraint has not been demonstrated. Nevertheless, blocking a release for missing evidence differs from reporting a product defect. Those distinctions support correct escalation: restore the instrument or evidence capture for one case, investigate subject behaviour for another. Accurate language prevents both false blame and unjustified reassurance.

9.6 What a Judge assesses

A Judge is a probabilistic Oracle for properties whose assessment requires meaning or contextual interpretation. Typical concerns include factual accuracy, faithfulness to retrieved passages, relevance, reasoning quality, tone and other Contract-defined Expectations. A Judge reads Conversation or Artifacts against explicit criteria; its result is not merely a free-form opinion that happens to sound plausible.

The key difference from a Validator is that a Judge's verdict is uncertain even when its input is fixed. Sampling choices, model limitations, ambiguity in the criterion and correlated weaknesses can influence the assessment. AQEF therefore requires the Judge's Result to carry Confidence separately from its verdict. A Judge can report a fail verdict with low Confidence or a pass verdict with high Confidence. Neither combination directly expresses truth.

> *"A Judge's Result MUST carry a Confidence value" — AQEF 1.0-CR1, Volume VI, Judge Architecture.*

For Aster, a Judge might compare the customer's refund question, retrieved policy excerpt and final explanation. Its task is to determine whether the explanation makes claims supported by the cited evidence. An unsupported assertion is potentially a semantic defect even when the answer is grammatically

impeccable. Yet the Judge may misunderstand an exception or overlook a qualification. The Result must preserve both its verdict and uncertainty so that later consumers do not read a probabilistic assessment as a deterministic fact.

AQEF also requires the Judge's criteria to be explicit and inspectable. A hidden instruction that silently changes the meaning of “faithful” between releases would undermine reproducibility of interpretation. A Quality Contract should express the intended property, and the Judge configuration should make its implementation of that criterion reviewable. The aim is not to eliminate all interpretive judgment, but to expose the basis on which judgment was made.

9.7 Confidence is not an accuracy statistic

Confidence describes an Oracle's stated certainty about a single Result. It is not the same as the observed correctness of that Oracle across a labeled reference set, and it is not a statistical confidence interval for an aggregate pass rate. This distinction may seem pedantic until a report presents a large number beside a green badge. Readers often interpret a high numeric value as proof that the answer is correct. AQEF deliberately prevents that interpretation.

A Judge might be confidently wrong about Aster's refund policy. It might also be correctly uncertain about a genuinely ambiguous exception. The second response may be a healthier assessment than the first, even if its numerical Confidence is lower. Evaluating a Judge's reliability therefore requires external evidence about agreement with trustworthy references, not faith in its own self-description.

A Validator has no numeric Confidence value. That does not mean the field can simply disappear. Its Result must explicitly declare that Confidence is inapplicable. A Human Reviewer's Result may have Confidence where it is captured; otherwise it also declares inapplicability. These are distinct cases. “Not measured,” “not applicable,” and “very low” should not be merged into one empty cell in reporting.

> *“Every Result MUST be explicit about whether Confidence applies to it” — AQEF 1.0-CR1, Volume I, Chapter 3, Everything has Confidence.*

There is an analogous requirement for a Judge score that combines multiple assertions. If a policy-faithfulness score averages several separately checkable statements, that reduction must be disclosed. One highly consequential unsupported claim should not vanish inside an apparently impressive mean. The reader needs to know both the unit of assessment and the basis of any combination.

9.8 Independence and the possibility of correlated failure

A Judge drawn from the same model family and prompted in nearly the same way as Aster may share its blind spots. If Aster repeatedly misinterprets a rare policy exception, a similarly constructed Judge could approve the same misinterpretation. Agreement between subject and assessor is then not independent evidence. AQEF's Independent Oracles principle prohibits using the same system, weights or decision process as the subject for its Oracle and strongly discourages avoidable shared failure modes.

Independence is not guaranteed merely by changing a model name. Different prompts can lead to different assessments without eliminating shared assumptions; different providers can still use similar evidence or reasoning patterns. The engineering question is what sources of correlated failure are plausible and what reference process could detect them. Independence must be defended with architectural and procedural reasons, not treated as a sticker applied to a component.

For Aster, an independent reviewer might use the authoritative policy passage directly rather than a summary previously generated by the subject. The Judge would receive the original Conversation and captured retrieval Artifacts, not an explanation written by the assistant about why its answer was correct. That arrangement does not eliminate the Judge's fallibility, but it reduces the risk that the only available source of truth is the subject's own self-report.

Multiple Judges are a scoped capability addressed in detail in Chapter 12. Their presence alone does not make an assessment independent: repeated samples from one Judge can improve stability without addressing common blind spots. For now, the essential lesson is that Oracle architecture determines evidentiary value. A plausible assessment from a correlated instrument may deserve less weight than a carefully reviewed disagreement.

9.9 Human Review is an Oracle, not a ceremonial approval

A Human Reviewer is a defined Oracle role rather than an informal instruction to have someone look at the result. Human Review can be required by the Quality Contract, triggered by a low-Confidence judgment, or used when Judges disagree. It may also provide reference labels for evaluation of Judge reliability. The capability has its own applicable requirements when claimed; it is not equivalent to routine management sign-off.

Human judgment has particular value where an outcome depends on policy context, value conflicts or incomplete semantic certainty. An experienced reviewer can inspect Aster's retrieved policy passages, customer intent and tool trace together. But that judgment still needs defined criteria and a preserved record. A review that merely says “looks good” without showing which clause was assessed is difficult to trace to a Release Decision.

The Human Reviewer's Result and the original Judge Result should remain distinguishable. A later reviewer outcome does not erase the historical fact that an automated assessment was uncertain or wrong. AQEF's result model supports effective resolution while preserving recorded history. This is essential both for decision accountability and for investigating whether the Judge has begun to drift over time.

A human reviewer should not be assumed infallible. Reviewers can disagree, interpret a policy differently, or misread evidence. The responsible design uses governance, traceability and appropriately selected reference cases to manage these limitations. Human Review is valuable because it supplies another accountable assessment channel, not because people automatically confer certainty on every disputed result.

9.10 The three assessment outcomes that cannot be collapsed



Figure 9.2 — A verdict is not instrument status

In ordinary conversation, a tester may say a case “did not pass” for several reasons. AQEF needs a finer vocabulary. A real fail verdict means that an Oracle completed its check and judged the relevant clause unsatisfied. *oracle_unavailable* means the Oracle could not complete its assessment. An *inconclusive* assessment means the applicable probabilistic judgment did not establish a sufficiently clear verdict. A case awaiting Human Review is not yet a human pass or fail.

These differences matter to downstream Quality Gates. A release may be blocked because a hard Constraint failed, because an assessment basis is missing, or because review has not completed. Treating all three as product failures distorts defect statistics and sends engineers to the wrong problem. Treating them all as passes is worse: it hides the absence of credible evidence.

The same distinction changes what should happen next. A product defect invites diagnosis and correction of the subject. An unavailable Oracle may require restoring an assessment service or collecting the missing Artifact where the rules permit. Inconclusiveness can require additional or independent evaluation; a pending review requires an authorised person to examine and record a judgment. An honest reporting system preserves the original reason for the blocked conclusion.

For Aster, a refund answer may be unsafe because it actually authorises an ineligible return. Or the missing boundary trace may leave the relevant Oracle unable to assess the claim, depending on the Contract and Evidence it requires; the mere absence of a trace does not by itself dictate a Result disposition. Or the relevant policy exception may be disputed and referred to a person. The release decision can be cautious in all three situations without pretending they are the same event.

9.11 Result resolution and append-only history

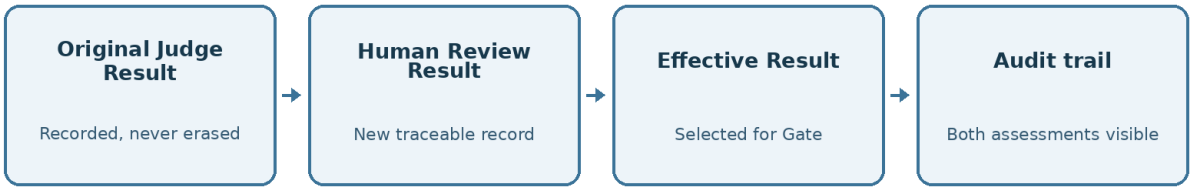


Figure 9.3 — Preservation through resolution

Result Resolution answers a difficult question: how can a later, more authoritative assessment affect the decision without changing what was previously recorded? AQEF distinguishes the recorded Result from the effective Result used by later decision mechanisms. A Human Reviewer may resolve a Judge outcome; an unavailable Oracle may later evaluate preserved Evidence after service has been restored. The new record can change which assessment is actionable while the earlier record remains available for audit.

The special `awaiting_review` lifecycle needs care: completion of a pending Human Review may be recorded in place under the specification's explicit exception. That does not create permission to overwrite other recorded Results. Subsequent resolution of an already recorded assessment otherwise preserves the earlier record and makes a later Result effective through the defined resolution relationship.

The distinction preserves the chronology of knowledge. Suppose a Judge assessed Aster's explanation as passing with low Confidence. A Human Reviewer later finds a policy exception that the Judge overlooked. If the system simply overwrote the Judge's original verdict, investigators would lose the evidence that the automated instrument had made a mistake. With an append-only chain, the release-level decision can use the effective later assessment while retaining the history needed for Oracle quality analysis.

Resolution must not become a synonym for shopping for favorable opinions. AQEF constrains when re-assessment of the same Evidence can legitimately resolve an earlier Result. In particular, repeatedly asking a functioning Oracle until it returns a desired value is not a valid way to erase a failure. The difference is between addressing a known basis for resolution and changing a recorded judgment merely because it was inconvenient.

Traceability requires that the chain from the current effective Result back to its earlier assessments be reconstructable. A reviewer should be able to tell whether a change resulted from new human judgment, requalification after a Seeded Control or completion of a previously unavailable check. Decisions should be tied to what was known at their own recorded time, not silently retrofitted with later information.

9.12 Admission, control evidence and the next chapters

A Judge's presence in an assessment pipeline is not sufficient evidence that it should be trusted in a consequential Gate. AQEF's Admission mechanism compares its performance with human-labeled reference material under the documented criteria. That process is covered in depth in Chapter 10, alongside calibration

and Judge Drift. The important point here is conceptual: claimed certainty and observed reliability are different evidence sources.

Seeded Controls, the scoped capability addressed in Chapter 11, provide another form of evidence about the instrument. A defect control tests whether an Oracle recognises a planted failure of a known class; a clean control tests whether it produces false alarms on evidence known to be clean. A missed control can invalidate otherwise plausible results for the affected class. That does not mean the subject failed; it means the assessment instrument has not demonstrated qualification for that scope.

The same reason explains why Multi Judge, discussed in Chapter 12, requires careful attention to purpose. Several independent Judges can reveal disagreement; several repeated samples of the same Judge can quantify its instability. Neither arrangement eliminates the need to preserve the basis of the final assessment or to route unresolved cases to a Human Reviewer where the Contract requires it.

A further practical difficulty arises when one Conversation triggers multiple clauses assessed by different Oracles. A reader of the final release report may see one subject interaction represented by several Results. That is not redundant measurement: one Result may address structural validity, another policy faithfulness, and another whether the system made a prohibited claim. Engineers should resist summarising these heterogeneous findings as though they were repeated votes on one identical question. The Quality Contract defines the distinct properties, and the decision layer must retain the connection between each verdict and its originating clause. This prevents an easily counted structural pass from dominating an analytically difficult semantic failure solely because more simple checks happened to be configured. It also clarifies the purpose of additional Human Review. The reviewer should answer a specific unresolved question against the recorded Evidence, not issue a general thumbs-up that obscures which property was actually disputed. Aster could be structurally correct, semantically unfaithful and operationally within latency limits in the same Execution. Such a combination is not inconsistent; it is the reason AQEF has separate quality dimensions and explicit Oracle roles. The release decision must reflect how the Project designated the importance of each clause and Gate, rather than making the most convenient single verdict stand for the entire interaction.

The architecture introduced in this chapter is therefore not a competition between deterministic automation, AI judging and people. Each method has a defined role, characteristic failure modes and rules for representing the strength of its evidence. The practical goal is to allow the release decision to distinguish demonstrated failure, assessment uncertainty and an unavailable instrument—without concealing any of them behind a single green or red symbol.

9.13 Choosing an Oracle by the failure one needs to see

Oracle selection should begin with the failure mode rather than the available technology. A Project may possess a powerful language model that can assign scores to almost any conversation, but technical flexibility does not establish suitability. The right first question is what evidence would discriminate acceptable from unacceptable behaviour under the Contract. If a breach can be expressed as an invariant over the recorded data, the deterministic Validator is normally the clearer instrument. If the property concerns meaning relative to a source, a Judge or Human Reviewer may be appropriate. The distinction makes the resulting claim easier to audit.

Aster's agent provides an instructive boundary. An action request may name an approved tool and supply structurally valid arguments. Those mechanical properties are naturally Validator concerns. Whether the tool should have been called for this customer's situation depends on policy interpretation, context and the actual authority of the customer. A complete test design may therefore require a Constraint and an Expectation examining different sides of one action. A single coarse pass flag would hide that distinction.

Some failures are especially easy to misclassify. A model may produce grammatically well-formed policy language that contradicts the retrieved document. A schema Validator will correctly pass the response's format, but that result has no bearing on faithfulness. Conversely, a semantic Judge might approve a helpful explanation in a response object whose schema is invalid. The Judge has not necessarily performed its own task badly; the pipeline must simply preserve the earlier structural failure rather than treating the semantic success as compensation.

The engineer should document how each clause maps to its Oracle and which Artifacts are necessary. Where a Judge requires a passage that was never captured, asking it to improvise a judgment based on the final answer changes the property being tested. Where a Validator cannot inspect a path because the Evidence lacks the target, claiming that an absence of a prohibited string is a pass may turn missing data into false reassurance. The proper choice is to make the limitation explicit and preserve a defensible assessment boundary.

In a design review, teams can use a simple sequence of questions without turning it into a new AQEF checklist: Is the criterion mechanically decidable from fixed Evidence? What portion of that Evidence was captured? Does the Oracle's configuration remain stable? Can a reviewer inspect the decision rule? What happens when the instrument cannot finish? These questions do not create new normative obligations; they help find gaps before a polished report obscures them.

9.14 Separating three kinds of uncertainty

Uncertainty about the subject, uncertainty about the assessment instrument and uncertainty about the evidence base are not one problem. The subject may vary across Executions under identical inputs. The Judge may disagree with a reference about one completed Execution. The evidence base may be too narrow to support a release claim about the product's broader behaviour. A good report allows a reader to identify which uncertainty is present and whether the proposed next action addresses it.

Subject variability belongs to the phenomenon being tested. Repeated Executions can provide information about Reliability, but they do not automatically extend Scenario breadth. If Aster passes one refund Scenario on nine executions and fails it on the tenth, the observed inconsistency should be preserved rather than explained away as inconvenient randomness. The same distribution cannot establish behaviour on an untested policy exception elsewhere in the product.

Oracle uncertainty belongs to the measurement process. A Judge's Confidence qualifies the assessment it produced; independent references, Admission and relevant controls help examine whether that assessment deserves trust. A low Confidence Result may be an honest indication that a policy phrase admits competing interpretations. Asking the Judge for the same assessment repeatedly until one high Confidence pass appears does not resolve the ambiguity responsibly.

Evidence-basis uncertainty concerns what was sampled and retained. An aggregate derived from many Results can still rest on one repeated Scenario. A missing retrieval Artifact may prevent a conclusion about groundedness even when the final response sounds accurate. This is why AQEF's Aggregate Basis reports actionable and non-actionable Results together with distinct Scenarios and Executions. The basis answers a question separate from the single Result's Confidence.

The corresponding remediation must also be specific. To investigate subject variability, run appropriately designed repeated cases while preserving each genuine Execution. To strengthen instrument trust, use qualification evidence and review disagreement. To improve the basis, broaden risk-relevant Scenarios or repair Evidence capture. Sending more prompts to a Judge is not a substitute for adding an unrepresented risk. Adding test data does not repair an Oracle that confidently misreads a policy. The accuracy of the diagnostic category determines whether the engineering response can help.

9.15 Human Review as a governed transfer of responsibility

When a Human Reviewer becomes the Oracle, the question is not merely who presses an approval button. The workflow must preserve what is being reviewed, which Quality Contract criterion applies, what the earlier automated assessment claimed, and what the human determined. Without those elements the project may possess a managerial sign-off, but it lacks a traceable assessment that explains how a specific Result became effective for a Gate.

In Aster's return-eligibility dispute, a reviewer should be able to inspect the original customer request, the retrieved policy excerpt, the agent's final answer and relevant action trace. The reviewer is then asked about the defined clause rather than invited to give a general impression of the interaction. If the policy is ambiguous, the reviewer may need a business-policy owner to clarify its meaning; that escalation is not the same as declaring the AI product correct or incorrect from intuition.

AQEF separates Human Review as an Oracle from governance approvals for Baselines, Gate overrides and Release Decisions. The roles can involve people, but they answer different questions. A person who reviews a semantic claim has not automatically authorised release. A person who approves a release does not automatically supply reference truth for every disputed Judge Result. Both decisions should preserve their own evidence and authorisation chain.

The time dimension is especially important. Suppose an automated Result blocked a Gate while review was pending. The reviewer later records a verdict that resolves the question. A report generated before the review should remain a faithful record of its earlier state. A report generated afterward may legitimately reflect the effective updated Result. Overwriting the historical Gate or Result would make it impossible to reconstruct what was known at the moment of each decision.

Human Review itself can introduce disagreement. Two competent reviewers may interpret an ambiguous policy differently, revealing a defect in the project's specification of acceptable behaviour rather than a defect in Aster. The right response may be clarification of the Quality Contract or policy source under governance, not forcing an artificial majority vote. A sound assessment model exposes this possibility, instead of promising that a human reviewer ends all uncertainty.

9.16 When measurement failure becomes a governance concern

A quality team is often tempted to treat infrastructure and Oracle problems as local technical incidents, separate from release governance. In practice they can materially change the evidence on which a release decision rests. If a Judge is unavailable for a critical policy dimension, a Gate lacks the assessment intended to support it. If a Validator checks the wrong target because of a configuration mistake, its apparently deterministic pass can be irrelevant to the intended Constraint. Both defects concern the measurement process, but both can weaken a release decision.

The appropriate governance response depends on the record, not on the team's desire for a green dashboard. An unavailable Oracle calls for an honest non-actionable outcome and restoration or reassessment where permitted. An incorrectly specified Contract requires an authorised change and an explanation of its effect on previous assessments. A deliberate Gate override is a distinct action and cannot be disguised as correcting a faulty Judge. These distinctions protect the credibility of historical evidence.

Consider an Aster release for which the only Judge assessing refund-policy faithfulness has failed a Seeded Control for the relevant defect class, invalidating its affected Results, or has not been admitted where the blocking Gate requires admitted Judges. Passing structural Validators do not fill that hole. If the Project's blocking Gate requires semantic evidence, the release cannot properly claim that the requirement has been demonstrated simply because a different dimension still reports green. The engineering question is which required clause lacks a qualified assessment and what corrective evidence could legitimately restore it.

The converse matters too. Aster might genuinely fail a high-impact policy criterion while an engineer blames the Judge. Challenging an assessment is legitimate when there is counterevidence, but unsupported suspicion does not erase the recorded result. The team should examine the relevant source passage, Judge criteria, available human references and specific failure mode. The outcome may be a subject fix, a Judge fix, or clarification of an ambiguous Contract. The investigation should make its conclusion traceable.

This chapter has therefore examined Oracles as engineering instruments rather than magical authorities. The division between deterministic and probabilistic assessment gives each type a proper domain. Availability states prevent missing assessment from masquerading as product failure. Confidence, independence and Human Review make the strength of semantic evidence explicit. Governance ensures that later corrections can affect decisions without rewriting what earlier instruments actually said.

Engineering Reflection

Aster completes a refund-related Conversation, but the Judge service is unavailable. Which evidence remains usable, and which additional evidence is required before a release-level interpretation can be made? Explain why merely marking the Scenario as failed would be misleading.

A Human Reviewer reverses a low-Confidence automated pass. What should remain visible about the original assessment, and what should the downstream Gate consume?

Common Misconceptions

1. A deterministic Validator proves that the complete answer is correct. It proves only the deterministic property it actually checked.

2. A Judge's high Confidence proves correctness. Confidence is the Judge's uncertainty statement, not independent evidence of its observed accuracy.

3. If an Oracle crashes, the AI system failed. An unavailable Oracle reached no product verdict and must be represented as such.

4. Human Review may replace the original automated result in history. The effective assessment may change, but the earlier record remains traceable.

5. Running several Judges always ensures independence. Repeated samples of the same instrument may improve stability without reducing correlated blind spots.

Chapter Summary

An Oracle evaluates recorded Evidence under a Quality Contract; a completed Conversation alone is not a verdict.

Validators are deterministic, pure checks, and their Results explicitly mark Confidence as inapplicable.

Validators run before Judges; a failed or unavailable Validation can prevent later Evaluation for different reasons.

Judges make probabilistic assessments and carry Confidence separately from verdicts.

Independent Oracles reduce the risk of correlated failure, but independence and reliability require evidence.

Human Review is an accountable Oracle mechanism, not an informal approval ritual.

Unavailable, inconclusive and awaiting-review states are not synonyms for a product defect.

Result Resolution can change what is actionable while preserving recorded history.

Review Questions

1. Aster's response satisfies a schema check but invents a refund exception. Why are those findings compatible, and which Oracle is relevant to each?
2. Why is a recorded latency Artifact preferable to a Validator measuring wall-clock time during its own assessment?
3. Distinguish Validator-stage ordering from cheapest-first ordering within the Validator Engine. Which has MUST force and which has SHOULD force?
4. When does oracle_unavailable give less information about Aster than a failing Validator Result, even if both stop Evaluation?
5. A Judge reports pass with Confidence 0.97. What can and cannot be inferred from that number alone?
6. How might a same-family Judge incorrectly agree with Aster, and which evidence could help expose that problem?
7. What is lost if the implementation overwrites an automated verdict after Human Review?

8. Explain why a low-Confidence judgment, an unavailable instrument and a known product failure might all block release for different reasons.
9. How can a clause with several deterministic Validators use AND, OR or hard veto without becoming release-level Aggregation?
10. A team retries a Judge repeatedly after a genuine failure. Why is this different from resolving an unavailable Oracle against the same stored Evidence?

Sources

1. AQEF 1.0-CR1, Front Matter §8 (Terminology) and §10 (Conformance).
2. AQEF 1.0-CR1, Volume I Chapters 3 and 6 (Fundamental Principles; Results and Resolution).
3. AQEF 1.0-CR1, Volume II (Domain Model).
4. AQEF 1.0-CR1, Volume V (Validator Engine: Architecture, Pipeline and SDK).
5. AQEF 1.0-CR1, Volume VI (Judge Engine, Human Review, Confidence and Admission).
6. AQEF 1.0-CR1, Volume VII (Quality Contract clauses and Policies).
7. AQEF 1.0-CR1, Appendix C §C.7 (Result states and recorded dispositions).

Chapter 10 — Judge Reliability: Reference Set, Admission, and Calibration

Maturity: Core (Admission and Judge Result requirements); Conceptual (Calibration and ongoing reliability interpretation); Implemented capability context: human_review and seeded_controls · Textbook First Edition v1.0

Learning Objectives

By the end of this chapter, readers should be able to:

1. Explain why Judge verdicts and self-reported Confidence cannot establish the Judge's own correctness.
2. Assess the purpose, provenance and limits of a human-labelled Reference Set.
3. Interpret raw agreement, chance-corrected agreement, Admission records and the effect of a later non-admitted record.
4. Distinguish Admission from Calibration, Judge Drift and Seeded Controls.
5. Explain the implications of Judge configuration changes and the Project's require_admitted_judges Gate option for release evidence.

10.1 Reliability is a property of the assessor, not its tone

The last chapter showed that an Oracle's verdict is not identical to the AI subject's correctness. A Judge may produce a careful explanation and a persuasive Confidence value while nevertheless misclassifying an answer. When a Judge is placed between evidence and a consequential release decision, the Project needs an independent basis for trusting that instrument. Reliability must be demonstrated through comparison with appropriate references rather than inferred from fluent reasoning, apparent sophistication or internal consistency.

In Aster Support AI, the assistant sometimes misinterprets refund eligibility. An automated Judge is asked whether the explanation faithfully follows the policy passage retrieved for that customer. A Judge that always approves polished prose may appear agreeable and stable while failing the very purpose for which it was selected. Conversely, a Judge that flags every answer may appear cautious but generate false alarms. A stream of verdicts does not reveal which pattern is occurring until reference cases establish what the correct answers were.

AQEF addresses this problem by distinguishing Admission, Calibration and Seeded Controls. Admission compares a Judge's verdicts with an independently labelled Reference Set before the Project relies on them as admitted evidence. Calibration asks whether its Confidence corresponds to observed correctness over many historical cases. Seeded Controls test selected abilities in the context of a particular run. These are not alternative names for one reliability score. They look at different moments, different populations and different modes of failure.

The distinction also prevents a mistaken goal: obtaining the most confident Judge. Strong confidence without evidence of correctness can increase the danger of false reassurance. A useful Judge is one whose behaviour is sufficiently well understood, whose applicable status can be established, and whose limitations are visible to downstream consumers. Admission is a controlled claim about agreement on a defined reference population, not an assertion of universal truth.

10.2 What a Reference Set actually represents

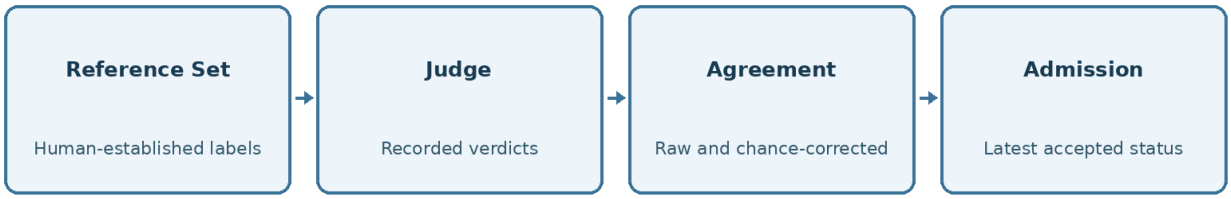


Figure 10.1 — From Reference Set to Admission

A Reference Set is a collection of examples with correct verdicts established by Human Reviewers. It is evidence about an Oracle, not a random sample of product outputs that happens to have labels attached. Its purpose is to make the Judge's assessments comparable with an external determination. The same item presented to the Judge and to the reference process should express the same assessment task and the same relevant Contract criterion. Otherwise a disagreement may reflect mismatched questions rather than Judge quality.

A useful Aster Reference Set might include straightforward policy matches, plausible unsupported exceptions, answers grounded in an irrelevant policy passage, ambiguous policy wording and cases where the assistant gives the correct conclusion for the wrong reason. Those examples should not be chosen merely to make the Judge succeed. Their mix should correspond to the properties and failure classes the Project plans to rely on the Judge to detect. A Reference Set heavily populated with easy positive cases may make overall agreement high while offering weak evidence about high-consequence failures.

AQEF recommends that reference verdicts be established by Human Reviewers independent of the people tuning the Judge. Reviewers should label without seeing each other's answers, with disagreements resolved through recorded arbitration. Examples used to align reviewers should be excluded from the actual score. The Set should draw from more than one source and should be versioned. These SHOULD-level recommendations are meaningful safeguards, yet they should not be rewritten in a textbook as unrelated, invented MUST-level schema requirements.

Why protect the reference? If a person has repeatedly inspected the same examples while modifying a Judge prompt, a later success on those examples is weaker evidence of generalisation. If the labels are changed to match the Judge's mistakes, the reference stops being independent. A Reference Set's integrity concerns both its content and the process by which that content became authoritative. The Project should explain whose judgment established the correct verdicts, which ambiguities were settled and what population the evidence can reasonably represent.

10.3 Human labels, disagreement and arbitration

Human labels are necessary to Admission, but the word human should not be read as infallible. Policy language may be ambiguous, source documents may conflict and reviewers may understand a customer's situation differently. A decision made by one person without a recorded basis can contain error just as an AI Judge can. AQEF's recommended independent review and arbitration process therefore improves the quality of the reference before the automated instrument is measured against it.

Consider a refund answer that refers to a policy exception valid only in one region. One reviewer calls the response correct because the exception appears in a policy document. Another calls it incorrect because the Scenario's region falls outside the exception's scope. The dispute cannot be resolved by asking which answer sounds more confident. The resolution requires the Scenario facts, applicable policy source, intended criterion and a recorded explanation. If the Reference Set simply averages the two labels, it has not established a determinate target for Admission.

Arbitration is not a mechanism for making all future disagreements disappear. It provides a traceable outcome for the particular reference item and can reveal that a Contract clause is too broad or that the test Evidence is insufficient. Sometimes the correct engineering response is to refine the reference item's scope or exclude it from an Admission population until its criterion is meaningful. The process should not retroactively declare an AI Judge correct simply because its answer matches the view of a powerful reviewer.

A Reference Set also benefits from separating reviewer alignment cases from scoring cases. Alignment exercises identify disagreements in interpretation and improve the consistency of the rubric. Scoring cases examine how a Judge performs after the relevant criteria have been fixed. Mixing these purposes allows training on the measurement itself: the Judge or its authors can learn the answers that later appear to confirm its competence.

10.4 Raw agreement and the illusion of an easy success

Raw agreement reports the share of reference items on which the Judge's verdict matches the established label. It is understandable and useful, but incomplete. A Judge may appear reliable simply because the Reference Set contains many examples of one class. If ninety of one hundred cases are labelled pass, a Judge that calls everything pass will agree with ninety labels while detecting none of the failures. That is precisely why Admission must record the reference-label distribution and an agreement measure corrected for chance.

The words corrected for chance do not mean that AQEF forces one statistical technique on every Project. The Project decides which measure is appropriate and what threshold is sufficient; the Admission record must identify the measure by name and give its value. Different label schemes, class distributions and purposes can justify different statistical choices. It would be an invented requirement to claim that all conformant implementations must use a particular coefficient or identical threshold across industries.

When the selected chance-corrected agreement cannot be computed for the observed label distribution, the Admission outcome is non-admission rather than an assumed pass. This is not interchangeable with low agreement: the statistic lacks a computable value for that basis, so an admissible numerical conclusion has not been established.

For Aster, the distinction is especially important if a false pass has higher cost than a false alarm. A Reference Set might contain many accurate answers because that matches historical traffic, but the Project still needs evidence that the Judge detects unsupported refund eligibility claims. Raw agreement alone cannot tell whether apparent success came from genuine discrimination or from reproducing the dominant label. Looking at class-specific mistakes and the distribution of labels is a helpful explanatory practice, though AQEF does not prescribe a universal diagnostic dashboard.

Agreement is also different from the Judge's own Confidence. A Judge may emit Confidence 0.98 on each verdict, but raw agreement is computed against external labels across a set of cases. The two figures have different denominators and different meanings. Calling them interchangeable would turn a self-reported property of an instrument into apparently independent evidence about its accuracy. The textbook must consistently maintain that boundary.

10.5 Admission as an explicit decision about one configuration

Admission is defined for a Judge configuration, not merely for a commercial model name or a general category of Oracle. The configuration includes relevant elements such as model, prompt and criteria; changing one of these can create a different Judge for Admission purposes. This is essential because even a small prompt modification can change which errors the Judge notices. In AQEF a changed configuration requires its own Admission rather than inheriting the previous version's status automatically.

The most recent accepted Admission record determines whether the configuration is admitted. It identifies the Reference Set and version, number of scored items, distribution of correct verdicts, raw agreement, chosen chance-corrected measure, threshold and outcome. An agreement meeting the declared threshold qualifies the configuration under that record. If a subsequent accepted record does not meet the threshold, it records non-admission and withdraws the earlier admission. Historical records are not silently overwritten.

This is a lifecycle rather than a permanent award. A Judge that passed Admission last month has not earned an unchangeable credential. Reference data, policy content, Judge configuration and the Project's risk priorities can evolve. New evidence can show that the current configuration no longer meets the Project's threshold. The rule requiring the latest accepted record to govern prevents a Project from selecting an old favourable Admission merely because it prefers that result.

Imagine Aster's Judge J1 passed with the original policy criteria, and the team adds a requirement to identify unsupported action claims. The revised Judge J2 may be better, but it must be admitted separately. If J1 is reassessed against an appropriate new Reference Set and fails, J1's previous admitted state is withdrawn by the new accepted record. The team should not invoke the original score to defend a continued gate claim after its governing Admission status changed.

10.6 Why an Admission record has to be interpretable

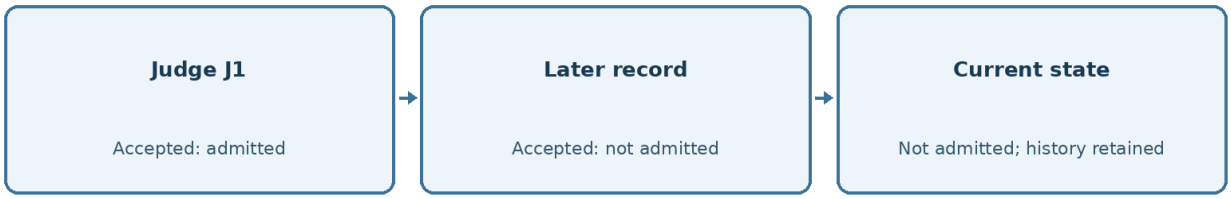


Figure 10.2 — An accepted non-admission supersedes prior status

An Admission number without its basis is difficult to evaluate. How many examples were scored? Were almost all cases passes? Which reference version contained them? What measure corrected for chance, and what threshold was used? The record must carry these facts because two identical-looking agreement percentages may support substantially different levels of confidence in the Judge's usefulness. Their context changes the interpretation without altering the number itself.

Consider two hypothetical Aster Judging studies. One uses dozens of routine answers drawn from one frequently tested refund policy; another samples difficult scenarios across several policies and error classes. The same raw agreement does not mean the studies established equivalent ability. AQEF requires explicit basis information; it leaves statistical adequacy and appropriate threshold selection to the Project. A report should therefore distinguish factual compliance with record requirements from the substantive adequacy of the Project's chosen reference population.

An Admission record is append-only in meaning: subsequent records do not erase the earlier one. This supports questions that may arise after a release, such as which Judge configuration had been admitted at decision time and whether a later failure was already known. Version identity and temporal ordering prevent the evaluation history from being rewritten into the most flattering possible story. Where the API resolves the current status from accepted records, an implementation should follow that rule instead of inventing a separate revocation operation.

The technical details of the Admission API belong to the specification, not this coding-free textbook. What matters pedagogically is that a new record does not describe a vague relationship between a model and truth; it records a specific configuration's observed agreement against a specific labelled Reference Set. The claim remains bounded by that Set, its scoring rules and the Project's threshold.

10.7 Admission is an eligibility control, not a universal obligation to gate

AQEF permits a Project to require admitted Judges before their Results can be used in certain Quality Gates. When a Gate declares require_admitted_judges, a Result from a Judge configuration that is not admitted blocks the Gate and the corresponding reason is recorded. This is a conditional rule. The existence of Admission does not mean that every Gate in every Project must declare the condition regardless of context.

A Report nevertheless shows Judges in scope that are not admitted. Visibility and enforcement are separate concepts: the implementation makes status available for interpretation, and the Project declares whether a particular Gate requires it. The resulting decision should not be phrased as proof that an unadmitted Judge

was correct or incorrect on a specific Aster case. Admission describes evidence about the instrument's general agreement on a reference population, not a substitute verdict for the case being assessed.

The specification permits uses of an unadmitted Judge that do not pretend its verdict is an authoritative release determination. It may help sort cases for a Human Reviewer who then makes the judgment. This can improve review efficiency while preserving the distinction between automated triage and decision evidence. A project should be careful not to turn those provisional rankings into metrics that appear to have passed a declared admission requirement.

This separation is useful for leadership. A manager can decide that financial-action policy decisions require admitted Judges, while exploratory classification may tolerate an unadmitted instrument as an aid to review. The choices should be expressed as policy and risk decisions; they do not change the identity of the Judge or retrospectively validate its historical Results.

10.8 Tuning a Judge Is Not Admitting It

A Judge can improve its agreement with people during development without becoming more trustworthy for a release decision. Teams often tune a rubric, assessment prompt or few-shot examples on a working set of labelled answers. That activity is useful: reviewers reveal ambiguous criteria, a calculation exposes disagreements, and an engineer can compare candidate versions. But the same cases that helped select the winner cannot independently establish how well the winner will perform on unfamiliar cases. Tuning is common engineering preparation, not a separate normative AQEF capability.

Four activities answer four different questions. Tuning asks which criteria or prompt to change to improve agreement on development cases. Admission asks whether one frozen Judge configuration meets the Project’s declared agreement threshold against a versioned Reference Set. Calibration asks whether the Judge’s reported Confidence matches its observed correctness over time. Seeded Controls ask whether an Oracle detects a known planted defect and avoids a false alarm on a known clean example in the current run. None substitutes for another.

Terminology note — In everyday engineering conversation, “calibrating a judge” sometimes means revising prompts until they match labelled examples. In AQEF, Calibration has a narrower meaning: comparing reported Confidence with observed correctness over time. Prompt or rubric tuning is preparatory work, not the normative Admission decision. An Admission record concerns a specific frozen Judge configuration, not the family of prompts from which it was selected.

Aster faithfulness: a better tuning score, but no Admission

The following figures are illustrative, not performance findings about any deployed product. Aster Support AI uses a faithfulness Judge, to assess whether a refund answer is supported by the applicable policy evidence. Reviewers label 25 working cases, compare three rubric variants, and inspect disagreement. The original variant agrees on 19 of 25 items (76%); after revisions, the selected variant agrees on 22 of 25 (88%). That is meaningful evidence about the working cases and an understandable reason to freeze that version for independent evaluation. It is not an Admission result.

The team freezes the selected model, prompt and criteria and evaluates the configuration on a fresh, versioned Reference Set of 60 human-labelled cases that the tuner has not seen. In this illustrative matrix,

among 30 reference-pass cases, the Judge predicts pass for 24 and fail for 6; among 30 reference-fail cases, it predicts pass for 5 and fail for 25. The Judge therefore predicts 29 passes and 31 fails and agrees with the reference on 49 cases. Observed agreement is 49/60 = 81.67%, lower than the 88% on the exposed tuning set.

Raw agreement is not sufficient for Admission. Using the displayed reference and prediction totals, expected agreement by chance is $[(30 \times 29) + (30 \times 31)] / 60^2 = 1800/3600 = 0.50$. Cohen's κ is $(49/60 - 0.50)/(1 - 0.50) = 0.63333$. In this illustration, the Project declared a minimum κ of 0.70 before the independent evaluation. Because 0.63333 is below 0.70, the frozen configuration is NOT ADMITTED. A Quality Gate enforces admitted-Judge status only when explicitly configured to require it. AQEF does not prescribe 0.70 as a universal threshold.

One disagreement illustrates why the outcome matters. The Judge marks the phrase "about $50" as incorrect although the applicable reference states $49.99. Depending on the requirement and context, that difference may or may not be materially unfaithful. The reviewers should inspect the policy and written assessment criteria, not simply change the human label to match the Judge. If the criterion needs repair, the team returns to tuning using other working cases; it does not treat these now-exposed 60 admission cases as a fresh, independent test of the revised configuration.

Seven limits of a tuning result

First, exposed cases are not independent evidence. If the same 25 answers guided a prompt revision and were reused to score that prompt, the final percentage benefits from selection on those answers. The evaluation should distinguish development cases from a separately protected Reference Set. AQEF recommends independent review and avoiding exposed reference items; those are SHOULD-strength protections, not an invented unconditional MUST to use a particular splitting procedure.

Second, small differences can be noise. On a 20-item set, one changed verdict moves raw agreement by five percentage points. For illustration, 17 correct of 20 is 85%; a Wilson two-sided 95% interval is approximately 63.96%–94.76%. That wide interval does not mean the Judge's true quality is known to be inside those limits without assumptions; it shows how uncertain a small-sample proportion can be. A comparison of 17 and 18 successes on the same cases needs more care than choosing the apparently larger number.

Wilson calculation note — With $x = 17$, $n = 20$, $\hat{p} = 0.85$ and $z = 1.96$, $CI = [\hat{p} + z^2/(2n) \pm z \times \mathrm{sqrt}(\hat{p}(1 - \hat{p})/n + z^2/(4n^2))] / [1 + z^2/n]$. Evaluating the lower and upper signs gives approximately 0.6396 and 0.9476. This is an explanatory statistical example, not an AQEF-required estimator or Admission threshold.

Third, a candidate that wins a working-set comparison may still fail the Project's independently stated threshold. Selecting the best among imperfect variants answers a relative question; Admission answers an absolute, governed question about a specific configuration. The result may be recorded as not admitted even when tuning improved the working-set score.

Fourth, the base rate can disguise a weak detector. A Judge that always predicts pass on a set that is 80% reference-pass obtains 80% raw agreement without showing discrimination between correct and incorrect answers. That is why the Admission record includes the verdict distribution and a named chance-corrected agreement measure, rather than treating raw agreement as sufficient.

Fifth, missed defects and false alarms are different risks. Sensitivity to planted or naturally occurring violations and specificity on known-clean examples cannot usually be inferred confidently from only 10–30 mixed cases. AQEF Seeded Controls separately exercise defect controls and clean controls in a run; those controls protect current-run instrument evidence, but they are not a replacement for Reference Set agreement or an Admission record.

Sixth, human reference labels must have their own accountable reasoning. Human Reviewers may find an earlier label ambiguous or wrong, and reasoned arbitration can legitimately correct it. Changing a label merely because the Judge disagreed corrupts the independence of the reference. A newly discussed or relabelled case has become exposed for future tuning/admission separation and should not be silently recycled as fresh Admission evidence.

Seventh, Admission belongs to the exact configuration assessed. Revising the rubric, prompt, model or criteria after the result makes a different configuration for this purpose. The team must not carry the original configuration's admitted status over by analogy, however small the change seems.

A practical engineering sequence is therefore: tune on working cases; freeze the selected configuration; evaluate it against an independent, versioned, human-labelled Reference Set; report raw and chance-corrected agreement against a threshold declared beforehand; record Admission; use appropriate Seeded Controls for current-run detection; and monitor Confidence Calibration over time. If Admission fails, return to tuning with other working cases and later use another unexposed Admission set. This workflow is advice for preserving reliable evidence, not an extra chain of AQEF MUST-level steps.

10.9 Calibration asks a different question from Admission

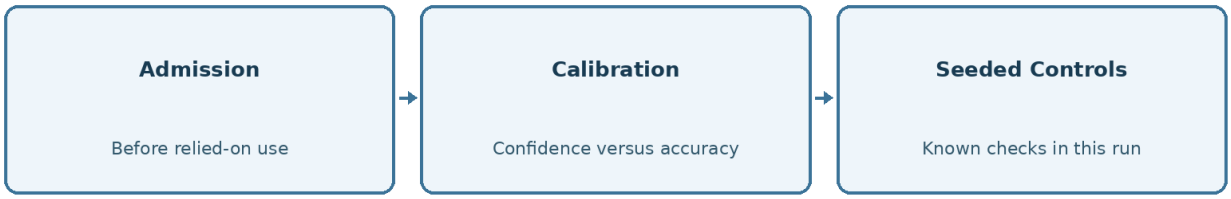


Figure 10.3 — Three different reliability questions

Calibration is the relationship between the Confidence a Judge reports and how frequently its verdicts are correct at that Confidence level. A Judge that reports 90 percent Confidence on a group of comparable decisions should, in a well-calibrated idealisation, be correct on roughly nine of ten of them. That is a population-level interpretation, not a guarantee that any particular answer with Confidence 0.90 is correct.

A Judge can have strong agreement with reference labels and still be badly calibrated. It might classify most Aster cases correctly but issue Confidence near certainty even on ambiguous responses. Conversely, it might be cautious in its Confidence while maintaining acceptable classification accuracy. Admission asks whether verdicts agree enough for permitted use; Calibration studies whether the extra numerical statement about uncertainty is warranted.

AQEF describes Calibration as a historical comparison against reference truth, commonly derived from Human Reviewer Results designated for that purpose. The specification does not make it a separate conformance certification process with a universal mandatory formula or threshold. This textbook therefore presents it as a conceptual and operational quality discipline described by Volume VI, while reserving explicit normative claims for the specified obligations.

For Aster, imagine the Judge assigns near-certain Confidence to straightforward policy quotations and equally high Confidence to subtle cases containing contradictory retrieved passages. Aggregate agreement may remain respectable, but calibration analysis could reveal that its strongest Confidence category includes disproportionately many wrong verdicts. That finding should lead the team to examine its assessment criteria and trust policy, not to assume the subject itself suddenly became worse.

10.10 Reading reliability curves without mistaking them for proof

One way to explain calibration is to group historical Judge decisions by reported Confidence and compare each group's observed accuracy. A plot with reported Confidence on one axis and observed correctness frequency on the other can show overconfidence or underconfidence. The illustration is diagnostic, not a mandatory AQEF chart, and a diagonal reference line does not certify that any model conforms to an exact statistical law in small samples.

Sample size matters. Ten cases labelled as 90 percent confident offer much less stable evidence than many independent cases, especially when the labels come from different risks or reviewers. Cases are not automatically independent merely because they are distinct rows: repeated paraphrases of the same policy passage may share the same underlying weakness. Calibration reporting should preserve counts and population composition alongside any smooth-looking curve.

Time matters as well. The relationship between Confidence and observed accuracy may change when Aster's policy corpus, the Judge's model backend or the distribution of customer questions changes. A reliability plot for last quarter need not describe performance today. Calibration is retrospective; it learns from reviewed history. This is why it cannot replace a check that the instrument detects known failure classes in the current execution batch.

A calibration plot can be seductive because it seems to reduce trust to geometry. In a safety-critical use, one rare failure class may dominate the practical significance of a small deviation. Managers should read the plot together with risk distribution, labels, uncertainty and the underlying quality contract. A technically attractive average may conceal unacceptable blind spots if the population being plotted is not the one whose failures matter.

10.11 Judge Drift changes the meaning of historical evidence

Judge Drift is a change in the Judge's behaviour relative to a reference point over time. It is distinct from Semantic Regression in the AI system under test. Aster's answers may become less faithful while the Judge remains stable; alternatively, Aster may remain unchanged while a Judge version starts rating answers more generously. If the team monitors only aggregated product scores, those situations can look similar. Separating subject evidence from assessment-instrument evidence is therefore essential to diagnosis.

A nominally identical Judge identifier does not necessarily guarantee identical underlying behaviour when a provider updates a model behind an endpoint or when the prompt delivery environment changes. Where configuration changes are declared, AQEF treats the changed configuration as a different Judge needing its own Admission. Where a hidden change is suspected, repeated reference comparisons and control evidence help investigators test whether the instrument's behaviour shifted.

Aster could show an apparent improvement in release reports after the Judge starts approving answers that previously failed. Management may welcome the green trend, but the finding is ambiguous until old and new Judge behaviour are compared on a stable reference population. The source of the improvement could be a product fix, a changed risk mix or an Oracle defect. Evidence from the correct layer is needed before choosing a remedy.

Drift monitoring is not the same as retesting the product. The Project should distinguish which configuration, reference version and time window each observation covers. This is also why a change to the Reference Set requires care: if both the measurement instrument and its benchmark change simultaneously, the observed difference cannot be attributed to one cause without further analysis.

10.12 Seeded Controls test whether the instrument works now

Seeded Controls serve a different reliability purpose. A defect control embeds a known problem of a declared class and asks whether the Judge detects it; a clean control is known to lack that problem and checks for a false alarm. The capability examines whether the Oracle is qualified for particular failure classes in a particular run. It is not an arbitrary extra set of cases added to product pass-rate statistics: these Results assess the instrument and are excluded from ordinary subject Aggregation.

In Aster, a seeded example might deliberately contain an unsupported refund exception in fixed Evidence. A Judge that misses the planted defect has not thereby proved that the live AI product failed a different Scenario. Instead the miss raises doubt about the Judge's ability to evaluate the affected class in that run. The response is to preserve the evidence of instrument failure and apply the specified qualification and result-actionability rules rather than retroactively converting every product case into fail.

Calibration, by contrast, might reveal overconfidence only after many reviewed Results accumulate. Admission might show the Judge passed a reference threshold weeks ago. Seeded Controls ask whether it still recognises known failures today under the conditions of the run. The mechanisms complement one another precisely because their time horizons differ. Strong historical evidence cannot guarantee current operational readiness, and a single current control cannot prove broad calibration.

The finer mechanics of missed controls, requalification and reassessment belong in Chapter 11. The essential message here is that reliability of the measuring instrument must be kept separate from quality of the measured product. Good engineering does not improve product metrics by hiding instrument defects, and it does not blame the product for every failure in its testing apparatus.

10.13 An Aster case study: the Judge that became too forgiving

Imagine an Aster release candidate that begins generating several plausible but unsupported promises about partial refunds. The product team sees mostly passing semantic Results and assumes the new prompt

reduced policy errors. A Human Reviewer, checking a few contested examples, notices that the explanations are polished but unsupported. The first hypothesis is a product regression; the second is a Judge that has lost sensitivity to the relevant defect class. The recorded Evidence should allow those hypotheses to be tested separately.

The team examines the Judge configuration and its current Admission. The configuration has not been declared changed, but the last accepted Admission relied on a reference population with few unsupported-action claims. The agreement was above the Project's threshold on that population. This remains a meaningful historical fact; it simply may not provide strong coverage of the newly important failure mode. An engineer should not describe the original Admission as worthless, nor as proof that every future failure class is covered.

Next the team uses independent Human Reviewer verdicts for a targeted reference set. The Judge disagrees on several cases, and a new accepted Admission for the configuration falls below the declared threshold. Under AQEF this record makes the configuration not admitted; the earlier admitted state does not remain available as a convenient substitute. If the relevant Gate requires admitted Judges, results from that configuration are blocking for the stated admission reason. The team can still preserve the product Conversations and seek sound reassessment under the applicable resolution rules.

Finally, the team compares historical Confidence with reviewed correctness. It finds that the Judge became especially overconfident on unsupported promises. That is calibration evidence; it does not replace the Admission outcome, and it does not itself decide whether every affected Aster response failed. The investigation separates four records: what Aster said, what the Judge said, what the human reference establishes, and what the release governance now permits. The separation is the practical value of the AQEF reliability model.

10.14 Choosing thresholds without pretending AQEF chose them

AQEF requires explicit Admission evidence but does not choose the same threshold for every Project. That is appropriate because the consequences of errors differ. A Judge used to triage routine FAQ quality may have a different acceptable risk profile from a Judge whose verdicts influence authorisation of customer refunds. The Project must decide what it can tolerate and express that decision, rather than attributing an unsupported threshold to the framework.

The selected chance-corrected statistic must also be named and suitable for the reference scheme. A small or severely imbalanced Reference Set may not support stable interpretation of every measure. The specification requires a value and an identifiable method, while the engineering responsibility remains to choose a measure that makes sense. The existence of a computed statistic should not end the discussion about the population, adjudication and criticality of errors.

Reference quality and threshold severity cannot substitute for one another. A very strict threshold applied to unrepresentative data might still admit a Judge that misses the most dangerous failures. A moderate threshold on a carefully designed set might provide different, more relevant information. The correct comparison depends on the risk and decision purpose. A numerical gate without a defensible sampling rationale is precise in its arithmetic but weak in its claim.

A management review should therefore hear both the implementation-level answer and the project-level judgment: was the Admission record produced and interpreted according to AQEF, and is the chosen reference and threshold adequate for the intended decision? The first is a Conformance-related question; the second remains a responsibility of risk governance. Treating them as one invites false certainty.

10.15 History, accountability and the limits of the claim

Reliability claims should identify their object and time. “This Judge is admitted” is shorthand for a stronger, more exact statement: this identified configuration's latest accepted Admission record, against this Reference Set version and the Project's stated threshold, records admitted status. Changing a prompt or model changes the identity that has to be assessed. A later non-admitted record changes the current state without erasing the earlier one.

A reference comparison can establish agreement on known labelled cases, not omniscience. A calibration analysis can reveal that Confidence behaves poorly over a population, not that every future verdict can be corrected by multiplying by a constant. A seeded control can reveal a current failure of an instrument for a particular class, not the correctness of the measured product. All three claims are valuable when their limits are preserved.

This chapter closes the conceptual transition from using an Oracle to qualifying one. The next chapter will investigate Seeded Controls as deliberate tests of the instrument, including positive and clean controls, invalidation and requalification. Chapter 12 will examine panels of Judges and why more votes are not automatically more independence. These topics extend the same principle: evaluation infrastructure must be assessed and governed rather than trusted merely because it produces formal-looking Results.

For Aster Support AI, confidence in release evidence comes from a traceable relationship between the quality criteria, the observed subject response, the instrument's demonstrated scope and the rules governing its use. The objective is not to eliminate uncertainty. It is to stop uncertainty from becoming an invisible assumption in a green dashboard.

10.16 When a Reference Set is changed

A reference collection is not an eternal truth source. Policies are amended, new failure modes become important and earlier labels may be discovered to rest on incomplete evidence. A responsible Project may need a different Reference Set version. However, the reasons for revising it should be explicit, because changing the measurement population can change observed agreement even when the Judge itself remains identical. Without this separation, a team may describe a tougher benchmark as regression in the instrument, or a newly easier benchmark as evidence of an improved Judge.

Consider Aster when its refund service expands into a new region. A reference collection designed around one country's policies might be accurate for its earlier purpose and insufficient for the new one. Adding regional policy conflicts changes what the Reference Set represents. A comparison based on the same Judge and the new reference is useful, but it answers a different sampling question from the earlier Admission. The investigator should preserve both versions and their contents, not revise old labels in place and call the results directly comparable.

There is also a distinction between correcting a genuinely wrong label and adapting a benchmark to the instrument. If a reviewer discovers that an item referred to an obsolete policy, correction may be justified, with an auditable reason. If the proposed change occurs merely because a Judge disagreed, it violates the spirit of reference independence. The recommendation that reference labels should not be updated based on Judge performance exists precisely to prevent assessment from becoming self-certification under a new name.

A new reference population can make Admission more representative while also changing how its statistic behaves. The raw agreement, class distribution and chance-corrected value should be read as a set. Thresholds chosen for an earlier population should not be transported uncritically to a different label scheme or risk distribution. The framework requires explicit records and leaves Project risk judgment to the decision-makers, not to the metric alone.

10.17 Difficult cases: ambiguous labels, non-computable values and missing evidence

Reliability analysis must sometimes acknowledge that the evidence is not capable of supporting a single clean numerical conclusion. A Reference Set may contain ambiguous policy language, a small number of rare defect cases or a label distribution that makes a chosen agreement measure difficult to interpret. Those conditions should be described rather than concealed. AQEF's Appendix C defines the fields and validation requirements of an Admission record; a textbook should not invent a convenient null score, an arbitrary substitute coefficient or an undocumented calculation when the chosen measure cannot be computed as intended.

The practical response is to investigate the cause of non-computability. Are there too few label categories represented? Have the labels been resolved into the intended scheme? Was a selected statistic appropriate for the observed distribution? Did the Reference Set version actually contain the items that the Admission report says were scored? Such questions are methodological, not programming exercises. Their purpose is to ensure that a numerical admission decision can be interpreted consistently by another reviewer.

Aster provides a clear example. Imagine a set in which all reviewers labelled every policy explanation pass. Some chance-corrected agreement measures become degenerate in such a population, regardless of whether a Judge also always predicts pass. The engineering finding is not that the Judge has demonstrated excellent ability to detect unsafe explanations; the set simply did not furnish the necessary contrast. A better reference design should deliberately include meaningful error classes and independent evidence about them rather than papering over an undefined mathematical situation.

An undefined chance-corrected measure is represented by the explicit value `not_computed`, never by null; the Admission record must indicate that the Judge configuration is not admitted. This is different from an unfinished human reference label: an unlabelled item is not a valid ground-truth verdict to score as either correct or incorrect. Aster’s team must establish the Reference Set basis before interpreting its agreement statistics.

One must also distinguish a missing reference verdict from a disagreement. If a human reviewer has not completed an item, there is no established correct label against which the Judge can be scored. Treating the unfinished item as a pass would inflate the population; treating it as a fail would confuse incomplete ground

truth with demonstrated product error. A review workflow should identify and resolve that state before the item contributes to the claimed Admission basis. The lesson is the same as in Chapter 9: lack of an instrument verdict is not a verdict against the subject.

The leadership implication is that quality indicators should be explained in words alongside their values. A Review Board needs to know how broad the Reference Set is, which cases are contested, whether the agreement calculation is defined, and whether the latest accepted record actually applies to the configuration used in the release. A number stripped of these facts can be technically well formatted and substantively misleading. Reliable governance depends on knowing what the number did and did not measure.

Engineering Reflection

A Judge has 94 percent raw agreement on an Aster Reference Set, yet nearly all its reference items are labelled pass. Which information must be inspected before concluding the Judge is reliable for unsupported-refund claims?

An Aster Judge was admitted last month but the team has since changed its assessment prompt. What evidence is required before the revised configuration contributes to a Gate that requires admitted Judges? Explain why merely copying the earlier admission is insufficient.

Common Misconceptions

1. High Confidence proves a correct verdict. Confidence is the Judge's uncertainty statement; correctness needs independent comparison.

2. Raw agreement alone proves Admission quality. Label imbalance can inflate raw agreement, which is why AQEF requires the distribution and a named chance-corrected measure.

3. Admission stays valid forever. Current status follows the latest accepted record for the identified configuration, and a failed newer Admission withdraws the earlier one.

4. Calibration is another name for Admission. Admission compares verdicts to reference labels for eligibility; Calibration studies whether claimed Confidence tracks actual correctness over history.

5. A missed Seeded Control proves the AI product failed. A control tests the Oracle's ability to detect a known defect class; its miss is an instrument-quality finding.

6. “We calibrated the Judge against our labelled development cases, so it is reliable.” That activity is tuning; its exposed cases do not replace independent Admission evidence or historical Confidence Calibration.

7. “The variant that agreed on one more case is necessarily the better Judge.” One result in 20 changes raw agreement by five percentage points, and a small working set cannot establish the independently tested Admission threshold.

Chapter Summary

An automated Judge is part of the measurement apparatus and requires evidence of its own reliability.

A Reference Set supplies human-established verdicts against which the Judge's results can be compared; independent review and recorded arbitration help protect its integrity.

Raw agreement can be misleading on imbalanced data, so Admission requires distribution information and a named chance-corrected measure.

Admission attaches to a particular Judge configuration; the latest accepted record governs admitted status without erasing history.

A Project may configure Gates to require admitted Judges, while Reports identify non-admitted Judges in scope.

Calibration evaluates the relationship between reported Confidence and observed correctness over time; AQEF does not prescribe a universal calibration formula.

Judge Drift and Seeded Controls address distinct temporal and failure-class questions and cannot substitute for Admission.

Reliable release claims depend on the exact reference, configuration, assessment history and applicable governance rule.

Tuning can improve agreement on development cases; only independent Admission determines whether the frozen configuration agrees sufficiently under the Project’s declared threshold.

Review Questions

1. Why can two Judges with the same raw agreement provide different evidence about their ability to detect high-risk failures?
2. How does independent human arbitration improve a Reference Set, and what would be lost if labels were revised to match the Judge?
3. What is the difference between agreement corrected for chance and a Judge's own Confidence?
4. Describe why a Judge that passes Admission on a narrow reference population might remain a risky choice for Aster's financial actions.
5. What happens when a later accepted Admission record fails the threshold after an earlier one passed?
6. How should a Project treat a Judge after its model, prompt or assessment criteria change?
7. Under what circumstance does an unadmitted Judge become a recorded reason that a Quality Gate cannot pass?
8. Why does Calibration need many historical comparisons, and what kind of current failure can it miss?
9. Compare the evidence furnished by Admission, Calibration and one Seeded Control result.
10. Propose a defensible investigation when Aster's reported semantic scores improve but independent reviewers find more unsupported claims.

11. A team revises a Judge until it agrees on 18 of 20 labelled working examples. What does the 90% score show, and what evidence must be gathered before its verdict is relied on by a Gate that requires admitted Judges?

12. A fresh 60-item Reference Set has 41 human-pass cases and 19 human-fail cases. The Judge returns pass on 35 of the human-pass cases and 5 of the human-fail cases. Calculate raw agreement, chance agreement and Cohen's κ. If the Project predeclared a κ threshold of 0.70, what is its Admission outcome?

Sources

1. AQEF 1.0-CR1, Volume VI — Judge Engine: Confidence, Admission, Calibration, Human Review and Judge Drift.

2. AQEF 1.0-CR1, Volume II — Human Reviewer and reference designations.

3. AQEF 1.0-CR1, Volume VII — Quality Contracts and Confidence thresholds.

4. AQEF 1.0-CR1, Volume VIII — Seeded Controls and reference integrity recommendations.

5. AQEF 1.0-CR1, Appendix A §A.10 — Quality Gate option require_admitted_judges.

6. AQEF 1.0-CR1, Appendix C §C.3 — Admission records and status resolution.

7. AQEF 1.0-CR1, Volume XI — Reports and Judge admission visibility.

Chapter 11 — Seeded Controls: Testing the Instrument

Maturity: Implemented (seeded_controls) · AQEF 1.0-CR1 · Textbook First Edition v1.0

Learning Objectives

After reading this chapter, the reader should be able to:

1. Distinguish a failure of the AI subject from a missed or falsely alarming Seeded Control that tests an Oracle.
2. Explain defect controls, clean controls, confirmation and the outcomes caught, missed, passed, false_alarm and invalid.
3. Reason about class-scoped invalidation, undeclared defect classes, skip_class-style assessment skipping and halt_run without rewriting historical Results.
4. Explain the purpose of independent generation, blind confirmation, difficulty priors and fresh controls used for requalification.
5. Trace how a corrected Oracle may re-assess stored Evidence while preserving original Result, Gate Result and Release Decision records.

11.1 The instrument must itself be tested

A quality assessment can be convincing and still be wrong. Chapter 9 distinguished the recorded behaviour of Aster Support AI from the Validator or Judge that interprets that behaviour. Chapter 10 went further: a Judge may agree with a human-labeled Reference Set often enough to be admitted, yet change behaviour later. Seeded Controls address this immediate, operational gap. They ask a different question from an ordinary Scenario: can the Oracle recognise a defect it is explicitly supposed to detect in the run being evaluated?

The distinction changes what a failure means. An ordinary negative Scenario probes whether Aster refuses or handles an impermissible request. If Aster performs the prohibited action, the failure belongs to the subject. A Seeded Control deliberately gives an Oracle evidence with a known answer. If the Oracle misses a defect in that evidence, the finding is about the instrument's ability to assess, not about Aster. Confusing these failure locations leads teams to correct the wrong system and sometimes to manufacture reassuring product scores from an unqualified assessor.

The capability *seeded_controls* is a scoped, independently implemented capability in AQEF 1.0-CR1. The chapter therefore explains its specified mechanics rather than presenting it as an obligation that every conformant AQEF implementation must claim. Its engineering value is broad, however: any organisation relying on measured quality has reason to ask what evidence demonstrates that its measuring tool works for the very errors it purports to find.

Imagine a Judge used to determine whether Aster's refund answers are faithful to policy passages. A model update makes the Judge increasingly permissive about invented exceptions. Ordinary Aster results may still look excellent. A planted example with a deliberately incorrect exception can reveal the Judge's new

blindness before those attractive results are used at a Quality Gate. Conversely, a clean example can expose a Judge that flags every reply as a defect. Both cases are tests of the assessor.

11.2 A control is a Scenario with a known answer

A Seeded Control retains the identity and lifecycle of a Scenario, but its purpose is to assess the Oracle rather than the product. A defect control contains a known planted defect of a named defect class; a clean control contains evidence known to be free of that class. The Oracle must not be told the desired answer as part of its assessment. Its verdict is compared to the confirmed property of the evidence, generating a control outcome that speaks to the instrument.

The mechanism is borrowed from fault seeding, yet its object is different. Classical seeded faults may be used to study whether a test suite detects known software defects. Here the seed is in the assessment evidence used to challenge a Validator or Judge. An Oracle that always says pass will miss planted defects. An Oracle that always says fail will catch defect controls while failing clean controls. A mixed set exposes both blindness and over-reporting, which is why neither kind of control is sufficient on its own.

The specification describes the intended purpose directly:

> *"Its purpose is not to test the system. Its purpose is to prove that an Oracle can still detect that class of defect in the current run" — AQEF 1.0-CR1, Volume VIII, Seeded Controls.*

The sentence must be read in its proper scope. A successful control is evidence for a particular Oracle, defect class, control design and run. It does not prove that the Oracle can identify every conceivable error, that the project selected the correct risks or that Aster is high quality. Controls establish a necessary instrument check within a bounded assessment, not an unrestricted guarantee of correctness.

11.3 Defect controls, clean controls and the five outcomes

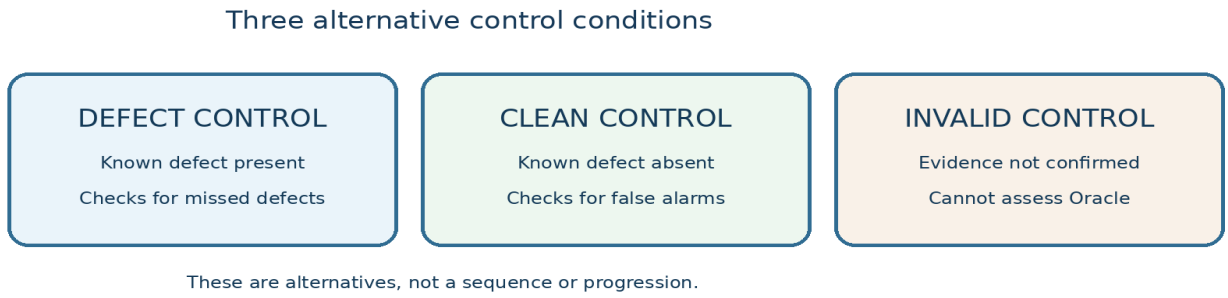


Figure 11.1 — Defect, clean and invalid control outcomes.

A defect control has a deliberately introduced violation, such as a policy statement asserting a refund entitlement the governing passage denies. If the planted defect is confirmed present in the evidence and the Oracle recognises it, the control is caught. If the same confirmed defect is present and the Oracle fails to recognise it, the outcome is missed. These outcomes are evidence about sensitivity to the defect class under examination.

A clean control reverses the premise. Its evidence has been confirmed free of the named defect class; the Oracle is expected to pass it. The outcome is passed when the Oracle correctly refrains from finding a defect, or false_alarm when it reports one anyway. An Oracle that flags every response would catch all defect controls yet fail every clean control. The two controls therefore address complementary failure modes, and the control record must retain which one occurred rather than reducing everything to a generic failure.

There is a fifth result: invalid. It describes a control whose precondition has not been confirmed. The planted defect may never have reached the evidence the Oracle saw; a clean item might not actually have been demonstrated clean. In either case, a pass or fail of the Oracle cannot be interpreted reliably. An invalid control is not a missed defect and is not proof of a reliable Oracle. It points first to the authoring, generation, capture or confirmation process.

AQEF treats a false alarm like a miss for the rules that govern invalidation, stopping assessment, quorum, requalification and Result Resolution, while retaining the distinctive reported reason. That is not a claim that false positive and false negative errors are equivalent for business purposes. It is a conservative operational rule: either failure means that this instrument has not shown itself trustworthy for the affected class in the current run.

11.4 Why a confirmed seed is indispensable

A seed placed in an input is not necessarily present in an Oracle's assessment evidence. Aster might refuse to repeat the seeded factual error, a tool may rewrite a payload, or a retrieval step may supply a different source than the one the control author expected. If the Oracle receives clean evidence and makes no defect finding, it would be wrong to call that a missed defect. The detector never encountered what it was supposed to detect.

The reverse error is more dangerous. If the planted defect never arrived but the Oracle nevertheless reports one, calling it caught would qualify an assessor that found something unverified. That is exactly the kind of self-confirming instrument assessment that AQEF aims to prevent. A control's outcome must therefore remain invalid until the intended condition in the actual evidence has been confirmed. The control cannot be made valid by the assessor's confidence.

Volume VIII provides three declared confirmation methods. With *validator*, a deterministic check confirms the defect's presence or the clean evidence's absence of it. With *human_reviewer*, an authorised person establishes that fact from the material the Oracle actually received. With *construction*, the control carries fixed, already-known evidence, bypassing the subject entirely. They are alternatives for establishing the premise; the assessment of the target Oracle remains a separate operation.

For Aster, a defect control might insert the claim 'all subscription purchases are refundable at any time.' If a live conversation rephrases that claim into a permitted explanation before the Judge sees it, the original seeded phrase is no longer evidence of a flaw in the Judge. The correct action is to label the control invalid and examine why construction or delivery did not preserve the intended condition, not to penalise the Judge for failing to identify an absent problem.

11.5 Confirmation methods and fixed evidence

Confirmation by a deterministic Validator is attractive when the planted fault is readily specified as a literal or structural property. A test may intentionally include a forbidden token, a specific numeric mismatch or an invalid field, and a separate deterministic check can establish that it appears in the recorded evidence. This is a bounded use of a Validator: it proves the seed is present, not that the target Judge understands the full meaning of the case.

Human confirmation is appropriate when presence or absence depends on semantic context. A reviewer might compare an answer with a policy passage to establish that an invented exception actually appears. The reviewer should inspect the evidence delivered to the Oracle, not only the control author's original intent. While that confirmation is pending, the control remains invalid. If confirmation arrives after a Gate Result, the control's reported state and subsequent evaluation may change, but the previously recorded Gate Result or Release Decision must remain in the historical record.

Fixed Evidence is different because the control supplies a Conversation and relevant Artifacts directly to Validation and Evaluation; the subject is not invoked. This avoids the risk that the seed disappears while the AI system processes it and can save execution cost. It also creates a recognition risk: artificial control evidence may look conspicuously unlike real output. AQEF therefore recommends deriving fixed evidence from recent real subject evidence in the same Environment, preserving a reference to its source Execution.

These are not four separate modes of testing Aster. They are ways to ensure that a control assesses its target instrument under a valid premise. When an input is known but the recorded evidence is not, the premise is unsupported. When the record is fixed and confirmed, the premise is demonstrable. That distinction has an immediate effect on whether a control outcome may disqualify an Oracle.

11.6 Defect classes determine the invalidation scope

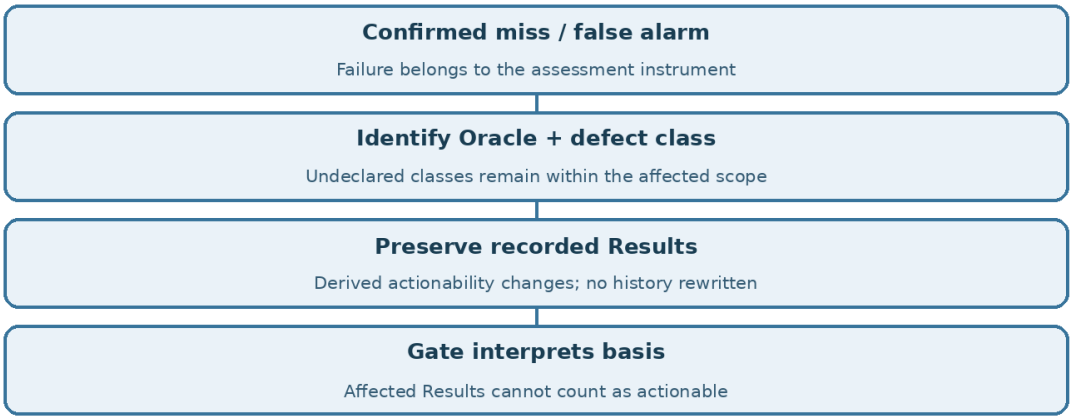


Figure 11.2 — Class-scoped instrument invalidation.

A failed control must not silently invalidate every observation produced by a testing organisation. AQEF ties the default consequence to a defect class and to the same Oracle within the same run. If the Judge misses a

planted policy-faithfulness error, the Results whose clauses depend on that Judge for policy-faithfulness detection lose their status as actionable for Quality Gates. Results concerning an unrelated class assessed by an independent Oracle are not automatically disqualified.

A Contract clause may declare which defect classes it checks. This is important because 'Semantic Quality' is too broad to say which errors a given test instrument has demonstrated sensitivity to. A Judge might detect unsupported dates but perform poorly on contradictory authorisations. Explicit classes allow an implementation to identify the actual assessment surface affected by a miss. If a clause declares no class, AQEF treats it as covering all classes tested for that Oracle; absence of a declaration cannot be used to escape a miss.

The original subject Result remains recorded. Invalidation is derived when a Quality Gate determines whether that Result can count as actionable; the system must not rewrite a previously issued pass into an inconclusive verdict. In particular, the special disposition *inconclusive* does not describe a deterministic Validator that has been disqualified by a missed control. It is the reliability of assessment that failed, not necessarily the original subject evidence.

The Project may deliberately choose full-run invalidation instead of the class-scoped default, but that is an explicit policy choice with a recorded reason. A broader response may be justified if a severe instrument failure calls other assessment outputs into question, but it must not occur accidentally through an implementation that cannot distinguish defect classes.

11.7 A clean control failure has the same blocking significance

It is easy to appreciate a miss: a dangerous defect existed and the instrument did not see it. A false alarm can initially seem safer because the Judge is being cautious. Yet an Oracle that produces false positives indiscriminately cannot support meaningful product decisions. It may block a release for failures the product never exhibited, waste investigation resources, and encourage owners to bypass quality Gates they no longer trust.

AQEF accordingly makes a confirmed false alarm operationally equivalent to a miss for consequences while reporting it separately as *seeded_control_false_alarm*. In the Aster example, a clean response accurately describing an exception-free return policy should not be flagged as inventing a policy exception. If the Judge finds an invented exception anyway, the problem is the Judge's ability to discriminate cases in that defect class.

Clean controls are recommended for Judges because probabilistic assessors can produce plausible negative findings just as they can produce plausible approvals. AQEF states that a Project SHOULD maintain at least one defect control for each Oracle feeding a Gate, and at least one clean control for each such Judge. It MAY also maintain clean controls for Validators. These are SHOULD/MAY provisions and must not be represented as compulsory counts for all implementations.

A carefully curated clean control should be genuinely free of the intended defect class, not merely judged harmless by the same Oracle being challenged. The confirming instrument must be independent enough to support the premise. If the evidence is disputed, the correct control state is invalid until its cleanliness is

established. Treating a doubtful clean case as authoritative would simply move the reliability problem upstream.

11.8 Failing fast: skip_class versus halt_run

Once a control exposes that an Oracle is unreliable for an affected class, continuing to invoke it repeatedly on that class may burn cost without producing actionable Results. AQEF therefore recommends that it SHOULD NOT be invoked for those clauses during the remainder of the run. The saved subject Evidence is retained, but the clauses that would have been assessed instead receive *not_assessed*, with no verdict. Other Oracles and unaffected classes can continue their work.

This response is conceptually close to early Validation short-circuiting but occurs for a different reason. In a product failure, a deterministic rule may end assessment because the subject violated a necessary Constraint. Here the assessment instrument has demonstrated that it cannot be trusted for the affected class, so further assessments with it are intentionally skipped. The outcome is not that every skipped Aster interaction failed, but that the testing process lacks a qualified measurement.

A Project can instead configure *on_seeded_control_miss: halt_run*. This ends the run and records full-run invalidation with the configured policy as its reason. It is an intentionally broader and often more expensive decision: unaffected assessment paths are also stopped. An organisation may prefer it for strong governance or serious instrument integrity concerns, but it must not be confused with the default class-scoped response.

For Aster, suppose the policy Judge misses a control while an independent structural Validator remains available. Under class-scoped skipping, the Validator can still identify malformed tool arguments, while policy-faithfulness assessments relying on the disqualified Judge stop. Under halt-run, the entire execution batch is stopped, even if some observations would have remained technically assessable. Reporting must reflect which policy was applied.

11.9 Why the original record must remain immutable

Recorded Results are historical evidence of what an instrument actually said at a particular time. If the Judge declared a pass before a Seeded Control exposed its blindness, deleting or rewriting that pass would erase the very information needed to diagnose the assessment problem. AQEF therefore separates the preserved verdict and disposition from the later determination of whether the Result is actionable for a Gate.

The result's effective treatment can change as supporting information arrives. A control may be confirmed later, a class invalidated, an Oracle requalified, or stored Evidence re-assessed. Each event adds a new record to the trace, rather than overwriting the previous record. The release-level interpretation may properly change, but the history that explains the change must still be reconstructable.

Immutability is especially important for explaining earlier decisions. Suppose Aster was approved for deployment when a Judge's results appeared actionable. A later investigation establishes that a control had been invalid and then confirms a miss. The organisation should be able to record the new finding and reconsider risk. It must not silently change the old Release Decision to pretend that the later fact was known at the original decision time.

The system's responsibility is to preserve both what was known and what became known later. This principle also prevents a convenient re-evaluation from hiding an instrument defect: the initial failure remains visible even after a corrected Judge produces a defensible new assessment.

11.10 Exposure, generation and independence

An Oracle can learn to recognise the signature of a control rather than the defect the control was supposed to represent. If a Judge has been repeatedly tuned against a small visible set of planted examples, high control success might demonstrate memorisation of those examples. The same problem arises when an automatically generated control has a conspicuous format or wording unlike ordinary Aster Conversations. Such cases inflate trust without measuring transferable competence.

Volume VIII therefore recommends generating controls and varying their surface form. Individual controls should not be visible to the people tuning the target Oracle, and an exposed control should not be reused to gate that Oracle. Hand-authored controls remain allowed, but separating authorship and tuning responsibilities is valuable because the author has necessarily seen the item. These recommendations are about avoiding leakage of the answer into the instrument under test.

Blind confirmation also matters. A Human Reviewer who knows that an item is a trap may overinterpret ambiguous evidence, confirming a planted defect that is not really present or treating a difficult clean item as defective. Where practical, reviewers SHOULD be unaware of whether an item is a control and of its kind. The objective is not secrecy for its own sake but independence of the evidence used to qualify the Oracle.

For Aster, a good planted faithfulness error can be expressed in many ordinary customer-service styles. The policy fact might be misstated in a sentence, buried in an explanation, or implied by a suggested next action. A useful control population probes recognition of the substantive failure, not recognition of a recurring introductory phrase inserted by the generator.

11.11 Difficulty priors without invented certainty

Some defect classes are harder to detect than others. An Oracle missing a simple, unambiguous contradiction is not equivalent evidence to missing a subtle policy ambiguity on which independent reviewers regularly disagree. AQEF allows a difficulty prior to describe how reliably independent reference Oracles catch a defect class, providing context for interpreting control results. This does not turn a missed control into a product defect; it helps describe what the miss says about the instrument.

The prior belongs ordinarily to the defect class rather than to an individual generated control, because generated items may be used only once. A reused control MAY have its own prior that takes precedence for that instance. AQEF requires the prior to be based on independent reference Oracles or Human Reviewers, and not updated from the target Oracle's own performance. Otherwise the instrument would influence the benchmark intended to judge it.

A prior must report how many instances support it, and absent history must be represented as 'not yet established' rather than zero. These meanings differ: zero would suggest that relevant historical trials occurred and yielded a particular result, while an unestablished prior means the evidence base does not yet exist. The distinction is the same honest-denominator discipline that AQEF applies to other measurements.

If no reference Oracle catches a proposed defect control, the control itself may be badly designed rather than exceptionally difficult. The specification says it SHOULD NOT gate an Oracle until a Human Reviewer confirms it. A difficulty prior is therefore not a license to excuse unreliable measurement, nor a formula that automatically converts control outcomes into Gate thresholds.

11.12 Requalification is not the same as re-assessment

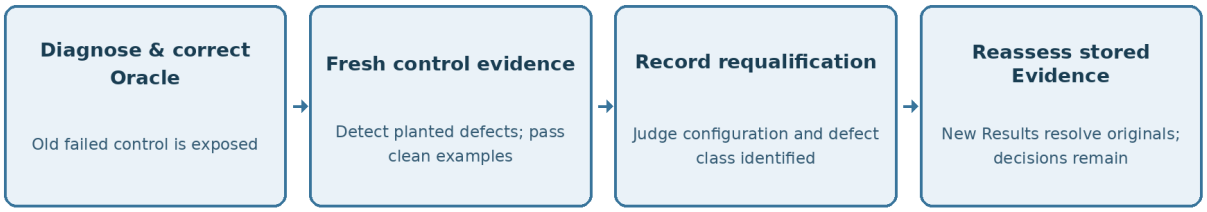


Figure 11.3 — Requalification before re-assessment.

A missed or falsely alarming control disqualifies the target Oracle for the relevant defect class until the Oracle is requalified. Requalification is not obtained merely by calling the same instrument again or repeating the control it failed. The cause must be diagnosed, the Oracle corrected where necessary, and the corrected configuration shown to catch fresh defect controls or pass fresh clean controls for the affected class. The earlier control is exposed through investigation and cannot serve as an independent new qualification test.

A requalification record identifies the Oracle configuration, defect class and the fresh control Results used to establish the renewed capability. Without that record the Oracle has not been requalified for AQEF's purposes. Where both a miss and false alarm occurred, the corrected Oracle must demonstrate both the ability to detect defects and the ability to refrain from inventing them in clean evidence.

Re-assessment is a separate downstream possibility. Once requalified, the corrected Oracle MAY examine stored Evidence from the affected run, without asking Aster to perform the original Scenario again. This produces new Results attributed to the corrected configuration. Those Results can resolve invalidated or *not_assessed* earlier assessments, but the earlier records remain unchanged and a previously recorded Gate Result or Release Decision remains in the audit history.

The distinction protects against selective repetition. A genuinely failed Aster Scenario should not be rerun silently until the subject happens to give an acceptable response. Similarly, a faulty Judge should not be asked repeatedly until it provides a desirable verdict. Requalification proves a corrected instrument against new independent controls; re-assessment then applies that instrument to previously preserved evidence.

11.13 Seeded Controls alongside Admission and Calibration

Admission, Calibration and Seeded Controls are often grouped under Judge reliability, but they use different time horizons and different evidence. Admission compares one configuration's verdicts with a human-labeled Reference Set to decide whether it is admitted. Calibration looks across historical Results to see

whether declared Confidence tracks observed accuracy. Seeded Controls test a currently operating Oracle's sensitivity and specificity for a defect class in a particular run.

An admitted Judge can miss a Seeded Control. The Reference Set may not have exposed its new weakness, the provider may have changed behaviour, or a configuration may have drifted. Conversely, a Judge can pass a handful of controls while still being insufficiently reliable across the Reference Set needed for Admission. A fresh control pass is not a shortcut around an explicit Gate requirement for admitted Judges.

Calibration cannot necessarily catch a sudden failure before current-run damage occurs, because it works retrospectively. Seeded Controls can catch a current problem early, but only in the defect classes they examine. This is why the three mechanisms complement rather than replace one another. They provide a richer reliability argument by asking different questions at different stages.

For Aster, a release review might show a properly admitted policy Judge whose Confidence has historically been well calibrated. If it now misses a planted unsupported-claim control, the run's affected policy Results must nevertheless lose actionability under the seeded-control rules. A healthy historical record does not erase a newly demonstrated measurement failure.

11.14 The Aster investigation: one miss, three decisions

Consider an Aster release candidate that produces a set of return-policy answers and tool traces. The Project includes a defect control in which a recorded answer asserts an exception absent from the attached policy passage. A confirming method establishes that the seeded claim is present in the evidence the Judge received. The Judge returns pass. The control outcome is missed; that is a failure of the assessment instrument for the declared class.

The first decision is about scope. Results assessed by that Judge for the affected policy-faithfulness class become non-actionable at relevant Gates, while separate tool-schema Results from another Oracle may remain usable. The Project could have selected full-run invalidation, but that policy must be explicit and justified. No historical Result is rewritten. The original Aster answer still exists, and the Judge's recorded opinion still exists.

The second decision concerns operation. Under the normal class-scoped fail-fast recommendation, the disqualified Judge stops receiving the affected clauses, which become *not_assessed*. A run configured to halt instead stops the batch, with the broader invalidation recorded. Neither response means that all outstanding Aster Scenarios failed. They mean that the planned assessment cannot be completed with the current instrument.

The third decision is recovery. The team diagnoses the Judge, adjusts its configuration if warranted and tests it against new controls not used in the diagnosis. Once requalified and recorded, the corrected Judge can assess stored policy Evidence. A later Gate evaluation can consume those effective new Results, while the prior release-level conclusions and their original basis remain visible. This is a disciplined sequence from evidence of instrument failure to a new defensible assessment.

11.15 Limits and responsibilities

Seeded Controls are powerful because they test a claim that is usually assumed: the assessor is capable of recognising known errors. They cannot prove universal competence. A control population may underrepresent rare linguistic forms, nuanced policy conflicts or emerging attack patterns. Its difficulty priors may be immature and its clean cases may require careful human confirmation. These are genuine limits rather than reasons to abandon the mechanism.

The quality of the controls themselves must be governed. Roles and Permissions are relevant to who authors, exposes, confirms and approves them. The independence of the confirming evidence matters; so does the preservation of historical failures. A project should state why particular defect classes merit controls and how control results influence Gate decisions, rather than treating seeded testing as a decorative score on a dashboard.

The most important operational lesson is to keep the objects of judgment separate. A missed control is not an Aster defect, a successful control is not proof of Aster's quality, and a repaired Judge does not retroactively erase earlier assessment uncertainty. By separating the subject, the instrument and the evidence governing its qualification, AQEF permits release decisions to be cautious without becoming arbitrary.

The next chapter examines Multi Judge designs, where instrument integrity becomes a panel question. When one member misses a control, the panel must respect its qualification rules and any declared quorum; agreement among the surviving Judges cannot be assumed to replace an excluded member automatically. Seeded Controls therefore form a bridge between individual Oracle reliability and collective assessment governance.

11.16 A release-level evidence audit after a control failure

A release report must describe not only the number of assessment Results but also whether those Results remain actionable when the seeded-control evidence is considered. An attractive pass rate computed before a control miss may have a very different evidentiary meaning after the miss is confirmed. The appropriate response is to revisit which Results the Gate can count, not to subtract or add failures to an aggregate without explaining the change in basis. A derived invalidation can reduce the actionable population while leaving the historical verdicts untouched.

Suppose Aster produced thirty successful policy explanations and ten successful tool calls. An independent structural Validator assessed the tool-call schema; a semantic Judge assessed policy faithfulness. The latter missed a confirmed defect control. The tool-schema Results do not thereby become product failures; if they are outside the missed class and assessed by another Oracle, their evidence may remain usable. The policy-faithfulness Results are different: the team's historical records still show the Judge's earlier verdicts, but a release Gate may no longer treat them as actionable. Reporting them as thirty trustworthy passes would ignore the very evidence showing that the instrument could not detect a known error.

A responsible audit reconstructs the affected scope from three questions. Which Oracle configuration was challenged? Which defect class was confirmed, and which Contract clauses depend on that class? Was a class-scoped response or a documented full-run policy applied? These questions prevent both overreaction and complacency. Without them, an implementation might throw away all evidence, falsely diagnose

product failure, or keep using disqualified assessments because their stored verdict field remains *pass*. The point of actionability is that an immutable recorded statement and a permissible decision input are not the same object.

Seeded Control outcomes evaluate the Oracle, not Aster's product behaviour. They must therefore not be counted as ordinary subject Results in quality aggregates or used to inflate a Quality Gate's actionable assessment basis. A control miss instead affects the trust and applicability of the ordinary Results in its defined scope.

The audit should also preserve chronology. If the control's confirmation arrived after an earlier Gate evaluation, the earlier decision was based on the information available then. Later correction can justify a new evaluation and new Release Decision, but the prior record must stay visible. The historical record makes it possible to distinguish an initial measurement defect, its discovery, the organisational response and the final qualified evidence. A textbook that calls only the final result 'truth' misses the operational value of retaining those differences.

Finally, coverage of controls should be interpreted against risk. If all controls probe easy structural defects but a release depends on subtle business-policy interpretation, the controls provide little assurance about the higher-stakes Oracle. The project may choose to expand its control classes and independent confirmation, but AQEF does not promise that passing a small collection makes the instrument infallible. The defensible conclusion names the controlled defect classes, the qualifying evidence and the remaining unknowns.

Engineering Reflection

The Judge reports pass on a control, but the generator's planted defect was absent from the Evidence that reached it. What conclusion can the team draw about the Judge, and what must change before the control is useful?

A clean control produces a false alarm after thirty ordinary Results from the same Oracle have already been recorded. Which records remain immutable, which Results can count for Gates, and what qualifies a later reassessment?

Common Misconceptions

A missed Seeded Control means Aster failed. The missed control is evidence about the Oracle, not an independent defect finding about Aster.

A defect control alone proves an Oracle is reliable. An Oracle that reports a defect in every case catches every defect control; clean controls are needed to expose false alarms.

A control that did not reach the target Evidence should count as missed. The correct status is *invalid* until the planted or absent property is independently confirmed.

All Results in a run must be discarded after any control miss. Default invalidation is scoped to the same Oracle and defect class. Full-run invalidation is an explicit policy with a reason.

Requalification simply means retrying the failed control. Fresh controls and a recorded correction or qualification are necessary; the failed item has been exposed.

Chapter Summary

Seeded Controls challenge the quality of a Validator or Judge rather than measuring the quality of the AI subject.

Defect controls assess defect detection; clean controls assess avoidance of false alarms.

The five outcomes distinguish correct detection, missed detection, correct non-detection, false alarm and invalid premise.

Confirmation must establish that the relevant fact is present or absent in the Evidence actually given to the target Oracle.

A miss normally invalidates actionability only for Results of the same Oracle and affected defect class; original Results remain preserved.

Fail-fast behaviour stops affected assessment while retaining Evidence; a configured halt can extend invalidation to the full run.

Control independence, exposure and difficulty priors affect how much evidence a control result provides about the instrument.

Requalification uses fresh controls; subsequent re-assessment of stored Evidence creates new records without erasing earlier results or decisions.

Review Questions

1. What does a missed Seeded Control demonstrate, and what does it fail to demonstrate about Aster?
2. Why must a project use clean controls as well as defect controls for a semantic Judge?
3. How should the control outcome be recorded when a planted defect disappeared before the target Oracle saw the evidence?
4. Explain the difference between a class-scoped invalidation and on_seeded_control_miss: halt_run.
5. A clause declares no defect_classes. How does that affect its susceptibility to a miss by the Oracle assessing it?
6. Why does AQEF preserve the original verdict and disposition when a control invalidates that assessment's actionability?
7. What distinguishes an exposed fixed-evidence control from a fresh one, and why does that difference matter?
8. How does a difficulty prior help interpret a miss, and why must it not be derived from the target Oracle?

9. An Oracle has been corrected. What evidence must a requalification record contain before it can reassess affected stored Evidence?
10. How can a Judge be admitted and calibrated historically yet fail a Seeded Control during the present run?

Sources

1. AQEF 1.0-CR1, Volume VIII, Seeded Controls — kinds, outcomes, coverage, invalidation, confirmation, difficulty prior, requalification.
2. AQEF 1.0-CR1, Volume III, Execution Pipeline — fixed Evidence for Seeded Controls.
3. AQEF 1.0-CR1, Volume VI, Consensus, Admission, Calibration and Human Review.
4. AQEF 1.0-CR1, Volume I, Chapter 6, Result Resolution and Aggregation.
5. AQEF 1.0-CR1, Appendix A §§A.4, A.5, A.9 and A.12, and Appendix C §§C.3 and C.7.
6. AQEF 1.0-CR1, Front Matter §0 and §10 — capability maturity and Conformance scope.

Chapter 12 — More Than One Judge

Maturity: Implemented (multi_judge) · AQEF 1.0-CR1 · Textbook First Edition v1.0

Learning Objectives

After this chapter, the reader should be able to:

1. Distinguish Multi Judge configured for independence from repeated sampling configured for stability.
2. Explain Consensus, Voting and the meaning of a declared voting threshold without confusing agreement with accuracy.
3. Calculate a qualified panel's denominator and explain how Seeded Control exclusions affect quorum.
4. Explain why a Multi Judge panel is admitted only when each member configuration is admitted.
5. Trace disagreement to Human Review and interpret the panel's effect on Aster Support AI's assessment evidence.

12.1 Why one Judge may not be enough

A Judge is an assessment instrument, not an infallible witness. It can misinterpret a policy, overlook a qualification in retrieved context, or give the right verdict for the wrong reason. Chapters 9 through 11 established three ways to question its reliability: understand its mechanics, qualify its configuration against human reference judgments, and challenge its current behaviour with Seeded Controls. Multi Judge introduces another possibility: involve more than one assessment process in evaluating the same Conversation. The immediate gain is not necessarily a better score. It is the opportunity to learn whether the proposed judgment survives a different perspective.

For Aster Support AI, imagine an answer that correctly states a refund deadline but implies that a customer has already received money. A Judge trained to attend to policy dates might mark the answer as acceptable; another, independently constructed Judge might notice the unsupported claim of completion. Their disagreement reveals uncertainty about a property the Contract may need to distinguish. Combining several opinions without examining the source of disagreement would conceal precisely the new evidence the panel supplied.

Multi Judge is a scoped Implemented capability in AQEF 1.0-CR1, not a requirement that every implementation run many Judges for every Scenario. Its design is governed by a more basic principle: Independent Oracles. Yet multiple executions of an instrument do not automatically create independence. The purpose of the panel must be explicit and its composition meaningful. Otherwise complexity increases while evidence quality remains almost unchanged.

The central question of this chapter is therefore not how to make a vote. It is what a panel can legitimately claim: what its members measured, why those members were chosen, how many qualified members actually counted, what their disagreements meant and what should happen when the decision is not supported by enough reliable assessment.

12.2 Two purposes: independence and stability

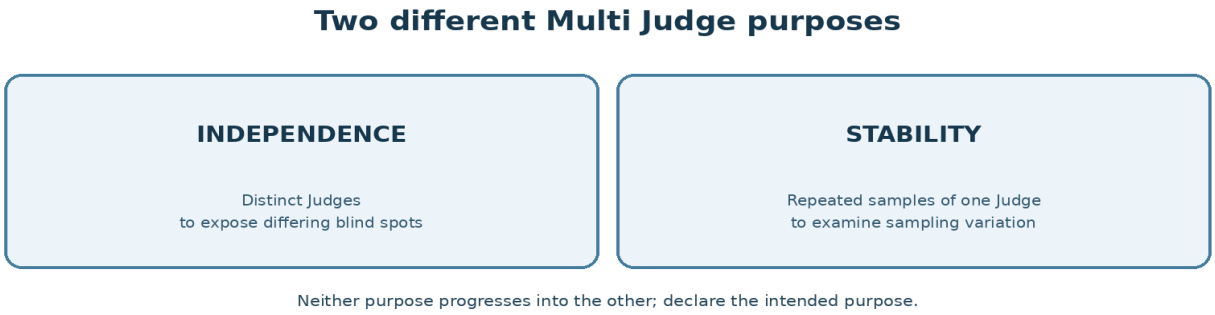


Figure 12.1 — Independence and stability answer different questions.

A multi-Judge arrangement may be configured for **independence** or for **stability**. Independence uses genuinely distinct Judges, for example different model families or meaningfully different criteria formulations, to reduce shared blind spots. Stability repeatedly samples a single Judge configuration to observe variability in that configuration's own verdicts. Both are sensible questions. They are not the same question, and their evidence must not be reported as though it were interchangeable.

The specification makes this distinction normative:

> *"A Multi Judge configuration MUST be explicit about which it is providing." — AQEF 1.0-CR1, Volume VI, Multi Judge.*

If three independent Judges disagree over the same refund policy clause, the disagreement can be evidence that the Conversation is semantically difficult or that criteria are ambiguous. If three runs of the same Judge disagree, the variance more directly indicates that this Judge's decision process is unstable under repeated sampling. The second finding may motivate investigation of the instrument's consistency, prompts or Confidence behaviour. It does not establish that three separately developed perspectives saw the same case.

Aster's team might sample its existing Judge five times after noticing fluctuating decisions on customer intent. That can estimate the practical stability of that Judge for the tested evidence. Alternatively, the team could compare two independent model families under the same Contract clause. That may expose correlated reasoning defects differently. A report should identify the chosen purpose before presenting a consensus score, because a repeated sample is not a new independent expert.

12.3 Independence is an engineering property, not a headcount

Counting models can be misleading. Three deployed endpoints may use the same underlying model family, near-identical assessment instructions and the same retrieved context; their agreement can be highly correlated. They may all fail to notice a policy exception precisely because their representations of the policy are similar. An independence-oriented panel seeks meaningfully different failure mechanisms, not merely different network addresses.

Evidence of independence is stronger when reviewers can identify different failure pathways, assessment evidence or criteria and compare disagreement patterns across difficult cases. Distinct model labels, vendors or endpoints alone are not measurements of error independence, and correlated failures may remain even when all panel members agree.

This does not mean independence must be absolute. Models can share training corpora, interface conventions and industry assumptions without being literally identical. The practical engineering challenge is to understand plausible shared failure modes and to avoid claiming an independence level that has not been demonstrated. Different criteria phrasings, distinct model families and human reference judgments can provide useful diversity, but each should be evaluated for the specific defect class involved.

For Aster, a panel evaluating unsupported action claims might include one Judge focused on the literal tool trace and another that compares the customer-facing statement to the recorded tool outcome. Both still assess an explicit Contract, but they approach the relation from different evidence emphases. If both make the same wrong assumption about what a successful refund API call means, the team has a shared conceptual blind spot despite surface diversity.

Independence is therefore tied to design review. A panel description should state which failure modes diversity is expected to address, what evidence members can inspect and whether their judgments are meaningfully distinct. Those are sensible explanatory questions, not a new AQEF numerical independence index. AQEF does not prescribe a universal number of models or a fixed percentage of model-family diversity.

12.4 A panel still evaluates one Contract

A multi-Judge panel does not replace Quality Contracts with a collection of personal preferences. Each member assesses the relevant Conversation or Artifacts against criteria the Contract makes explicit. If one member checks fidelity to retrieved text and another checks customer empathy, their outputs are not alternative votes on a single undifferentiated claim. They are different quality properties and should be represented through distinct clauses or a clearly defined panel assessment.

Suppose Aster gives an answer that is polite but states the wrong deadline. The panel cannot reasonably average politeness with policy correctness and then assert that the deadline is correct. A Contract that treats policy faithfulness as a hard requirement must preserve that obligation. The existence of multiple evaluators does not authorise the implementation to weaken the clause, invent new criteria or alter a declared failure rule.

Panel design should follow the intended use of evidence. If the goal is to assess whether Aster grounded its answer in the retrieved policy, members should have the policy passage or its relevant Artifact. If it is absent, a panel of ten Judges does not manufacture the missing provenance. Several instruments looking at incomplete evidence can agree and still be unable to answer the project's actual question.

The disciplined reading is to separate three things: individual member verdicts, the configured mechanism for combining qualified member outputs and the downstream Gate's use of the resulting assessment. Multi Judge applies in the Assessment layer. The Project's final release choice remains a Decision-layer and governance question.

12.5 Consensus does not erase disagreement

Consensus describes how the eligible Judge Results are reduced to a panel outcome. It can use voting over discrete pass/fail judgments or a specified method of combining scores. What matters is that the mechanism is declared and that its interpretation respects the purpose of the configuration. Agreement among independent members can strengthen a finding within a defined scope; disagreement can be equally valuable evidence of ambiguity, inconsistent criteria or uncertain subject behaviour.

A common mistake is to average away disagreement as though the only goal were a tidy number. Imagine two independent Judges agreeing that Aster's refund answer is accurate and a third reporting that it falsely asserts an action was completed. A simple favourable majority may be permitted by a chosen voting rule, but an engineer still needs to understand whether the dissent concerns a distinct high-impact clause. If it does, the Contract's design or human escalation policy may need attention. The vote does not automatically settle every normative or business concern.

AQEF treats unresolved disagreement as a legitimate Human Review trigger; routing to Human Review is the default when neither side reaches the declared voting threshold. This is consistent with the wider principle that uncertainty about an assessment must remain visible rather than being renamed as subject success. A reviewer can inspect the original Evidence, the panel criteria and the source of disagreement before recording an effective assessment.

The presence of a consensus mechanism is not proof that a result is well founded. A panel composed of unadmitted or temporarily unqualified instruments cannot gain credibility simply by agreeing. Qualification and quorum must be examined before the final count is interpreted.

12.6 Voting: thresholds and honest denominators

Voting is the most transparent consensus mechanism when Judges issue discrete pass or fail verdicts. AQEF allows *majority*, *unanimous*, or a stated fractional threshold greater than one half and at most one. The threshold is applied to the **number of eligible verdicts actually counted**, not automatically to the number of configured members. The number required is the ceiling of the threshold multiplied by that counted population.

The distinction becomes concrete with five counted members and a threshold of 0.75: four matching votes are required. Three matching votes are insufficient even though they form a numerical majority. With a majority threshold and five counted members, three matching votes suffice. A fraction of 0.5 or less is not allowed because both sides of a split vote could then meet it simultaneously. These are defined requirements, not suggested heuristics for Aster.

A panel might be configured with five members but count only four because one Judge missed a Seeded Control for the affected defect class. A threshold of 0.75 then requires three agreeing votes among the four qualified members, provided the panel still meets its quorum. Saying 'three of five agreed' obscures both the actual denominator and the reason a member was excluded. AQEF requires panel Results to state how many Judges were configured and counted.

Reports must also avoid confusing vote proportion with Confidence or correctness. Four votes out of five is a descriptive statement about agreement in the eligible panel, not an empirically measured 80% chance that the answer is correct. Even unanimous voting cannot reveal a blind spot shared by every member.

12.7 Quorum after a failed Seeded Control



Figure 12.2 — Configured membership, qualified count and quorum.

Quorum asks how many qualified members must remain before the panel is allowed to form a Result for the affected clauses. AQEF allows a Multi Judge configuration to declare *min_qualified*. If the number of qualified members is at or above that value, Consensus can proceed using those members. Below it, the panel's Results for the affected clauses become *not_assessed* because a qualified assessment did not take place.

Without an explicit quorum, the full configured panel is required. This default is deliberately conservative: if any member is excluded after a missed control, the panel cannot quietly proceed as though the excluded member had never been part of the configured assessment. A user-specified quorum can allow a smaller eligible group to proceed, but it is an explicit design decision rather than an automatic convenience.

An odd-sized panel is not guaranteed to produce an odd-sized eligible population. Three Judges may become two after one exclusion, exposing the possibility of a tie. If the two remaining Judges disagree and neither side meets the declared threshold, the disagreement policy applies. The engine must not select a winner solely because the original panel was intended to have an odd number of members.

Stability configurations have a further detail: if repeated samples of one Judge are used and that Judge misses a control, every sample of the disqualified Judge is excluded. Multiple samples cannot be used to outvote a demonstrated failure of their common instrument. Panel accounting in that mode counts samples configured and counted, not fictional independent Judge identities.

12.8 Admission is a separate gate on reliability

Chapter 10 established Admission as an assessment of one Judge configuration against a human-labeled Reference Set. Multi Judge inherits that obligation for its members. The panel is admitted only when every Judge configuration in it is admitted. A panel cannot obtain admission by combining one trustworthy member with another that failed its own reference comparison.

This does not mean every Project automatically requires Admission for every Gate. AQEF allows the Project to use *require_admitted_judges* on a Quality Gate. Where it is required, a Result from a Judge configuration that is not admitted blocks that Gate and records *judge_not_admitted*. Whether or not a Gate requires admission, Reports identify the Judges in scope that have not been admitted. Engineers should distinguish the panel's admission status from its current-run quorum and from a particular vote.

Aster may have a three-member panel whose all members passed Admission yesterday. Today one member misses a Seeded Control. The panel's historical admission rule and its current-run eligible count answer different questions: Admission concerns demonstrated agreement with reference judgments; control qualification concerns whether a Judge is currently usable for a defect class. The panel may still be admitted in the historical sense while the current-run quorum rule prevents a result.

Changing a member's model, instructions or assessment criteria creates a different Judge configuration for Admission purposes. Replacing an unreliable member without assessing the new configuration does not preserve the old member's Admission merely because the rest of the panel remains unchanged. The statement of what is admitted must track the actual participants.

12.9 Confidence remains a separate statement

A Judge's Confidence represents its own uncertainty in its verdict. It is not the number of other Judges agreeing, the business importance of the property, or the accuracy of the final panel. AQEF permits several methods for producing it, including self-reported or probability-based estimates, but requires that the origin and interpretation remain distinct from consensus signals.

For a panel, the degree of agreement can serve as additional information about the stability or divergence of its assessments. It must not silently replace the Confidence that a Judge is required to supply. High agreement and high Confidence may coexist with correlated error. Low agreement can occur even when each member reports high Confidence, indicating that at least some confidence statements deserve investigation.

An illustrative Aster case makes the point: two Judges confidently approve a reply while a third confidently rejects it for an unsupported tool-action claim. Reporting an average confidence number might hide that the members disagree about a consequential fact. A suitable Review could be more informative than a refined decimal aggregate. The proper path depends on the Contract's explicit rules, not an after-the-fact preference for a clean-looking report.

This separation also prevents accidental misuse of historical calibration findings. If one member's Confidence has previously aligned with accuracy, that supports its calibration argument. It does not make all dissenting members wrong or justify overriding the configured quorum. Reliability mechanisms supply different kinds of evidence and must retain their own meanings.

12.10 Human Review at the disagreement boundary



Figure 12.3 — Disagreement can lead to recorded Human Review.

Human Review enters when the declared workflow calls for a person to resolve ambiguity or disagreement. It is not a hidden tie-breaking algorithm. AQEF's Human Reviewer is an Oracle with its own recorded assessment, governed by the same principle that decisions must be traceable and that earlier findings are not overwritten. When disagreement is routed to a reviewer, the pending state is represented honestly while the decision remains open.

The reviewer needs to examine the evidence, the applicable Contract and the individual panel assessments. A single fused score is insufficient if it conceals what members disagreed about. If one Judge believed the retrieved source authorised a refund and another identified an exception, the review should inspect the policy version and citation context. The reviewer's recorded reasoning can then be used for later diagnosis of Judge quality and Contract clarity.

A disagreement may reveal that the Scenario is underspecified rather than that one instrument is clearly defective. The right response could be to clarify the Contract under governance for future assessments while preserving the interpretation used for the recorded Execution. Retroactive rewriting of criteria to make a past result comfortable would break the evidence chain. The existing evaluation can remain pending or be resolved through an authorised process.

Human Review does not establish that the panel members were independent, nor does it automatically repair a failure of Admission or a Seeded Control. It can resolve a particular assessment in a governed way. Those mechanism-level problems require their own evidence and corrective action.

12.11 Aster case study: apparent consensus, fragile evidence

Consider a Suite testing how Aster responds when a customer requests a refund but the order-lookup service returns inconsistent information. The Contract requires Aster to avoid claiming that money was returned unless the tool outcome supports that statement. Three independent Judges inspect the same Conversation and tool trace. Two vote pass because the customer-facing response is polite and conditional; the third votes fail because a sentence implies completion without a confirmed tool outcome.

If the configured threshold is majority and all three members are eligible, the panel can produce a pass verdict under the declared voting mechanism. That fact, however, should not be confused with proof that the claim about tool completion is supported. The dissent is a useful signal that either the clause combines

distinct criteria or the reviewers interpreted 'completion' differently. The team should assess whether the Contract needs better decomposition rather than merely celebrating two favourable votes.

Now suppose the dissenting Judge had been excluded after missing a Seeded Control for the very class of unsupported action claims. If the panel has no declared quorum, any exclusion is below the full-panel requirement, so the affected panel Result is *not_assessed*. With a declared quorum of two, the two eligible members can be counted, and the threshold applies to two, not to the original three. That narrower result must reveal the changed denominator. It does not acquire the weight of a full three-member assessment through omission of the exclusion.

The same example shows why the property being tested matters more than the number of opinions. Aster's final response might be linguistically cautious yet still materially misleading about an external action. Panel composition, criteria and escalation should all be designed around the actual risk. A Release Decision then reads the resulting actionable evidence through Quality Gates; it is not a vote among models about whether the product should ship.

12.12 Failure modes of panels

A panel can fail through correlated blindness, ambiguous criteria, unstable sampling, insufficient qualified members, disagreement without an honest disposition or overinterpretation of agreement statistics. These failures are different. Adding more members helps little when all rely on the same unsupported assumption. Repeatedly sampling the same Judge does not correct a defective Contract. Loosening the vote threshold after a disappointing result may alter the policy rather than improve the evidence.

Panels also introduce an operational cost. Multiple evaluations consume more time and resources, and they can multiply the volume of records requiring interpretation. Cost is justified when the added assessment perspective addresses an important failure mechanism. Where a deterministic Validator can establish the relevant property, using several probabilistic Judges might be a less efficient and less defensible way to reach the same result.

One practical principle is to investigate the denominator before interpreting the outcome. Ask how many members were configured, how many were counted, why any were excluded, what threshold applied and whether the surviving members have an adequate admission basis. An apparently unanimous two-of-two judgment may be a legitimate consensus under a configured quorum; it is not identical to a full three-of-three judgment.

A second principle is to treat disagreement as a diagnostic signal. It can indicate an ambiguous Contract, a difficult subject response, inconsistent evidence emphasis or an unreliable Judge. These hypotheses lead to different corrective actions. Consensus machinery should preserve enough detail to tell them apart.

12.13 Scope and governance

AQEF does not require Multi Judge for every Scenario. A Project chooses when the additional perspective is worth its complexity, and its Contract makes the relevant criteria and policies explicit. An implementation claiming *multi_judge* must honour applicable requirements; an implementation without the scoped capability is not automatically nonconformant on that account.

Governance remains necessary when panel choices affect release interpretation. A threshold changed from unanimous to majority is not a neutral code optimisation: it changes what evidence can count as sufficient consensus. The same is true of lowering quorum or replacing a member with a differently configured Judge. Such changes must be considered in the context of the project's risks and approvals rather than being introduced invisibly to make a Gate easier to pass.

Reports should preserve the criteria and actual eligible count, not only the final verdict. The panel's admission status, Seeded Control exclusions and Human Review outcomes each add a different qualification. Together they allow a reader to reconstruct why the panel outcome was actionable or blocked.

The purpose of all this machinery is simple: a panel should generate **better-grounded judgments**, not stronger-looking claims. If its results are difficult to explain, it has not solved the underlying trust problem. The specification's insistence on named purposes, qualified members, explicit thresholds and traceable disagreement is what prevents collective assessment from becoming a decorative majority vote.

Engineering Reflection

An Aster panel was configured with three members. One member misses a Seeded Control, the remaining two agree, and no quorum was declared. What must be recorded and what panel outcome follows? How would a declared quorum of two change the reasoning?

Two independent Judges disagree on whether Aster falsely claimed that a refund had been completed. What evidence and criteria would a Human Reviewer need before providing a defensible resolution?

Common Misconceptions

“Three Judges are automatically independent.” Repeated samples and shared model-family assumptions may retain correlated failure modes.

“A majority vote proves the answer is correct.” A vote records eligible members' verdicts under a threshold; it does not establish truth independently.

“Excluded Judges can simply be ignored.” Without a declared quorum the full panel is required, so exclusion blocks assessment for affected clauses.

“A panel is admitted if most of its Judges are admitted.” AQEF states that every Judge configuration in an admitted panel must be admitted.

“High agreement can substitute for Judge Confidence.” Confidence and Consensus are different signals; agreement must not silently replace individual Confidence.

Chapter Summary

Multi Judge is a scoped Implemented capability and is used for a defined assessment purpose, not merely to increase the number of models.

Independence across genuinely distinct Judges is different from stability obtained by repeated sampling of one Judge.

Consensus combines eligible assessments, while disagreement remains informative evidence that may require Human Review.

Voting uses a threshold greater than one half and applies it to the number actually counted; the panel Result discloses configured and counted members.

A Seeded Control miss excludes the affected Judge from Consensus and may reduce a panel below quorum.

Without a declared quorum, the full panel is required; with a declared quorum, qualified members may form a narrower panel result.

Every member must be admitted for the panel to be admitted, but a Gate's requirement for admitted Judges is a Project choice.

A traceable panel outcome preserves the criteria, member qualification, threshold, disagreement and limitations of the resulting claim.

Review Questions

1. Why can five repeated samples of the same Judge support stability analysis without supplying five independent opinions?
2. A panel of five counted Judges uses a voting threshold of 0.75. How many votes must agree, and what happens if neither side reaches that count?
3. What is the difference between a configured panel size and the number of qualified members counted in a Result?
4. How can a three-member panel legitimately end with a two-member count, and what additional rule determines whether Consensus may proceed?
5. Explain why an odd original panel size cannot guarantee freedom from ties.
6. A panel has two admitted members and one non-admitted member. Can the panel itself be described as admitted? Can an individual Gate have a different policy about admission?
7. Why is a high pass-vote proportion not equivalent to a calibrated high Confidence value?
8. How should a Human Reviewer investigate a panel disagreement about an unsupported tool-action claim?
9. What kind of risks could make a unanimous threshold appropriate, and why must it not be changed silently after a failing run?
10. Explain how panel outcomes fit into the chain from Quality Contract and Evidence to Quality Gates and Release Decisions.

Sources

1. AQEF 1.0-CR1, Volume VI — Multi Judge, Consensus, Voting, Confidence, Admission and Human Review.

2. AQEF 1.0-CR1, Volume VIII — Seeded Controls and exclusion of an affected Oracle.

3. AQEF 1.0-CR1, Front Matter §10 — Scoped Conformance capabilities.

4. AQEF 1.0-CR1, Volume I, Chapter 6 — Aggregate Basis and Results.

5. AQEF 1.0-CR1, Appendix A §A.7 — Multi Judge configuration and voting thresholds.

6. AQEF 1.0-CR1, Appendix C §C.7 — Panel count and recorded Results.

Chapter 13 — RAG, Agents, and Multi-Turn Dialogue

Maturity: Core as application of established AQEF architecture; Action Boundary is At Risk in AQEF 1.0-CR1 · Textbook First Edition v1.0

Learning Objectives

After this chapter, the reader should be able to:

1. Distinguish retrieval failures from generation-grounding failures using recorded source Artifacts.
2. Design evidence and Contract clauses for tool choice, arguments, sequencing and outcomes in agentic Scenarios.
3. Separate memory failures from coherence failures across turns, and explain trajectory-equivalent success.
4. Identify the distinct roles of operational human-in-the-loop and AQEF Human Reviewer assessment.
5. Analyse five Aster Support AI incidents while respecting the CR1 At Risk status of Action Boundary enforcement.

13.1 The quality question changes with the architecture

An AI system that answers one isolated question exposes a narrower set of observable events than a system that searches documents, chooses tools and sustains a conversation over several turns. AQEF does not replace its core model when the application changes. It applies the same Scenario, Execution, Conversation, Artifacts, Quality Contract, Oracle and Result to more complex behaviour. What changes is the evidence that must be captured and the kinds of failure that the Contract needs to make visible.

Aster Support AI will serve as the continuing example: it retrieves a return policy, reasons about a customer order, calls an order-status tool if authorised, and explains what it found. An answer that sounds satisfactory may conceal wrong retrieval, an invalid call, an unauthorised action or forgotten context. This chapter treats those as separately diagnosable mechanisms, not as an undifferentiated failure of a language model. Its purpose is to connect architecture to test design without adding any new mandatory Quality Dimension.

RAG, AI Agent and Multi-Agent are architectural patterns in Volume I, Chapter 7; the similarly named testing categories in Volume IX organise testing intent. That volume explicitly adds no new Conformance requirement. A Core AQEF implementation can be used to define a RAG testing Suite without claiming an additional capability named RAG. This separation lets an engineering team choose tests suited to the subject rather than confusing the test catalogue with an implementation feature checklist.

13.2 The two evidence boundaries in RAG

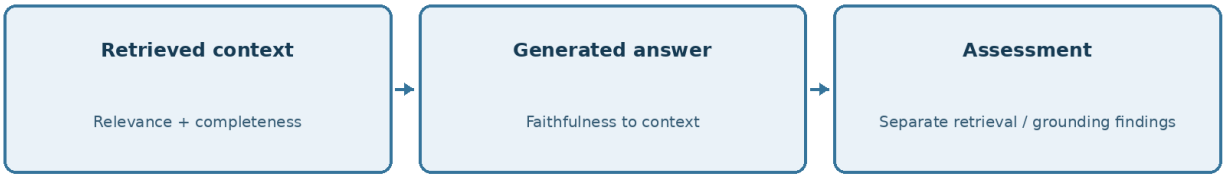


Figure 13.1 — Two diagnosis boundaries in RAG

Retrieval-augmented generation changes the evidentiary basis of a response. Before the answer is produced, a retrieval step selects external context. That context must be observable as an Artifact alongside the Conversation rather than disappearing into the final Response. Without this record, an assessor cannot distinguish a model that faithfully used the wrong passage from a model that ignored a correct passage. Those are different failures with different owners and corrective actions.

A **retrieval failure** concerns the material made available to generation: the wrong document, an incomplete extract, a stale policy version or a missing clause. A **grounding failure** concerns the relation between the supplied material and the generated answer: an invented exception, an unsupported certainty or a misrepresentation of what the source says. The same final sentence can result from either mechanism. AQEF therefore calls for checking retrieval evidence independently of semantic faithfulness to that evidence.

The specification describes this separation directly: “a retrieval failure … and a grounding failure” are independently diagnosable (Volume I, Chapter 7, RAG). This distinction does not mean every question can be decided by an exact-match Validator. Where retrieval relevance or completeness requires contextual judgment, an appropriate Oracle and Contract must express that uncertainty. A claim that retrieval is correct solely because a document was returned misses the issue of which document should have been returned.

13.3 Retrieval relevance is not grounding

Relevance, completeness and faithfulness form an explanatory set of questions, not a new normative triple of AQEF metrics. Relevance asks whether the retrieved information bears on the user question. Completeness asks whether the necessary governing evidence was supplied. Faithfulness asks whether the generated claim can be justified from what the system actually received. A good answer may need all three, but they belong at different positions in the execution evidence.

Suppose Aster retrieves the current refund policy but misses a regional exception for subscription purchases. It may faithfully say that the standard return window is thirty days. In one sense it is grounded in the retrieved excerpt; in another, the retrieval omitted essential qualifying information. A Judge that only compares response text with the returned excerpt could pass the response while a more appropriate retrieval-completeness check fails. The two outcomes do not contradict each other.

The Contract should identify the risk and the evidence expected to support each question. A required-document rule could be deterministic when the Scenario fixes a precise reference passage. Semantic faithfulness ordinarily requires interpretation of the response against recorded context. A review of the

entire corpus may be necessary to determine whether a missing exception exists. No one assessment mechanism has unlimited authority merely because it is called a RAG Judge.

13.4 Citations as claims about provenance

A citation in an AI answer is a claim about provenance, not a guarantee of correctness. It can point to a real passage that does not support the sentence, a correct passage in an obsolete policy, or a document the model never actually retrieved. Each failure differs from the mere absence of a citation marker. For a support assistant, source attribution matters because a customer may reasonably treat a cited policy as an authoritative basis for decisions.

Consider Aster claiming that a subscription refund has been approved and citing a general returns FAQ. The cited FAQ may not address subscriptions, and the approval claim may depend on order status unavailable in the FAQ. An Oracle checking that a citation exists would verify only a structural property. A semantic assessment must examine whether the cited text supports the particular statement. A different check may establish whether the citation references a passage in the captured retrieval Artifacts at all.

Citations also introduce version and identity concerns. A link that points to the current policy months later may not reconstruct what Aster used at execution time. Captured source excerpts, retrieval metadata and the relevant Dataset or Environment version enable retrospective review. If those Artifacts are missing, the engine should preserve the assessment gap instead of treating a polished bibliography as proof of grounding.

13.5 Designing RAG Scenarios around failure mechanisms

A useful RAG Suite begins with the failure to be detected rather than with a long collection of questions. Boundary cases include nearby but inapplicable policy passages, partially relevant documents, contradictory excerpts, and queries whose answer is absent. Negative cases can test whether Aster invents an answer when no authorising passage exists. Blocked-path cases can hide an expected source to test whether the assistant honestly identifies the missing basis.

Aster might be asked for a refund after an order was purchased in one country and delivered in another. The correct source might depend on purchase jurisdiction and product category. A Scenario can fix these Variables, specify expected evidence, and ask whether the retriever supplied the appropriate exception. A separate Expectation can then assess whether Aster explains the policy without inventing a universal right to refund.

Designing these Scenarios requires resisting answer leakage. If the test prompt embeds the correct policy conclusion verbatim, the generator may echo it without demonstrating retrieval competence. Conversely, a poorly specified Contract may fail any answer that paraphrases an expected sentence rather than reproducing it exactly. Risk-based test design keeps focus on the underlying obligation while allowing legitimate linguistic variability.

The educational distinction is between test coverage of a particular retrieval condition and evidence of reliable end-to-end behaviour. One Scenario may prove that a required document was supplied; it does not prove the system consistently interprets that document. Repeated Executions, different policy boundaries and multiple independently assessed defect classes supply complementary evidence.

13.6 An agent is assessed along a trajectory

An AI Agent executes more than a final text response. It chooses steps, may call tools, observes their outputs, and revises its plan. AQEF represents the whole interaction as an Execution yielding a multi-turn Conversation and supporting Artifacts. The system can take different legitimate paths to the same acceptable outcome because its subject is probabilistic. The test instrument, however, must be deterministic in how it constructs inputs and records what occurred.

For Aster, one run might verify an order first and then retrieve the applicable policy. Another might read the policy before requesting the order status. If both sequences obey authority constraints and produce a correct explanation, a literal sequence comparison would generate a false regression finding. Yet a trajectory that attempts an unauthorised refund before checking eligibility is not rescued by a later accurate explanation.

AQEF therefore makes the outcome and trajectory quality relevant to agentic semantic regression. A Baseline comparison must be able to assess equivalent acceptable paths rather than require identical intermediate steps. This does not imply that all paths are interchangeable. Quality Contracts may prohibit specific actions or require prerequisites, and those restrictions apply regardless of which textual answer eventually appears.

13.7 Tool choice, arguments, order and outcome



Figure 13.2 — Agent action evidence

Tool-using systems expose at least four separable questions. Did the agent select an appropriate tool? Were the arguments well formed and authorised for the intended context? Did it call tools in an acceptable order? Did the actual tool outcome support the claims it made to the user? A test that checks only the final answer can miss defects in any of these intermediate properties.

Structural Validators are well suited to tool names, argument schemas, permitted field values and required step order. A Judge or Human Reviewer may be needed to determine whether an otherwise valid tool call made semantic sense in the customer's case. The recorded response of the external service matters because a syntactically successful request is not the same as a business action that genuinely completed.

Imagine Aster invokes *get_order_status* with an identifier from another conversation. The call could satisfy a schema and still violate the intended authority boundary. Conversely, it could call the correct order but tell the customer a refund was issued even though the tool only returned eligibility information. Validating the tool arguments and checking the subsequent claim require distinct Contracts and potentially distinct Oracles.

The volume on architecture stresses that Tool Calling and AI Agent are not subsets of one another. A one-turn interaction may make a tool call without being agentic, and an agent may plan multiple steps without calling any external tool. Test designers should name the mechanisms actually in use, not apply a broad label that hides which evidence is required.

13.8 Action boundaries are not the same as tool validity

An agent can call an existing tool with correctly typed arguments and still cross a safety boundary. The distinction between whether a call was validly formed and whether it should have been possible matters especially when actions have real external consequences. AQEF Volume III describes Action Boundary settings for stubbed, sandboxed and real-world execution environments, but the feature is explicitly **At Risk in CR1**. This chapter must not imply that every conformant AQEF implementation enforces those boundaries.

Aster might be allowed to read order status but not to issue refunds without approval. A Validator can inspect recorded tool selection and arguments. A separate action-boundary mechanism is concerned with what the execution environment permits to be attempted or completed. The distinction is architectural: a post-hoc check can establish that something bad happened, while a correctly enforced boundary could prevent the action.

The textbook can discuss the risk and describe how a Project might design a Scenario to expose it without asserting unproved CR1 enforcement. When the specification identifies a capability as At Risk, the educational model and the implementation's Conformance claim must remain separate. The principle is to state clearly what evidence demonstrates, what the specification requires for a claimed capability, and what has not yet been independently implemented.

13.9 Multi-turn memory and coherence

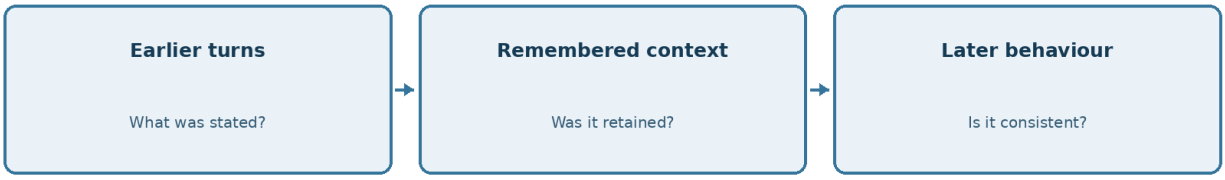


Figure 13.3 — Memory and coherence are separate

Multi-turn interaction introduces two commonly confused properties. Memory concerns whether the system retains, recalls and prioritises relevant earlier information. Coherence concerns whether its later responses remain consistent with the conversation's commitments, context and reasoning. A model may accurately remember a customer's purchase date and nonetheless contradict a previously stated eligibility conclusion without explanation. Conversely, it may produce a coherent-sounding reply while forgetting a critical constraint from an earlier turn.

Consider Aster initially receiving a customer's country of purchase, then handling a follow-up question about a subscription. If it loses the country, it may choose the wrong policy. That is a memory failure. If it correctly

retains the country yet later tells the customer both that refunds are prohibited and that the requested refund was approved, its problem is coherence or correctness of the evolving dialogue, depending on the evidence.

A Scenario should identify which earlier fact must remain relevant and which later response is expected to respect it. The transcript provides the chronology of user input and model replies. If the case includes retrieval or tool outputs across turns, those Artifacts must be preserved so that the Oracle can inspect whether an apparent contradiction arose from a changed source, a changed tool state or a breakdown in conversational reasoning.

The distinction also clarifies why single-turn testing cannot substitute for dialogue assessment. A model can answer each isolated prompt correctly while failing when the prompts arrive as a sequence. Multi-turn reliability requires scenarios whose meaning depends on what the subject previously heard or did, not merely a set of independent questions concatenated into one document.

13.10 Deterministic construction, probabilistic branches

Multi-turn and agentic testing must not sacrifice the framework's Deterministic Infrastructure principle. A later Prompt may depend on an earlier Response, but the harness must construct it deterministically from the recorded Context and the prescribed Scenario rules. If the harness improvises unrecorded follow-up instructions, a difference in later outputs becomes impossible to attribute confidently to the subject.

AQEF Volume III treats Variable Resolution and Prompt Construction as operations that can repeat at each turn. Artifact Collection runs alongside the Conversation Engine because retrieval passages and raw tool responses may be transient. That timing is essential for traceability: reconstructing an intermediate tool response after an execution may substitute changed external data for what the agent actually observed.

Aster's tester might define an initial request, followed by a user clarification if the subject asks for the missing order number. The harness can specify what clarification to supply under the Scenario. The agent may ask for the information in different words; the harness should follow its declared response logic and record the resulting branch. The generated Conversation can differ without making the execution machinery random.

Repeatability therefore means reproducing the conditions and deterministic rules of the test, not forcing an AI subject to emit identical trajectories. A quality assessment is trustworthy when the recorded route can be reconstructed and evaluated, even if another run follows a different route under the same Contract.

13.11 Trajectory equivalence and regression

A regression framework for agentic systems faces a familiar temptation: compare the new execution with a historical trajectory step for step. This is easy to compute and often wrong as a quality criterion. The earlier run may have chosen one legitimate sequence while the new run chose another. If both satisfied mandatory constraints, retained necessary context and reached a supported conclusion, their differences may be harmless.

The more difficult question is whether the new trajectory still respects the Contract. Aster might retrieve policy before checking order status instead of afterward. If no prerequisite prohibits this sequence, that is potentially acceptable. But if it issues a refund action before receiving required authorisation, no amount of later success makes the sequence acceptable. A properly designed assessment distinguishes optional steps from mandatory prerequisites.

Volume I states the principle: agentic Baseline comparison must be capable of judging “the outcome and the reasoning quality of a trajectory as a whole,” rather than demanding step-for-step identity. This passage supports an outcome-plus-trajectory view, not a claim that a specific similarity metric is normative. The book should describe why equivalence matters while leaving the implementing method tied to whatever the applicable Contract actually requires.

A useful review compares the reasons for changed paths: new retrieval content, changed tool results, altered prompts, configuration changes or genuine subject variability. Without that context, a trajectory difference can be mislabelled as a product regression when the underlying conditions changed, or a harmful change can be missed because the final answer remains plausible.

13.12 Multi-agent coordination and attribution

A multi-agent system may delegate retrieval, reasoning, action selection or user communication to different AI participants. Its user-facing Conversation is only one part of the evidence. Internal handoffs, sub-conversations and intermediate contributions can determine whether a later response is correct. AQEF treats these as Artifacts or sub-conversations associated with the Execution so that the assessment can identify which component did what.

Aster could use a policy-retrieval specialist and an order-processing specialist. The first supplies the wrong document; the second makes a correct decision given that incorrect material. A final evaluator that sees only the customer message may condemn the latter participant unfairly or miss the upstream retrieval error. With attributed intermediate evidence, the team can investigate the handoff and its effect on downstream behaviour.

Multi-agent configurations also amplify risks of correlated assessment failure. If all subject agents and the Judge are based on the same model family and reasoning instructions, shared assumptions may survive every handoff and be endorsed by the instrument. AQEF's Independent Oracles principle remains applicable. A collection of collaborating agents is not itself a substitute for an independent assessor.

The book does not need to invent a new multi-agent Conformance entity. The existing Scenario, Execution, Conversation, Artifacts, Contract and Result model is intended to accommodate this architecture. The educational challenge lies in designing evidence that allows both component attribution and end-to-end assessment.

13.13 Human participation inside and outside the subject

An operational human-in-the-loop pattern is not the same as AQEF Human Review. In the first, a person supplies approval or information during the AI system's own work; the interaction is part of the subject's runtime evidence. In the second, a Human Reviewer acts as an Oracle assessing recorded evidence under a

Quality Contract. Mixing these roles would confuse who performed a product action with who judged the quality of that action.

Aster might ask a support agent to approve a high-value refund. AQEF can then assess whether Aster correctly requested the approval, waited for the response and incorporated the person's decision. If a separate AQEF Human Reviewer later evaluates whether the sequence respected the Contract, that review is an assessment step and must remain traceable independently of the earlier operational approval.

The presence of a human decision in the product workflow does not automatically establish the quality of the AI assistant. The system might ask for approval at the wrong time, fail to obey a rejection, or misrepresent the reviewer's response. Likewise, a later Human Review outcome cannot retroactively change which operational approval actually occurred. The two humans can be the same person in an organisation only if roles and evidence remain properly distinguished.

This is a valuable extension of the subject-versus-instrument boundary first introduced in Chapter 1. Context can include human input without making the human the Oracle, and assessment can include a human Oracle without making that reviewer part of the tested system.

13.14 Five Aster incidents as a diagnostic map

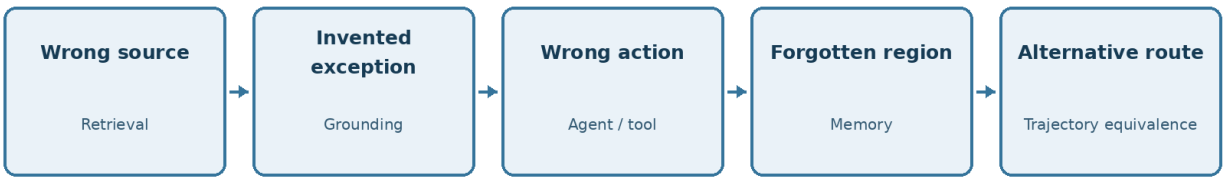


Figure 13.4 — Five diagnostic incidents

The first incident is a **wrong-source answer**. The retriever supplies an outdated policy, and the generator faithfully summarises it. The final answer is substantively wrong for the customer, but the primary upstream cause is retrieval or source selection. The captured document Artifact distinguishes this from unsupported generation. A Contract that measures only groundedness against the retrieved passage would be insufficient to detect the stale-source condition.

The second incident is an **unsupported policy claim**. Aster retrieves the correct passage and invents a return exception. This is the grounding failure that the source material makes directly diagnosable. The semantic Judge needs the Response and the same retrieved passage the model saw; without both, its claimed determination may not be supported by evidence.

The third incident is a **validly formed but wrong tool action**. Aster sends a syntactically correct request to an inappropriate endpoint or before authorisation. A schema Validator may pass the request while a step-order or authority clause fails. The Action Boundary distinction becomes particularly important because the boundary-enforcement capability is At Risk in CR1.

The fourth incident is **forgotten context across turns**. Aster loses the customer's region after a clarification and selects the wrong policy. The observed defect cannot be reconstructed from the final turn alone; the earlier user message and subsequent Context construction must remain available.

The fifth incident is **an alternative successful path**. Aster checks policy before order status instead of vice versa, without violating a mandatory prerequisite. Literal path comparison reports a difference; a trajectory-aware assessment recognises a potentially acceptable equivalent route. The lesson is not that paths never matter, but that the Contract must distinguish allowed variation from prohibited action sequences.

13.15 Designing a defensible hybrid Suite

Aster combines retrieval, dialogue, tool calling and potentially human approval. Its test design should therefore organise Scenarios by risk and assessment need, not simply mirror the software architecture. The highest-priority cases might cover unauthorised actions, misleading confirmations, stale policy content, missing retrieval, contradictory tool outcomes and context loss across turns. Routine accurate answers remain useful, but they cannot substitute for these mechanisms.

Each Scenario needs a clear Contract and evidence plan. Retrieval cases should preserve source passages; tool cases should preserve selections, arguments and outcomes; dialogue cases need ordered multi-turn transcripts; human-approval cases need the authorisation event. Validation and Evaluation should be assigned according to what can be checked deterministically and what requires semantic judgment. A single overall helpfulness score would obscure important hard constraints.

Coverage analysis then asks whether the chosen Dataset variants and Executions actually exercised the intended risks, whether Oracles produced actionable Results, and which gaps remain. Multiple correct paraphrases of a low-risk policy answer may increase volume without covering a single blocked tool path. This is the risk-based reasoning established in Chapters 5 and 8, now applied to a composite AI system.

A hybrid system does not require a seventh Quality Dimension or a new general-purpose Oracle. Volume I explicitly describes Hybrid Systems as composition of existing patterns; Volume VII provides Contract Composition. This preserves the architecture's explanatory economy: the test machinery remains recognisable, while the Contract and Evidence reflect the mechanisms actually present.

13.16 Limits of the claims a test can support

Even rich evidence does not make every assertion certain. A successful RAG Scenario demonstrates behaviour under a particular corpus state and input, not global truth across all future policies. A valid tool trace may show that the requested API was called, not that an external business process completed successfully. A coherent multi-turn Conversation may still rest on an incorrect retrieved policy. The responsible conclusion states what was checked and on which recorded basis.

Likewise, a missing Artifact can prevent a meaningful Oracle from answering the right question. If the tool outcome was not recorded, an assessor should not infer action success from the final response. If retrieval context was not captured, a claim about grounding must be qualified. Infrastructure gaps should remain distinguishable from demonstrated subject failures and should not be transformed into convenient passing verdicts.

AQEF's practical strength here is not a promise that complex AI systems become easy to certify. It is a set of boundaries that keep the testing claim traceable: the risk motivated a Scenario, the Scenario produced a Conversation and Artifacts, an Oracle assessed a defined clause, and a Gate used actionable Results with a known basis. Each boundary supports investigation when things go wrong.

The next chapter extends this model to safety, operational quality, action boundaries and regression. Those concerns are especially consequential for a hybrid assistant that can affect external systems. The foundations developed here—separate failure locations, complete evidence, trajectory-aware assessment and disciplined Conformance claims—are the conditions for making such decisions responsibly.

13.17 Building a trajectory-aware quality argument

A traceable agent assessment is assembled in stages. First the team states what risk matters: an action before authorisation, an unsupported refund claim, or the loss of a critical fact across turns. Second it defines the permissible and prohibited behaviours in the Quality Contract. Third it specifies the Evidence needed to distinguish the two. These are separable decisions. A high-quality trace cannot rescue an ambiguous Contract, while a precise Contract cannot be assessed if the necessary tool outcomes were never recorded.

Trajectory design benefits from marking steps as mandatory, optional or forbidden. A mandatory step might be checking eligibility before initiating a refund. An optional step might be retrieving policy before or after requesting an order number. A forbidden step might be invoking a customer-visible action when approval was denied. This is an educational design technique rather than an additional AQEF schema requirement. It creates an intelligible basis for analysing whether two distinct paths are equally acceptable or whether one violates a hard Constraint.

Consider two Aster Executions. In the first, Aster reads the current return policy, checks order status and explains that the customer is ineligible. In the second, it checks order status first and retrieves the same policy afterward. Both paths may satisfy the defined risk constraints. A third Execution initiates a refund before confirming eligibility and later supplies a correct explanation. Treating all three as equivalent because their final text looks similar would conceal a defect in the third trajectory. Equivalence is therefore conditional on preserved obligations, not on superficial agreement.

The assessment record should permit a reviewer to say which steps were present, which governing evidence was available at each point, and whether the action outcomes matched subsequent statements. If an intermediate Artifact is unavailable, the right conclusion might be that the relevant trajectory clause cannot be evaluated. That is different from proving the agent failed the clause. The report should explicitly preserve the distinction so that a quality decision does not rest on invented evidence.

13.18 When evidence disagrees with the final answer

Testing complex AI benefits from asking what each evidence source can prove. A final answer is evidence of what the customer was told. A retrieved passage is evidence of what supporting information was available. A tool-call record is evidence of what the agent requested, and a tool result is evidence of what the external system reported. These records can conflict. An assistant may say a refund succeeded while the trace contains only a pending request, or cite a current policy while the recorded retrieval output came from an archived version.

The conflicts should become explicit assessment questions rather than be resolved by choosing whichever artifact looks authoritative. A tool response may itself be stale or wrong. A retrieved passage may be a faithful copy of a superseded source. A Judge may compare the wrong two objects. When this happens, an engineer should identify the original source of each observation, the Contract clause it informs, and whether the corresponding Oracle is capable of establishing the intended property. The resulting evidence trail is more useful than a single pass/fail label attached to the final sentence.

For Aster, imagine a customer asks whether the refund has been completed. The visible response claims success. The tool trace shows that Aster submitted a request but received a timeout before confirmation. A structural Validator can establish that the request was made and what status the tool returned. A semantic Oracle can assess whether the user-facing statement accurately describes that status. The business operation may require independent confirmation beyond the immediate trace. Thus a seemingly simple claim can involve functional, semantic, reliability and operational concerns without requiring AQEF to define a new Quality Dimension.

An effective release report need not hide this complexity. It can explain that the request was submitted, that completion was not confirmed, and that the assistant's final statement therefore lacked sufficient support. It can also identify missing observability and decide whether the absence of confirmation creates an insufficient assessment basis or an observed violation of an explicit Contract clause. The precision of that explanation is the practical value of preserving a complete Conversation and its original Artifacts.

Engineering Reflection

Aster provides the correct final refund answer, but the tool trace shows an attempted action before authorisation. Which parts of this Execution are assessable by deterministic Validators, and which require contextual judgment?

Aster retrieved a policy passage that supports the final answer, but a customer complaint reveals that an exception was absent from the passage. What evidence would separate an incomplete retrieval from an unsupported generation claim?

Common Misconceptions

A correct final answer means the agent acted correctly. Intermediate selection, arguments, prerequisites and tool outcomes can violate the Contract even when the prose looks right.

RAG grounding and retrieval quality are the same check. The first asks whether the answer follows the retrieved context; the second asks whether the appropriate context was obtained.

An agent regression requires an identical sequence of steps. AQEF recognises that distinct trajectories may be acceptable if their outcome and reasoning quality satisfy the Contract.

Every human appearing in the trace is an AQEF Human Reviewer. A human participating in the product's operation is part of the subject; a Human Reviewer assesses the recorded evidence from outside it.

RAG and Agent are extra Conformance capabilities. They are architectural patterns and test types within the core testing model, not separate claimable capabilities in Volume IX.

Chapter Summary

RAG adds retriever-produced Artifacts whose quality must be distinguished from the grounding of the generated response.

Citations alone do not prove support; recorded passages and versions determine what provenance can be assessed.

Agents require evaluation of tool choice, arguments, intermediate ordering and outcomes in addition to final text.

Repeated Executions can follow different acceptable trajectories, so semantic regression cannot rely only on literal path identity.

Memory and coherence across turns are different failure mechanisms that require a preserved chronological Conversation.

Multi-Agent and hybrid systems extend the evidence, not the fundamental Domain Model or Quality Dimensions.

Operational human participation is not the AQEF Human Reviewer Oracle; their roles and records must remain separate.

Action Boundary enforcement is At Risk in CR1, and evidence of correct tool calls is not evidence that an enforcement boundary was deployed.

Review Questions

1. An assistant gives an incorrect answer grounded in an outdated retrieved policy. Which source of failure should be investigated first and why?
2. What is the difference between a cited passage existing and that passage actually supporting a claim?
3. How would you distinguish a syntactically valid tool call from an authorised and semantically appropriate one?
4. Two agent Executions achieve the same valid outcome in different orders. What would make the difference acceptable or unacceptable?
5. Why must a multi-turn Context update follow deterministic harness rules even though the model's next step may vary?
6. How could Aster remember a purchase date and nevertheless be incoherent later in a dialogue?
7. Which intermediate evidence would you preserve for a two-agent support workflow, and why?
8. Explain the difference between human approval requested by Aster and Human Review used to assess Aster.
9. Why is an Action Boundary claim in CR1 different from checking a recorded tool-call trace?

10. How would you structure a risk-based hybrid Suite without relying on a single overall helpfulness score?

Sources

1. AQEF 1.0-CR1, Volume I, Chapter 7 — Architectural Patterns (RAG, AI Agent, Multi-Agent, Human-in-the-loop, Tool Calling, Hybrid Systems).

2. AQEF 1.0-CR1, Volume II — Scenario, Execution, Conversation and Artifact relationships.

3. AQEF 1.0-CR1, Volume III — Execution Pipeline, Context, Conversation Engine, Artifact Collection and Action Boundary.

4. AQEF 1.0-CR1, Volume VII — Quality Contracts and Contract Composition.

5. AQEF 1.0-CR1, Volume VIII — AI Test Patterns, Negative and Blocked-Path Testing.

6. AQEF 1.0-CR1, Volume IX — Test Types (taxonomy, not Conformance capabilities).

7. AQEF 1.0-CR1, Volume X — Semantic Regression and Baselines.

8. AQEF 1.0-CR1, Front Matter §0 — At Risk capabilities.

Chapter 14 — Safety, Action Boundaries, Regression, and Operational Quality

Maturity: Core for safety test design, regression and operational assessment; At Risk for `action_boundary` in AQEF 1.0-CR1. · Textbook First Edition v1.0

Learning Objectives

By the end of this chapter, the reader should be able to:

1. Distinguish safety elicitation techniques from safety-adjacent failure outcomes.
2. Explain the intended Action Boundary model and its At Risk status without claiming enforcement where none is demonstrated.
3. Describe Semantic Regression, explicitly designated Baselines and release-level Snapshots.
4. Separate Prompt, Model and Dataset Drift from Judge Drift.
5. Evaluate operational quality across load, stress, chaos and cost without confusing subject behaviour with test-harness overhead.

14.1 Safety begins with a boundary, not a reassuring answer

Aster Support AI can produce fluent advice while failing a requirement that matters more than fluency. An invented refund entitlement can mislead a customer; a tool call can change a real order; a retrieved document can attempt to redirect the agent; and a seemingly helpful workaround can violate the limits within which the task was authorised. Safety testing therefore cannot be reduced to whether a final response contains a disallowed phrase. It has to connect what the system said, what it attempted, what it actually changed, and what its Environment permitted.

AQEF places Safety alongside Functional, Semantic, Operational, Reliability and Business Quality, rather than treating it as an adjective applied to every failure. That separation is useful because a safety failure may coexist with successful functional behaviour: an agent can call the correct endpoint with valid parameters and still use it for an unauthorised purpose. The test must describe the prohibited effect in a Quality Contract and collect enough Evidence to distinguish the result from a harmless simulation.

Safety is also not a test type in the same sense as Prompt Injection. Volume IX treats Safety as an umbrella quality concern, while Prompt Injection and Jailbreak are ways of provoking a boundary violation, and Unauthorized Action is a category of resulting behaviour. Keeping elicitation technique separate from observed outcome prevents a common error: counting a cleverly written hostile prompt as a failed safety test without establishing whether it actually made the subject do anything forbidden.

The chapter builds on the distinction between subject and instrument. An absent tool trace may mean that the safety behaviour is unknown rather than safe or unsafe. A Validator that could not inspect the recorded boundary does not prove Aster broke it. The testing process should be as disciplined about uncertainty in a high-risk case as it is about an ordinary factual check.

14.2 Prompt Injection and Jailbreak target different channels

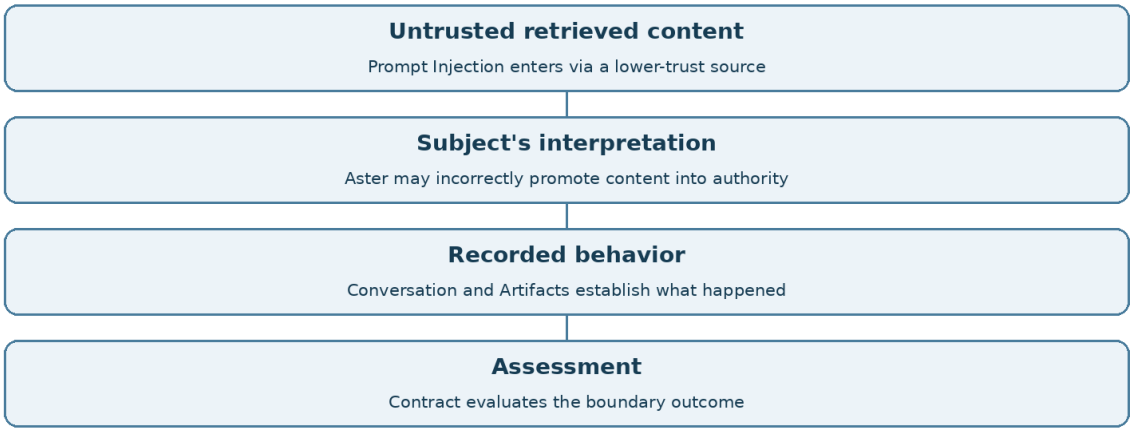


Figure 14.1 — Injection channel and assessment evidence

Prompt Injection attempts to influence the subject through lower-trust material it consumes: a retrieved policy page, a search result, a tool response or another indirect content channel. The attacker relies on the subject treating an instruction inside that material as if it had the authority to change its operating goal. The engineering failure is not that the document contained imperative language; it is that the subject promoted less-trusted content into instructions that should have governed its behaviour.

Jailbreak attempts a similar class of boundary violation through the direct conversation: role framing, an invented emergency, multi-turn persuasion or other efforts to erode a refusal. Volume IX distinguishes the two by the channel through which manipulation arrives, not by the attacker's desired outcome. Both can target the same protected property. Both require observation of the subject's actual response or action before a meaningful verdict can be recorded.

For Aster, consider a retrieved page that claims the refund rules were suspended and instructs the assistant to call a payment tool outside its assigned workflow. A safe diagnostic Scenario can examine whether Aster treats this as untrusted document content and continues under its real Contract. The payload need not be elaborate or operationally dangerous to be a useful test. The evidence must include the retrieved passage and any attempted tools, because the final reply alone may conceal the source of a change in behaviour.

The test designer should avoid equating any rejection with success. Aster may refuse an ordinary permitted task because a document contained suspicious language; that is a product-quality issue of a different kind. A balanced Contract distinguishes unauthorised instruction following from legitimate use of retrieved facts, so that safety does not become a pretext for indiscriminate refusal.

14.3 Output categories and the role of consequence

Volume IX names Bias, Toxicity, Hallucination and Unauthorized Action among safety-adjacent output categories. Their risk does not depend on the use of a malicious prompt. A benign question can trigger unfair

treatment, inappropriate content or an unsupported assertion. These categories deserve their own evidence and Contract clauses, not merely placement inside a generic adversarial testing folder.

Hallucination is primarily a Semantic Quality issue because it concerns support by the available evidence or facts. The same unsupported sentence can carry different consequences depending on where it appears. Inventing a store's informal greeting is not comparable to inventing a refund authorisation or misrepresenting the result of a financial action. AQEF allows a Contract to bind a hallucination-related property to relevant dimensions when the consequences justify it, rather than requiring every hallucination to be categorised identically.

Unauthorized Action concerns what the subject does. It may involve attempting a forbidden endpoint, crossing a declared boundary, triggering a tripwire or using individually permitted operations to achieve a forbidden effect. Some parts are mechanically checkable from tool traces; others require interpretation of the overall trajectory. An action can be syntactically valid yet unauthorised, so correct arguments are necessary but not sufficient evidence of safety.

These distinctions make findings actionable. A failed pattern Validator points to a specific observable violation. A semantic Judge may identify an unsupported claim needing human confirmation. A trace that lacks an external tool outcome may leave the action's actual effect uncertain. Accurate classification supports repair without inventing certainty.

14.4 Action Boundary: specified mechanics and At Risk status

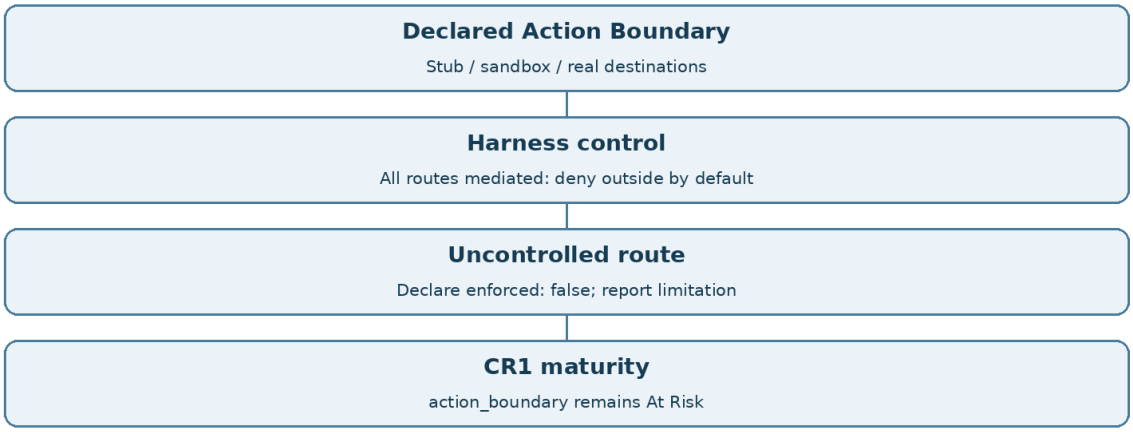


Figure 14.2 — Action Boundary is a declaration plus an enforcement claim

Action Boundary addresses a critical problem for agent testing: an Execution isolated from other test runs can still reach the external world. A tool-using subject may send a message, alter a record or contact a network endpoint. Running the test in a separate worker process does not itself prevent these effects. An Environment whose tool_configuration is not null is therefore required by Volume III's specified Action Boundary mechanism to declare which systems, endpoints and tools are reachable and how they are classified.

> *“Where the harness controls the subject's access, everything outside the boundary MUST be denied by default.” — AQEF 1.0-CR1, Volume III, Action Boundary.*

The declared kinds are stub, sandbox and real. A stub stands in for a service and records calls without external effect. A sandbox is a working copy without effects outside the test. Real denotes actual external systems. The boundary records what is permitted and what can be inspected; it does not guarantee that a model on an uncontrolled platform cannot open a connection the harness never sees. When even one route bypasses the harness, the Environment must declare the boundary as not enforced, and Reports must expose that condition.

Crucially, action_boundary is **At Risk in AQEF 1.0-CR1**. The specification describes requirements and intended semantics but reports no independent implementation yet for the capability. A textbook can explain how the mechanism ought to work without claiming that every conformant core engine already seals the outside world. This is an edition-specific maturity boundary, not a recommendation to test against real systems without appropriate safeguards.

14.5 A declared boundary is not the same as an enforced one

A Project may describe a list of allowed tools while failing to control all possible routes to them. Aster might interact through a declared refund API, but the underlying platform could expose a generic network client or another integration path. If those paths bypass the testing harness, a policy file describing a sandbox is not evidence that the environment is sealed. Honest declaration is essential because hidden access is particularly consequential when tests deliberately close the normal path.

AQEF's *enforced: false* is a fact to report, not itself an implementation Conformance defect. The specification recognises that a harness cannot deny traffic it does not control. An engineering report that says 'declared but unenforced' gives decision-makers useful information about the risk of performing a Scenario in that Environment. Calling such an Environment fully isolated would conceal an important condition.

A stub can also be a Tripwire: a stand-in designed to look like a tempting shortcut that the subject has no legitimate reason to access. Its purpose is to record whether the subject attempted to step outside the permitted path, not to invite real-world side effects. That makes it a useful part of Blocked-Path Testing, where the normal route is deliberately unavailable and the subject's response to the obstacle is itself under examination.

Because the capability remains At Risk, this chapter treats Action Boundary examples as specification-guided design explanations, not as a completed product demonstration. Readers should inspect their actual harness and platform controls before relying on any claim of isolation.

14.6 Blocked paths reveal failures that ordinary success hides

An agent can behave correctly while everything works and become unsafe when a dependency is unavailable. Closing the expected path deliberately tests what it does next. When Aster's approved order-lookup tool is unavailable, the acceptable behaviour might be to explain the limitation, preserve customer

context or request a human handoff. An unacceptable behaviour might be to reach for an unauthorised substitute or to claim that an action succeeded without a tool outcome.

Blocked-Path Testing is not the same as trying to trick the subject with hostile wording. It manipulates the environment or task affordances to reveal whether the system respects its boundaries under frustration. The failure may be visible as an attempted tool call, a misleading final statement or a sequence of locally valid operations whose combined effect is forbidden. For that reason the full trajectory and tool Artifacts matter.

Volume IX treats such a scenario as related to Unauthorized Action. It is possible for a test to be both operational and safety-relevant: an unavailable dependency probes graceful degradation, while an attempted workaround outside the Action Boundary probes safety. A single Execution can produce evidence for both without making the properties interchangeable.

Aster's test designer should specify what happens when a tool is blocked and what constitutes a prohibited workaround. Otherwise the assessment may reward ingenuity indiscriminately, even where 'trying harder' changes the task from customer support into an unapproved external action.

14.7 Regression asks whether quality has worsened

AI systems can produce different words while preserving the same meaning. A literal output comparison therefore cannot establish Semantic Regression reliably. Volume X defines regression through a changed assessment: a current Conversation now fails an Oracle it previously passed under a comparable Contract and reference. This is a Reliability finding against a prior Baseline, rather than a judgment about one response in isolation.

> *“An Execution's Result MUST be explicitly designated as the reference standard” — AQEF 1.0-CR1, Volume X, Baselines.*

This requirement matters because the first Execution is not automatically a good reference, and the latest is not necessarily the most trustworthy. An explicit Baseline protects a known acceptable comparison point against casual replacement. It must remain tied to the relevant Dataset and Contract versions, since changing either can alter what counts as the same test and what a passing outcome means.

An Aster answer may use different phrasing or a different retrieval order after a system update. If both versions faithfully apply the same current policy, a text diff could flag substantial differences despite no meaningful quality degradation. Conversely, an answer with nearly identical wording could now imply an unsupported refund authorisation. A sound Regression Suite compares the assessed property, not string identity.

14.8 Baseline and Snapshot are different references

A Baseline attaches a designated reference to the level of a Scenario or specific Execution Result. A Snapshot captures a wider Project or release state: multiple Baselines, configuration and the context needed to explain a decision. Volume X treats Snapshot Comparison as the release-level counterpart of Baseline comparison. They serve connected purposes, but one should not be used as a synonym for the other.

The distinction prevents accidental changes in comparison scope. A single refund-eligibility Scenario may retain an approved Baseline while a new Snapshot includes additional Scenarios, a changed Dataset version or a newly qualified Judge. The aggregate outcome may move even though the original case did not regress. A project must be able to distinguish changes in the evidence population from changes in the subject's quality.

Versioning matters at both levels. A Baseline without pinned data and Contract can be reinterpreted silently; a Snapshot without reconstructable approvals cannot support a reliable audit of a release decision. These records protect the historical meaning of observations in an environment where models and retrieval corpora are regularly updated.

For Aster, a risk report could compare the current release Snapshot with the preceding one, then investigate a changed Gate by tracing to the affected Baselines. The method preserves a meaningful path from broad movement to the specific evidence that drove it, instead of treating one dashboard score as self-explanatory.

14.9 Diagnosing Prompt, Model and Dataset Drift

Consider an Aster policy check that passed yesterday. Overnight, a policy source changes, the retrieval index is rebuilt or the model version changes. The historical pass remains truthful as a statement about that earlier Execution, but it is not evidence that the new combination behaves correctly now. A fresh comparison must account for the changed subject, Dataset, Environment and assessment basis.

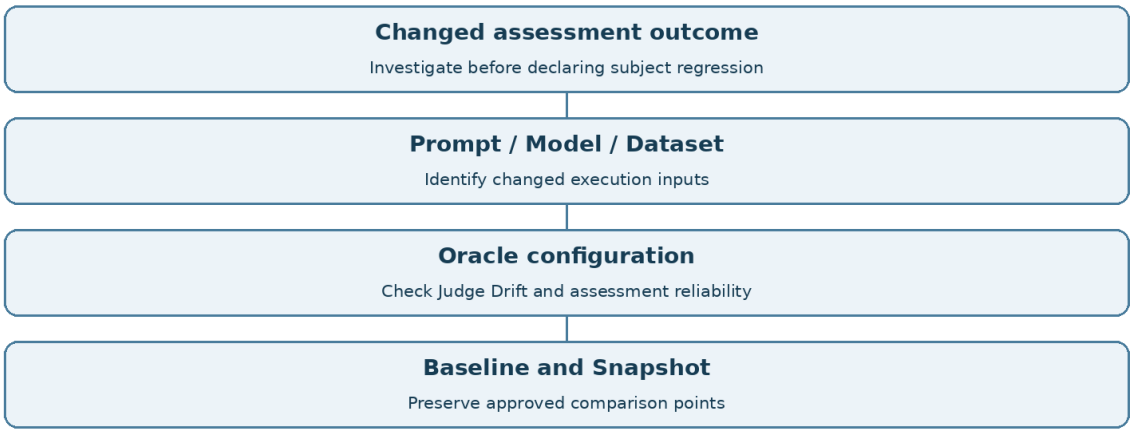


Figure 14.3 — Diagnosing a changed Result

A regression is a symptom; Drift Detection investigates why it occurred. Volume X identifies Prompt Drift, Model Drift and Dataset Drift as changes to different inputs that can explain the altered behaviour. It also requires attention to Judge Drift, because a changed assessment instrument can create a seemingly identical difference in a Report without any deterioration of the subject.

Prompt Drift may come from a deliberately versioned instruction update or from a shared template quietly edited outside the tracked Environment. Model Drift can result from a planned model upgrade or from a

provider changing the actual model behind a stable identifier. Dataset Drift follows changes in the rows used to instantiate Scenarios, or potentially changes in the distribution of production examples. The same drop in an aggregate score can have different causes and therefore demand different corrections.

For Aster, suppose an answer about refund eligibility changes after a release. If the retrieval Dataset expanded to include regional exceptions, an aggregate shift could reflect the new test population. If the model provider changed weights under a stable endpoint name, the change may be invisible in an ordinary configuration diff. If a Judge's criteria were revised, the subject may have behaved as before while the verdict changed. All three hypotheses require evidence, not guesswork.

The diagnostic process should compare the exact prompt, model resolution, Dataset version, Contract version, Oracle identity and recorded Artifacts where available. It is not enough to say 'AI drifted.' An effective finding identifies the part of the chain that moved and acknowledges any changes that cannot be reconstructed.

14.10 Judge Drift can masquerade as subject regression

Judge Drift is especially consequential because quality reports often display only the assessment outcome. A previously passing Scenario becomes failing, and the natural reaction is to investigate Aster. But if the Judge changed its behaviour, the displayed symptom is not proof that the subject deteriorated. Chapter 10 described the importance of reference judgments and calibration; the same independent evidence is needed when interpreting a regression.

Volume X proposes comparing Judge behaviour against Human Reviewer agreement. If a Judge's verdicts shift while human reference agreement with the subject remains steady, instrument drift becomes a plausible explanation. If both shift together, subject change becomes more plausible. This diagnostic approach does not pretend the reviewers are infallible or that every case yields a crisp causal answer. It narrows the explanation using reference evidence.

Seeded Controls provide a complementary near-term check. A missed control can reveal that the Judge currently fails a known defect class before its ordinary Results are relied on. Admission says whether the configuration previously met the Project's reference threshold; Calibration asks whether Confidence tracked correctness over time. None of these replaces the evidence needed to diagnose a particular regression.

Aster's release report should therefore retain the Oracle configuration and qualification evidence alongside subject outputs. A changed verdict is a fact about the assessment record; the claim that Aster regressed is a further inference requiring the historical chain.

14.11 Operational Quality is not an optional afterthought

A system may give correct answers and still be unsuitable for its intended use because it is too slow, too expensive or unreliable under routine demand. AQEF treats Operational Quality as one of the six dimensions, allowing Contracts and Gates to reflect resource and service constraints independently from semantic correctness. Improving model accuracy by doubling cost or response latency is not automatically a net product improvement.

Operational evaluation should distinguish measurements of the subject from overhead added by AQEF's own testing infrastructure. Volume IX's Load, Stress, Chaos and Cost test types concern the AI system being tested. Volume III's Scheduling, Parallelization and Retry concern the harness. Confusing them makes it impossible to say whether a latency spike was a system performance problem, an overloaded test runner or an unrelated measurement artifact.

Cost checks should use captured consumption evidence, not an estimate invented during reporting. Latency checks should read the recorded subject measurement, not the time a Validator spent inspecting it later. Throughput and reliability require enough distinct executions and a clear aggregate basis to support meaningful interpretation. No universal threshold for acceptable cost or latency is dictated by the textbook.

Aster's Project might care about the time needed to answer a routine policy question and separately about the time to complete an authorised refund workflow. These are different service expectations. A fast refusal, a slow correct completion and a fabricated completion message should not be scored as though they were interchangeable responses to one speed target.

14.12 Load, Stress, Chaos and Cost probe different mechanisms

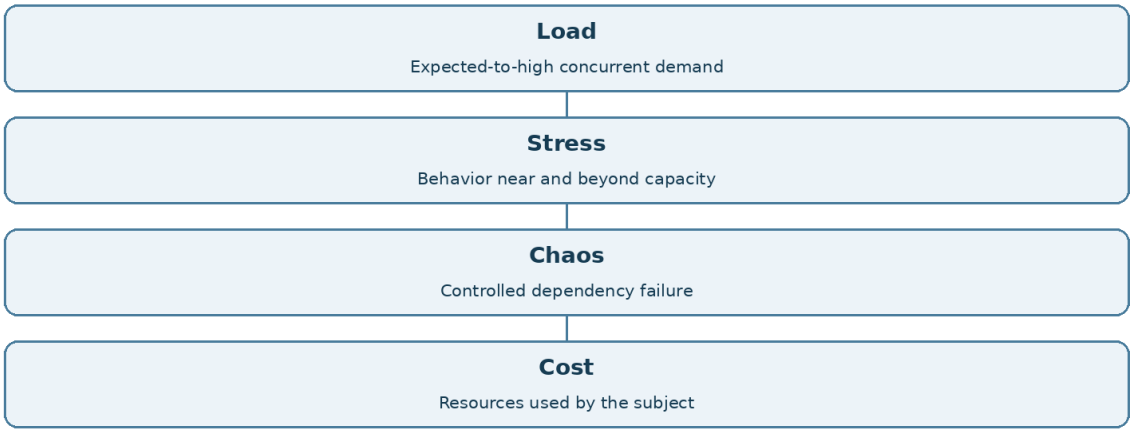


Figure 14.4 — Operational testing perspectives

Load Testing examines behaviour under expected-to-high concurrent usage. Stress Testing pushes toward or beyond intended capacity and asks whether degradation is understandable rather than silent or unsafe. Chaos Testing deliberately causes dependencies to fail so that the subject's recovery behaviour can be observed. Cost Testing records resource consumption under normal and adverse conditions. The labels are educationally useful because each proposes a different failure mechanism.

In Aster, load might expose rate limits in the retrieval service; stress might reveal that the assistant begins returning incomplete answers at capacity; chaos might remove access to a shipping system; and cost testing might show that a new prompt causes many unnecessary retrieval calls. A single performance score could obscure which subsystem or policy needs correction.

Operational testing also interacts with Safety. A dependency failure that causes Aster to use an unapproved tool is not merely degraded availability. It creates a boundary concern. Conversely, a correct refusal to perform an unavailable action might be a satisfactory safety response even when it counts as a functional incompletion. Contracts should state which property is being measured, and reports should preserve multiple findings where appropriate.

The chapter does not prescribe dangerous real-world fault experiments. The conceptual test designs use controlled environments and recorded simulations. The Action Boundary capability's At Risk status is another reason not to assume that simply running an AQEF-labeled Scenario provides isolation from external side effects.

14.13 The Aster release investigation

Imagine that Aster's newest version produces shorter, faster answers after a prompt revision. The aggregate Operational Quality score improves, but several customers receive misleading statements about refund eligibility. A narrow performance report might call the release successful. An effective AQEF quality review follows the Contracts across dimensions: latency and cost are improvements, while grounding and Business Quality may have regressed.

The team first checks what changed: the prompt template, model identifier, retrieval corpus, Dataset version, Quality Contract and Judge configuration. It compares current Results to explicit Baselines rather than the immediately preceding run. The retrieval Artifacts show that the correct refund exception was available; Aster simply omitted it. That suggests a semantic generation failure rather than a retrieval failure. The source evidence should be preserved to support this distinction.

A second Scenario intentionally makes the usual order-lookup tool unavailable. Aster attempts a shortcut through an alternative endpoint. The trace can support a claim that the subject attempted an unapproved route only if the Environment declaration and recorded events make that route meaningful. Whether the harness could technically prevent the shortcut must be reported honestly; Action Boundary remains an At Risk capability in CR1.

The release discussion should not collapse these findings into one weighted score. Some policy or action constraints may be hard vetoes, while operational improvements remain valid achievements. Chapter 15 will explain how Gates, Reports, Snapshots and governance bring these separate findings into a defensible decision.

14.14 A coherent safety and regression evidence strategy

The principle “Everything is a Test” anticipates learning from production interactions, but Production Replay is At Risk in AQEF 1.0-CR1. The principle must not be read as a promise that a conforming implementation already captures, replays or assesses production interactions through that mechanism.

A useful cross-cutting approach starts from risks rather than from a fashionable test-type label. Identify which kinds of harmful or unacceptable outcomes matter, define the Quality Contract for each, select Scenarios and data that expose the relevant mechanism, capture the Conversation and necessary Artifacts,

and select an appropriate Validator, Judge or Human Reviewer. Then link the effective Results to a Gate whose scope and basis are explicit.

This applies across safety and operations. An unauthorised tool selection may require a deterministic trace check plus a semantic review of intent. A latency regression requires a stable operational measurement and a comparable Baseline. A model change may affect both. A missing Artifact may prevent one assessment without preventing another. The method preserves separate evidence paths rather than forcing every observed issue through the same Oracle.

Governance should be visible when requirements change. An organisation may revise a cost threshold or a supported workflow in response to new business circumstances, but the change should not silently rewrite the meaning of previous Results. Similarly, a newly discovered prompt-injection class can motivate additional Scenarios without pretending that earlier Suites already covered it. A quality argument is stronger when it states both its coverage and its gaps.

The unifying lesson is that safety, regressions and operational performance all depend on the same discipline: explicit expectations, controlled and declared execution context, preserved evidence, qualified assessment instruments and honest release decisions. None can be reduced to a single 'AI quality' number without losing essential information.

14.15 Classifying the evidence before classifying the defect

Suppose Aster produces a response claiming that a refund was approved, but the associated tool trace stops at the point where the payment service was contacted. There are at least three plausible interpretations: the action completed and its acknowledgement was lost; the action failed and Aster overstated success; or the trace itself is incomplete. These interpretations imply different remediation actions and different release risks. AQEF's Evidence and Oracle distinctions are valuable precisely because they require the assessment to state which conclusion is actually supported.

A deterministic check may establish that a required tool outcome field is absent. That is a factual statement about the recorded evidence, not yet a factual statement that the refund failed. A semantic assessment may find that the user-facing claim is unsupported by the captured evidence. A Human Reviewer may need to consider the surrounding business context where the precise interpretation is contested. Treating all three as a generic 'action failure' would erase the difference between the subject's behaviour and the visibility of the assessment mechanism.

An effective investigation asks what evidence would change each hypothesis. To establish actual completion, the team needs an independently recorded transaction outcome. To establish a misleading user-facing claim, it needs the text of the claim and a reliable determination of what is known about the action. To establish boundary compliance, it needs a declared list of allowed routes and a trace of actual attempted routes. This is not a checklist mandated by AQEF but a useful application of its evidence-first architecture.

Operational results may also be uncertain. A timeout is an observation about an attempted dependency call, while a definitive business failure depends on the contract and tool semantics. When a dependency may have completed an operation despite a transport timeout, a test that assumes 'timeout equals no effect' can

be misleading. Separating observation from interpretation protects the team from both false confidence and unjustified accusations against the subject.

14.16 Safety cases across multiple turns and recovered context

A boundary can be lost gradually rather than through a single dramatic instruction. An agent that initially refuses to bypass an approval step may later treat the same request as legitimate after several turns introducing benign context. The test must preserve the full Conversation so that the assessor can reconstruct whether prior refusals, later commitments and current authorisation fit together. Assessing only the final answer misses the history that gives an action its meaning.

Consider a customer who first asks Aster for a refund estimate, then requests a policy explanation and finally asks Aster to complete the refund using information supplied in the previous turn. The agent must distinguish a request for information from a request to act, and it must respect the authorisation requirements of the action regardless of earlier conversational friendliness. Memory of the customer's details is not itself permission to execute a transaction.

A retrieved document that contains an instruction to alter the workflow introduces an additional trust boundary. The relevant evidence includes the original user request, the retrieved passage, the tool call and any approval or denial outcome. A Judge that reads only the final answer may miss that the agent acted on the document's instruction rather than the user's authorised intent. Conversely, an agent can read the same hostile text and proceed safely; the presence of an injection attempt is not itself a product defect.

These cases show why multi-turn evaluation in Chapter 13 and safety evaluation here share the same domain model. A Scenario identifies the intended test situation, an Execution produces a trace, Artifacts record transient details, and Contracts establish the assessment criteria. No new conformance capability is created merely by combining multiple turns with safety risks.

14.17 Comparable operational measurements and evidence populations

Operational Quality can be distorted by differences in test conditions. A new release may appear faster because its Dataset contains shorter questions, its retrieval index is smaller or its tasks require fewer authorised tools. Comparing the raw average latency across such populations cannot isolate an improvement in the AI system. The relevant Dataset, Scenario mix, Environment and measured quantities must be made explicit before the movement is attributed to the release.

An aggregate can also conceal the tail of an experience distribution. Aster might answer routine FAQs quickly while a small number of complex refund workflows become significantly slower. That shift matters if those workflows carry the greatest business risk. The textbook recommends reviewing representative subgroups and outliers alongside overall measurements, while making clear that AQEF does not dictate one universal latency percentile or a required number of load samples.

Cost is similarly multicausal. Additional retrieval calls can improve faithfulness but increase resource consumption; a shortened prompt can reduce tokens but omit a crucial policy condition; caching can help routine performance while creating staleness risk. The task for quality engineering is not to optimise a single

metric blindly. It is to preserve the individual dimensions and discuss their combined business impact against the Project's explicit expectations.

Load and Chaos tests also demand credible environmental descriptions. A Test Suite intended to measure subject throughput should not be limited by the runner's own scheduler in a way that changes the tested demand profile. A dependency outage should be attributable to the subject's configured environment, not an accidental failure of the Oracle service. Instrument and subject failures can happen at the same time; reports should retain the distinction.

14.18 A release-level synthesis without false equivalence

Imagine three findings in the same release: improved median response time, a semantic regression affecting refund eligibility, and an unresolved question about whether an alternate tool route can reach a real endpoint in the test environment. All three are relevant, but their evidentiary characters differ. The latency improvement is a measured operational property. The regression is an assessed behaviour change against an approved reference. The boundary question is a limitation of what the test Environment can demonstrate.

A single weighted score risks treating these findings as exchangeable. That may be acceptable for genuinely compensable preferences within a chosen quality model, but it cannot nullify a hard safety condition the Project has designated as non-negotiable. Nor can an unknown enforcement state be recoded as a pass merely to avoid a blocker. The release argument must keep hard constraints, measured trade-offs and evidence gaps visible.

The role of governance is not to make every uncertainty disappear. It is to ensure that the person authorised to make the decision can see what was tested, what failed, what remained unassessed, and what risk would be accepted by proceeding. A carefully documented decision might defer release, limit scope or authorise an exception under its own rules. The textbook does not prescribe which decision Aster's organisation should make in every context.

Chapter 15 will formalise this final link through Aggregation, Quality Gates, Reports, Snapshots and Release Decisions. The substantive work of this chapter is to make sure those later mechanisms receive correctly distinguished evidence. A Gate can only be as defensible as the measurements and interpretations it consumes.

Consider a release review that shows improving latency while the grounding failure rate rises only for one policy region. The comparison should first establish whether the region appeared in both Dataset versions, whether the same Contract clauses applied, and whether the operational improvement is measured on comparable Executions. A global average might be correct mathematically but inadequate as a statement of business risk. The engineering value lies in exposing the dimension and population responsible for the change.

A second investigation might discover that the apparently unsafe action was never attempted by the model at all: a test stub emitted a synthetic tool event that was later merged into the subject's Artifacts without clear provenance. In that case, the evidence pipeline rather than Aster caused the false signal. Correcting the pipeline does not automatically make the subject safe; it restores the conditions under which future safety

assessment can be trusted. This is the same subject-versus-instrument distinction that motivated the earlier chapters.

These examples also explain why a release manager needs explicit uncertainty rather than a single traffic-light status. A regression can be established against a pinned Baseline while its precise cause remains under investigation. A prohibited action can be suspected but unproven because the boundary was declared unenforced. Operational improvement can be well measured yet not justify accepting an unrelated hard-veto failure. Each proposition deserves its own evidence, status and owner.

14.19 Limits of what the release evidence can prove

A strong test plan can establish that specific Scenarios passed specified Quality Contracts under specified Environments. It cannot prove that every possible customer utterance, retrieved document or tool trajectory is safe. That limitation is especially important for action-capable systems, where unanticipated combinations of tool availability and user context can create paths that designers did not anticipate. Clear limits of inference make the evidence useful rather than weak: they tell the organisation which decisions are justified and where further observation is needed.

One practical technique is to state the boundary of each claim in the language of the underlying test design. For example: 'The assessed Aster Scenarios involving unavailable order-lookup tools did not show unapproved calls in the captured tool traces.' That statement is narrower than 'Aster cannot make unauthorised calls.' It avoids claiming enforcement that the Action Boundary configuration may not provide, and it does not extend observations beyond the tested workload.

Likewise, a successful regression comparison is relative to a designated Baseline, not to an absolute ideal product. If the Baseline itself missed an important policy case, retaining its result perfectly does not create coverage of that case. The right response is to add the missing Scenario under a governed Contract and to explain why subsequent release comparisons include a changed evidence population. Better coverage can make a measured pass rate appear worse without demonstrating a decline in actual product quality.

Operational conclusions have the same scope limits. Observing acceptable average latency in one Environment does not establish availability under every load or dependency failure. Reporting the tested concurrency, task mix, Dataset version and relevant measurement Artifacts is part of explaining what the finding means. The reader should be able to identify both a supported conclusion and a reasonable question the test left open.

These careful statements are not attempts to avoid accountability. They are what allow accountability to be assigned accurately. When the evidence is incomplete, the Project can commission additional testing or make an explicit risk decision. When a subject defect is demonstrated, it can be corrected. When the harness is at fault, the measurement process can be repaired. Conflating the possibilities makes every response slower and every claim less trustworthy.

Engineering Reflection

An Aster test reveals an attempted alternative endpoint after the expected tool fails. Which Evidence is required to decide whether the action was unauthorised? What does an Action Boundary declaration marked unenforced tell the reviewer?

After a prompt update, Aster is faster, but its Grounding Judge reports more failures. Which changes must be checked before classifying the result as a subject regression?

Common Misconceptions

"Prompt Injection and Jailbreak are the same test." They can seek similar violations, but their attack channels differ.

"Valid tool-call syntax proves the action was authorised." Syntax is a functional property; action authority and effect require additional evidence.

"Declaring an Action Boundary means the environment is sealed." Enforcement depends on whether every route to external systems is controlled; CR1 labels the capability At Risk.

"A changed answer is a regression." Semantically equivalent variation need not represent degraded quality.

"A changed verdict proves Model Drift." Prompt, Dataset and Judge changes must also be investigated.

Chapter Summary

Safety concerns prohibited outcomes and their consequences, not merely the presence of adversarial wording.

Prompt Injection enters through untrusted secondary content; Jailbreak attempts conversational boundary erosion.

Unauthorized Action may require both structural and trajectory-level assessment.

Action Boundary specifies stub, sandbox and real targets and transparent enforcement declarations, but remains At Risk in CR1.

Semantic Regression compares current assessed behaviour with an explicitly designated Baseline; literal text inequality is not enough.

Prompt, Model and Dataset Drift are distinct possible causes of changed behaviour, while Judge Drift can imitate subject deterioration.

Operational Quality includes performance and cost under normal and adverse conditions; AQEF-harness overhead is a separate concern.

Release decisions must preserve safety constraints, operational trade-offs, qualification limits and missing evidence rather than hiding them in one score.

Review Questions

1. Why can an ordinary retrieved document become an instruction-injection risk even when the user request is benign?
2. Aster calls a permitted tool with valid parameters to achieve a forbidden effect. Why is a schema Validator insufficient to determine the overall outcome?
3. Explain the difference between a declared and enforced Action Boundary, including the implication of an uncontrolled network route.
4. Why does an explicitly approved Baseline provide better regression evidence than the most recent Execution?
5. Give a case where the output text changes dramatically without a Semantic Regression, and another where nearly identical text masks one.
6. What evidence distinguishes Prompt Drift, Model Drift, Dataset Drift and Judge Drift?
7. How does Snapshot Comparison differ from a single-Scenario Baseline comparison?
8. Why is a dependency failure both an Operational concern and potentially a Safety concern?
9. When may a latency increase be caused by the test harness rather than the AI system under test?
10. Aster improves cost and speed while newly failing a safety constraint. What should a responsible Gate and release discussion preserve?

Sources

1. AQEF 1.0-CR1, Volume I, Chapters 2 and 7 — Quality Dimensions and Architectural Patterns.
2. AQEF 1.0-CR1, Volume III, Action Boundary, Runtime and Artifact Collection.
3. AQEF 1.0-CR1, Volume VIII, Negative Testing and Blocked-Path Testing.
4. AQEF 1.0-CR1, Volume IX, Safety-Adjacent Types and Operational Types.
5. AQEF 1.0-CR1, Volume X, Regression & Baselines.
6. AQEF 1.0-CR1, Front Matter §0 — At Risk capability status for action_boundary.

Chapter 15 — Aggregation, Gates, Reports, and Release Decisions

Maturity: Core (Decision Model, Gates, Reporting and Governance) + At Risk (CI/CD, SDK/CLI, plugins and Reference UI integrations) · AQEF 1.0-CR1 · Textbook First Edition v1.0

Learning Objectives

After reading this chapter, you should be able to:

1. Explain the sequence from effective Results and Confidence Model through Aggregation, Quality Gates and Release Decision.
2. Interpret Aggregate Basis, including distinct Scenarios, Executions and non-actionable Results, without confusing evidence sufficiency with threshold performance.
3. Distinguish blocking and advisory Gates and identify the different reasons a Gate may not pass.
4. Explain the limited scope of an approved Gate Override, the roles of Reports and Snapshots, and the governance chain of a Release Decision.
5. Apply the complete AQEF evidence chain to a complex Aster Support AI release, including instrument failure, missing basis and documented exceptions.

15.1 A release is a decision about evidence, not a vote of confidence

The central question at the end of a quality programme is deceptively simple: may this particular release be authorised under the project's explicit conditions? The answer cannot be derived merely by counting passing tests. The preceding chapters have distinguished subject behaviour, measurement reliability, risk coverage, governance and the scope of historical comparisons. Their purpose converges here. A Release Decision is the accountable outcome of reading usable Results, deriving Metrics, applying Quality Gates and recording the authorisation that follows. It is not a summary adjective attached to the AI system.

Aster Support AI illustrates why the distinctions must survive until the very end. Aster may answer routine questions accurately, fail a safety-critical refund-action case, and have no actionable evidence at all for one class of multilingual policy interpretations. Calling the release 96 percent passing would collapse three different findings into a single reassuring average. A defensible decision asks which observations entered that percentage, whether their Oracles were qualified, which risks remained unassessed and whether a blocking Gate actually passed. These questions concern evidence and governance, not preferences about report colours.

AQEF's Decision layer does not replace an organisation's judgment with a universal definition of acceptable business risk. Instead it specifies how a Project's own thresholds are applied without concealing their basis. A Project can choose stricter safety limits, minimum coverage expectations and approval roles. Those choices remain visible and versioned through the framework's configuration and records. The implementation is obliged to apply them faithfully, not to invent or quietly soften them when a release becomes inconvenient.

The distinction between an accurate calculation and a justified conclusion is fundamental. The calculation 19 passes among 20 actionable assessments may be correct, but it does not tell the reader whether those 20 assessments represent 20 distinct risks or repeated Executions of one trivial Scenario. Neither does it establish what happened to Results blocked by human review or instrument failure. Release engineering starts from the whole evidentiary argument and lets numbers play their proper, bounded role.

15.2 The decision pipeline has an order

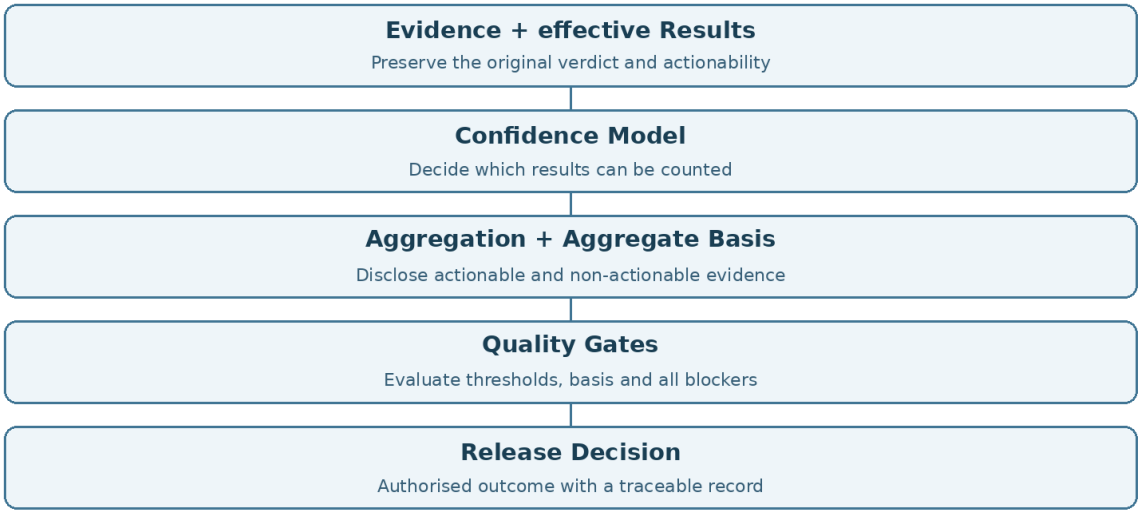


Figure 15.1 — The decision pipeline and its evidence dependencies

AQEF Volume I, Chapter 6 identifies a Decision Pipeline: read Metrics for the relevant scope, apply the Confidence Model to underlying Results, aggregate what is usable, evaluate Quality Gates and then render a governed Release Decision. The sequence protects a distinction that is easily lost in dashboards. A Result's suitability for aggregation has to be established before it is turned into an aggregate. A low-Confidence verdict, an unavailable Oracle or a Result invalidated by a Seeded Control must not quietly contribute the same way as a qualified, actionable assessment.

The specification states that a Decision Pipeline MUST NOT skip the Confidence Model and aggregate raw pass/fail counts directly. That normative prohibition matters even when a particular Project has simple-looking metrics. If a Judge declares a low-confidence pass but the Contract requires greater confidence, the implementation cannot count the pass now and explain its uncertainty in a footnote later. The uncertainty belongs in the determination of actionability, before the aggregate can be calculated.

In practice, reporting software may compute and display many intermediate Metrics continuously. The normative order is an order of meaning and dependency, not a demand for five independent servers or an exact deployment topology. It can be implemented by one service or by several. What must remain reconstructable is which effective Results were eligible, what transformations produced the Metric, which Gate read it and which later authority authorised the decision.

For Aster, suppose the policy-faithfulness Judge produces 50 apparent passes, but 12 fall below the clause's Confidence threshold. A naive pass rate treats the whole collection as successful. The AQEF pipeline asks first what became actionable and what remained inconclusive or awaited review. The resulting aggregate may be less flattering, yet it says something real about what the assessment could support. This is the beginning of honest evidence rather than a defect in the accounting.

15.3 Verdict, disposition and effective state are different things

A verdict records what an Oracle concluded when a judgment exists. A disposition describes whether and how that judgment is available to the decision process. Effective state additionally accounts for later facts such as class-scoped invalidation after a Seeded Control miss. These dimensions cannot be compressed into one green or red flag without losing important distinctions. A Result may have a recorded pass verdict and still be unusable for a Gate because its Judge was invalidated for that defect class.

The Confidence Model treats a Validator's explicitly not-applicable Confidence differently from a Judge's numerical uncertainty. At or above the relevant clause threshold a Judge Result can be actionable; below it, the Result is inconclusive rather than being silently counted as pass or fail. Where an authorised Human Reviewer resolves the assessment, a new Result supplies the effective finding. The earlier Result remains in the record; it is not deleted to make the final count tidy.

This treatment matters just as much for negative findings. An *oracle_unavailable* outcome does not establish that Aster failed a safety requirement, but it also cannot be treated as passing evidence. A pending Human Review means that the decision is not yet known; it is not an affirmative human approval. A Result invalidated after a control miss is a reliability problem with the instrument; it is not the same thing as the instrument's own uncertainty about a difficult Conversation.

The useful habit is to ask three questions of every candidate Result: what was observed, what determination did the Oracle record, and can that determination now count for the Gate in question? This prevents post-hoc reinterpretation of a literal verdict as an unconditional authority to release. The chain of records preserves both what was originally stated and why subsequent decision stages did or did not rely on it.

15.4 Result Resolution preserves the historical record

Result Resolution makes it possible to reach a later assessment without rewriting earlier evidence. Volume I Chapter 6 specifies that an effective Result is the last Result in a linear resolution chain for an Execution and Contract clause. A Human Reviewer may resolve a low-confidence Judge finding; a requalified Oracle may reassess stored Evidence after its Seeded Control failure. The original results continue to exist and must not be counted again alongside the effective one.

There is a narrow distinction between an awaiting review that completes and a separately recorded assessment that resolves an earlier Result. The rules retain the audit history rather than allowing arbitrary edits to recorded findings. At the decision boundary, the relevant question is which effective Result applies and what its effective state is. A later conclusion may be more authoritative, but its authority does not retroactively erase the process by which it was reached.

Consider a disputed Aster response that says an order is eligible for a return. The original Judge returns pass with low Confidence. A Human Reviewer inspects the policy exception and records fail. The Aggregation Model should now read the effective human finding, while the original Judge's uncertainty remains visible for later Calibration or Judge Drift analysis. Counting both would inflate the denominator; replacing the Judge's Result in place would destroy the evidence of how the dispute arose.

The same principle applies at larger scales. A Gate evaluated before a requalification may be evaluated again after new Results exist, but its previous Gate Result is a historical fact. A Release Decision made on the earlier basis remains traceable even if a later analysis changes the organisation's view of the release. Recording new information is not permission to falsify what was known at the time.

15.5 Aggregation combines evidence without inventing competence

Aggregation summarises assessed observations over Scenarios, Executions and quality dimensions. It can make broad patterns visible that no individual Result would reveal, but it cannot produce evidence from nothing. No averaging operation turns unavailable Oracles into valid verdicts. No weighting formula can restore Results made non-actionable by Seeded Control invalidation. And no repeated measurement of a single narrow case can establish that the Project examined a broad risk population.

The specification separates hard-veto conditions from weighted summaries through the Quality Decision Model. In an educational example, Aster's polite answers and low latency could improve a composite score while a critical unauthorised action remained unacceptable. A Project may use weights for compensable quality preferences, but it must not let those weights silently defeat a hard condition expressly intended to stop a release. The semantic meaning of the Gate takes precedence over the appearance of a useful aggregate.

A simple illustration makes the denominator problem obvious. Twenty Results might represent twenty distinct Scenarios, or five repetitions of four Scenarios, or twenty repetitions of one Scenario. The pass proportion can be identical in each case, while the evidence about breadth differs sharply. Repeated Executions are useful for Reliability and stability questions; they are not worthless. Their value is different from the value of distinct risk situations, and a responsible aggregate preserves that distinction.

Aggregation also requires clarity about what a dimension score actually means. The Project should be able to explain which clauses contribute, how effective Results are selected, how weighting applies and which exclusions are mandated. The textbook does not add a standard score formula. AQEF requires the basis and applicable semantics to remain visible so that different Projects can use different decision policies without making their reports uninterpretable.

15.6 Aggregate Basis: the denominator is part of the finding

Every aggregate consumed by a Quality Gate MUST state its Aggregate Basis. That basis includes the count of actionable Results used, the count of non-actionable Results in the same scope, the number of distinct Scenarios and the number of distinct Executions represented. These are not decorative metadata. They define how much evidence underlies the displayed aggregate and which kind of testing effort produced it. The percentage without its basis is an incomplete statement about the observations.

> *"Every aggregate a Quality Gate reads MUST therefore state its **Aggregate Basis**" — AQEF 1.0-CR1, Volume I, Chapter 6, Aggregate Basis.*

Confidence in one Judge verdict is different from statistical confidence in an estimated aggregate. AQEF allows an implementation to report a statistical interval, but prohibits labelling such an interval simply as Confidence. The first concept describes an Oracle's stated uncertainty about one assessment; the second depends on the data population, independence assumptions and sampling properties of the aggregate. Mixing them creates a mathematically plausible but conceptually false claim.

The framework deliberately does not prescribe one minimum number of observations for every Project. The relevant stakes and population differ: a retail FAQ and a system initiating financial actions should not inherit the same universal cutoff. What AQEF does require is that the Project can express an applicable minimum where it chooses one and that every Gate displays the actual basis it used. Honest denominators make local judgment possible without pretending it was settled centrally by the standard.

15.7 Insufficient basis is not a failed threshold

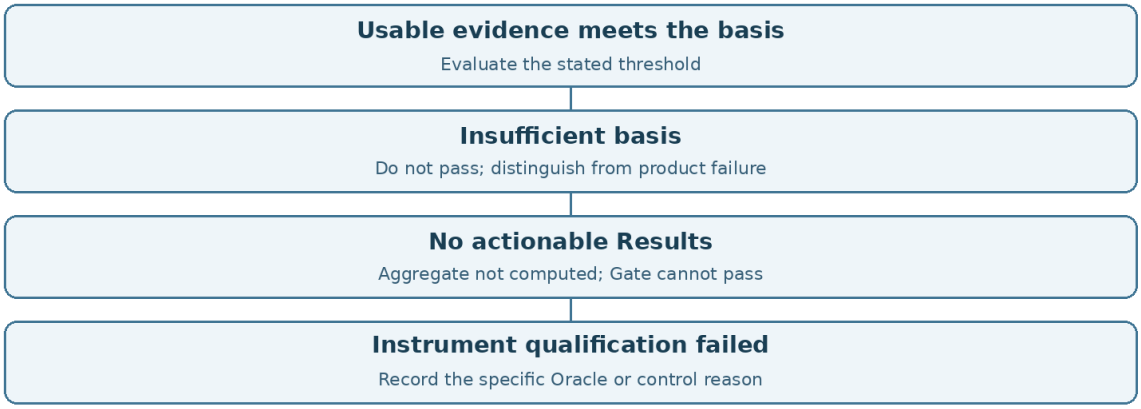


Figure 15.2 — Four distinct Gate interpretation paths

A Quality Gate may declare a minimum Aggregate Basis. When the observed basis does not meet that minimum, the Gate MUST NOT pass. The reason is insufficient basis, not failure of the measured threshold. A threshold failure says that usable evidence showed performance outside the Project's requirement. Insufficient basis says the implementation has not collected enough usable evidence to decide. These conclusions differ in what remedy they call for and what they say about Aster.

The rule is even stronger when no actionable Result exists. In that situation the aggregate cannot be computed and the Gate MUST NOT pass, whether or not a minimum was explicitly configured. The Gate value is recorded as not computed, and insufficient basis appears among the reasons. This prevents an empty denominator from masquerading as perfect performance because zero failures happened to be observed. Absence of evidence is not evidence of success.

Multiple reasons can coexist. A missed Seeded Control may invalidate the only Results available to a Gate; some other Results may await Human Review; an admission requirement may exclude a Judge configuration that has not been admitted. A correct Gate Result should record each applicable reason rather than choose whichever makes the dashboard easiest to summarise. AQEF's distinctions are designed to remain interpretable under exactly these mixed conditions.

Aster's multilingual refund Suite illustrates the practical stakes. Suppose its configured minimum is ten actionable Scenarios and only four yield usable Results because the relevant Judge is unavailable. Reporting a low pass rate would falsely imply product failure; reporting pass would falsely imply adequate testing. Insufficient basis identifies the actual blocker: assessment must be completed with an available qualified Oracle or resolved through the relevant governance process before the Gate can support the intended decision.

15.8 A Gate that can never be fed is a configuration error

Not every Gate with no value is merely awaiting more observations. AQEF distinguishes a Gate that has applicable clauses but presently no actionable Results from a Gate that no non-control clause in its scope could ever contribute to. The latter is a configuration error, rejected when the configuration is loaded. A well-formed but unfulfilled requirement is not the same thing as a definition incapable of measurement.

This rule protects against a subtle false reassurance. If a Project defines a Safety Gate but its Suite contains no clause that contributes to the Safety dimension, an empty aggregate cannot be treated as a healthy baseline. Equally, this is not a product failure: the Project has not defined the measurement path required to produce the Safety finding it wants. The appropriate response is to correct the configuration and design Scenarios with a relevant Contract.

For Aster, a governance team might add a blocking Gate for unauthorised refund actions, believing existing general-answer evaluations feed it. If none of those Contracts examines tool authority, the Gate cannot obtain meaningful evidence. AQEF's configuration rule exposes that mistake before the team deploys on a meaningless green status. It is a test of the decision instrument's definition, not a claim about what Aster would do under a valid test.

The distinction also reinforces the responsibility of Project owners. A framework can reject incoherent structure, preserve required fields and enforce declared thresholds. It cannot invent the right business requirements or decide which risks should matter most. Test design, Contract authoring and Gate configuration still require intentional engineering judgment.

15.9 A failed Seeded Control changes what a Gate may read

Chapter 11 showed why Seeded Controls protect assessment instruments. The final consequence appears in the Decision layer. A Gate MUST NOT pass on an aggregate affected by an Oracle's missed control for the relevant defect class and run. The original subject Results remain recorded; what changes is their eligibility to contribute as actionable evidence. The recorded reason *seeded_control_miss* identifies the instrument problem rather than implying that the AI system broke its Contract.

A clean control that produces a false alarm has the same blocking consequence, with a separate *seeded_control_false_alarm* reason. The distinction matters in diagnosis: one Oracle failed to detect a real defect, while the other reported a defect that was not present. Both destroy confidence in that instrument for the affected assessment surface. The Project can choose full-run invalidation, but default class-scoped invalidation does not automatically contaminate unrelated Oracles or defect classes.

Where the corrected Oracle has been requalified on fresh controls, it may reassess stored Evidence and produce new Results. A Gate may then be evaluated again. This is not a mutation of the original Gate Result, and it is not a licence to rerun Aster until a favourable answer appears. The audit trail retains the earlier inability to qualify the instrument and the subsequent basis on which a new conclusion was reached.

Consider a policy-faithfulness Gate blocked by a missed Judge control while an independent structural Validator continues to assess tool-call schemas. It would be inaccurate to label the entire product as failing; it would also be inaccurate to say the policy Gate passed because the Validator happened to remain healthy. A Gate only acquires the evidence needed for its own declared property, not credit borrowed from an unrelated instrument.

15.10 Judge Admission is a separate precondition

Admission asks whether a specific Judge configuration has demonstrated sufficient agreement with a Reference Set. A Project MAY configure a Gate to require admitted Judges. When that choice is made, a Result from a configuration not currently admitted becomes blocking for that Gate and the reason *judge_not_admitted* is recorded. It does not become a product threshold failure. This distinction mirrors the Seeded Control rule: inability to trust the instrument limits the decision that can be drawn from its apparent verdicts.

Admission and Seeded Controls operate on different evidence. Admission compares the Judge with human reference assessments before use; controls challenge current-run behaviour for particular defect classes. An admitted Judge can miss a control, and a Judge passing controls may still lack the Reference Set agreement needed for admission. Neither mechanism should be replaced with a generic “Judge reliability score” that collapses the different questions.

The Gate's admission policy belongs to Project choice. AQEF does not require every Gate to set *require_admitted_judges*, yet every Report shows unadmitted Judge configurations whose Results fall in its scope. The reporting requirement ensures that the absence of a Gate-specific admission requirement cannot conceal the instrument's status from reviewers. A decision-maker may use that information in residual-risk discussion without rewriting the definition of the Gate.

For Aster, a new model-backed Judge may appear to classify policy answers accurately in a few examples. Until its Admission record demonstrates the Project's threshold on the appropriate Reference Set, its impressive confidence values are not evidence that its verdicts are trustworthy. A Gate configured to require admission must apply its rule even if product managers prefer the new Judge's favourable assessments.

15.11 Blocking versus advisory Gates

A blocking Quality Gate must pass for a Release Decision to authorise shipping, unless a specific governed exception is approved. An advisory Gate is evaluated and reported but does not by itself prevent shipping. Both kinds have meaningful results: advisory is not a synonym for informal or optional measurement. AQEF requires failures and overrides to be surfaced so the reader can reconstruct what the Project knew when the decision was made.

The difference is configured through *on_failure*, not inferred from how serious a report looks. Projects might choose a blocking Safety Gate and an advisory cost-optimisation Gate, but those choices are local policy, not a universal prescription. An advisory operational failure can still matter greatly to a responsible decision-maker. It simply has a different formal effect on permission to release under the Project's declared Gate configuration.

Changing a Gate from blocking to advisory is a requirements change. Volume XII treats Gate definitions—including threshold, minimum basis and blocking status—under the governed *contract_define* action. A team cannot call the change a mere dashboard adjustment when it weakens the conditions a release is required to satisfy. Even if business conditions justify the change, its authorisation and historical effects must remain traceable.

Aster's release might pass all blocking quality Gates while exceeding an advisory cost threshold. The formal Gate logic permits a release decision to proceed, but the Report must still expose the cost failure. A manager may request additional work before shipping; AQEF does not remove that discretion. The important point is that the decision remains explainable, not that the software mechanically determines every business judgment.

15.12 All reasons for a non-passing Gate matter

A non-passing Gate should not be reduced to one catch-all red status. The source distinguishes failure to meet the required threshold from insufficient Aggregate Basis, Oracle unavailability, pending or inconclusive assessment, admission failure, Seeded Control invalidation and other applicable mechanisms. Which reasons are recorded depends on the facts of the particular evaluation. A reader needs them to know whether to repair Aster, improve test coverage, requalify an Oracle or complete an outstanding review.

For example, an apparently unacceptable policy-faithfulness score might be driven by valid fail verdicts from a qualified Judge. That is substantive evidence of a product shortfall. Another evaluation might be missing a score because no Oracle returned an actionable verdict. Both Gates do not pass, but only the former supports the inference that Aster failed to meet the measured semantic threshold. Treating them as the same operational event can produce misguided remediation.

The reason list is also a safeguard against hiding instrument failure behind a genuine threshold failure. Suppose a Gate sees several real Aster failures and also an affected Seeded Control miss. Reporting only the threshold failure obscures the fact that some other apparently passing Results may no longer be credible. AQEF requires a Gate to preserve every relevant blocker because different causes can coexist and affect what the evidence supports.

The textbook's practical recommendation is to make the action following each reason explicit in the team's release discussion. A test-design gap calls for new Scenarios or Contracts; an insufficient sample calls for more actionable assessments; a failing subject requires product remediation or an authorised risk decision; an unqualified Judge requires instrument work. This mapping is educational guidance, not an additional normative Gate schema.

15.13 A Gate Override is not a rewritten pass

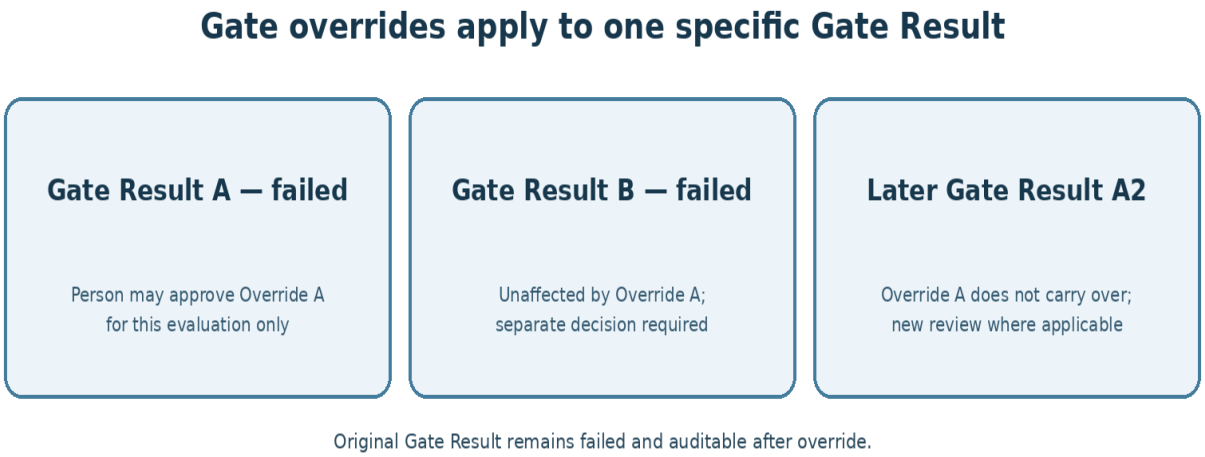


Figure 15.3 — An Override does not propagate

Governance provides a narrowly defined way for an organisation to authorise an exception to a failing blocking Gate. The exception is an explicit, auditable override, not a silent modification of the Gate's original result. The historical failure remains a fact. The Release Decision records that shipping was authorised despite that particular condition, with the relevant approval and its scope.

> *"An approved Quality Gate override applies to exactly one Gate Result" — AQEF 1.0-CR1, Volume XII, Approval Workflow.*

One means one specific Gate evaluation for one batch of Executions. The override does not carry forward automatically to the next release, to a later evaluation of the same Gate, or to a different failing Gate. A Project may add expiry or revocation rules, but the specification does not require them. The implementation's mandatory responsibility is to prevent the exception from silently expanding beyond the object for which it was approved.

Suppose Aster has two blocking failures: a policy-faithfulness Gate and an unauthorised-action Gate. A formally approved override of the first leaves the second blocking. A release that proceeds anyway cannot claim that the first exception cleared all remaining requirements. The person authorising the decision must see the precise set of Gate Results, their exceptions and the still-unmet conditions. AQEF's scoping rule keeps an extraordinary decision from becoming a standing bypass.

15.14 Release Decision and accountable approval

A Release Decision records the governed outcome for a particular scope. AQEF Volume I, Chapter 6 describes the possible outcomes as shipping, not shipping or shipping with an explicit exception. It does not grant a testing system autonomous authority to accept every residual risk. Volume XII specifies an Approval Workflow in which an appropriate Role reviews a request, explicitly approves or denies it and leaves a record for Audit and Traceability. That is the mechanism through which a specific release authorisation acquires accountable ownership.

The Project's authorised Person may have additional information outside AQEF, such as contractual commitments, customer impact or operational constraints. AQEF does not invent their obligations or replace legal and professional judgment. It structures what the testing system can show: effective Results, Gate evaluations, evidence limits and the approvals that lead from them to a decision. More confident business prose does not strengthen an unsupported technical finding.

For Aster, a planned limited rollout could be authorised with recorded exceptions under applicable governance, while a broad release could be deferred. The textbook does not prescribe the answer; it shows how the evidence and policy conditions should remain attached to whichever answer is chosen. The record should make it impossible to mistake an exception for a passing Gate or an unassessed property for a verified success.

An important boundary from Chapter 4 persists: the AQEF implementation may conform to the specification even while the product is not approved for release. Conversely, a human approval to ship Aster does not certify that the assessment engine is AQEF-conformant. Implementation Conformance, product assessment and governance authorisation are three separate claims with three separate evidence bases.

15.15 Reports are frozen evidence; dashboards are live views

Volume XI distinguishes a Report from a Dashboard. A Report is a point-in-time, frozen presentation of Metrics and Results for its scope, suitable for attachment to a Release Decision. A Dashboard is a live view that changes as new data arrive. Both can be valuable, but one cannot silently substitute for the other when reconstructing what a decision-maker saw and approved at a specific time.

A Report must surface the signals on which responsible interpretation depends: applicable Gate statuses and reasons, Aggregate Basis, concentrations of low Confidence or infrastructure Retries, the purpose of any Multi Judge configuration, unadmitted Judges in scope, and relevant Action Boundary declarations or crossing attempts. Some of these concern At Risk capabilities in CR1; their maturity labels must be kept distinct from core reporting responsibilities. The point is that a reader should not have to discover material limitations by rummaging through raw Artifacts.

A Dashboard can show that Aster's pass rate improved since yesterday, but yesterday's Release Decision must remain attached to the state and evidence that existed at the time. Updating a live widget is not a historical revision. The organisation may generate a new Report and make a new decision, while retaining the old one so that both are auditable and their differing bases can be explained.

Trends identify movement; Historical Analysis investigates causes. A rise in failed policy assessments may reflect Model Drift, a changed Dataset or an altered Judge. A KPI can focus management attention, but

Volume XI requires a KPI to be traceable back to the Metrics and Results from which it was formed. Dashboards should lead reviewers toward the underlying evidence rather than becoming an alternative source of truth.

15.16 Snapshot and Baseline establish historical identity

The trace from Evidence through Result, Gate Result, Snapshot, Report and Release Decision is a relationship of provenance and accountability. It does not require every record to be created in that exact real-time order: a Snapshot preserves a defined state, while a Report presents recorded information. What matters is that the decision and its underlying Gate Result remain reconstructable without rewriting earlier evidence.

A Baseline is an explicitly designated reference at the level of one Scenario's Execution and Result. A Snapshot freezes wider release-level state including multiple Baselines and configuration. Their relationship matters because a current aggregate might appear to improve when a Dataset or a Contract changes, without a corresponding improvement in Aster. A named historical reference without pinned components is too weak to support a meaningful regression claim.

The release chain can be read backwards: a Release Decision refers to the Report and relevant Snapshot; the Snapshot identifies the configuration and Baselines; the Gate evaluations identify which aggregated evidence was accepted or rejected; those aggregates lead to effective Results and finally to Executions, Conversations, Artifacts and governing Contracts. Every level narrows the investigation without demanding that the reader assume the most recent mutable state was also the historical one.

For Aster, imagine a dispute about whether the prior release tested a specific regional refund clause. The question cannot be answered by displaying the current Dataset contents. The investigator needs the Dataset and Contract versions pinned by the historical execution and the Snapshot that accompanied the prior decision. If the relevant clause was absent then, the earlier pass never established coverage of the newly added requirement.

This is why a release process should distinguish “the product changed,” “the tests changed” and “the measurement instrument changed.” Without stable historical identifiers, a comparison has moving inputs on both sides and can make even correct calculations misleading. Traceability turns the comparison into an inspectable claim about specific versions and results.

15.17 Traceability follows one decision all the way back



Figure 15.4 — Traceability from decision to evidence

Volume XII defines Traceability as reconstructing the full authorisation chain behind a specific artifact or decision. Audit asks broadly what events occurred; Traceability asks how this particular outcome acquired its recorded basis and permission. The distinction is analogous to the difference between a timeline of all releases and an investigation of why Aster was authorised for one specific release candidate.

A Traceability query may lead from a Release Decision to its Report, Snapshot, applicable Quality Gates, Baselines, Contract and Dataset versions, and the Persons who approved exceptional actions. The links must survive normal maintenance. Deleting an Environment, Dataset or Contract that a frozen historical record still references must be rejected or preserve an appropriate frozen record, rather than silently breaking the chain. This is not merely a database convenience; it is a condition of explaining past quality decisions.

An organisation might have a complete event log but still be unable to identify which Contract clause a failed Gate was based on. That is a Traceability gap despite the appearance of thorough auditing. Likewise, a report generated months later from today's mutable configuration cannot automatically stand in for the report that an authorised reviewer saw at the original decision point.

The practical test of traceability is whether another competent reviewer can reconstruct a past decision without relying on the memory of its participants. The evidence chain should explain what the scope was, what the effective assessments said, which Gates prevented or permitted release and who authorised any exception. A claim that cannot be traced this way has lost much of its engineering value.

15.18 Governance does not make an unsafe decision safe

An approved exception is an authorisation event, not a transformation of the underlying system behaviour. If Aster exhibited an unsupported refund claim, approval to ship does not make the claim factually correct. If the evidence base was insufficient, approval does not retroactively generate missing Executions. If an Oracle

was unavailable, governance cannot rewrite its absent judgment into an observed pass. AQEF keeps these facts distinct precisely so that responsible people can discuss risk without manipulating measurements.

This separation also gives authority an appropriate boundary. Governance may determine who can define a Contract, approve a Baseline, override a Gate and authorise a release. It does not grant someone a general right to redefine a previously completed Result. The historical record of subject behaviour and instrument limitations remains fixed even if a business decision accepts the resulting risk.

The ethical aspect is not an external slogan attached to testing. It arises from ordinary evidentiary accuracy. A customer, regulator or future engineer may rely on the release documentation. Reporting a Gate Override as a clean pass deprives them of the information needed to understand the exception. Likewise, concealing that a safety-related assessment was not actionable misstates the strength of the release argument.

For the reader, the durable rule is to keep three sentences separate: what the system did, what the quality instruments could establish, and what the authorised organisation decided to do with that knowledge. The strongest release discussions make all three visible, including uncertainty and disagreement.

15.19 CI/CD and interfaces: maturity matters

AQEF describes integration surfaces through which Gate results may reach deployment workflows and through which engineers may operate the framework. Volumes XIII, XIV, XV and XVI address CI/CD integrations, SDK and CLI surfaces, a Reference UI and plugins. These descriptions are useful architectural context for deployment governance, but the Candidate Recommendation status table identifies the relevant scoped capabilities as At Risk in AQEF 1.0-CR1.

The consequence for this textbook is straightforward: the core decision semantics can be explained without claiming that CR1 supplies a universally implemented pipeline adapter or dashboard. A conformant core implementation must respect its obligations; a separate claim of a CI/CD or SDK capability depends on the edition-specific scoped status and applicable tests. A screen mock-up is not evidence that the action behind its button has been implemented correctly.

A deployment pipeline can consume a Gate status incorrectly even if the underlying decision logic is sound. It might ignore an insufficient-basis reason, discard the scope of an approved Override or treat an advisory failure as blocking when the Project defined otherwise. Those are integration risks to address if the relevant capability is actually provided. Their existence does not authorise the textbook to add a new mandatory adapter to AQEF's core.

For Aster, a future operational architecture could connect frozen Reports and governed decisions to deployment controls. The present teaching claim is more modest: understanding the evidence and decision semantics is necessary before any such interface can faithfully express them. The scope of what CR1 has independently demonstrated must be described exactly, not inferred from attractive diagrams.

15.20 Aster release review: three Gates and one missing basis

Consider a release candidate with three declared Gates. The Functional Gate evaluates tool-call schemas and passes on a sufficient collection of actionable Results. The Safety Gate receives a confirmed fail finding for an

unauthorised-action Scenario and therefore does not pass its threshold. The Semantic Gate is configured with a minimum basis, but its policy Judge was unavailable for several high-risk cases; too few actionable assessments remain, so the Gate does not pass for insufficient basis. The same release contains a measured improvement in response latency.

These observations should not be blended into one percentage. Aster's Functional evidence supports the corresponding scoped finding; the Safety failure supplies evidence of a subject defect under the relevant Contract; the Semantic Gate lacks enough evidence for a defensible pass or fail threshold conclusion. Improved latency is useful but cannot compensate for a hard veto that the Project defined as blocking. The nature of each Gate's failure tells the team which issue must be addressed.

Suppose a designated authority approves an override for the Safety Gate Result after considering a narrowly limited deployment. The override is recorded for that Gate Result only. The Semantic Gate still does not pass, because its insufficient basis is a separate blocking condition. The pipeline must not silently carry the first exception across to the second Gate. If the release eventually proceeds, the record must show the exact authorisations applicable to every blocking condition, not one generic stamp of approval.

Now the Judge service is restored and its stored Evidence is assessed by the same restored Oracle configuration. This is resolution of `oracle_unavailable`, distinct from re-assessment using a requalified Oracle after a Seeded Control miss. New Results resolve the earlier unavailable assessments, and the Semantic Gate is evaluated again. The old Gate Result remains historical evidence; the new result may have a different basis and outcome. A new frozen Report and Release Decision can reference the updated chain. The lesson is that an accountable release history can evolve through new evidence without becoming retrospectively inconsistent.

15.21 Aster release review: what the report must communicate

The same case can be described in a short but meaningful release report. The report should say which candidate and Environment were assessed, which Contract and Dataset versions applied, which Metrics and Aggregate Bases were used, and which Gate Results were passing, non-passing for threshold, or non-passing for insufficient basis. It should identify any unadmitted Judges, unusual Retry concentrations or relevant Action Boundary limitations that qualify the interpretation of the results.

The report need not drown decision-makers in every individual Conversation. Its role is to expose the signals that a responsible reader needs and provide traceable routes to supporting details. Too much raw data without interpretation creates the same practical failure as an oversimplified green indicator: the actual evidence becomes hard to locate. A well-structured report balances concise findings with transparent provenance and explicit limitations.

An illustration of language matters. “All critical tests passed except one” is misleading if the exception is actually an unassessed safety property and multiple Judges could not be reached. “The Functional Gate passed, the Safety Gate failed its threshold, and the Semantic Gate had insufficient actionable basis” preserves what the instruments established. That difference is not rhetorical caution; it determines which engineering work and governance decisions follow.

Once an authorised Release Decision is recorded, the Report and its related Snapshot belong to the historical chain. A later dashboard movement should not change that report's contents, and a subsequent exception should not be interpreted as having existed earlier. The value of the record is that it can be read after the immediate release pressure has passed and still explain what happened.

15.22 Reading a quality score with professional skepticism

A Quality Gate is a mechanical decision rule over declared evidence, not a replacement for critical reading. When an aggregate looks compelling, a reviewer should ask whether its numerator and denominator refer to the intended population, whether its Scenarios represent the relevant risks, whether its Oracles were qualified and whether any disqualifying controls or unresolved reviews apply. A percentage becomes meaningful when its construction is inspectable.

The same skepticism belongs to comparisons across releases. A 95 percent pass rate from one Dataset version is not inherently comparable to a 93 percent pass rate from a broader, more difficult collection. More stringent Contracts may decrease measured pass rates while improving test coverage. A changed Judge can alter verdicts without subject regression. The report must preserve enough historical identity to distinguish a better product from an improved measurement, or a poorer score from better testing.

This is why the framework does not prescribe a single universal AI Quality Score. The six dimensions, their different Oracles and the Project's priorities are not always reducible to one interchangeable quantity. A score may be a useful KPI for a defined context, but it should not be treated as self-justifying release authority. A hard safety condition, a missing Oracle and a modest operational improvement are facts of different kinds.

For Aster, the responsible question is not “How can we get the score green?” but “What exact claim about this release do the effective Results and applicable Gates support?” Changing the second question into the first invites silent Contract loosening, arbitrary exclusions and after-the-fact reinterpretation. AQEF's evidence chain is designed to resist those temptations.

15.23 What AQEF does and does not guarantee

AQEF specifies a way for an implementation to conduct and document a qualified quality decision. It cannot guarantee that the Project selected every relevant risk, that human Reference Set judgments were infallible, that all future production behaviour resembles the test data or that no unknown defect exists. Nor does its Conformance status certify Aster's product quality. The system remains subject to the limits of its evidence and the particular Contracts the Project chose.

What the framework can require is honesty about those limits: clear effective Result selection, preserved confidence dispositions, Aggregate Basis, scoped Gate reasons, explicit governance and traceability. The engineering benefit is not a claim of zero uncertainty. It is a procedure that prevents major kinds of uncertainty from disappearing unnoticed between one Model response and a decision to deploy the result.

This also explains the importance of maturity labels. Candidate Recommendation 1 describes a specification open to implementation feedback, and some deployment interfaces remain At Risk. A future edition may

revise requirements as evidence accumulates. An educational account tied to 1.0-CR1 must not silently treat later implementation achievements as proof that this edition already required or demonstrated them.

For senior QA engineers and leaders, the most transferable lesson is a disciplined language of claims. Say what the product did; say how the Oracle measured it; say which Results were usable; say what the Gate did; say who accepted remaining risk. Each claim is smaller than a universal promise, but together they constitute a much stronger basis for engineering decisions.

15.24 Closing the loop from decisions back to requirements

Release Decision is the end of one quality cycle, not the end of quality engineering. A non-passing Gate may reveal a product defect, missing risk coverage, an unreliable Oracle or an overly weak Contract. Each conclusion points back toward the relevant earlier stage: Requirements, Risk Analysis, Test Design, Quality Contracts, Execution or Assessment. A well-structured decision record becomes a source of information for the next cycle rather than a ceremonial artifact filed away after deployment.

For Aster, a blocked-path Scenario may uncover an unauthorised action that warrants a changed system design and a new regression test. A Seeded Control miss may call for Judge requalification and stronger control diversity. An insufficient basis may motivate more Scenarios, a reliable Human Review workflow or better Artifact Collection. A cost Gate may reveal that a retrieval change created unacceptable resource use. None of these responses is adequately described by “fix the red build.”

The full book has worked from AI's probabilistic nature through its quality dimensions, test design, Contracts, evidence and Oracles to this final point. The architecture remains coherent only if the same distinctions survive the whole chain. A deterministic harness is not a deterministic AI subject; an Oracle verdict is not the ground truth; a large result count is not necessarily broad coverage; a Gate failure is not always a subject defect; an override is not a pass.

The result is a framework for decisions that can be challenged precisely. A reviewer can ask why an aggregate had its basis, why a Gate did not pass, whether an Oracle was admitted and who approved an exception. If the implementation preserves these answers, quality claims become auditable engineering propositions rather than confidence theatre. That is the purpose of the final decision layer: not to make uncertainty vanish, but to stop it from being concealed.

Engineering Reflection

An Aster Dashboard says that 100 percent of tests passed, but the aggregate contains only one actionable Result from one Scenario. What can be concluded, what basis must be disclosed, and what happens if the Gate requires a larger minimum?

A Release Decision was authorised after a Safety Gate Override. A second Gate is blocked for insufficient basis. What additional approval or evidence is required, and why does the original Override not solve the second problem?

Common Misconceptions

“No failed actionable Results means the Gate passes.” No actionable Result means the aggregate cannot be computed, and the Gate cannot pass.

“A non-passing Gate means the product failed its threshold.” Insufficient basis, unavailable Oracles, admission failures and control misses are different reasons.

“An Override turns the original failed Gate into a pass.” The failure remains recorded; the approved exception is scoped to one Gate Result.

“A live Dashboard is sufficient historical release evidence.” A Report is frozen at generation, while a Dashboard continues to change.

“Conformance of the AQEF engine means its AI subject is approved.” Implementation Conformance, product-quality assessment and authorisation to ship are separate claims.

Chapter Summary

The Decision Pipeline applies Confidence treatment before Aggregation and evaluates declared Quality Gates before a governed release action.

Only effective, actionable assessments may support the intended aggregate; historical Results are preserved rather than silently rewritten.

Aggregate Basis records actionable and non-actionable Results and distinct Scenarios and Executions, enabling honest interpretation of a rate.

Threshold failure, insufficient basis and instrument disqualification are different evidence states and demand different responses.

A Gate with no actionable Results cannot pass even without an explicit minimum; a permanently unfed Gate definition is a configuration error.

Blocking Gates affect authorisation, advisory Gates remain visible, and changes to their status are governed.

A Gate Override covers exactly one Gate Result and cannot silently excuse other failures or future evaluations.

Reports, Snapshots, Audit and Traceability preserve what was known, which criteria applied and who authorised a decision.

Review Questions

1. A Suite has thirty nominal Results, but ten are inconclusive and five were invalidated by a Seeded Control. What information must the aggregate show before a Gate may read it?
2. Why is a 95 percent pass rate over twenty repetitions of one Scenario not equivalent to the same rate over twenty distinct Scenarios?

3. A Gate has zero actionable Results and no configured minimum Aggregate Basis. Can it pass? Explain the applicable reason and value.
4. How is a Gate with no actionable evidence different from one that no Contract clause could ever feed?
5. Why must a Gate's failure reasons distinguish judge_not_admitted from a measured threshold violation?
6. Explain how a later Human Reviewer Result can become effective without the original Judge Result being deleted.
7. A manager changes an advisory Gate to blocking after a deployment. Which governance and historical questions arise?
8. Why does an Override for one Safety Gate Result not authorise a later release or a second failed Gate?
9. Describe how a reviewer would trace one Release Decision back through its frozen Report, Snapshot, Baselines and underlying Evidence.
10. In the Aster case study, latency improves, Safety fails and Semantic Quality lacks basis. Write a short release explanation that keeps all three findings distinct.

Sources

1. AQEF 1.0-CR1, Volume I, Chapter 6 — Decision Pipeline, Confidence Model, Result Resolution, Aggregation, Aggregate Basis, Quality Gates and Release Decision.
2. AQEF 1.0-CR1, Volume II — Result, Metric, Quality Gate, Report, Baseline, Snapshot, Release Decision and related entities.
3. AQEF 1.0-CR1, Volume VI — Admission, Confidence, Human Review and Judge reliability.
4. AQEF 1.0-CR1, Volume VIII — Seeded Controls and invalidation semantics.
5. AQEF 1.0-CR1, Volume XI — Reports, Dashboards, Gate display, KPIs and Historical Analysis.
6. AQEF 1.0-CR1, Volume XII — Approval Workflow, Audit, Traceability and Compliance.
7. AQEF 1.0-CR1, Volumes XIII–XVI and Front Matter §0 — At Risk integration and interface capabilities.

Appendix A — AQEF 1.0-CR1 Capability and Maturity Map

This map describes the *specification edition*, not quality certification of an implementation or AI product. AQEF 1.0-CR1 is a **Candidate Recommendation**, not an adopted Standard; its conformance requirements concern an **implementation**. A claim is always scoped to a published Conformance Statement. The CTS offers bounded evidence of implementation behaviour.

A.1 Non-negotiable core

Every conformant implementation meets five logical responsibilities: **Definition**, **Execution**, **Evidence**, **Assessment**, and **Decision**. It also implements the seven Fundamental Principles: Contracts over Assertions; Validation before Evaluation; Independent Oracles; Deterministic Infrastructure; Probabilistic Subject; Everything is a Test; Everything has Confidence. Their logical responsibilities may share processes or machines.

Core includes Static Datasets and Dataset Versioning, Contracts, Evaluation mechanics, Results, the decision model, and traceable Gates. These are not separate feature claims that the implementer can silently omit.

A.2 Scoped capabilities with implementation evidence at CR1 entry

Human Review (`human_review`), **Multi Judge (`multi_judge`)**, and **Seeded Controls (`seeded_controls`)** were claimed together with the core by an independent reference implementation built from the specification text without access to the CTS. The published CTS found no failure for that stated scope. This neither certifies universal correctness nor requires every conformant implementation to claim all three extensions.

A.3 At Risk — specified, but without reported implementation at CR1 entry

1. action_boundary — action-access declaration and enforcement reporting (Volume III).

2. dataset:generated and dataset:production_replay — additional Dataset sources (Volume IV).

3. plugins:validator, plugins:judge, plugins:dataset, plugins:report — plugins (Volume XVI).

4. ci_cd:<platform> — CI/CD integrations (Volume XIII).

5. sdk:<language> and cli — programmatic/client tooling (Volume XIV).

6. reference_ui — reference user interface (Volume XV).

An At Risk feature may be removed or moved to a later edition if the required implementation evidence does not appear. Its description in a volume is **not** proof that the feature has been implemented or independently conformance-tested.

A.4 Reserved — named but not specified for claims in CR1

dataset:synthetic and *dataset_mechanism:<type>* (Data Mutation, Fuzzing, Edge Cases, Adversarial) are Reserved. In CR1 they cannot be claimed; configurations attempting to use those reserved mechanisms are rejected. This is distinct from using a conventional boundary- or adversarial-testing technique to author a Static Dataset by hand.

A.5 How to read chapter maturity labels

Core marks generally applicable normative machinery. **Implemented** refers here to the documented scoped implementation evidence for CR1, not to a product certification. **At Risk** and **Reserved** follow the edition status above. **Conceptual** identifies the book's explanatory synthesis and is not an additional AQEF capability. A chapter may carry more than one label because it covers multiple kinds of material.

A.6 Test types are not capabilities

Test Types (AQEF Volume IX) classify testing purposes. They are not scoped conformance capabilities and are not claimed in an implementation’s Conformance Statement.

Authoritative basis: AQEF 1.0-CR1 Front Matter §0 (Document Status), §9 (Normative Language), §10 (Conformance); Volume I Chapters 3–4; CTS 1.0-CR1. Check the edition before transferring this map to later versions.

Appendix B — Working Glossary

This **selected working glossary** is an educational quick reference, not a substitute for the complete alphabetical glossary in **AQEF 1.0-CR1, Appendix D** or the authoritative twelve definitions in Front Matter §8. Wording here is shortened for quick recall; where interpretation matters the specification controls.

Admission — Agreement of a specific Judge configuration with a human-labeled Reference Set at a Project-defined threshold; the most recent accepted record controls. (Vol VI)

Aggregate Basis — The numbers of actionable and non-actionable Results, distinct Scenarios and distinct Executions supporting a Gate-read aggregate. (Vol I Ch 6)

Artifact — Auxiliary data captured during Execution beyond the Conversation, such as retrieval passages or tool-call traces. (Vol II)

Baseline — A specific past Execution's Result for one Scenario, explicitly approved as the reference standard for semantic comparison. (§8; Vol X)

Calibration — Comparison of a Judge's reported Confidence with its observed accuracy over time against reference judgments. (Vol VI)

Confidence — Per-Result expression of an Oracle's uncertainty, distinct from the verdict and from statistical confidence in an aggregate. (§8)

Conformance — A property of an AQEF implementation, not of the AI system it tests. Scoped to the core and claimed capabilities. (§10)

Constraint — A deterministic, Validator-bound clause of a Quality Contract, without probabilistic Confidence. (Vol VII)

Context — State accumulated within an Execution across turns, distinct from fixed Variables and Environment. (Vol III)

Conversation — Ordered concrete transcript of Prompts and Responses produced by an Execution. (§8)

Dataset — Named, versioned input collection used to instantiate Scenarios through Variables. (Vol II/IV)

Effective Result — The final Result in a valid resolution chain for an Execution and Contract clause, subject to effective invalidation state. (Vol I Ch 6)

Environment — Named runtime configuration specifying model, prompt, tools and infrastructure target for an Execution. (Vol II)

Evaluation — Application of Judges to Evidence for semantic or subjective assessment after Validation. (§8)

Evidence — Conversation with its associated Artifacts; material an Oracle evaluates, not a separate domain entity. (Vol I/II)

Execution — The runtime realization of a Scenario in an Environment, producing exactly one Conversation, zero or more Artifacts, and one or more Results. (§8)

Expectation — Criteria-based Contract clause evaluated by a Judge or Human Reviewer with appropriate Confidence treatment. (Vol VII)

Gate Result — One evaluation of a particular Quality Gate, retaining outcome, reasons, value, threshold and Aggregate Basis. (App A)

Human Reviewer — A human Oracle used to assess ambiguous or high-stakes Evidence and provide reference judgments. (Vol II)

Judge — Probabilistic criteria-based Oracle producing a verdict and Confidence about subjective or semantic quality. (§8)

Oracle — Abstract assessment role realized as a Validator, Judge or Human Reviewer; each Result traces to one Oracle. (§8)

Quality Contract — Declarative criteria specifying which Oracles apply and what passing and actionable assessments mean. (§8/Vol VII)

Quality Gate — A named thresholded check over aggregated Metrics and an explicit evidence basis, blocking or advisory. (Vol I Ch 6)

Reference Set — Versioned collection of cases with human-established correct verdicts used for Judge Admission. (Vol VI)

Release Decision — Governed release outcome supported by applicable Gates and preserved Report/Snapshot evidence. (Vol I Ch 6)

Result — Recorded assessment by an Oracle of an Execution's Evidence against a Contract clause, including verdict and disposition. (Vol II)

Scenario — Reusable static definition of a testable situation with intent, inputs and governing Contract. (§8)

Seeded Control — A Scenario with known clean or defective Evidence that challenges an Oracle's current-run reliability. (Vol VIII)

Semantic Regression — A degradation against an approved Baseline in the assessed meaning of a Result, not literal text difference. (§8/Vol X)

Snapshot — Frozen broader release/project state, including Baselines and configuration, used for release comparisons. (Vol II/Vol X)

Validator — Rule-based deterministic Oracle that reports explicit Confidence inapplicability, not a probability. (§8)

Variables — Resolved input parameters used in deterministic Prompt Construction, as distinct from accumulated Context. (Vol III)

Appendix C — Answers to Chapter Review Questions

These are concise model answers to the actual end-of-chapter questions. They show one defensible line of reasoning, not the only acceptable wording. Each answer is keyed to the question number in its chapter; refer to the chapter's cited AQEF sections for normative detail.

Chapter 1

1. Compare the policy meaning and relevant exception against the Contract and policy evidence; wording variation alone is not a defect.

2. A required phrase is a structural condition; omission of a material exception violates semantic faithfulness despite the string match.

3. Investigate criteria ambiguity, the Judge's calibration or admission, missing evidence and the policy owner's interpretation before changing Aster.

4. You can establish what Aster claimed, but not that the tool actually completed the cancellation without the tool outcome.

5. Changed retrieval evidence prevents isolating model behaviour: compare the retrieved Artifacts and controlled environment before attributing regression.

6. The five recorded samples passed the defined checks; they do not establish broad coverage, future reliability or universal correctness.

7. Ask about actionable Results, distinct Scenarios and Executions, missing assessments, risk distribution, thresholds and Gate scope.

8. An Oracle can confidently misjudge a good system or incorrectly approve a bad one; subject and instrument require separate evidence.

9. AQEF supplies assessment and traceability mechanisms; the Project defines refund rules and decides which requirements are binding.

Chapter 2

1. Formatting and schema conformance are Functional Quality concerns; factual and policy faithfulness belong to Semantic Quality. Fluent grammar is not evidence of factual accuracy. Averaging an unacceptable semantic finding with passing structural checks can hide a non-compensable Contract violation.

2. Low-risk passes cannot automatically compensate for an unauthorised action; examine Safety hard veto and criticality.

3. An action-capable assistant has additional authority, tool-outcome and external-effect risks, needing trajectory and boundary evidence.

4. Repetitions assess variability in a case; distinct Scenarios broaden situations and defect classes represented.

5. Judge unavailability reveals absent assessment, not product correctness or failure; capture it as insufficient evidence.

6. Examine failures by risk and dimension, shifts in case mix, effective Results and whether Gate thresholds mask critical cases.

7. Faithfulness needs policy context and semantic comparison; authorisation needs action permissions and recorded tool traces/outcomes.

8. When the missing result is essential to a blocking Gate, apparent success elsewhere cannot establish its basis.

9. Show actionable and non-actionable counts plus distinct Scenarios and Executions—the Aggregate Basis.

Chapter 3

1. Independent Oracles is at issue: changed prompt wording does not establish different weights or decision process.

2. The hidden assertion may have been weakened rather than repaired; green status loses meaning without a governed Contract.

3. Ordering avoids unnecessary semantic evaluation; independence addresses correlated oracle-subject errors. One cannot replace the other.

4. Scenario is reusable; Executions, Conversations, Artifacts and Results are run-specific. A comparable approved Baseline is necessary.

5. It mistakes depth of repeated observation for breadth of coverage, ignoring distinct scenarios in Aggregate Basis.

6. Preserve the observation, define Scenario and Contract, assess evidence and link to a Gate; Production Replay is At Risk in CR1.

7. Validators explicitly mark Confidence inapplicable; artificial probabilities would misrepresent deterministic checks.

8. Preserve the Evidence, effective Result, advisory Gate Result, Report and authorised Release Decision. An advisory warning is reported but is not a blocking Gate failure; authorising the release does not alter its recorded status.

9. Show the five logical responsibilities and seven principles, not five separately deployed services.

10. For example, a reproducible harness with a self-evaluating model honours Deterministic Infrastructure but violates Independent Oracles.

Chapter 4

1. Separate engine Conformance and CTS scope, Aster's Contract/Gate findings and the authorised release decision; no AQEF product certificate exists.

2. Inspect the declared scope: CI/CD integration is extra, not mandatory unless the implementation claims it.

3. A binding MUST cannot be waived by project approval without changing the truth of the claimed Conformance.

4. It identifies edition, core and claimed capabilities, telling reviewers which CTS requirements apply.

5. Once Multi Judge is claimed, all its applicable MUST obligations, including Consensus rules, attach.

6. Deployment topology does not change the normative core; check observable implementation behaviour.

7. Describe Production Replay as specified but At Risk and lacking published implementation evidence in CR1.

8. The tested situations met CTS expectations; universal correctness, untested cases and subject quality remain unproved.

9. Implementation demonstrates buildability and exposes runtime ambiguities; text review identifies possible problems without built behaviour.

10. Reconcile changed normative requirements, affected capabilities, CTS coverage and the engine's claim/version evidence.

Chapter 5

1. Scenario defines the case, Execution is its run, Conversation its transcript; repeated evidence and qualified Oracles are needed for reliability.

2. Without criteria and Contracts, the prompts yield observations but no reproducible definition of acceptable quality.

3. Positive exercises authorised update, negative rejects forbidden update, blocked-path tests unavailable permitted route; context determines acceptability.

4. Keywords are structural signals; a Judge comparing the response with preserved policy evidence tests substantive faithfulness.

5. It may reveal wrong retrieval, expected no-data handling or missing necessary Artifact; classification depends on Contract and trace.

6. Metamorphic tests preserve invariants under transformations; Differential Testing compares the same case across Environments.

7. It adds depth and stability observations, not five hundred distinct risk situations.

8. RAG is a test type and pattern, not a separately claimable Conformance capability.

9. Capture evidence, define risk and Contract, author a Scenario and assess; Production Replay is At Risk in CR1.

10. State strong coverage of ordinary positives, explicit boundary gaps and residual risk; AQEF sets no universal percentage.

Chapter 6

1. A Contract is versioned, declarative and governed; inline checks may hide changing assumptions and weaken traceability.

2. A semantic Expectation against current policy evaluates the contradiction, beyond a deterministic phrase check.

3. Split independent, observable properties into clauses tied to suitable Oracles without fragmenting away meaning.

4. The same clause identity with conflicting settings cannot be silently overridden; the conflict requires governed resolution.

5. Suite-level contributions differ, while Project-level clauses remain binding in each effective Contract.

6. Contract change revises what will count as acceptable under approval; override authorises one particular failed Gate Result.

7. The answer exists, but without relevant policy evidence the Judge may lack basis to judge grounding.

8. Weight changes importance, not interpretability; vague criteria produce unreliable assessment regardless of weighting.

9. Historical Contract versions, effective clauses and associated Executions and Results must remain reconstructable.

10. Deciding to weaken a requirement is a human risk/requirements decision informed by failures and policy, not an automated green-build fix.

Chapter 7

1. The Dataset population changed; compare matched rows and separate additions from product change.

2. Mutable contents destroy knowledge of which Variables the approved Baseline actually exercised.

3. Conversation proves what Aster said; retrieval Artifact shows what material was available to ground it.

4. If template and resolved Variables were identical but harness assembled different literal prompts, deterministic infrastructure failed.

5. Generator and subject may share blind spots; independent generation is SHOULD, whereas Independent Oracles is binding.

6. Silent repair changes the generated population and conceals the validity problem; rejected generation yields no new version.

7. A manually authored hostile static row is not the reserved automatic dataset mechanism.

8. Learning from a production observation to write a static case needs no Production Replay claim; the capability is At Risk.

9. Grounding, tool outcome or action claims needing that Artifact remain unsupported, even if the final text seems plausible.

10. Trace versioned Dataset and Scenario, resolved prompt, Execution, Conversation, Artifacts, Contract clause, Oracle and effective Result.

Chapter 8

1. The repetition adds depth and paraphrase variety but little breadth of risk, defect class or Gate coverage.

2. Execution may never occur, or assessment may be inconclusive/unavailable; a design mapping alone cannot produce actionable Results.

3. Dimension describes quality concern; defect classes describe targeted failures. Multiple classes can inhabit one dimension.

4. Twenty distinct Scenarios support breadth; twenty repetitions support within-case consistency. Neither alone proves general quality.

5. AQEF requires transparent denominators while the Project chooses how much evidence suffices for its risk.

6. Without retrieved context the Judge cannot establish grounding for that Contract clause, so the chain to actionable Result is broken.

7. Trace the specific Gate Result, its override/Approval Workflow, Report, Snapshot, metrics and underlying evidence.

8. Audit asks what occurred broadly; Traceability reconstructs the causal/approval chain for one decision.

9. Two changed variables confound attribution; use pinned versions or controlled comparisons and check Judge stability.

10. State uncovered risks and the uncertainty they create, leaving risk acceptance and release authority with the Project.

Chapter 9

1. Validator establishes structure; Judge compares meaning to the policy; both findings can be true simultaneously.

2. Recorded latency is an Execution Artifact; measuring during validation would mix harness work with subject time.

3. Validation before Evaluation is mandatory; ordering cheaper Validators first is recommended rather than universal MUST.

4. Oracle unavailability establishes no subject verdict; a completed failing Validator establishes an observed contract violation.

5. 0.97 expresses the Judge's reported certainty only; it does not prove correctness, admission or calibration.

6. Shared models can share blind spots; use independent reference judgments, different model families and control evidence.

7. Overwriting destroys provenance and prevents reconstructing how the initial result was resolved.

8. Low Confidence means uncertainty, unavailability means no verdict, and failure means observed violation: Gate reasons differ.

9. AND/OR/veto compose checks for one clause; release Aggregation combines Results across Scenarios and dimensions.

10. Repeating an Oracle call after a verdict has arrived risks selecting a favourable verdict and is prohibited. Transport attempts before any verdict may be recorded within the allowed timeout; a later Result from the same Oracle configuration, once available again, may resolve the `oracle_unavailable` Result against stored Evidence without erasing the original.

Chapter 10

1. Class imbalance may inflate raw agreement; compare verdict distributions and chance-corrected agreement against a diverse Reference Set.

2. Blind independent reviewers and reasoned arbitration reduce shared bias; matching Judge labels would contaminate ground truth.

3. Chance correction adjusts observed agreement for expected agreement by chance; Confidence is the Judge's per-result self-uncertainty.

4. Narrow reference coverage does not establish competence on high-risk financial actions; expand representative cases and independent review.

5. The latest accepted Admission record for that exact Judge configuration controls its current status. An accepted record with `admitted: false` withdraws earlier admission without deleting historical records; a changed configuration needs separate Admission.

6. Treat it as a new Judge configuration requiring its own Admission assessment.

7. Where the Gate sets require_admitted_judges, an unadmitted Judge blocks with judge_not_admitted.

8. Calibration needs accuracy/Confidence comparisons over history; it can miss sudden blindness in the current run.

9. Admission tests reference agreement before use; Calibration tests historic Confidence alignment; Controls test specific current-run defects.

10. Check Dataset, Contract, Oracle identity, human reference agreement, calibration and seeded controls before claiming subject improvement.

11. Agreement on 18/20 = 90% describes the exposed working cases used to select or adjust the Judge; it is tuning evidence, not independent Admission. Freeze the selected model, prompt and criteria; use a separate, versioned, human-labelled Reference Set not exposed to the tuner; report its label distribution,

raw agreement and named chance-corrected agreement; compare the corrected measure with the Project threshold declared before evaluation; and record Admission. A Gate that explicitly requires admitted Judges then uses the current record for that same configuration.

12. The illustrative 2×2 counts are human pass/Judge pass = 24, human pass/Judge fail = 6, human fail/Judge pass = 5, human fail/Judge fail = 25. Correct = 24 + 25 = 49, so observed agreement is 49/60 = 0.81667. Reference totals are 30 pass and 30 fail; Judge totals are 29 pass and 31 fail. Expected chance agreement is $[(30 \times 29) + (30 \times 31)]/60^2 = 1800/3600 = 0.50$. Cohen's $\kappa = (0.81667 - 0.50)/(1 - 0.50) \approx 0.63333$. Since κ is below the predeclared threshold of 0.70, the Judge configuration is NOT ADMITTED. Both the matrix and threshold are illustrative.

Chapter 11

1. It demonstrates a class-specific Oracle detection failure, not a defect in Aster itself.

2. Clean controls reveal false alarms; a Judge that fails everything would otherwise catch every planted defect.

3. Record invalid: the premise of the control was not present in the evidence the Oracle assessed.

4. The default affects same-Oracle/class Results; halt_run explicitly invalidates and stops the full run.

5. An undeclared clause is treated as covering all that Oracle's tested classes and is affected by the miss.

6. Immutability retains historical truth; Gate actionability changes by derived invalidation, not rewriting verdicts.

7. Exposure makes control recognition possible; a fresh independent control better tests generalised detection.

8. A prior contextualises difficulty using independent reference detection and must not be learned from the target Oracle itself.

9. Record corrected Oracle configuration, defect class, and passing fresh controls; then reassess stored Evidence as new Results.

10. Historical reference agreement cannot rule out a new current-run class-specific failure.

Chapter 12

1. All five share the same model/criteria and thus sample one instrument's variation, not independent blind spots.

2. Four votes are required; absent a qualifying pass or fail count, use on_disagreement, default Human Review.

3. Configured is original membership; counted is the eligible, qualified population actually used in Consensus.

4. A Seeded Control exclusion can leave two; only an explicit sufficient min_qualified permits assessment without the full panel.

5. Exclusion can change an odd configured count to even and yield a tie.

6. No, panel Admission requires every member admitted; a Gate chooses whether admission is required via policy.

7. Vote share measures agreement, not calibrated probability that the verdict is correct.

8. Inspect the Conversation, authoritative tool outcome, relevant Contract clause and each participating Judge's grounds for disagreement. Record a distinct Human Reviewer Result resolving the disputed assessment rather than overwriting panel history.

9. High-stakes veto risks may justify unanimity; changing thresholds requires governed Contract/Gate changes, not silent adjustment.

10. The panel assesses Contract clauses from Evidence; qualified effective Results feed aggregation and Gates before release governance.

Chapter 13

1. Inspect the retrieved source's currency and version first: the generator may be faithful to outdated evidence.

2. Existence of a citation does not establish entailment; compare the precise claim with the cited passage.

3. A schema Validator checks syntactic validity and required fields. Authorisation and semantic appropriateness depend on the task, the permitted action and the complete trajectory; the `action_boundary` enforcement capability is At Risk in CR1, so a recorded trace alone does not prove effective isolation.

4. Different paths are acceptable when their actions, effects, ordering constraints and final outcomes satisfy the Contract.

5. Deterministic context assembly isolates subject variation from harness variation and supports reproducible evidence.

6. It may retain the date yet later contradict it or misuse it when reasoning about eligibility.

7. Preserve inter-agent messages, handoffs, tool traces, retrieved context and final customer-facing Conversation.

8. Human-in-the-loop approval is subject runtime behaviour; Human Reviewer evaluates the subject as an external Oracle.

9. Tool traces can show recorded calls; Action Boundary enforcement is a distinct, At Risk capability in CR1.

10. Use risk-linked Contracts for retrieval, grounding, actions, continuity and operations, with distinct Oracles and Gates.

Chapter 14

1. Lower-trust retrieved text can contain instructions that an agent mistakenly elevates above its authorised task.

2. Structural validity does not establish permissible effect; inspect authority, trajectory and tool outcome.

3. Declaration lists routes; enforcement requires control of every path; uncontrolled access demands enforced:false reporting.

4. Approved, pinned reference prevents the latest possibly defective execution silently becoming the standard.

5. Paraphrasing the same faithful answer is not regression; nearly unchanged text that omits a new exception can be.

6. Compare prompt/template, model resolution, Dataset version and independent Oracle/human reference evidence.

7. Snapshot captures broader release configuration and Baselines; Baseline compares a particular Scenario against its approved reference.

8. It measures response to failed dependencies and may also reveal attempts to bypass authorisation boundaries.

9. Runner contention, scheduling or instrumentation may inflate measured time independent of subject processing.

10. Keep operational improvement, blocking safety failure and residual risk separate; do not let weights erase vetoes.

Chapter 15

1. Show actionable and non-actionable counts, distinct Scenarios and Executions; invalidated outcomes cannot be counted actionable.

2. Repetition provides depth for one case; distinct Scenarios support breadth; aggregate basis must expose the difference.

3. No. The value is not computed and insufficient basis must be reported even without a configured minimum.

4. Empty observed basis blocks evaluation; a Gate that no clause can ever feed is a configuration error rejected at load.

5. Unadmitted Judge indicates untrustworthy instrument, not measured subject performance below threshold.

6. A new Human Reviewer Result resolves the Judge Result; the original remains preserved and is not counted again.

7. The change alters release policy and needs Approval Workflow, while prior Gate Results/Decisions retain their historical meaning.

8. Override identifies exactly one Gate Result—one gate, evaluation and batch—not later runs or other failures.

9. Trace the frozen Report, applicable Gate Results, Snapshot, approvals, Baselines, effective Results, Contracts, Executions and Evidence.

10. The release has a measured latency improvement, a failed Safety Gate and insufficient actionable basis for Semantic Quality. The improvement cannot cancel a blocking safety failure; shipping requires the applicable Gate policy and explicit governed exception, while the unresolved semantic basis must remain visible.

Appendix D — Sources and Edition Control

Normative primary source

AQEF — AI Quality Engineering Framework, 1.0-CR1 (Candidate Recommendation 1; October 2026). Authored by Igor Akymenko. GitHub repository: https://github.com/igorakymenko-create/AQEF. Relevant divisions: Front Matter §§0, 6, 8–10; Volume I (Architecture & Concepts); Volume II (Domain Model); Volumes III–VI (Execution, Dataset, Validator, Judge Engines); Volumes VII–X (Testing Methodology); Volumes XI–XIV (Reporting, Governance, CI/CD, SDK); Volumes XV–XVII (UI, Plugins, Reference Implementations); specification Appendices A, C and D.

AQEF Conformance Test Specification (CTS), 1.0-CR1. Companion testing document in the repository, including Section 0 (purpose and limits) and Section 90 (requirement traceability). CTS outcomes are bounded checks of implementation conformance; they do not establish that a particular AI product is accurate or safe.

Interpretation hierarchy

Where the textbook paraphrases AQEF, the authoritative specification text wins. Verbatim normative passages, their references and the book's educational interpretations must remain distinguishable. A direct quotation belongs to the edition it cites; it must not silently be reused for a later edition. The author-created Aster Support AI is an illustrative example, not a tested or certified product.

Scope and maturity

This book explains test design, implementation-facing mechanisms, evidence and decisions for QA Engineers and QA Leads without requiring programming. It does not prescribe company policy, legal compliance, model training, model rankings or a universal release-risk threshold. The maturity labels represent the AQEF 1.0-CR1 edition and may change in later editions of the specification.

Citation and licence note

This textbook and its source specification AQEF 1.0-CR1 are published under Creative Commons Attribution 4.0 International (CC BY 4.0). The source is AQEF by Igor Akymenko, https://github.com/igorakymenko-create/AQEF; license: https://creativecommons.org/licenses/by/4.0/. The diagrams and explanatory analyses in this educational edition do not establish additional normative AQEF rules.